Introduction à l'Informatique

Bernard Boigelot
Université de Liège

2026

Table des matières

Avant-propos **ix**

1 Les ordinateurs, les algorithmes et les programmes **1**

1.1 Introduction 1

1.1.1 Qu'est-ce qu'un ordinateur ? 1

1.1.2 Une petite histoire de l'informatique 4

1.2 L'algorithmique 7

1.2.1 La notion d'algorithme 7

1.2.2 Exemples de problèmes algorithmiques 8

1.3 La programmation 12

1.3.1 L'implémentation d'un algorithme 12

1.3.2 Les langages de programmation 13

1.3.3 Un premier programme 14

1.3.4 La compilation et l'exécution d'un programme 19

1.4 Quelques considérations théoriques 20

1.4.1 Les ressources consommées 20

1.4.2 Les limitations de l'informatique 21

1.5 Exercices 22

2 Les bases du langage C **23**

2.1 Introduction 23

2.2 Les variables 24

2.2.1 Les types de base 25

2.2.2 La déclaration des variables 28

2.2.3 La portée d'une déclaration 29

2.3 Les expressions 30

2.3.1 Les opérateurs arithmétiques 31

2.3.2 Les opérateurs de comparaison 32

2.3.3 Les opérateurs booléens 33

2.3.4 Les opérateurs d'affectation 34

2.3.5 Les opérateurs d'incrément et de décrément 35

2.3.6 L'opérateur virgule 36

2.3.7 La conversion de type 37

2.3.8 Les expressions conditionnelles 38

2.4 Les instructions de contrôle 39

2.4.1 Le choix conditionnel binaire 39

2.4.2 Le choix conditionnel multiple 43

2.4.3 Les instructions de boucle 46

2.4.4 Les instructions de rupture de séquence 50

2.5 Les commentaires 53

2.6 Exercices 54

3 Quelques notions d'algorithmique **57**

3.1 La recherche de nombres parfaits 57

3.1.1 Première solution 58
3.1.2 Deuxième solution 59
3.1.3 Troisième solution 60
3.1.4 Comparaison des performances 64
3.2 La complexité en temps 65
3.2.1 Principes 65
3.2.2 La notation “grand-O” 66
3.2.3 Les classes de complexité 69
3.2.4 Application aux programmes de recherche de nombres parfaits 70
3.3 L’analyse d’un programme 73
3.3.1 Les triplets de Hoare 74
3.3.2 Les invariants de boucle 77
3.3.3 Illustration 79
3.3.4 La terminaison d’un programme 87
3.4 Exercices 89

4 Les fonctions et les procédures 91

4.1 Introduction 91
4.2 La programmation des fonctions 92
4.2.1 La définition d’une fonction 92
4.2.2 Les paramètres d’une fonction 95
4.2.3 La déclaration d’une fonction 96
4.2.4 L’invocation d’une fonction 98
4.2.5 Les notions d’interface et d’implémentation 99
4.3 La récursivité 101

4.3.1 Principes . . . 101
4.3.2 La complexité en espace . . . 103
4.3.3 La terminaison d'une fonction récursive . . . 106
4.3.4 Application : les tours de Hanoï . . . 106
4.4 Les variables globales . . . 113
4.4.1 Définition et déclaration . . . 113
4.4.2 Exemples . . . 115
4.5 Les macros . . . 118
4.6 Exercices . . . 121
4.6.1 Principes . . . 121
4.6.2 Énoncés . . . 122
4.7 Exercices supplémentaires . . . 126

5 Les tableaux et les chaînes de caractères 133

5.1 Les vecteurs . . . 133
5.1.1 La définition et l'utilisation des vecteurs . . . 134
5.1.2 Application : le crible d'Ératosthène . . . 135
5.2 Le passage de tableaux à une fonction . . . 138
5.2.1 Principes . . . 138
5.2.2 Illustration . . . 140
5.2.3 L'arithmétique sur les pointeurs . . . 140
5.3 Le tri d'un vecteur . . . 141
5.3.1 Le tri par insertion . . . 142
5.3.2 Le tri par fusion . . . 145
5.4 Les chaînes de caractères . . . 151

5.4.1 Principes 151
5.4.2 Exemples 153
5.5 Les tableaux multidimensionnels 156
5.5.1 Définition et utilisation 156
5.5.2 Le passage d'un tableau multidimensionnel à une fonction 157
5.5.3 Application : bibliothèque de transformations géométriques 158
5.6 Exercices 161

6 Les structures et les pointeurs **177**

6.1 Les données structurées 177
6.1.1 La définition d'un type de données structuré 178
6.1.2 L'utilisation des données structurées 181
6.1.3 Application : manipulation de nombres complexes 183
6.2 Les pointeurs 186
6.2.1 Le passage d'arguments par variable 186
6.2.2 Les pointeurs en langage C 186
6.2.3 Les pointeurs vers des données structurées 190
6.2.4 Les pointeurs et les tableaux 193
6.3 L'allocation dynamique de mémoire 195
6.3.1 Principes 195
6.3.2 La copie de blocs en mémoire 197
6.3.3 Application : retournement d'une séquence de valeurs 198
6.3.4 L'allocation dynamique de tableaux 202
6.4 Les paramètres de `main` 209
6.5 Exercices 210

7 Les piles et les files **233**

7.1 Les structures de données 233

7.2 Les piles 234

7.2.1 Définition 234

7.2.2 Interface 234

7.2.3 Implémentation à l'aide d'un vecteur 236

7.2.4 Application : appariement de parenthèses 240

7.2.5 Implémentation à l'aide d'une liste liée 244

7.3 Les files 245

7.3.1 Définition 245

7.3.2 Interface 249

7.3.3 Implémentation à l'aide d'un vecteur 250

7.3.4 Implémentation à l'aide d'une liste liée 256

7.4 Exercices avancés 260

Avant-propos

Les ordinateurs sont devenus des objets indispensables de la vie quotidienne. Nous en dépendons pour nous informer, nous divertir, communiquer et travailler. Il existe aussi des ordinateurs moins directement visibles qui sont employés comme composants essentiels de dispositifs plus complexes ; par exemple, une voiture moderne en comprend plusieurs dizaines. De nos jours, la plupart des ingénieurs, quelle que soit leur discipline, sont amenés à devoir réaliser ou gérer des développements informatiques.

Ce cours a pour objectif d'apprendre à utiliser un ordinateur pour résoudre des problèmes. Il comprend une introduction à l'*algorithmique*, qui étudie comment passer d'un problème à une procédure effective permettant de le résoudre, et à la *programmation*, qui vise à exprimer une telle procédure sous la forme d'un *programme* pouvant être exécuté par un ordinateur. Dans ce cours, la programmation est introduite à l'aide du langage de programmation C, qui a l'avantage d'être simple, très utilisé, et de permettre d'introduire aisément des concepts et des mécanismes possédant une portée plus générale.

Ces notes de cours sont structurées de la façon suivante :

— Le chapitre 1 introduit la matière du cours, en commençant par une brève histoire de l'informatique et en définissant les notions d'algorithme et de programme.

— Le chapitre 2 décrit les bases du langage C, en fournissant suffisamment d'éléments pour pouvoir commencer à rédiger des programmes simples.

— Le chapitre 3 est consacré à l'algorithmique. Il aborde un problème élémentaire (la recherche de nombres parfaits), et discute de la façon de le résoudre en développant des solutions de plus en plus élaborées. Des mécanismes permettant de comparer les performances des solutions obtenues et de prouver rigoureusement que ces dernières sont correctes sont également introduits.

— Le chapitre 4 définit le concept de fonction. Celui-ci permet de structurer un programme en une combinaison de fragments plus simples, et de réutiliser facilement des solutions

développées pour des sous-problèmes. Ce chapitre présente ensuite le mécanisme d'exécution récursive d'une fonction, qui permet de résoudre élégamment certains problèmes algorithmiques.

— Le chapitre 5 décrit les éléments du langage C liés à la manipulation de tableaux et de chaînes de caractères, et illustre leur utilisation dans le cadre de deux problèmes algorithmiques : le calcul de nombres premiers et le tri d'un ensemble de valeurs.

— Le chapitre 6 aborde la représentation de données structurées, et introduit la notion fondamentale de pointeur. Il présente également les mécanismes permettant d'allouer dynamiquement de la mémoire pour accommoder des données de taille variable.

— Le chapitre 7 étudie deux structures de données importantes : les piles et les files, et discute de leurs implémentations possibles. Le fait qu'une même structure puisse être implémentée de différentes façons est un mécanisme essentiel au développement de programmes possédant de bonnes propriétés de modularité.

Chacun de ces chapitres se termine par une série d'exercices, servant en partie de base aux séances de travaux pratiques. Les exercices du dernier chapitre ne font pas partie de la matière du cours d'*Introduction à l'informatique* ; ils visent à donner quelques exemples de problèmes d'algorithmique et de programmation plus avancés, susceptibles d'intéresser les étudiants qui souhaiteraient aller plus loin.

Ce cours ne constitue qu'une première étape dans l'apprentissage de l'algorithmique et de la programmation ; sa matière est destinée à être approfondie et complétée par d'autres enseignements. En particulier, les cours de *Structures de données et algorithmes* et de *Compléments de programmation* continuent l'apprentissage de l'algorithmique et de la programmation. *Object-oriented programming* et *Programmation fonctionnelle* explorent d'autres paradigmes de programmation qui complètent celui introduit dans ce cours. *Organisation des ordinateurs* s'intéresse quant à lui aux principes de fonctionnement des machines capables d'exécuter les programmes tels que ceux développés dans ce cours ; il s'agit d'un sujet qui est ensuite approfondi dans *Computation structures*. Cette liste de cours n'est pas exhaustive.

Chapitre 1

Les ordinateurs, les algorithmes et les programmes

1.1 Introduction

Dans ce cours, nous allons apprendre à programmer un ordinateur. Avant de pouvoir expliquer ce que cela signifie exactement, la première étape est de définir précisément ce que l'on entend par "ordinateur".

1.1.1 Qu'est-ce qu'un ordinateur ?

Si l'on demande à des personnes prises au hasard d'expliquer ce qu'est un ordinateur, il est probable que la plupart d'entre elles vont évoquer le PC (*Personal Computer*, ordinateur personnel) qu'elles utilisent pour naviguer sur Internet, interroger des moteurs de recherche ou des outils d'intelligence artificielle, jouer à des jeux ou communiquer avec d'autres personnes. Une autre forme d'ordinateur que nous connaissons tous est bien sûr le *smartphone* qui a de plus en plus tendance à remplacer le PC pour les activités que nous venons de citer. On peut aussi mentionner les grands systèmes informatiques que l'on trouve par exemple dans le siège central des banques, chargés de gérer les données financières relatives à un ensemble de comptes. Un autre exemple d'ordinateurs est donné par ceux qui permettent de résoudre des problèmes scientifiques, par exemple simuler l'évolution des paramètres de l'atmosphère en vue de prévoir la météo, ou l'écoulement de l'air à la surface d'une aile d'avion.

À côté de ces types d'ordinateurs qui sont connus du grand public, il existe d'autres formes de systèmes informatiques qui sont en fait beaucoup plus répandus, mais moins directement visibles : il s'agit des systèmes *enfouis (embedded systems)* ou *embarqués*, qui sont des ordinateurs

utilisés comme composants de dispositifs plus complexes. On trouve de tels systèmes enfouis dans de nombreux objets et appareils de la vie quotidienne : télévisions, électroménager, thermostats, centrales d'alarme, véhicules, dispositifs médicaux, ascenseurs, … Une voiture moderne comprend ainsi plusieurs dizaines d'ordinateurs dont la fonction va du contrôle en temps-réel des paramètres du moteur et des dispositifs d'aide à la conduite à la gestion des phares et des essuie-glaces.

Une question naturelle consiste donc à déterminer quel est le point commun entre tous les types d'"ordinateurs" que nous venons de citer. Ce point commun est qu'ils sont tous amenés à *traiter des données*. Par exemple, un ordinateur employé pour prévoir la météo est alimenté par un ensemble de données fournissant des informations de température, de pression atmosphérique et d'humidité en un grand nombre d'endroits, et aura pour tâche de traiter ces données en leur faisant subir un certain nombre de calculs afin de déterminer la meilleure estimation possible de l'évolution de ces paramètres au cours du temps.

Cette notion de traitement de données possède une portée plus générale. Le fait de résoudre un problème peut être vu comme un cas particulier de traitement de données. Par exemple, l'équation du second degré

$$2x^2 + 3x - 2 = 0$$

est entièrement caractérisée par ses trois coefficients $[2, 3, -2]$. On peut donc considérer que l'opération consistant à résoudre cette équation est un traitement des données d'entrée $[2, 3, -2]$, dont le résultat est formé par les données de sortie $\left\{-2, \frac{1}{2}\right\}$.

Un ordinateur est donc une machine capable de traiter des données, c'est-à-dire recevoir ces données en entrée, leur faire subir un certain traitement, et produire des données de sortie représentant le résultat de ce traitement. Comme le montrent les exemples d'ordinateurs discutés précédemment, les données traitées peuvent prendre différentes formes ; il peut s'agir de nombres, de textes, d'images, de sons, …

Cette définition de l'ordinateur n'est cependant pas complète. Une caractéristique fondamentale d'un ordinateur est que tous les traitements de données qu'il effectue sont basés sur des *opérations préétablies*. Cela signifie que l'ordinateur n'est pas directement capable de déterminer lui-même quelles opérations de traitement il doit effectuer : l'ensemble de ces opérations et l'ordre dans lequel elles doivent être réalisées doivent lui être explicitement fournis.

Dans le cas d'une équation du second degré comme celle que nous avons déjà évoquée, nous avons appris à l'école secondaire quelles opérations effectuer afin de la résoudre dans le cas général : étant donné l'équation

$$ax^2 + bx + c = 0,$$

on commence par déterminer si $a \neq 0$. Si c'est le cas, on calcule $\Delta = b^2 - 4ac$. On teste ensuite si Δ est négatif, nul ou positif, et ainsi de suite. La solution générale de ce problème prend la forme d'une procédure qui décrit précisément quelles opérations effectuer et dans quel ordre afin d'obtenir l'ensemble des solutions de l'équation. Une telle procédure ne laisse aucune li-

berté d'interprétation, et peut être suivie par quiconque comprend les opérations à réaliser et est capable de les effectuer sans erreur.

C'est ainsi que fonctionne un ordinateur : si on lui fournit un *programme* qui décrit rigoureusement un ensemble d'opérations de traitement de données à effectuer dans un certain ordre, l'ordinateur est capable d'*exécuter* ce programme rapidement et correctement. Nous avons donc la définition suivante.

> Un ordinateur est une machine capable de résoudre des problèmes et de traiter des données en effectuant des opérations préétablies.

Au cinéma, on voit souvent des ordinateurs qui font preuve d'intuition ou d'intelligence. Dans la réalité, à leur niveau le plus fondamental, les ordinateurs ne sont ni intelligents, ni capables d'initiative : ils se contentent d'exécuter mécaniquement les programmes qu'on leur fournit.

Cette constatation peut sembler entrer en contradiction avec l'émergence de l'*intelligence artificielle*, qui permet aux systèmes informatiques modernes de tenir une conversation, répondre à des questions complexes, résoudre des problèmes scientifiques, battre le meilleur joueur humain aux échecs ou au jeu de go, traduire un texte ou conduire une voiture de façon autonome. Cependant, si des ordinateurs sont capables de telles prouesses, c'est uniquement parce qu'on les a dotés de programmes qui précisent quelles opérations doivent être effectuées pour y arriver. Un ordinateur jouant aux échecs fonctionne de la même façon que celui qui résout une équation du second degré : tous deux se contentent d'exécuter sans réfléchir les instructions de leur programme. L'"intelligence" dont font preuve de tels systèmes informatiques ne réside pas dans les circuits de leur ordinateur, mais plutôt dans les programmes et les données qui sont fournis à cet ordinateur.

Le but de ce cours est d'apprendre à programmer un ordinateur, c'est-à-dire rédiger un programme qui permet à un ordinateur d'effectuer un traitement de données particulier ou de résoudre un problème donné. Pour réaliser cela, deux étapes sont nécessaires. Premièrement, étant donné un problème à résoudre, il s'agit de trouver une procédure spécifiant quelles opérations de traitement doivent être effectuées pour arriver à une solution. Une telle procédure s'appelle un *algorithme*, et la discipline qui étudie le développement d'algorithmes est l'*algorithmique*.

Quand un algorithme a été obtenu pour un problème donné, l'étape suivante consiste à le *programmer*, c'est-à-dire l'exprimer sous une forme, le *programme*, qui peut être fournie à un ordinateur en vue d'être exécutée. L'objectif dans ce cours est d'explorer à la fois l'aspect algorithmique et l'aspect programmation de l'informatique, afin d'être capable de passer de l'énoncé d'un problème à une solution effectivement exécutable par un ordinateur. Le cours fournira suffisamment d'éléments pour pouvoir tester concrètement à l'aide d'un ordinateur personnel les solutions que nous développerons aux problèmes abordés.

1.1.2 Une petite histoire de l'informatique

L'idée d'essayer d'automatiser le traitement des données, en particulier les opérations de calcul, est très ancienne. Il y a plus de 4000 ans, les Sumériens et puis les Babyloniens représentaient déjà les nombres en *notation positionnelle*, qui est un système d'écriture conduisant à des procédés de calcul simples pour des opérations arithmétiques telles que l'addition ou la multiplication. Ce système est proche de celui qui est toujours utilisé de nos jours, si ce n'est que la notation babylonienne reposait sur la base 60 plutôt que la base 10 que nous employons maintenant [1].

Considérablement plus tard, au 9ème siècle de notre ère, le mathématicien persan Muhammad al-Khwārizmī a publié un traité établissant les bases de l'algèbre. En particulier, cet ouvrage fournit une procédure effective permettant de résoudre n'importe quelle équation du second degré, ce que l'on peut considérer comme étant une des premières publications d'un algorithme. Les termes *algorithme* et *algorithmique* sont d'ailleurs dérivés du nom latinisé de ce mathématicien.

Au 17ème siècle, le mathématicien français Blaise Pascal a construit une machine capable de manipuler des nombres, dans le but de faciliter le travail de son père qui était comptable. Cette machine permettait de calculer automatiquement des additions et des soustractions, en implémentant les règles du calcul écrit grâce à un procédé mécanique reposant sur un système d'engrenages. On peut y voir un des premiers dispositifs spécifiquement construits pour traiter automatiquement des données.

Au 19ème siècle, un ingénieur anglais, Charles Babbage, a conçu, construit et fait fonctionner une machine arithmétique mécanique, la *machine à différences (difference engine)*, basée sur les mêmes principes que celle de Pascal, mais capable d'effectuer des opérations plus complexes. L'objectif consistait à générer automatiquement des tables de valeurs pour une fonction polynomiale donnée. Étant donné que n'importe quelle fonction réelle peut être approximée avec un degré de précision arbitraire par un polynôme, une telle machine permet d'imprimer automatiquement et sans erreur des tables de valeurs pour les fonctions élémentaires telles que *exp*, *log*, *sin*, *cos*, *tan*, ... utilisées en mathématique et en ingénierie.

Tout comme la machine de Pascal, on ne peut pas considérer que la machine à différences de Babbage constitue un ordinateur au sens général du terme, car il n'est pas possible de programmer cette machine dans le but de lui faire effectuer n'importe quel traitement de données : la seule opération qu'elle est capable de réaliser est de générer des tables de valeurs pour des polynômes. Par exemple, on ne peut pas l'utiliser pour résoudre une équation du second degré ou jouer aux échecs. Babbage a cependant réalisé qu'il serait possible de concevoir une machine qui pourrait être programmée de façon à effectuer n'importe quelle opération de traitement. Il a alors consacré le reste de sa vie à mettre au point une telle machine, appelée la *machine analytique (analytical engine)*. Cette machine, beaucoup plus complexe que la machine à différences, n'a

1. L'influence de la notation babylonienne est toujours présente de nos jours dans la représentation des angles et des heures, qui utilisent la base 60.

jamais pu être entièrement construite. Conceptuellement, il s'agit cependant du premier modèle d'ordinateur ayant été imaginé.

Les programmes destinés à la machine analytique devaient être représentés sous la forme de *cartes perforées*, c'est-à-dire de cartes en carton percées de trous dont la disposition encode les instructions à effectuer. Ce mécanisme, développé à l'origine pour spécifier les motifs dessinés par des métiers à tisser, a été largement utilisé en informatique jusqu'aux années 1980.

En parallèle à ces travaux visant à construire une machine capable de traiter les données de la façon la plus générale possible, et avant même qu'une telle machine ne soit disponible, des mathématiciens se sont intéressés aux propriétés théoriques des ordinateurs. En particulier, un chercheur anglais, Alan Turing, a établi en 1936 en collaboration avec son promoteur de doctorat Alonzo Church qu'un très petit ensemble d'opérations élémentaires suffit pour implémenter n'importe quel algorithme. Alan Turing a également mis en évidence certaines limitations de l'informatique, en démontrant qu'il existe des problèmes qui ne peuvent pas être résolus par un ordinateur, quels que soient les mécanismes de celui-ci. En d'autres termes, ce résultat prouve l'existence de problèmes qui n'admettent pas de solution algorithmique [2].

Il faut attendre 1941 pour voir arriver la première machine entièrement fonctionnelle possédant toutes les caractéristiques d'un ordinateur : le *Z3*, construit par l'ingénieur allemand Konrad Zuse. À la différence de la machine analytique dont les mécanismes étaient basés sur des jeux complexes d'engrenages, le Z3 était un dispositif électromécanique composé essentiellement de quelques milliers de relais. L'avantage de cette approche est qu'une telle machine est plus facile à construire et à mettre au point qu'un appareil purement mécanique. Bien sûr, il n'est pas immédiat de comprendre comment il est possible d'assembler des composants tels que des relais pour obtenir une machine capable d'exécuter des programmes. Cette question est explorée plus en détail dans les cours d'*Organisation des ordinateurs* et de *Computation structures*.

La puissance de calcul du Z3 était très modeste : une opération d'addition de deux nombres nécessitait de l'ordre d'une seconde, et une multiplication trois fois plus de temps. Les programmes étaient fournis sous la forme de films perforés. Ce mécanisme a été amélioré une dizaine d'années plus tard, quand sont apparus des ordinateurs qui considéraient leurs programmes comme étant des données particulières. Ces ordinateurs étaient donc capables d'effectuer des opérations de traitement qui produisaient des programmes pouvant ensuite être exécutés. Un exemple de tel ordinateur est l'*EDVAC (Electronic Discrete Variable Automatic Computer)* mis au point à l'Université de Pennsylvanie (USA) en 1951. Cet ordinateur était construit à l'aide d'environ 18000 tubes à vide, qui sont des composants purement électroniques capables de remplacer les relais, et atteignait une puissance de calcul de l'ordre de 1200 opérations arithmétiques par seconde. Il pesait 7,8 tonnes, consommait 56 kW de puissance électrique, et coûtait un demi-million de dollars de l'époque.

2. Un exemple de tel problème est le *problème de l'arrêt*, qui vise à déterminer si l'exécution d'un programme informatique donné en entrée finit par s'arrêter, ou bien continue ses opérations indéfiniment.

Une avancée technologique majeure qui a révolutionné l'informatique est l'invention du *transistor* en 1947. Il s'agit d'un composant électronique pouvant être utilisé à l'instar des relais et des tubes à vide pour construire des circuits capables de traiter les données. L'avantage des transistors est cependant qu'ils sont faciles à miniaturiser et beaucoup plus économes en énergie. Il devient alors possible d'équiper les ordinateurs de circuits considérablement plus complexes, ainsi que plus compacts. Le premier ordinateur construit à l'aide de transistors voit le jour en 1953. En 1958, l'invention du *circuit intégré* permet ensuite de miniaturiser davantage les circuits des ordinateurs. Le premier *microprocesseur*, qui intègre au sein d'un seul composant tous les circuits d'un ordinateur chargés d'exécuter les programmes est produit en 1971. Il comprend environ 2300 transistors et est capable d'effectuer environ 92000 opérations par seconde, pour un prix de l'ordre de 200 dollars de l'époque.

À côté de cette évolution technologique dans le domaine de l'électronique, des progrès sont également réalisés quant à la façon d'utiliser les ordinateurs. Des *langages de programmation* tels que *Fortran* (1954), *Lisp* (1958) et *COBOL* (1959) sont créés afin de faciliter le développement de logiciels de plus en plus complexes. Ces langages archaïques sont toujours en partie utilisés de nos jours. Le système d'exploitation *Unix* commence à être développé en 1969, et est toujours largement employé [3].

La première version du langage de programmation *C*, qui est celui étudié dans ce cours, apparaît en 1972. Il s'agit toujours actuellement du langage de programmation le plus utilisé. À la même époque voient également le jour les premiers ordinateurs destinés au grand public, sous la forme de micro-ordinateurs (1978) et d'ordinateurs personnels (*Personal Computer, PC*, 1981). Le système d'exploitation *Windows* fait son apparition en 1985, le *World-Wide Web* en 1989, et le langage de programmation *Java* en 1995.

Ce processus d'évolution technologique se poursuit. De nos jours, les microprocesseurs qui équipent les ordinateurs modernes comprennent des milliards de transistors, dont les dimensions ont été réduites jusqu'à être de l'ordre de la distance qui sépare quelques dizaines d'atomes de silicium. Ces microprocesseurs sont capables d'effectuer plusieurs centaines de milliards d'opérations par seconde, pour un prix allant de quelques dizaines à des centaines d'euros. De nouveaux langages et environnements de programmation sont également régulièrement créés afin de simplifier le processus de développement de logiciels. Enfin, l'essor des outils d'intelligence artificielle permet maintenant aux ordinateurs d'effectuer de plus en plus de tâches autrefois réservées à l'être humain.

3. Ce système est notamment à la base de *Linux*, sur lequel repose en particulier *Android*, ainsi que de *macOS*.

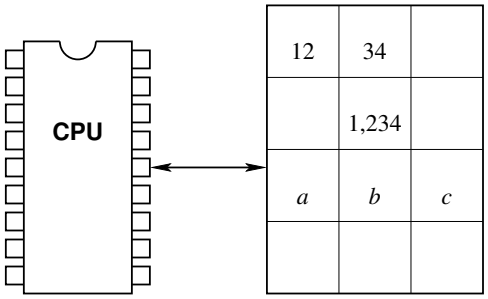


FIGURE 1.1 – Modèle conceptuel d'un ordinateur

1.2 L'algorithmique

L'algorithmique s'intéresse à la façon de produire, à partir de l'énoncé d'un problème, un algorithme capable de résoudre ce dernier, en d'autres termes une procédure pouvant être exécutée par un ordinateur.

1.2.1 La notion d'algorithme

Pour pouvoir raisonner sur des algorithmes, il est nécessaire de faire des hypothèses sur le type d'ordinateur dont on dispose ; en particulier, il faut spécifier la nature des opérations que cet ordinateur est capable d'effectuer. Notre but est de rester le plus indépendant possible des détails matériels de l'ordinateur utilisé, afin de pouvoir développer des algorithmes pouvant potentiellement être exécutés par n'importe quel ordinateur.

À un niveau d'abstraction élevé, on peut considérer qu'un ordinateur possède deux composants principaux, illustrés à la figure 1.1. Le premier composant est le *processeur (Central Processing Unit, CPU)*, qui est chargé d'exécuter les programmes en effectuant les opérations de traitement des données. Le deuxième composant est une *mémoire* qui retient toutes les données manipulées par le processeur. Ces données incluent les données d'entrée qui sont fournies aux programmes, les données de sortie produites par ceux-ci, et les données de travail dont les programmes ont besoin pour gérer les résultats intermédiaires des opérations qu'ils effectuent. Les programmes eux-mêmes peuvent être vus comme une donnée particulière qui est également placée en mémoire.

Les données présentes en mémoire peuvent être de différents types : il peut notamment s'agir de nombres, de textes, ou de données plus complexes. Conceptuellement, on peut considérer que la mémoire est structurée en un ensemble de cases contenant chacune une donnée élémentaire. Au cours de ses opérations, le processeur peut lire le contenu d'une case donnée de la mémoire,

ou modifier le contenu d'une case en y écrivant une nouvelle valeur qui remplace alors la précédente.

Par rapport à ce modèle conceptuel d'ordinateur, on peut maintenant définir ce que l'on entend par algorithme : un *algorithme* est une procédure permettant de résoudre un problème ou de traiter des données en décrivant précisément les opérations qui doivent être effectuées. Une opération effectuée par un algorithme peut correspondre à une lecture ou une écriture en mémoire, ou bien à une opération de traitement interne au processeur.

En reprenant l'exemple de la résolution d'une équation du second degré $ax^2 + bx + c = 0$, on pourrait convenir de placer dans les trois premières cases de la mémoire les données d'entrée a, b et c de ce problème. Un algorithme chargé de résoudre l'équation lirait alors le contenu de ces cases, et effectuerait les opérations de traitement correspondantes, par exemple, une de ses premières opérations consisterait à déterminer si le contenu a de la première case est nul ou non. Au cours de l'exécution de l'algorithme, d'autres cases de la mémoire seraient utilisées pour retenir des résultats intermédiaires, par exemple la valeur du discriminant $\Delta = b^2 - 4ac$. À la fin de l'exécution de l'algorithme, celui-ci laisserait dans des cases prédéterminées de la mémoire le nombre de solutions obtenues et leur valeur.

De nombreux problèmes se prêtent à une résolution algorithmique. Informellement, on peut citer comme exemples ceux qui consistent à calculer la racine carrée d'un nombre positif à une certaine précision près, de déterminer si un nombre naturel est premier ou non, de calculer le plus court chemin entre deux points dans un espace donné, de calculer le meilleur coup à jouer dans une configuration donnée du jeu d'échecs ou de go, de piloter un avion ou une voiture en fonction de consignes particulières et de données fournies par différents capteurs, de détecter des visages dans une image, de traduire un texte ou une conversation du français vers l'anglais, de simuler l'évolution d'un phénomène physique à partir d'une situation initiale connue, ... Dans la section suivante, nous allons examiner plus en détail certains problèmes, en essayant de développer des algorithmes permettant de les résoudre.

1.2.2 Exemples de problèmes algorithmiques

La fonction factorielle

L'énoncé du problème est le suivant : étant donné un entier $n \geq 1$, calculer

$$n! = n\,(n-1)\,(n-2)\,\ldots\,1.$$

Il s'agit d'un problème qui ne présente aucune difficulté car son énoncé mène directement à un algorithme permettant de le résoudre : pour obtenir $n!$ pour une valeur donnée de n, il suffit de calculer successivement n, $n-1$, $n-2$, ... et de multiplier ces nombres. (Une astuce utile est de réaliser que l'on peut terminer l'énumération des facteurs à 2 plutôt qu'à 1, pour $n > 1$.)

Le plus grand commun diviseur

Tous les problèmes n'admettent pas une solution aussi immédiate que le précédent. Nous nous intéressons à présent au calcul du *Plus Grand Commun Diviseur (PGCD)* de deux entiers $a, b \geq 1$ donnés, défini comme étant le plus grand nombre $\gcd(a, b)$ qui divise à la fois a et b.

On peut comme dans le cas précédent chercher à obtenir une solution algorithmique directement dérivée de l'énoncé du problème : puisque $\gcd(a, b)$ divise a et b, on a $1 \leq \gcd(a, b) \leq a$ et $1 \leq \gcd(a, b) \leq b$. Pour calculer $\gcd(a, b)$, il suffit donc d'énumérer tous les entiers appartenant à l'intervalle $[1, \min(a, b)]$, déterminer pour chacun d'entre eux s'il divise à la fois a et b, et garder le plus grand entier qui satisfait cette condition. Une façon simple d'implémenter ce dernier mécanisme consiste à énumérer les éléments de l'intervalle par ordre décroissant de valeur, et s'arrêter dès que la valeur courante divise à la fois a et b.

Cette solution algorithmique naïve est correcte, mais inefficace : pour calculer, par exemple, $\gcd(5229827045084057754, 1570889565818035132)$, elle conduit à devoir effectuer un nombre d'opérations qui est hors de portée des ordinateurs actuels. Nous allons cependant montrer qu'il est possible de développer un meilleur algorithme, capable de produire pratiquement instantanément le résultat de ce calcul [4].

Cette situation est représentative de la plupart des problèmes que l'on rencontre en algorithmique : les solutions que l'on peut directement dériver de leur énoncé sont souvent trop inefficaces pour être utilisables en pratique. Il devient alors nécessaire d'étudier plus en profondeur ces problèmes pour en comprendre la structure et découvrir des propriétés que l'on peut exploiter afin de réduire le nombre d'opérations à effectuer. Il n'existe cependant aucune méthode systématique pour faire cela ; l'algorithmique s'apprend en se construisant petit à petit une expérience de résolution de problèmes variés.

Pour le calcul du PGCD, un algorithme efficace est connu depuis longtemps. Au $3^{\text{ème}}$ siècle avant notre ère, le mathématicien grec Euclide a proposé une solution basée sur les principes suivants.

Premièrement, la définition de la fonction $\gcd(a, b)$ est symétrique en a et b : Pour tous $a, b \geq 1$, on a $\gcd(a, b) = \gcd(b, a)$. Cela permet de simplifier le problème en supposant sans perte de généralité que l'on a $a \leq b$. (Si cette condition n'est pas satisfaite pour des valeurs données de a et de b, il suffit de les permuter.)

Ensuite, on remarque que si un nombre divise à la fois a et b, alors il divise également $b - a$. De façon similaire, si un nombre divise à la fois a et $b-a$, alors il divise également b. On en déduit que l'on a $\gcd(a, b) = \gcd(a, b-a)$. Cette propriété est très intéressante, car elle permet de réduire la taille des instances du problème : remplacer le calcul de $\gcd(a, b)$ par celui de $\gcd(a, b - a)$

4. Ce résultat est égal à 67956946.

conduit à manipuler des nombres plus petits, ce qui est de nature à rendre le problème plus facile.

On peut continuer à appliquer cette propriété de réduction : on a

$$\begin{aligned}\gcd(a,b) &= \gcd(a, b-a)\\ &= \gcd(a, b-2a)\\ &= \gcd(a, b-3a)\\ &\;\;\vdots\\ &= \gcd(a, b-ka),\end{aligned}$$

où k est un entier tel que $ka \leq b$. En choisissant la plus grande valeur possible de k qui satisfait cette condition, on obtient finalement

$$\gcd(a,b) = \gcd(a, b \bmod a),$$

où $x \bmod y$ désigne le reste de la division de x par y. Notons que pour tout $a > 0$, on a $\gcd(a,0) = a$, car 0 est divisible par n'importe quelle valeur non nulle.

Les propriétés que nous avons établies conduisent à un algorithme efficace (l'*algorithme d'Euclide*) pour calculer $\gcd(a,b)$ pour n'importe quels entiers $a, b \geq 1$:

1. Si $a > b$, alors permuter a et b.
2. Tant que $a \neq 0$, répéter l'opération suivante :
   — Remplacer (a,b) par $(b \bmod a, a)$.
3. Retourner la valeur de b.

Le tableau suivant illustre les étapes successives de l'exécution de cet algorithme pour le calcul de $\gcd(24,18)$:

Étape	a	b
1	24	18
2	18	24
3	6	18
4	0	6

L'algorithme d'Euclide est très efficace. Il a été démontré [5] que le nombre d'étapes nécessaires au calcul de $\gcd(a,b)$, pour $a \leq b$, est inférieur ou égal à cinq fois le nombre de chiffres de l'écriture décimale de a.

5. Gabriel Lamé, 1844. *Note sur la limite du nombre des divisions dans la recherche du plus grand commun diviseur entre deux nombres entiers*. Comptes rendus de l'Académie des Sciences, 19 :867–870.

La racine carrée

Nous allons à présent aborder le problème de calculer la racine carrée d'un nombre réel positif ou nul donné. Avant de chercher à le résoudre, il convient d'abord d'en préciser exactement les détails. La difficulté est qu'un ordinateur n'est en toute généralité pas capable de manipuler des nombres réels arbitraires, car pour connaître la valeur d'un tel nombre, il est nécessaire de disposer d'une quantité infinie de données. En effet, si l'on représente les réels en notation positionnelle, chacun d'entre eux est caractérisé par une séquence infinie de chiffres qui sont tous significatifs.

En pratique, la plupart des applications ne nécessitent pas de connaître exactement la valeur des nombres que l'on est amené à manipuler. On opte alors pour une représentation approximative des réels, en fixant par exemple une tolérance affectant les données d'entrée et le résultat des opérations effectuées. On peut ainsi spécifier le problème de calculer la racine carrée d'un nombre réel de la façon suivante : étant donnés un réel $r \geq 0$ connu approximativement et une précision souhaitée $\varepsilon > 0$, le problème consiste à calculer une valeur $\mathrm{sqrt}(r)$ telle que

$$\left|\mathrm{sqrt}(r) - \sqrt{r}\right| < \varepsilon.$$

Contrairement aux deux précédents, l'énoncé de ce problème ne donne pas d'indications immédiates sur la façon de le résoudre algorithmiquement. L'étude des principes qui sous-tendent le développement de tels algorithmes sort du cadre de ce cours [6]. Nous nous contentons de donner ici un exemple de solution connue depuis plusieurs milliers d'années (la *méthode de Héron*) :

1. Partir d'une estimation initiale quelconque x de $\sqrt{r}$, par exemple $x = r$.
2. Remplacer x par $x' = \dfrac{x + \frac{r}{x}}{2}$.
3. Tant que les deux dernières estimations x et x' sont telles que $|x - x'| \geq \varepsilon$, répéter l'étape précédente.
4. Sinon, retourner la valeur de la dernière estimation x'.

Par exemple, pour les données d'entrée $r = 2$ et $\varepsilon = 10^{-6}$, les estimations de $\sqrt{2}$ successivement produites par cet algorithme sont les suivantes.

Étape	Estimation x
1	2.00000000
2	1.50000000
3	1.41666667
4	1.41421569
5	1.41421356
6	1.41421356

6. Ils font notamment partie de la matière du cours d'*Introduction à l'algorithmique numérique*.

L'algorithme de Héron est en fait particulièrement efficace ; on peut ainsi démontrer que chacune de ses étapes permet d'environ doubler le nombre de chiffres corrects du résultat. Signalons cependant que l'implémentation pratique de cet algorithme n'est pas immédiate : comme nous l'avons vu, les représentations des nombres réels utilisées en informatique sont approximatives, et il est nécessaire de garantir que les opérations arithmétiques réalisées aux différentes étapes de l'algorithme produisent un résultat suffisamment précis. Il s'agit également d'une problématique qui est abordée dans le cours d'*Introduction à l'algorithmique numérique*.

1.3 La programmation

Dans la section 1.2.2, nous avons présenté trois exemples de problèmes qui peuvent être résolus à l'aide d'algorithmes. Nous nous intéressons maintenant à l'opération qui consiste à traduire ces algorithmes en des *programmes* pouvant être exécutés par un ordinateur.

1.3.1 L'implémentation d'un algorithme

Nous avons vu que le *processeur* est le composant de l'ordinateur chargé d'exécuter les programmes. Techniquement, un processeur est capable d'exécuter des *instructions*, qui sont des opérations élémentaires que ses circuits sont directement à même d'effectuer. Le *jeu d'instructions* d'un processeur, c'est-à-dire l'ensemble des instructions qu'il est capable d'exécuter, dépend de son *architecture*. Il existe des architectures de processeurs très simples, que l'on rencontre notamment dans des applications embarquées de petite envergure, et d'autres beaucoup plus complexes, comme celle des processeurs qui équipent les ordinateurs personnels modernes [7].

Pour traduire un programme sous une forme qui lui permet d'être exécuté par un processeur, il faut donc exprimer ce programme comme une combinaison d'instructions appartenant au jeu d'instructions de ce processeur. Dans la section 1.1.2, nous avons mentionné qu'Alan Turing a établi que le jeu d'instructions d'un processeur peut être très simple : à l'aide d'un très petit nombre d'instructions [8], il est possible d'implémenter n'importe quel algorithme. Bien sûr, il s'agit de considérations théoriques ; en pratique, on emploie des architectures complexes dans un but d'efficacité, l'objectif étant que les programmes soient exécutés le plus rapidement possible par l'ordinateur.

7. Une telle architecture est étudiée dans le cours d'*Organisation des ordinateurs*.

8. Par exemple, des instructions permettant d'ajouter ou retirer un à des variables entières, et de tester si leur valeur est égale à zéro, suffisent.

1.3.2 Les langages de programmation

Il n'est pas très commode pour les programmeurs de devoir tenir compte de l'architecture des processeurs qui exécutent leurs programmes. Afin de pouvoir s'abstraire de ce genre de détail, et aussi de disposer de mécanismes de programmation plus confortables que les instructions directement exécutables par les processeurs, on a défini des *langages de programmation*. Ceux-ci permettent de programmer des algorithmes en les exprimant sous une forme qui reste relativement indépendante des détails matériels de l'ordinateur utilisé. Grâce à ces langages, il devient possible d'écrire des programmes pouvant être déployés sur n'importe quelle architecture de processeur.

Il existe plusieurs familles de langages de programmation. Dans ce cours, nous étudions les mécanismes d'un langage *impératif*, c'est-à-dire qui conduit à préciser explicitement les opérations devant être effectuées pendant l'exécution d'un programme. Il existe aussi des langages dits *déclaratifs*, pour lesquels ces opérations ne figurent pas directement dans les programmes ; de tels langages sont notamment utilisés pour interroger des bases de données (e.g., SQL). Certains langages sont de *haut niveau*, c'est-à-dire qu'ils mettent à la disposition du programmeur des mécanismes d'abstraction évolués ; d'autres sont plus proches du jeu d'instructions du processeur. Il existe des langages génériques, comme C, C++, Java, Python, C#, Rust et Go, et d'autres plus spécifiques à des domaines d'application particuliers.

Dans ce cours, nous allons étudier le langage de programmation C, qui a l'avantage d'être simple et très utilisé. Les concepts et mécanismes de programmation que nous allons introduire ont cependant une portée plus générale, et sont facilement transposables à d'autres langages de programmation impératifs.

Les programmes que l'on rédige dans un langage de programmation donné ne peuvent en général pas être directement exécutés par un processeur : le *code source* d'un programme doit d'abord être traduit en *code machine*, composé d'opérations appartenant au jeu d'instructions du processeur. Cette traduction du code source vers le code machine peut être réalisée soit avant l'exécution du programme (on parle alors de *compilation*), soit au fur et à mesure de cette exécution (il s'agit alors d'*interprétation*). Le mécanisme de compilation est en général plus efficace que l'interprétation, car il permet de rendre toute la puissance de calcul du processeur disponible pour l'exécution des opérations utiles d'un programme. Il s'agit de l'approche mise en œuvre par le langage C : avant de pouvoir exécuter un programme C, il faut employer un outil logiciel appelé *compilateur*, pour traduire le code source de ce programme en code machine exécutable par le processeur. Nous montrerons à la section 1.3.4 comment effectuer concrètement cette opération.

Tous les langages de programmation sont définis grâce à une *syntaxe* et une *sémantique*. La syntaxe précise un certain nombre de règles d'écriture que tous les programmes doivent respecter pour être *syntaxiquement valides*. Par exemple, la syntaxe du langage C impose de faire suivre la plupart des instructions d'un programme par un point-virgule "`;`". Si un programme n'est pas

syntaxiquement correct, alors sa compilation échoue en produisant un message d'erreur indiquant la nature du problème et l'endroit du code source où il s'est produit.

À côté de la syntaxe, la sémantique d'un langage donne un sens aux programmes qui sont syntaxiquement valides. On peut distinguer la *sémantique statique*, qui impose un certain nombre de règles devant être satisfaites par les programmes pour qu'ils puissent être considérés sémantiquement valides, et la *sémantique dynamique*, qui spécifie la nature des opérations qui seront effectuées lors de l'exécution du programme. Par exemple, la sémantique statique du langage C requiert que chaque variable soit déclarée [9] avant d'être utilisée. Les programmes qui ne respectent pas cette contrainte sont incorrects, même s'ils sont syntaxiquement valides, et leur compilation échoue en affichant un message d'erreur.

1.3.3 Un premier programme

La figure 1.2 fournit le code source d'un programme C complet implémentant l'algorithme d'Euclide étudié dans la section 1.2.2. Nous allons d'abord présenter de façon intuitive les éléments de ce programme ; l'ensemble des mécanismes et constructions utilisés seront étudiés plus en détail au chapitre 2. Nous verrons à la section 1.3.4 comment compiler ce programme afin de le faire exécuter concrètement par un ordinateur.

Ce programme est essentiellement composé d'*instructions*, comme `int a, b, c`, `scanf("%d %d", &a, &b)` ou `a = c`. Comme cela a déjà été mentionné, la syntaxe du langage C impose de faire suivre d'un point-virgule "`;`" la plupart des instructions d'un programme.

Les instructions d'un programme peuvent être combinées pour former des *séquences*. Par exemple, dans notre programme, les instructions `c = a`, `a = b` et `b = c` sont combinées de façon à former la séquence

```
c = a;
a = b;
b = c;
```

Une séquence peut comprendre n'importe quel nombre d'instructions. Ces instructions sont exécutées une à une, dans l'ordre où elles apparaissent. En entourant une séquence d'instructions

9. Les mécanismes correspondants seront introduits au chapitre 2.

```
#include <stdio.h>

int main()
{
  int a, b, c;

  printf("Entrez deux entiers strictement positifs: ");
  scanf("%d %d", &a, &b);

  if (a > b)
    {
      c = a;
      a = b;
      b = c;
    }

  while (a != 0)
    {
      c = a;
      a = b % a;
      b = c;
    }

  printf("Le PGCD est égal à %d.\n", b);
}
```

FIGURE 1.2 – Implémentation de l'algorithme d'Euclide

d'*accolades* "{" et "}", on obtient un *bloc* :

```
{
  c = a;
  a = b;
  b = c;
}
```

Un bloc joue le même rôle syntaxique qu'une instruction individuelle. Cela signifie qu'il est généralement permis, à quelques rares exceptions près, de remplacer une instruction d'un programme par un bloc. En particulier, une instruction figurant dans un bloc peut aussi être remplacée par un bloc, et ainsi de suite jusqu'à un niveau d'imbrication arbitraire. Les langages de programmation qui possèdent un tel mécanisme sont appelés les *langages structurés*.

Les espaces blancs comme le symbole d'espacement " ", les tabulations et les sauts de ligne n'ont pas de fonction syntaxique en C et sont en général ignorés par le compilateur. Leur usage permet cependant de rendre le code source d'un programme plus lisible ; en particulier, on *indente* les instructions d'un programme de façon à en expliciter la structure, en alignant les instructions de chaque bloc à une position horizontale décalée par rapport à celles du bloc qui contient ce dernier [10].

Dans le programme, on voit qu'à la suite de la ligne contenant `int main()` se trouve un bloc qui englobe le reste des instructions du programme. Remarquons que la convention d'indentation permet de déterminer facilement où ce bloc commence et où il se termine. Nous n'allons pas à ce stade expliquer précisément ce que signifie cette ligne de code [11], mais simplement mentionner que le bloc que nous venons de décrire est celui qui sera exécuté dès que le programme commencera son exécution.

La première instruction `int a, b, c` de ce bloc principal est une *déclaration de variables*. Une *variable* est un emplacement de mémoire reservé pendant une partie de l'exécution d'un programme, servant à retenir une donnée d'un certain *type*. Cette instruction déclare trois variables capables de retenir des valeurs entières. Les noms de ces variables sont spécifiés par le biais des *identificateurs* `a`, `b` et `c`. Ceux-ci apparaissent dans les instructions suivantes du programme qui sont amenées à manipuler ces variables.

Par exemple, l'instruction `c = a` est une instruction d'*affectation* dont l'effet est de lire la valeur courante de la variable `a` et d'*affecter* cette valeur à la variable `c`, ce qui revient à remplacer le contenu de `c` par celui de `a` (le contenu précédent de `c` est alors perdu).

10. Dans certains langages de programmation comme *Python*, l'indentation des instructions présente une valeur syntaxique.

11. Ces explications viendront au chapitre 4.

Cette instruction fait partie de la séquence d'instructions

```
c = a;
a = b;
b = c;
```

qui a pour effet de permuter les valeurs des variables `a` et `b`, en utilisant `c` comme un espace de travail auxiliaire.

L'instruction `printf(" ... ")` qui suit la déclaration des variables est une *invocation de fonction* : "`printf`" n'est pas un mot-clé du langage C, mais l'identificateur d'une *fonction*. Cette fonction correspond à un fragment de code défini ailleurs, dont l'exécution peut être déclenchée à cet endroit du programme. Il s'agit ici d'une fonction de la *bibliothèque standard (standard library)*, présente dans toutes les installations de l'environnement de programmation C, et qui contient l'implémentation d'opérations d'usage courant. Dans le cas présent, la fonction `printf` permet d'afficher un message à l'écran ; ce message, qui lui est fourni en *argument*, invite l'utilisateur à entrer deux entiers positifs.

De la même façon, l'instruction `scanf("%d %d", &a, &b)` invoque la fonction `scanf` de la bibliothèque standard. À ce stade du cours, il n'est pas encore possible d'expliquer le détail des mécanismes qui interviennent dans cette instruction d'invocation. Intuitivement, cette instruction attend que l'utilisateur introduise au clavier deux nombres entiers, et écrit la valeur de ces nombres dans les variables `a` et `b`.

Une fois la valeur de `a` et de `b` connue, la première étape de l'algorithme d'Euclide consiste à les permuter dans le cas particulier où l'on a `a` > `b`. C'est l'objet de l'instruction suivante du programme. Une instruction de la forme

`if` (*expression E*)
instruction I

évalue d'abord l'expression *E*. Une *expression* fournit le moyen de calculer une valeur, en partant de données élémentaires comme la valeur de variables ou de constantes, et en leur appliquant des *opérateurs*. Par exemple, l'expression `a < b`, obtenue en appliquant l'opérateur "<" aux sous-expressions élémentaires `a` et `b`, compare la valeur des variables `a` et `b`, et fournit un résultat considéré comme vrai si `a` possède une valeur strictement inférieure à celle de `b`. Ensuite, si l'évaluation de *E* fournit un résultat vrai, et seulement dans ce cas, alors l'instruction *I* est exécutée. Dans cette instruction, "`if`" est un *mot-clé* du langage, c'est-à-dire un mot réservé qui ne peut pas être utilisé comme identificateur. Remarquons que dans notre programme, l'instruction *I* est remplacée par un bloc, comme le permet le mécanisme de programmation structurée.

Globalement, l'effet de l'instruction

```
if (a > b)
  {
    c = a;
    a = b;
    b = c;
  }
```

consiste donc à permuter les valeurs de `a` et de `b`, en s'aidant de `c`, lorsque la condition `a > b` est initialement satisfaite.

L'instruction suivante est une instruction de *boucle*. Une instruction de la forme

`while` (*expression E*)
instruction I

basée sur le mot-clé `while` évalue d'abord l'expression *E*. Si le résultat obtenu est vrai, alors l'instruction *I* est exécutée, et le processus recommence : on évalue à nouveau *E*, on exécute *I* si le résultat est vrai, et ainsi de suite. L'opération se termine dès que l'évaluation de *E* cesse de fournir un résultat vrai. Si cela se produit à la première évaluation de *E*, alors *I* n'est pas exécutée du tout.

L'instruction `while` figurant dans le programme détermine si la valeur de `a` est non nulle (l'opérateur `!=` teste si ses deux opérandes possèdent une valeur différente) et, tant que cette condition est satisfaite, exécute la séquence

```
c = a;
a = b % a;
b = c;
```

La seule instruction inédite de cette séquence est `a = b % a`. Cette instruction évalue l'expression `b % a`, et écrit le résultat de cette évaluation dans `a`. L'opérateur "`%`" est l'opérateur de *modulo*, qui calcule le reste de la division de sa première opérande par la seconde. Notre séquence d'instructions a donc pour effet de remplacer la valeur de la paire (a, b) par $(b \bmod a, a)$, qui est bien l'opération devant être réalisée par une itération de l'algorithme d'Euclide.

Ensuite, la dernière instruction du programme invoque la fonction `printf` afin d'afficher à l'écran le résultat du calcul, contenu dans `b`. Une fois encore, les détails de cette instruction seront précisément expliqués dans la suite du cours.

Il reste pour terminer à expliquer une ligne du programme que nous avons négligée : celle qui contient `#include <stdio.h>`. Il ne s'agit pas à proprement parler d'une instruction du

langage C, mais d'une *directive de prétraitement* (ou *directive de compilation*). Son but consiste à indiquer au compilateur qu'il faut commencer par traiter le contenu d'un fichier appelé `stdio.h` avant de compiler la suite du programme. Ce fichier fait partie de la bibliothèque standard et contient, entre autres, la déclaration des fonctions `printf` et `scanf`, dont le compilateur doit disposer pour pouvoir gérer correctement les instructions d'invocation de ces fonctions. Nous reviendrons sur ces mécanismes au chapitre 4.

1.3.4 La compilation et l'exécution d'un programme

Montrons maintenant comment compiler et exécuter le programme de la figure 1.2 sur un ordinateur personnel. Il existe différents types de compilateurs et d'environnements de développement pour le langage C. Nous illustrerons les exemples de ce cours à l'aide de *GCC* (*the Gnu Compiler Collection*), qui est disponible pour une très grande variété d'architectures de processeurs et de systèmes d'exploitation, et permet également de compiler d'autres langages que C.

La façon la plus simple d'invoquer le compilateur GCC est de le faire en *ligne de commande* : en supposant que le code source du programme se trouve dans un fichier appelé `pgcd.c`, la commande

```
gcc -o pgcd pgcd.c
```

invoque le compilateur en lui passant en arguments le nom "`pgcd.c`" de ce fichier, précédé par l'option "`-o pgcd`". Cette option spécifie que le résultat de la compilation doit être écrit dans un fichier appelé `pgcd`. En d'autres termes, cette commande demande au compilateur de traduire le fichier source `pgcd.c` en un fichier exécutable `pgcd`. Ce mécanisme est illustré à la figure 1.3.

Si cette compilation réussit, ce qui doit être le cas si le contenu de `pgcd.c` correspond exactement au code fourni à la figure 1.2, alors elle génère un fichier `pgcd` qui peut être exécuté. Dans un environnement de programmation *Unix* (par exemple, *Linux*), cette exécution peut être lancée à l'aide de la commande

```
./pgcd
```

qui demande le chargement et l'exécution du programme contenu dans le fichier `pgcd` situé dans le répertoire courant ("`.`").

Un exemple complet de compilation et d'exécution de ce programme est donné ci-dessous (le préfixe "`%`" distingue les commandes entrées par l'utilisateur et ne fait pas partie de ces commandes) :

```
% gcc -o pgcd pgcd.c
% ./pgcd
Entrez deux entiers strictement positifs: 19675050 8178000
Le PGCD est égal à 4350.
```

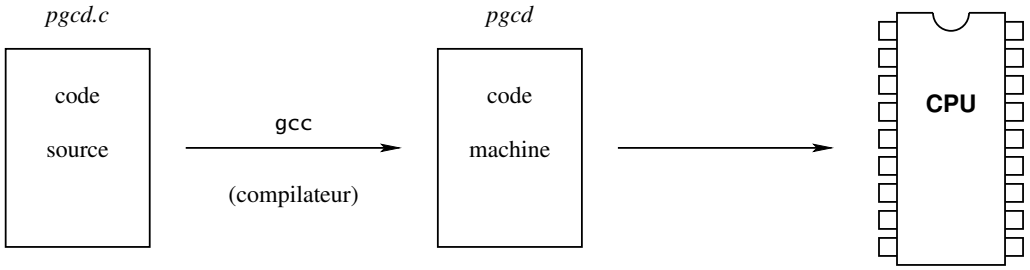


FIGURE 1.3 – Compilation d'un programme

1.4 Quelques considérations théoriques

1.4.1 Les ressources consommées

L'exécution de l'algorithme d'Euclide fourni à la figure 1.2 sur un ordinateur personnel donne l'impression de s'effectuer instantanément [12]. Ce n'est pas toujours le cas ; par exemple, si nous avions programmé le calcul du PGCD à l'aide de la première solution obtenue à la section 1.2.2, le temps nécessaire à l'exécution du programme aurait été considérablement plus élevé pour de grandes valeurs d'entrée.

On voit donc que différents algorithmes résolvant le même problème peuvent montrer des performances différentes. On abordera la question de mesurer les performances d'un algorithme au chapitre 3. De façon générale, il y a deux types de ressources quantitatives consommées au cours de l'exécution d'un programme : le *temps d'exécution* du programme, et la taille de l'*espace mémoire* nécessaire à cette exécution (c'est-à-dire la quantité des données à retenir). Nous introduirons dans la suite du cours une notion de *complexité* visant à décrire la quantité de ces ressources nécessaires à un algorithme, d'une façon la plus indépendante possible des détails de l'environnement d'exécution.

Un algorithme possède donc une *complexité en temps* et une *complexité en espace*. Celles-ci sont déterminées dans le cas le plus défavorable possible, en d'autres termes, pour les instances du problème qui conduisent au temps d'exécution (pour la complexité en temps) ou à la quantité de mémoire consommée (pour la complexité en espace) les plus élevés possibles. L'idée est que si l'on est à même d'établir qu'un algorithme est efficace dans le pire des cas, alors on est certain qu'il l'est toujours. Notons qu'il existe également des notions de complexité moyenne, calculées par rapport à une distribution de probabilité associée aux instances possibles du problème étudié. Ces notions de complexité moyenne sont beaucoup plus difficiles à mettre en œuvre.

12. La durée réelle de cette exécution se compte en microsecondes.

1.4.2 Les limitations de l'informatique

Comme cela a déjà été évoqué à la section 1.1.2, l'informatique possède des limitations fondamentales. En particulier, on peut démontrer l'existence de problèmes pour lesquels il n'existe aucune solution algorithmique efficace, voire même pas de solution du tout.

Un exemple de problème figurant dans la première catégorie est celui qui consiste à décider une formule de l'*arithmétique de Presburger*. Il s'agit de l'arithmétique des variables entières que l'on peut comparer et additionner, en combinant des sous-formules à l'aide d'opérateurs booléens et en quantifiant existentiellement ou universellement les variables. Par exemple la formule

$$\exists x \in \mathbb{N}\ \forall y \in \mathbb{N}\ (\exists z \in \mathbb{N}\ y = z + z) \Rightarrow y < x$$

exprime que l'ensemble des naturels pairs est borné. (Cette formule est donc fausse.) Il a été démontré que le problème consistant à déterminer si une telle formule est vraie ou fausse peut être résolu algorithmiquement, mais qu'il n'existe aucun algorithme efficace capable de le faire [13]. En d'autres termes, pour tout algorithme prenant une formule en entrée et déterminant correctement si cette formule est vraie ou fausse, il existe nécessairement des formules pour lesquelles le temps d'exécution de cet algorithme est prohibitivement élevé.

Pour des exemples de problèmes *indécidables*, c'est-à-dire n'admettant aucune solution algorithmique générale, on peut citer pour commencer le problème de l'arrêt déjà mentionné à la section 1.1.2, qui consiste à déterminer si l'exécution d'un programme fourni en entrée s'arrête ou non [14]. Un autre problème est celui de déterminer si une formule de la théorie additive et multiplicative des entiers est vraie ou non [15] ; il s'agit d'une extension de l'arithmétique de Presburger à laquelle on ajoute un opérateur permettant de multiplier des variables. Un dernier exemple est le *dixième problème d'Hilbert*, consistant à déterminer si un polynôme donné à plusieurs variables possède ou non un zéro entier [16] [17]. Les concepts et les principes permettant d'établir de tels résultats sont notamment étudiés dans le cours *Introduction to the theory of computation*.

13. Michael J. Fischer, Michael O. Rabin, 1974. *Super-exponential complexity of Presbuger arithmetic*. Proceedings of the SIAM-AMS Symposium in Applied Mathematics. 7:27–41.

14. Alan L. Turing, 1937. *On computable numbers, with an application to the Entscheidungsproblem*. Proceedings of the London Mathematical Society, s2-42 (1):230–265.

15. Kurt Gödel, 1931. *Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme, I*. Monatshefte für Mathematik und Physik, 38(1):173–198.

16. Martin D. Davis, Hilary W. Putnam, Julia H. Robinson, 1961. *The decision problem for exponential Diophantine equations*. Annals of mathematics, second series, 74(3):425–436.

17. Yuri V. Matiyasevich, 1970. *Enumerable sets are Diophantine*. Doklady Akademii Nauk SSSR, 191(2):279–282.

1.5 Exercices

Les algorithmes qui font l'objet des exercices suivants peuvent être exprimés en *pseudo-code*, c'est-à-dire dans un formalisme similaire à celui qui a été utilisé à la section 1.2.2 pour décrire l'algorithme d'Euclide et la méthode de Héron. La programmation en langage C de ces algorithmes fera partie des exercices proposés au chapitre 2.

1. Écrire un algorithme capable de résoudre une équation du second degré

   $$ax^2 + bx + c = 0$$

   pour des valeurs quelconques de a, b et c.

2. La *multifactorielle* $M(n, k)$ des deux entiers strictement positifs n et k est définie comme le produit $M(n, k) = n(n - k)(n - 2k) \cdots$ de tous les entiers strictement positifs m tels que $m \leq n$ et $n - m$ est divisible par k. Ecrire un algorithme capable de calculer la multifactorielle de deux nombres donnés.

3. Écrire un algorithme capable de calculer la somme, le produit, la moyenne arithmétique, le plus grand et le plus petit élément d'un ensemble (fini) quelconque de nombres réels fournis séquentiellement.

4. Un nombre entier positif est *premier* s'il possède exactement deux diviseurs : 1 et lui-même. Écrire un algorithme capable de déterminer si un nombre donné est premier ou non.

5. Écrire un algorithme capable de calculer la valeur de $\lfloor \log_2 n \rfloor$ pour un nombre entier strictement positif n donné, où $\lfloor x \rfloor$ dénote le plus grand entier inférieur ou égal à x.

6. Écrire un algorithme capable de calculer la valeur de $a^b \bmod m$, pour des nombres entiers a, b et m donnés tels que $b \geq 0$ et $m > 0$. *Note :* Il est possible d'effectuer cette opération beaucoup plus efficacement qu'en multipliant a par lui-même $b - 1$ fois (quand $b > 1$).

Chapitre 2

Les bases du langage C

2.1 Introduction

Ce chapitre contient une première introduction au langage de programmation C, suffisante pour commencer à écrire des programmes simples. Les mécanismes plus avancés de ce langage seront ensuite étudiés aux chapitres 4, 5 et 6.

Le langage C est un langage de programmation de bas niveau, ce qui signifie que les instructions figurant dans les programmes restent proches de celles que le processeur est directement capable d'exécuter. Apprendre à programmer en commençant par le langage C permet d'acquérir une bonne compréhension des mécanismes qui régissent l'exécution des programmes, notamment la façon dont le processeur accède à la mémoire de l'ordinateur. Un autre avantage de ce langage est qu'il est simple. Cette simplicité ne fait cependant pas obstacle à son utilisation pour des projets complexes : de très nombreux logiciels de grande taille sont programmés en C. Par exemple, le noyau du système d'exploitation *Linux*, qui équipe notamment une majorité des *smartphones* [1], est principalement rédigé en C, et comprend plusieurs dizaines de millions de lignes de code.

Un exemple de programme rédigé en langage C a déjà été donné à la figure 1.2 du chapitre 1. La structure de ce programme constitue la forme la plus simple de programme C que l'on peut écrire :

```
int main()
{
  ...
}
```

1. Ceux basés sur le système d'exploitation *Android*.

Dans ce fragment de code, "`...`" ne fait pas partie de la syntaxe du programme, mais marque l'endroit où l'on peut placer les *instructions exécutables* qui appartiennent au *bloc* délimité par les accolades "`{`" et "`}`". Ce bloc est celui qui est exécuté lorsque le programme est lancé. Comme cela a déjà été expliqué, les instructions d'un bloc sont terminées par des points-virgules "`;`" et sont exécutées en séquence. De plus, le langage C est *structuré*, ce qui signifie qu'un bloc peut dans la plupart des cas prendre la place d'une instruction exécutable individuelle.

Nous allons étudier dans ce chapitre les différentes sortes d'instructions exécutables permises par le langage C. La signification exacte de la ligne de code `int main()` sera expliquée au chapitre 4. Nous verrons également dans ce chapitre comment structurer des programmes plus complexes, en répartissant leur code source dans des fichiers séparés.

2.2 Les variables

Le premier élément de langage que nous allons étudier est la *variable*. Dans un programme, une variable représente un emplacement réservé au sein de la mémoire de l'ordinateur, capable de retenir une valeur pendant l'exécution d'un programme. Nous avons vu à la section 1.2.1 qu'à un certain niveau d'abstraction, on pouvait considérer que la mémoire de l'ordinateur est composée d'un ensemble de cases capables de retenir des données. Une variable correspond à une de ces cases. Comme son nom l'indique, une variable peut généralement changer de valeur pendant l'exécution d'un programme ; celui-ci dispose d'instructions lui permettant de consulter (*lecture* en mémoire) ou de modifier (*écriture*) les variables.

Une variable est caractérisée par deux éléments d'information. Premièrement, elle possède un *identificateur*, qui est un nom permettant d'y faire référence dans le programme. En langage C, les identificateurs sont formés par une suite de symboles comprenant les lettres minuscules (de "`a`" à "`z`"), les lettres majuscules (de "`A`" à "`Z`"), le caractère de soulignement ("`_`") et les chiffres (de "`0`" à "`9`"). Un identificateur ne peut cependant pas commencer par un chiffre. Un certain nombre de mots appelés *mots-clés* sont réservés par le langage et ne peuvent pas être utilisés comme identificateurs.

Dans les identificateurs, les majuscules et les minuscules sont distinguées ; par exemple, deux variables appelées `Delta` et `delta` sont considérées comme étant distinctes. Le caractère de soulignement est souvent utilisé pour écrire des identificateurs formés par plusieurs mots, par exemple `nb_chiffres`. Une bonne pratique de programmation est de veiller à ce que le nom des variables d'un programme soit cohérent, et décrive le plus précisément possible les données qu'elles contiennent et l'utilisation qui en est faite.

En plus d'être caractérisée par son identificateur, une variable possède un *type*. Le type d'une variable détermine la nature des valeurs que cette variable peut prendre, ainsi que la façon dont ces valeurs sont représentées dans la mémoire de l'ordinateur. Les opérations de traitement de

données que l'on peut appliquer au contenu d'une variable dépendent de son type. Les types que l'on peut employer sont très variés. Pour des données simples telles que des nombres entiers ou réels, on utilise des *types de base*, aussi appelés *types primitifs*, qui sont prédéfinis par le langage. Le programmeur a également la possibilité de définir ses propres types adaptés à des données plus complexes. Les mécanismes permettant de manipuler de tels types seront abordés aux chapitres 5 et 7.

2.2.1 Les types de base

Les principaux types de base du langage C sont les suivants.

`char`	Caractère, ou petit entier
`int`	Entier
`float`	Nombre réel (simple précision)
`double`	Nombre réel (double précision)

Ces types ne sont pas précisément définis, ce qui signifie que leurs détails d'implémentation peuvent différer d'un environnement de programmation à un autre. Dans ce cours, nous allons souvent prendre l'exemple de l'architecture x86-64, qui est celle de la plupart des ordinateurs personnels actuels, mais il ne faut pas être surpris de constater des différences si l'on programme dans un autre environnement.

Le type `char`

Le type de base `char` permet de représenter un *caractère*, c'est-à-dire un symbole dans un texte. Il correspond aussi dans les versions modernes du langage C à la plus petite unité adressable de la mémoire. Intuitivement, dans le modèle abstrait d'ordinateur que nous avons introduit à la section 1.2.1, on peut considérer qu'il s'agit d'une case de la mémoire.

Dans la très grande majorité des architectures de processeurs actuellement utilisées, l'unité élémentaire de mémoire contient 8 bits d'information, et donc le type `char` permet de représenter une valeur de 8 bits, aussi appelée *octet*. Un *bit* correspond à l'information contenue dans une variable binaire dont les deux valeurs possibles sont équiprobables. Une donnée représentée sur 1 bit possède donc deux valeurs possibles. Plus généralement, une donnée représentée sur n bits peut prendre $2 \times 2 \times \cdots \times 2 = 2^n$ valeurs distinctes.

Pour la plupart des architectures, une variable de type `char` peut donc prendre $2^8 = 256$ valeurs. Ces valeurs peuvent indifféremment être vues commes des nombres entiers (nous en préciserons l'intervalle un peu plus loin), ou comme des caractères. La correspondance entre ces entiers et les caractères est établie par l'intermédiaire d'une table dont les détails dépendent de

l'environnement utilisé. La majorité des systèmes informatiques actuels emploient une table de correspondance dérivée du *code ASCII (American Standard Code for Information Interchange)*. Dans ce code, par exemple, le caractère "`a`" possède le code 97. Cela signifie que si une variable de type `char` contient la valeur 97, alors le programmeur peut décider de l'interpréter comme représentant soit l'entier 97, soit le caractère "`a`", selon les besoins du programme qu'il est en train de rédiger. Des précisions supplémentaires sur ce procédé d'encodage des caractères seront données plus tard, à la section 5.4.1.

Bien sûr, 256 caractères ne suffisent pas à représenter tous les symboles qui peuvent apparaître dans un texte. Des procédés de représentation permettant d'encoder de très grands alphabets à l'aide d'un nombre variable d'octets existent mais sortent du cadre de ce cours [2]. À ce stade, il est suffisant de considérer que les caractères que l'on est amené à traiter appartiennent à un alphabet réduit comprenant moins de 256 symboles. Ces symboles comprennent en particulier ceux dont on a besoin pour programmer en C, ainsi que ceux qui sont nécessaires pour écrire des textes en français ou en anglais.

Le type `int`

Le type `int` est probablement le type de base le plus important, et sert à représenter une valeur entière. La norme du langage C n'impose pas la façon dont la valeur d'une variable de type `int` est représentée en mémoire ; cela permet aux concepteurs d'un compilateur de choisir la représentation qui est la plus facilement manipulable par le processeur, et donc la plus efficace. Dans une très grande majorité des environnements de programmation en langage C pour l'architecture x86-64, une variable de type `int` contient 32 bits d'information, et peut donc prendre 2^{32} valeurs distinctes.

Par défaut, une variable de type `int` est *signée*, ce qui signifie qu'elle peut prendre des valeurs aussi bien positives que négatives. Nous n'allons pas dans ce cours aborder les détails de la représentation interne des valeurs signées[2], mais seulement mentionner qu'avec une représentation sur 32 bits, un entier signé peut prendre n'importe quelle valeur dans l'intervalle

$$\begin{aligned} &[-2^{31},\ 2^{31}-1] \\ =\ &[-2147483648,\ 2147483647]. \end{aligned}$$

Pour certaines applications, il n'est cependant pas utile de considérer des valeurs négatives. On peut alors écrire le modificateur `unsigned` avant le type `int` ou `char`, afin signifier que les valeurs doivent être *non signées*, c'est-à-dire se limiter aux entiers positifs ou nuls. Le modificateur `signed` existe également pour indiquer explicitement qu'un type doit être signé.

2. Ils seront étudiés dans le cours d'*Organisation des ordinateurs*.

Pour des entiers sur 32 bits, le type `unsigned int` permet de représenter les nombres appartenant à l'intervalle

$$\begin{aligned}&[0,\ 2^{32}-1]\\ =\ &[0,\ 4294967295].\end{aligned}$$

Les types `unsigned char` et `signed char` correspondent quant à eux respectivement à

$$\begin{aligned}&[0,\ 2^{8}-1]\\ =\ &[0,\ 255].\end{aligned}$$

et

$$\begin{aligned}&[-2^{7},\ 2^{7}-1]\\ =\ &[-128,\ 127].\end{aligned}$$

Notons que le type `int` est signé par défaut, et que `signed` et `unsigned` employés seuls sont équivalents (respectivement) à `signed int` et `unsigned int`.

Il existe aussi d'autres modificateurs que l'on peut appliquer au type `int`, qui servent à augmenter (`long` ou `long long`) ou diminuer (`short`) la taille des représentations. Leur effet exact dépend de l'architecture et de l'environnement de programmation utilisés. Par exemple, pour l'architecture x86-64, le type `long long int` représente un entier sur 64 bits et le type `short int` sur 16 bits. Ces modificateurs peuvent être combinés avec `signed` ou `unsigned`, par exemple, les valeurs de type `unsigned long long int` appartiennent à l'intervalle

$$\begin{aligned}&[0,\ 2^{64}-1]\\ =\ &[0,\ 18446744073709551615].\end{aligned}$$

Signalons enfin que quand on utilise ces modificateurs, le type `int` est implicite et peut être omis. Par exemple, on peut abréger `unsigned long long int` en `unsigned long long`.

Les types `float` et `double`

Les types de base `float` et `double` servent à représenter des nombres réels. Comme cela a été discuté à la section 1.2.2, ces représentations ne sont pas exactes, mais approximent les nombres à une certaine précision près. Il est important de toujours garder cela à l'esprit quand on manipule des variables réelles.

Les types réels `float` et `double` correspondent respectivement à des représentations en *simple* et en *double précision*. Leur implémentation peut varier selon l'environnement de programmation, mais le standard le plus couramment employé, appelé *IEEE754*, correspond à des

représentations sur respectivement 32 et 64 bits. De nombreux processeurs modernes possèdent des circuits permettant de manipuler efficacement des valeurs de type `double` ; il s'agit donc du type que l'on utilisera le plus souvent pour représenter des réels. Ce type représente les nombres avec une précision supérieure à 15 chiffres décimaux significatifs, ce qui suffit largement pour la plupart des applications. Signalons enfin qu'il existe aussi le type `long double` permettant d'encore augmenter la précision des représentations.

2.2.2 La déclaration des variables

Pour pouvoir utiliser une variable dans un programme, il faut préalablement la *déclarer*. En langage C, l'instruction de déclaration d'une ou de plusieurs variables prend la forme suivante.

[`const` *]* *type identificateur [= valeur]*
[`,` *identificateur [= valeur]]*
⋮
[`,` *identificateur [= valeur]]* `;`

Dans une telle spécification de syntaxe, les crochets en italique "*[*" et "*]*" délimitent des éléments qui sont optionnels. La forme la plus simple de définition de variable est donc

type identificateur `;`

où *type* et *identificateur* sont bien sûr le type et l'identificateur de la variable déclarée. La forme générale de définition de variables permet de déclarer dans une même instruction plusieurs variables partageant le même type, et de leur attribuer une valeur initiale. Le mot-clé `const` permet de déclarer des *constantes*, c'est-à-dire des variables dont la valeur ne peut pas être modifiée.

Des exemples de déclarations de variables sont donnés ici :

```
int            i = 0, j = -1, k;
unsigned long  code_barre;
char           symbole = 'a', marqueur = '\n';
const double   avogadro = 6.02214179e23;
```

Dans cet exemple, toutes les variables sauf `k` et `code_barre` sont initialisées. Les expressions employées pour initialiser `symbole` et `marqueur` montrent la syntaxe particulière des constantes de type caractère : l'évaluation de l'expression `'a'` produit comme valeur le code du caractère "a" dans le jeu de symboles utilisé. Comme nous l'avons vu précédemment, ce code vaut 97 dans la plupart des environnements de programmation, donc les expressions `'a'` et `97` sont

synonymes. Le symbole “\” qui apparaît dans l’expression d’initialisation de `marqueur` est un *caractère d’échappement*. Il permet, en le faisant suivre d’un autre caractère, de spécifier des *caractères spéciaux* qui ne sont pas imprimables mais provoquent une action particulière quand ils sont affichés. Par exemple, le caractère désigné par “\n” représente un *retour chariot* et un *saut de ligne* ; en d’autres termes, il termine la ligne courante de texte et passe à la ligne suivante. Pour spécifier le caractère “\” lui-même, on écrit “\\”.

Pour terminer, l’expression d’initialisation de la constante `avogadro` montre comment écrire des constantes réelles en notation scientifique.

2.2.3 La portée d’une déclaration

En langage C, la déclaration d’une variable doit figurer avant les instructions qui utilisent cette variable. Cette déclaration sert à indiquer au compilateur comment la variable doit être représentée en mémoire et comment elle doit être manipulée. Par exemple, une opération d’addition ne s’effectue pas de la même façon selon que ses opérandes sont des nombres entiers ou des nombres réels.

Lorsqu’une variable est déclarée au sein d’un programme, la *portée* de cette variable désigne l’ensemble des endroits du programme où cette variable peut être utilisée. En langage C, la portée d’une variable commence à sa déclaration, et se termine à la fin du plus petit bloc qui contient cette déclaration. Il est permis que plusieurs variables déclarées dans des blocs différents partagent le même identificateur ; dans ce cas, la déclaration la plus interne masque celles qui figurent dans les blocs extérieurs.

Pour illustrer cela, nous considérons le fragment de programme de la figure 2.1. Nous ne nous intéressons qu’aux déclarations de variables, en remplaçant les autres instructions du programme par des pointillés “⋮”. Les numéros figurant au début des lignes de code ne font pas partie de la syntaxe du programme, mais sont ajoutés pour permettre de faire facilement référence aux instructions.

Ce morceau de programme définit plusieurs blocs imbriqués. L’instruction située à la ligne 8 déclare une variable appelée `x`, de type `long`. La portée de cette variable commence juste après cette déclaration, et se termine à la fin du plus petit bloc qui la contient, c’est-à-dire à la ligne 10. Cela signifie que l’emplacement de mémoire correspondant à la variable est alloué à l’exécution de la ligne 8, et libéré à la ligne 10. Les instructions qui peuvent faire référence à cette variable sont celles situées à la ligne 9 du programme.

Le programme définit également à la ligne 4 une autre variable appelée `x`, cette fois de type `int`. La portée de cette variable commence après cette déclaration, et se termine à la ligne 12. Remarquons qu’aux lignes 9 et 10, cette variable est masquée par cette déclarée à la ligne 8. Les instructions qui peuvent utiliser la variable déclarée à la ligne 4 sont donc celles figurant aux

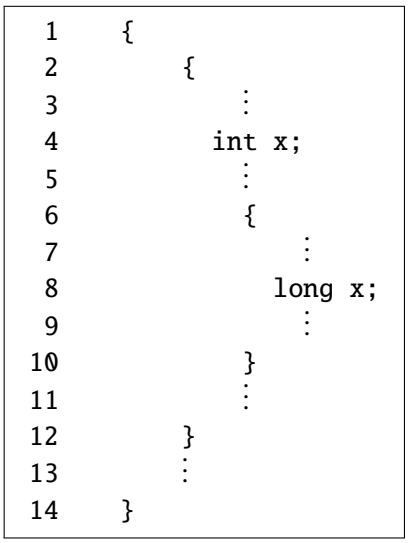


FIGURE 2.1 – Portée d'une variable

lignes de code 5, 6, 7 et 11. Cette variable est néanmoins toujours présente en mémoire même quand elle est masquée : si une instruction située à la ligne 5 y écrit une valeur, alors cette valeur pourra être lue à la ligne 11. L'emplacement de mémoire alloué à cette variable n'est libéré qu'à la ligne 12.

Pour des raisons de lisibilité, il est conseillé de placer les définitions de variables au début de leur bloc. Il ne s'agit cependant pas d'une contrainte imposée par le langage C.

2.3 Les expressions

Les expressions sont une catégorie particulière d'instructions exécutables, servant à exprimer des opérations de traitement de données. Lorsqu'une expression est *évaluée*, ces opérations sont effectuées, et fournissent une valeur qui constitue le résultat de cette évaluation.

Les expressions les plus simples que l'on peut écrire sont les *littéraux*. Il s'agit de valeurs constantes, par exemple `42`, `3.142592654` ou `'a'`. Leur évaluation fournit la valeur correspondante.

L'identificateur d'une variable forme également une expression, par exemple `avogadro` ou `nb_chiffres`. Bien sûr, pour qu'une telle expression soit valide, il faut qu'elle soit située dans la portée de la variable correspondante. L'évaluation d'une telle expression fournit la valeur courante de cette variable ; en d'autres termes, cette évaluation effectue une lecture de la variable.

On peut construire des expressions plus complexes en appliquant des *opérateurs* à des sous-expressions. Par exemple, l'expression

```
4.0 / 3.0
```

s'obtient en appliquant l'opérateur de division arithmétique "/" aux deux sous-expressions littérales `4.0` et `3.0`. L'évaluation de cette expression fournit une approximation de type `double` de la valeur réelle 1,333333 . . .

L'expression

```
(4.0 / 3.0) * pi * r * r * r
```
(2.1)

applique l'opérateur de multiplication arithmétique "*" aux sous-expressions `(4.0 / 3.0)`, qui possède la même valeur que celle de `4.0 / 3.0`, et `pi`, qui lit la valeur d'une variable nommée `pi`. Le résultat forme la sous-expression

```
(4.0 / 3.0) * pi
```
.

Ensuite, l'opérateur "*" est appliqué successivement trois fois à l'expression obtenue précédemment et à la sous-expression `r`. En supposant que la variable `pi` contienne une approximation de π, et que `r` représente le rayon d'une sphère, l'évaluation de l'expression (2.1) calcule donc le volume de cette sphère.

Nous allons à présent passer en revue les principaux opérateurs définis par le langage C.

2.3.1 Les opérateurs arithmétiques

Les opérateurs figurant dans le tableau suivant permettent d'effectuer des additions, des soustractions, des multiplications et des divisions.

`+`	Addition
`-`	Soustraction
`*`	Multiplication
`/`	Division
`%`	Reste de la division (modulo)

La nature de l'opération effectuée dépend du type de ses opérandes. Par exemple, l'évaluation de l'expression

```
2 / 3
```

calcule la *division entière* de 2 et de 3, qui vaut 0. En revanche, celle de l'expression

```
2.0 / 3.0
```

retourne 0,666666 . . ., car l'opérateur de division est maintenant appliqué à des valeurs réelles.

L'opérateur "%" n'est applicable qu'à des entiers ; il retourne le reste de la division de sa première opérande par la deuxième. (Nous avons déjà vu un exemple d'application de cet opérateur dans l'implémentation de l'algorithme d'Euclide au chapitre 1.) Cet opérateur correspond à l'opération mathématique de *modulo* quand il est appliqué à des valeurs positives [3].

Lorsqu'une expression comprend plusieurs opérateurs, ceux-ci sont évalués dans un ordre qui dépend de leur *priorité*, ou *précédence*. Celle-ci est définie d'une façon similaire aux règles de l'algèbre : les opérateurs "*", "/" et "%" sont plus prioritaires que "+" et "-". Des parenthèses peuvent être utilisées pour forcer explicitement un ordre d'évaluation des opérateurs. Par exemple, les expressions

```
a * a + b * b
```

et

```
(a * a) + (b * b)
```

sont équivalentes.

Enfin, les opérateurs "+" et "-" possèdent également une forme *unaire*, c'est-à-dire qui n'admet qu'une seule opérande. Par exemple, si `x` est une variable entière ou réelle, l'expression `-x` calcule l'opposé de la valeur de `x`.

2.3.2 Les opérateurs de comparaison

Les opérateurs suivants permettent de comparer des valeurs.

<	Plus petit que
>	Plus grand que
<=	Plus petit ou égal à
>=	Plus grand ou égal à
==	Égal à
!=	Différent de

Leur évaluation retourne une *valeur booléenne*, c'est-à-dire une valeur qui est soit vraie, soit fausse. En langage C, la valeur booléenne vraie est représentée [4] par n'importe quel entier différent de 0, et fausse par l'entier 0. Par exemple, l'évaluation de

```
5 <= 2
```

3. Le reste de la division diffère du modulo pour des opérandes négatives. Par exemple, on a $-7 \bmod 3 = 2$, alors que le reste de la division de -7 par 3 vaut -1.

4. Les versions modernes du langage C définissent aussi un type de base dédié aux valeurs booléennes. Nous ne l'emploierons pas dans ce cours, car il n'est pas universellement utilisé.

retourne 0. Celle de

```
10 == 5 + 5
```

retourne un entier non nul. (La valeur de celui-ci n'est pas précisément connue, et peut potentiellement varier selon l'environnement de programmation, ou encore d'une évaluation à une autre.) Notons que l'opérateur de test d'égalité s'écrit avec deux symboles "="; nous verrons à la section 2.3.4 que celui formé d'un seul symbole "=" possède une sémantique différente. La confusion entre ces deux opérateurs est une erreur de programmation fréquente en C.

2.3.3 Les opérateurs booléens

On est souvent amené à devoir combiner plusieurs tests au sein d'une même expression. Les opérateurs suivants sont destinés à être appliqués à des valeurs booléennes, en d'autres termes, à des entiers dont on s'intéresse seulement au fait qu'ils sont nuls ou non nuls.

`&&`	Et logique
`||`	Ou logique
`!`	Négation (opérateur unaire)

L'évaluation de l'opérateur "`&&`" retourne une valeur booléenne vraie si et seulement si ses deux opérandes sont vraies. Par exemple, l'expression

```
c >= 'a' && c <= 'z'
```

teste si la valeur de la variable `c` est une lettre minuscule, en faisant l'hypothèse que ces lettres apparaissent dans l'ordre du dictionnaire et de façon consécutive dans le jeu de caractères utilisé. (Cette propriété est vraie pour tous les systèmes informatiques modernes.)

L'opérateur "`||`" détermine si au moins une de ses deux opérandes est vraie. Par exemple, l'expression

```
x >= 0 || y >= 0
```

teste si les deux variables `x` et `y` ne possèdent pas toutes les deux une valeur strictement négative.

L'opérateur unaire "`!`" transforme une valeur vraie en une valeur fausse et réciproquement. Par exemple, si `x` est une variable entière [5], les expressions

```
!(x >= 1000)
```

5. Cette équivalence n'est pas vraie pour une variable réelle, qui peut prendre des *valeurs exceptionnelles* qui ne sont pas des nombres, et ne sont donc ni supérieures, ni inférieures à un nombre donné.

et

```
x < 1000
```

sont équivalentes.

Les opérateurs "&&" et "||" présentent une particularité importante : ils s'évaluent en *circuit court*. Cela signifie que leur évaluation s'effectue de gauche à droite (en d'autres termes, leur opérande de gauche est évaluée avant celle de droite), et se termine dès que la valeur finale de l'expression est connue. Par exemple, l'évaluation de

```
n != 0 && m / n > 1
```
(2.2)

commence par évaluer la sous-expression `n != 0` qui constitue l'opérande de gauche de l'opérateur "&&". Si le résultat obtenu est faux, alors il n'est pas nécessaire d'évaluer l'opérande de droite de "&&", car on sait déjà que l'évaluation de (2.2) produira un résultat faux. En revanche, si l'évaluation de `n != 0` retourne un résultat vrai, alors il devient nécessaire d'évaluer aussi la sous-expression `m / n > 1` pour déterminer le résultat de l'évaluation de (2.2). Ce mécanisme permet donc ici d'éviter d'effectuer une division par zéro lorsque la valeur de `n` est nulle. (L'effet d'une telle division est indéterminé, et peut aller jusqu'à interrompre l'exécution du programme.)

2.3.4 Les opérateurs d'affectation

L'opérateur d'*affectation* "=" permet d'écrire une valeur dans une variable. Son opérande de gauche est ce que l'on appelle une *valeur à gauche*, et sert à spécifier l'endroit dans la mémoire de l'ordinateur où l'opération d'écriture va être effectuée. Dans sa forme la plus simple, il s'agit de l'identificateur d'une variable. L'opérande de droite fournit la valeur qui sera écrite lors de l'évaluation de l'expression. Cette valeur est aussi celle qui constitue le résultat de cette évaluation.

Par exemple, pour évaluer l'expression

```
x = x + 2
```
,

on commence par évaluer l'opérande de droite `x + 2` de l'opérateur "=", ce qui revient à lire la valeur de `x` et lui ajouter 2. Ensuite, le résultat obtenu est écrit dans la variable `x`. En résumé, l'évaluation de cette expression *incrémente* la valeur de `x` de deux unités. Le résultat de cette évaluation correspond à la valeur écrite dans `x`. Il s'agit d'une évaluation qui présente un *effet de bord* : son effet ne consiste pas seulement à produire une valeur, mais aussi à réaliser une autre opération : modifier la valeur de `x`.

L'évaluation de l'expression

```
(a = b) < 10
```

recopie la valeur de la variable `b` dans `a`, et teste ensuite si cette valeur est strictement inférieure à 10. L'évaluation de l'expression

```
i = j = k = 0
```

affecte la valeur 0 à `k`. Ensuite, cette valeur est affectée à `j`, puis de la même façon à `i`. Cette expression permet donc d'initialiser les trois variables avec la même valeur 0.

Il existe d'autres opérateurs d'affectation qui offrent des raccourcis d'écriture, il s'agit des opérateurs "+=", "-=", "*=", "/=" et "%=". Leur principe est simple : une expression de la forme

var α= *expr*,

avec $\alpha \in \{+, -, *, /, \%\}$, est équivalente à

var = *var* α *expr*.

Par exemple, l'expression

```
x += 2
```

équivaut à

```
x = x + 2
```

.

2.3.5 Les opérateurs d'incrément et de décrément

Les opérateurs d'*incrément* "++" et de *décrément* "--" sont des opérateurs unaires qui s'appliquent à des valeurs à gauche, par exemple l'identificateur d'une variable. Il peuvent être placés à droite ou à gauche de leur opérande, leur sémantique étant différente dans les deux cas.

Ces opérateurs présentent un effet de bord qui consiste à incrémenter ("++") ou décrémenter ("--") leur opérande. S'ils sont placés à droite, ils retournent la valeur de cette opérande avant l'opération d'incrément ou de décrément. S'ils sont placés à gauche il retournent la valeur de l'opérande après son incrément ou son décrément.

Par exemple, en supposant que `x` et `y` sont deux variables entières initialisées à 0, l'évaluation de l'expression

```
x = y++
```

commence par évaluler la sous-expression `y++`, ce qui a pour effet d'incrémenter `y`. La valeur de cette variable devient donc égale à 1. Étant donné que l'opérateur "++" est placé à droite, l'évaluation de `y++` retourne la valeur de `y` avant l'opération d'incrément, c'est-à-dire 0. C'est cette valeur 0 qui est ensuite affectée à `x`.

En supposant toujours que `x` et `y` sont initialisées à 0, l'évaluation de l'expression

```
x = --y
```

décrémente `y`, dont la valeur devient donc égale à -1, et écrit ensuite cette valeur dans `x`, qui prend donc aussi la valeur -1.

Les opérateurs d'incrément et de décrément permettent d'écrire des expressions qui sont syntaxiquement correctes, mais dont la sémantique n'est pas précisément définie par la norme du langage C, par exemple

```
x = --x + x++
```

.

L'évaluation de telles expressions retourne un résultat indéfini. Leur usage est donc à proscrire.

Étant donné que les opérateurs d'incrément et de décrément présentent un effet de bord, leur influence dépend du fait qu'ils sont évalués ou non. Par exemple, l'évaluation de l'expression

```
x > 0 && --x
```

décrémente `x` seulement si sa valeur est initialement strictement positive, car l'opérateur "`&&`" fonctionne en circuit court comme expliqué à la section 2.3.3. L'évaluation de cette expression retourne une valeur booléenne vraie si et seulement si la valeur initiale de `x` est strictement supérieure à 0 et différente de 0 après avoir été décrémentée, c'est-à-dire si elle est strictement supérieure à 1.

2.3.6 L'opérateur virgule

L'opérateur virgule "`,`" n'est pas indispensable, mais permet de simplifier certaines constructions. Il est utilisé lorsqu'on souhaite évaluer plusieurs expressions dans un contexte où la syntaxe du langage C n'autorise d'en écrire qu'une seule.

Cet opérateur prend deux opérandes et évalue d'abord celle de gauche et puis celle de droite. Le résultat prend le type et la valeur de l'opérande de droite.

Considérons par exemple l'expression

```
x = (y++, y++)
```

où `x` et `y` sont deux variables entières initialisées à 0. Cette expression emploie des parenthèses car la priorité de l'opérateur "`,`" est inférieure à celle de "=".

L'évaluation du membre de droite de l'opérateur "=" va commencer par évaluer la sous-expression `y++` située à gauche de "`,`", ce qui va incrémenter `y` et retourner 0. Ensuite, on évalue l'opérande de droite `y++` de "`,`", ce qui incrémente à nouveau `y` et retourne 1. Cette derrnière valeur est celle qui constitue le résultat de l'évaluation de `y++, y++`. En définitive, `x` devient donc égal à 1 et `y` à 2.

2.3.7 La conversion de type

Chaque expression possède un *type*, qui caractérise la nature des valeurs produites lors de l'évaluation de cette expression, ainsi que les opérations que l'on peut faire subir à ces valeurs. Les principaux types de base définis par le langage C ont été introduits à la section 2.2.1.

Il est parfois permis, sous certaines conditions, d'affecter à une variable d'un type donné une valeur d'un autre type. Cela est par exemple autorisé pour tous les types numériques, c'est-à-dire les types entiers et réels ainsi que le type `char` qui est considéré comme un cas particulier de type entier. Dans ce cas, une opération de *conversion de type* est implicitement réalisée.

Par exemple, le fragment de code

```
double p = 3.1416;
int x;

x = p;
```

déclare une variable réelle `p` et une variable entière `x`, et affecte la valeur de la première à la deuxième. Cette opération provoquera la conversion de la valeur réelle 3,1416 en un entier, c'est-à-dire son arrondi vers la valeur 3, qui sera celle écrite dans `x`.

Dans certains cas, on souhaite spécifier explicitement qu'une telle conversion de type doit être effectuée. On emploie pour cela l'opérateur de conversion de type, dont la syntaxe est

(*type*) *expr* .

Cette expression s'évalue en évaluant la sous-expression *expr*, et en convertissant ensuite le résultat obtenu vers le type *type*. Le procédé utilisé pour convertir une valeur d'un type vers un autre dépend des types impliqués ; la spécification du langage C contient une description détaillée de cette opération pour un certain nombre de paires de types.

Pour illustrer cette opération, nous considérons la séquence d'instructions suivante.

```
double a = 2.0, b = 3.0, x, y;

x = a / b;
y = (int) a / (int) b;
```

Ces instructions affectent d'abord à la variable réelle `x` le résultat de la division de `a` par `b`. Étant donné que ces deux variables sont déclarées avec le type `double`, l'opération qui va être effectuée est une division réelle, et `x` va prendre la valeur 0,666666... qui est le quotient de 2 par 3.

La deuxième instruction d'affectation est différente : elle convertit d'abord le résultat de l'évaluation des deux sous-expressions `a` et `b` vers des entiers avant de leur appliquer l'opérateur de division "/". L'opération effectuée sera donc une division entière de 2 par 3, dont le résultat 0 sera la valeur écrite dans `y`. Remarquons que dans la sous-expression `(int) a / (int) b`, la priorité de l'opérateur de conversion de type est supérieure à celle de l'opérateur de division. De plus, l'évaluation de cette sous-expression produit un résultat de type entier. Une opération de conversion de ce résultat vers un réel est donc implicitement effectuée avant de l'affecter à `y`.

2.3.8 Les expressions conditionnelles

Nous allons maintenant introduire un opérateur qui, comme l'opérateur virgule, n'est pas essentiel mais permet parfois de simplifier l'écriture de certaines instructions. Une *expresssion conditionnelle* admet la syntaxe suivante.

cond ? *expr1* : *expr2*

Cette expression comporte un seul opérateur, représenté par les deux symboles "`?`" et "`:`", appliqué aux trois sous-expressions *cond*, *expr1* et *expr2*.

L'évaluation de cette expression commence par l'évaluation de *cond*, dont le résultat est interprété comme une valeur booléenne. Si cette valeur est vraie (en d'autres termes, si elle est différente de 0), alors *expr1* est évaluée, sinon c'est *expr2* qui l'est. Dans tous les cas, il n'y a qu'une et une seule sous-expression parmi *expr1* et *expr2* qui est évaluée. Le résultat de cette évaluation est celui qui est retourné par l'évaluation de l'expression complète.

Par exemple, l'expression

```
max = (a > b) ? a : b
```

s'évalue en déterminant d'abord si la condition `a > b` est vraie. Si c'est le cas, alors l'évaluation de la sous-expression `(a > b) ? a : b` retourne la valeur de `a`, sinon celle de `b`. L'expression complète a donc pour effet d'affecter à `max` la plus grande valeur parmi celles de `a` et de `b`.

Les sous-expressions qui apparaissent dans une expresssion conditionnelle peuvent produire des effets de bord. Par exemple, l'évaluation de l'expression

```
incr_x ? x++ : y++
```

incrémente `x` si la valeur de `incr_x` est vraie, et `y` dans le cas contraire.

2.4 Les instructions de contrôle

Nous avons vu que, par défaut, les instructions qui composent un bloc sont exécutées séquentiellement, dans l'ordre où elles apparaissent. Ce mécanisme n'est cependant pas suffisant ; pour implémenter la plupart des algorithmes, on a besoin de pouvoir exprimer que certaines instructions doivent être exécutées plusieurs fois, ou que leur exécution doit être conditionnée au résultat d'une opération précédemment effectuée. Par exemple, dans l'algorithme d'Euclide étudié à la section 1.3.3, l'opération consistant à permuter la valeur des deux variables `a` et `b` ne doit être effectuée que si la condition `a > b` est satisfaite. De plus, dans la suite de cet algorithme, les opérations de calcul constituant une itération doivent être répétées tant que la valeur de `a` est différente de 0.

Les *instructions de contrôle* sont celles qui permettent de modifier l'ordre séquentiel normal des instructions exécutables d'un bloc. Il en existe de plusieurs types.

2.4.1 Le choix conditionnel binaire

En langage C, l'instruction de *choix conditionnel binaire* possède la syntaxe suivante.

```
  if (expr)
    instr1 ;
[ else
    instr2 ; ]
```

Pour rappel, les crochets en italique "*[*" et "*]*" délimitent des éléments optionnels. Dans le cas présent, cela signifie que la partie de l'instruction commençant par le mot-clé `else` peut être omise. Dans cette spécification de syntaxe, *expr* représente une expression s'évaluant en une valeur entière, et *instr1* et *instr2* sont des instructions exécutables, pouvant être remplacées par des blocs. Remarquons que les parenthèses qui entourent *expr* font partie de la syntaxe de l'instruction et sont obligatoires.

Lorsque cette instruction est exécutée, elle commence par évaluer *expr*, et interprète le résultat de cette évaluation comme une valeur booléenne. Si celle ci est vraie, alors l'instruction *instr1* est exécutée. Sinon, l'instruction *instr2* est exécutée, à condition que la partie `else` de l'instruction soit présente. (Dans le cas contraire, l'instruction ne fait rien d'autre après avoir évalué *expr*.)

On peut représenter graphiquement la sémantique d'une telle instruction à l'aide d'un *organigramme*, qui est un diagramme montrant les différents chemins d'exécution possible d'un morceau de programme. L'organigramme de l'instruction `if` est donné à la figure 2.2. Pour

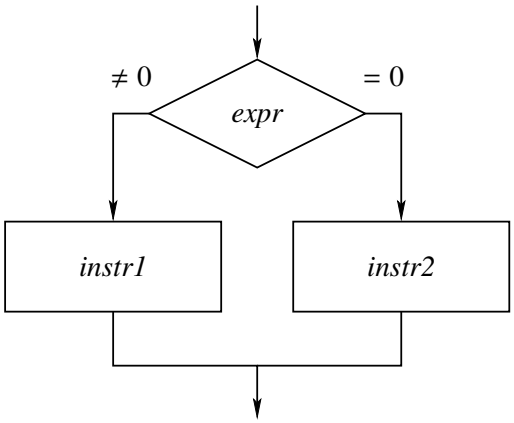


FIGURE 2.2 – Organigramme de l'instruction `if`

rappel, une valeur booléenne est considérée comme étant vraie si sa représentation entière est différente de 0.

En langage C, le mot-clé `else` ne peut être utilisé que pour commencer la deuxième partie d'une instruction `if`. Quand on utilise `else`, il faut toujours garder à l'esprit qu'il se rapporte toujours au mot-clé `if` le plus récent du même bloc (à condition que celui-ci ne possède déjà pas une partie `else`). Il s'agit d'une source d'erreur fréquente. Considérons par exemple le fragment de programme suivant.

```
if (expr1)
    if (expr2)
        {
            ⋮
        }
else
    {
        ⋮
    }
```

Dans cet exemple, le mot-clé `else` se rapporte au deuxième `if` et non au premier. L'indentation laisse cependant supposer que ce n'était pas l'intention du programmeur qui a rédigé ce code. Pour que le mot-clé `else` se rapporte au premier `if`, il aurait fallu écrire, par exemple :

```
if (expr1)
  {
    if (expr2)
        {
            ⋮
        }
  }
else
    {
        ⋮
    }
```

Pour illustrer l'utilisation de l'instruction `if`, nous allons montrer comment rédiger un programme C capable de déterminer si une année donnée est bissextile ou non. Les années bissextiles sont celles qui comprennent un 29 février. Elles sont ajoutées au calendrier de façon à rendre le nombre moyen de jours d'une année le plus proche possible de la durée d'un cycle complet des saisons.

La règle de base est qu'une année est bissextile si elle est divisible par 4. Sur le long terme, cette règle ne conduit cependant pas à une approximation suffisamment précise de la durée moyenne d'une année. On introduit donc une exception : les années divisibles par 100 ne sont pas bissextiles, sauf celles qui sont divisibles par 400.

On voit donc que pour déterminer si une année donnée est bissextile, il faut considérer 4 cas possibles : elle n'est pas divisible par 4, elle est divisible par 4 sans être divisible par 100, elle est divisible par 100 sans être divisible par 400, ou elle est divisible par 400. Un programme C implémentant ce processus de décision est donné à la figure 2.3.

La structure de ce programme et l'utilisation des fonctions `printf` et `scanf` sont semblables à celles de l'implémentation de l'algorithme d'Euclide à la section 1.3.3. La procédure de décision est implémentée en déterminant d'abord si l'année est divisible par 400, puis par 100, puis par 4 ; cette stratégie permet de minimiser le nombre de tests effectués. Pour tester si l'année est divisible par n, on évalue une expression de la forme

`!(annee % n)` .

En effet, si c'est le cas, alors le reste de la division d'`annee` par n vaut 0, qui représente une valeur booléenne fausse. L'opérateur "`!`" transforme alors cette valeur en une valeur vraie. Réciproquement, si `annee` n'est pas divisible par n, alors l'expression s'évalue en une valeur booléenne fausse.

Les deux premières instructions `if` du programme ont donc pour effet de placer dans la variable `est_bissextile` une valeur booléenne vraie ou fausse, selon que l'année est bissextile

```
#include <stdio.h>

int main()
{
  int annee, est_bissextile;

  printf("Entrez une année: ");
  scanf("%d", &annee);

  if (!(annee % 400))
    est_bissextile = 1;
  else
    if (!(annee % 100))
      est_bissextile = 0;
    else
      est_bissextile = !(annee % 4);

  printf("Cette année ");

  if (est_bissextile)
    printf("est");
  else
    printf("n'est pas");

  printf(" bissextile.\n");
}
```

FIGURE 2.3 – Détermination d'une année bissextile

ou non. La troisième et dernière instruction `if` sert alors à afficher le message approprié, en fonction de la valeur contenue dans `est_bissextile`.

2.4.2 Le choix conditionnel multiple

L'instruction `switch` permet d'évaluer une expression et d'ensuite aiguiller l'exécution du programme vers différents endroits d'un bloc en fonction du résultat obtenu. Sa syntaxe est la suivante.

```
switch (expr)
  {
    case val1:
      seq1;
      break;
    case val2:
      seq2;
      break;
        ⋮
  [ default:
      seqn; ]
  }
```

Le nombre d'occurrences de "`case`" est quelconque, et la présence de "`default:`" est optionnelle. L'expression *expr* doit être de type entier (en ce compris `char` qui est considéré comme étant un cas particulier de type entier). Les littéraux *val1*, *val2* doivent être des constantes entières. Les éléments *seq1*, *seq2*, ... de l'instruction sont des séquences [6] d'instructions exécutables.

Les étiquettes "`case` *val1*`:`", "`case` *val2*`:`", ... et "`default:`" peuvent être vues comme des *points d'entrée* dans le bloc qui suit "`switch (`*expr*`)`"; ce bloc constitue le *corps* de cette instruction. L'instruction `switch` fonctionne de la façon suivante : après avoir évalué *expr*, si la valeur obtenue est égale à *val1*, alors l'exécution du corps commence à l'endroit spécifié par cette étiquette, c'est-à-dire par *seq1*. Si l'évaluation de *expr* retourne *val2*, on entre dans le corps après l'étiquette correspondante, c'est-à-dire en exécutant *seq2*, et ainsi de suite en fonction du nombre d'étiquettes présentes. Si la valeur retournée par l'évaluation de *expr* est différente de *val1*, *val2*, ..., alors on entre dans le corps après "`default:`", à condition que cette étiquette figure dans l'instruction. Dans le cas contraire, aucune instruction du corps n'est exécutée.

6. La spécification du langage C autorise en fait une forme plus générale pour le corps d'une instruction `switch`, mais celle étudiée ici suffit dans la très grande majorité des cas.

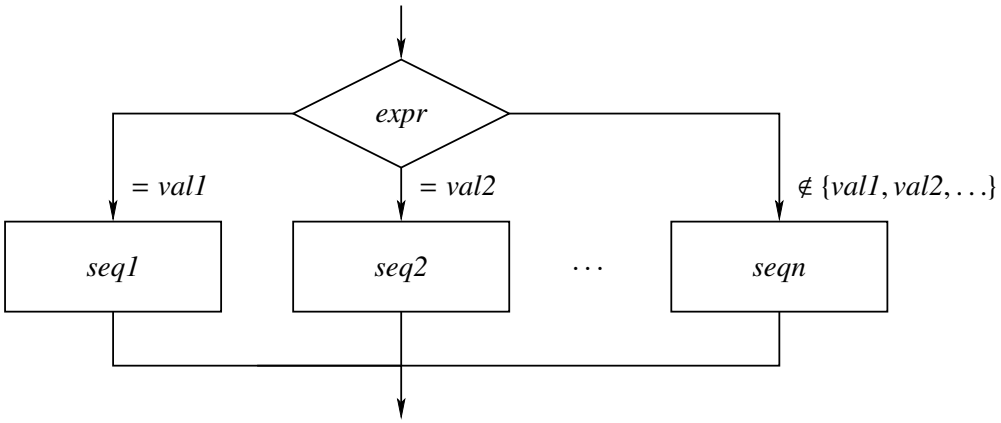


FIGURE 2.4 – Organigramme de l'instruction `switch`

Les instructions `break` qui séparent les différentes parties du corps ont pour effet de terminer l'exécution de celui-ci. (Nous reviendrons sur cette instruction à la section 2.4.4.) Si, par exemple, l'évaluation de *expr* retourne *val2*, alors cela garantit que seule la séquence *seq2* sera exécutée. Il est permis d'omettre ces instructions `break` lorsqu'on souhaite que l'exécution d'une séquence soit suivie par celle de la séquence suivante. Le plus souvent, ce n'est pas l'intention du programmeur, et l'oubli d'un `break` entre deux parties du corps d'un `switch` est une erreur de programmation fréquente. (C'est pour cette raison que nous n'avons pas indiqué les instructions `break` comme étant optionnelles dans la syntaxe de l'instruction `switch`.) Il est fortement conseillé, dans le cas où l'on omet délibérément une instruction `break` entre deux parties du corps d'un `switch`, d'écrire un commentaire signalant qu'il ne s'agit pas d'une erreur. (La façon d'exprimer des commentaires sera expliquée à la section 2.5.) Le seul cas où cela n'est pas nécessaire est celui de plusieurs étiquettes `case` qui correspondent au même point d'entrée, en d'autres termes qui sont séparées par des séquences vides.

Un organigramme précisant la sémantique de l'instruction `switch` est fourni à la figure 2.4.

Le programme de la figure 2.5 illustre l'utilisation de l'instruction `switch` pour implémenter un choix conditionnel possédant un grand nombre de branches. Ce programme lit le numéro d'une carte à jouer et en affiche la description. Le cas d'un numéro compris entre 2 et 10 montre comment grouper des étiquettes `case` en les séparant par des séquences vides. Dans ce programme, la partie "`default:`" du corps de l'instruction `switch` est un exemple de *programmation défensive* : elle permet au programme de gérer correctement le cas d'un numéro de carte invalide entré par l'utilisateur.

Il est probablement utile pour aider à comprendre ce programme de donner quelques explications sur le principe de fonctionnement de la fonction `printf` (nous y reviendrons plus en détail au chapitre 4). Cette fonction reçoit un premier *argument* qui est une *chaîne de caractères* ; l'effet de `printf` est d'afficher cette chaîne en y remplaçant les occurrences de *marqueurs de format*

```
#include <stdio.h>

int main()
{
  int carte;

  printf("Entrez un nombre entre 1 et 13: ");
  scanf("%d", &carte);

  printf("Cette carte est ");

  switch (carte)
  {
    case 1:
      printf("un as");
      break;

    case 2:
    case 3:
    case 4:
    case 5:
    case 6:
    case 7:
    case 8:
    case 9:
    case 10:
      printf("un %d", carte);
      break;

    case 11:
      printf("un valet");
      break;

    case 12:
      printf("une dame");
      break;

    case 13:
      printf("un roi");
      break;

    default:
      printf("invalide");
  }

  printf(".\n");
}
```

FIGURE 2.5 – Description d'une carte à jouer

par la valeur des arguments suivants. Par exemple, dans l'instruction

```
printf("un %d", carte)
```
,

la chaîne à afficher `"un %d"` contient un marqueur de format "%d" qui correspond à un entier signé. Ce marqueur sera donc remplacé par la valeur de l'argument suivant de l'appel, c'est-à-dire `carte`, affichée comme une valeur entière. Par exemple, si `carte` vaut 7, alors la chaîne `"un 7"` sera affichée.

Signalons pour terminer que l'on aurait pu écrire un programme équivalent à celui de la figure 2.5 en n'employant uniquement que des choix binaires, c'est-à-dire en imbriquant plusieurs instructions `if` les unes dans les autres. Utiliser une instruction `switch` est cependant plus efficace, car le compilateur est capable de la traduire en des instructions machine qui s'exécutent plus rapidement.

2.4.3 Les instructions de boucle

Nous passons maintenant aux *instructions de boucle*, qui permettent de répéter plusieurs fois l'exécution d'un morceau de code. En langage C, il en existe trois.

La boucle `while`

Nous avons déjà rencontré cette instruction dans l'implémentation de l'algorithme d'Euclide à la section 1.3.3. Sa syntaxe est proche de celle de l'instruction `if`, si ce n'est que l'instruction `while` ne possède pas de partie commençant par `else`.

`while` (*expr*)
instr ;

L'expression *expr*, appelée le *gardien* de la boucle, s'évalue en une valeur booléenne. L'instruction *instr* constitue le *corps* de la boucle ; elle peut bien sûr être remplacée par un bloc.

La sémantique de l'instruction `while` est donnée par l'organigramme de la figure 2.6. L'instruction commence par évaluer le gardien *expr*. Si le résultat obtenu est vrai, alors le corps *instr* de l'instruction est exécuté. Ensuite, le même processus se répète : l'expression *expr* est évaluée, *instr* est exécutée si le résultat obtenu est vrai, et ainsi de suite. L'instruction `while` se termine dès que l'évaluation du gardien *expr* retourne un résultat faux. Si cela se produit lors de sa première évaluation, alors le corps *instr* de l'instruction n'est pas exécuté du tout. Si cela ne se produit jamais, alors *instr* est exécutée indéfiniment, et l'instruction `while` ne se termine pas. (On parle alors de *boucle infinie*.)

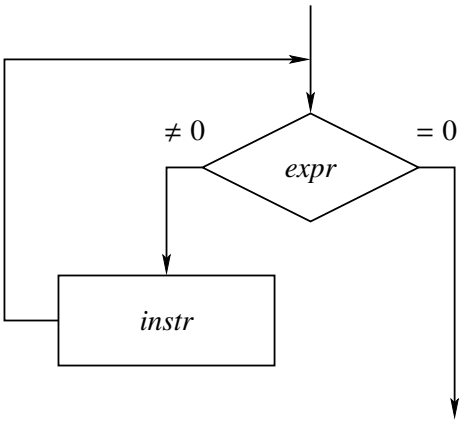


FIGURE 2.6 – Organigramme de l'instruction `while`

```
#include <stdio.h>

int main()
{
  unsigned i;

  i = 0;

  while (i < 100)
    {
      printf("%u\n", i);
      i++;
    }
}
```

FIGURE 2.7 – Compteur

Comme exemple de programme exploitant l'instruction `while`, nous proposons celui de la figure 2.7, qui énumère simplement tous les entiers de 0 à 99. Ce programme utilise un *compteur de boucle*[7] `i` initialisé à 0, et un gardien qui conditionne l'exécution d'une itération au fait que la valeur de ce compteur est strictement inférieure à 100. Le corps de la boucle affiche la valeur courante de `i`, et puis l'incrémente. Notons dans ce programme que le marqueur de format pour la fonction `printf` associé au type `unsigned` est "`%u`".

7. L'utilisation de variables nommées `i`, `j`, `k`, ... comme compteurs est une ancienne convention provenant du langage Fortran.

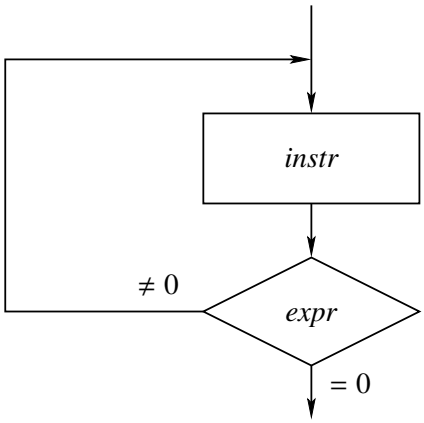


FIGURE 2.8 – Organigramme de l'instruction `do ... while`

La boucle `do ... while`

La deuxième instruction de boucle du langage C est l'instruction `do ... while`. Celle-ci partage l'usage d'un mot-clé avec l'instruction `while` que nous venons d'étudier, mais possède une syntaxe différente.

```
do
   instr;
while (expr);
```

Tout comme pour l'instruction `while`, le gardien *expr* s'évalue en une valeur booléenne, et le corps *instr* de la boucle peut être remplacé par un bloc.

La sémantique de cette opération est décrite par l'organigramme de la figure 2.8. L'instruction commence par exécuter le corps *instr* de la boucle, et puis seulement ensuite évalue *expr*. Si le résultat obtenu est vrai, alors on exécute à nouveau *instr*, on évalue *expr*, et ainsi de suite. Dès que l'évaluation de *expr* retourne une valeur fausse, l'instruction `do ... while` termine son exécution.

La différence entre les instructions `while` et `do ... while` est donc que la première évalue le gardien de boucle avant d'effectuer une itération, et la seconde après. Dans le cas particulier où le gardien est faux la première fois qu'il est évalué, l'instruction `while` n'effectue donc aucune itération, alors que l'instruction `do ... while` en effectue une.

On utilise l'instruction `while` quand on est amené à décider si une nouvelle itération est ou non nécessaire avant d'effectuer celle-ci. On emploie `do ... while` quand la décision de continuer ou non la boucle ne peut être prise qu'après une itération, par exemple parce qu'elle

dépend de données produites par cette itération. En pratique, on constate que le premier cas est plus fréquent que le second pour les problèmes de programmation courants.

Par exemple, quand on utilise une boucle pour appliquer une certaine opération de traitement aux éléments d'un ensemble, il est nécessaire de déterminer avant d'effectuer cette opération s'il reste au moins un élément à traiter. Si l'ensemble considéré est vide, alors aucun traitement ne doit être effectué. On emploie donc dans ce cas une instruction `while`.

Pour illustrer une application de `do ... while`, on peut considérer le problème qui consiste à tirer un nombre premier au hasard. Si l'on dispose d'un générateur de nombres aléatoires, une façon simple [8] de résoudre ce problème consiste à tirer des nombres au hasard en s'arrêtant dès que l'on obtient un nombre premier. On peut programmer cet algorithme à l'aide d'une boucle `do ... while` qui teste après chaque itération si celle-ci a produit ou non un nombre premier. Un autre exemple est donné par l'algorithme de Héron étudié à la section 1.2.2 : dans cet algorithme, on décide d'effectuer ou non une itération supplémentaire sur base d'une dernière estimation qui a été calculée par l'itération courante. Une instruction `do ... while` est donc tout indiquée pour implémenter cet algorithme.

La boucle `for`

La troisième instruction de boucle du langage C est l'instruction `for`, dont la syntaxe est plus compliquée :

```
for (expr1; expr2; expr3)
  instr;
```

Cette instruction comprend quatre composants. Tout comme pour les deux autres instructions de boucle, l'instruction *instr* constitue le corps de la boucle, et peut être remplacée par un bloc. Les parenthèses qui suivent le mot-clé `for` sont obligatoires, et doivent contenir exactement trois expressions [9] *expr1*, *expr2* et *expr3* séparées par des points-virgules "`;`".

8. Il ne s'agit pas de la meilleure solution.
9. Les versions modernes du langage C autorisent aussi de remplacer *expr1* par une déclaration de variable.

La sémantique de l'instruction `for` est identique à celle de la construction suivante.

```
{
  expr1;
  while (expr2)
    {
      instr;
      expr3;
    }
}
```

Cela signifie qu'*expr1* est évaluée une seule fois, avant de commencer à itérer la boucle. On emploie habituellement cette expression pour initialiser la valeur des variables manipulées dans la boucle. L'expression *expr2* est le gardien de boucle ; elle est évaluée avant chaque itération afin de déterminer si celle-ci doit avoir lieu. Cette évaluation doit retourner une valeur booléenne. Enfin, *expr3* est évaluée à la fin de chaque itération, après avoir exécuté le corps *instr* de la boucle. On exprime donc dans *expr3* les opérations qui doivent être effectuées pour passer d'une itération de la boucle à la suivante.

Il est permis de laisser *expr1*, *expr2*, *expr3* et/ou *instr* vides. Dans le cas d'*expr2*, cela revient à exprimer un gardien qui est toujours vrai. En particulier, l'instruction

```
for (;;);
```

implémente une boucle infinie qui n'effectue aucune opération utile.

L'instruction `for` permet de réécrire de façon un peu plus simple le programme de la figure 2.7. La nouvelle version est donnée à la figure 2.9. Il est facile de voir que ces deux programmes sont équivalents.

Un exemple un peu plus intéressant de programme utilisant l'instruction `for` est fourni à la figure 2.10. Ce programme est chargé de générer des tables de multiplication, et contient deux boucles imbriquées. La boucle sur `i` énumère tous les entiers de 1 à 9. Pour chacun d'entre eux, la boucle sur `j` multiplie successivement cet entier par toutes les valeurs de 1 à 9. Au total, l'instruction `printf(...)` la plus interne est donc exécutée 81 fois.

2.4.4 Les instructions de rupture de séquence

Nous avons vu à la section 2.4.2 qu'une instruction `break` située dans le corps d'une instruction `switch` termine immédiatement l'exécution de cette dernière. L'instruction `break` peut également être employée dans le corps d'une instruction `while`, `do ... while` ou `for`, avec le même effet : son exécution sort de la boucle en terminant immédiatement l'itération en cours.

```
#include <stdio.h>

int main()
{
  unsigned i;

  for (i = 0; i < 100; i++)
    printf("%u\n", i);
}
```

FIGURE 2.9 – Compteur (boucle `for`)

```
#include <stdio.h>

int main()
{
  unsigned i, j;

  for (i = 1; i <= 9; i++)
    {
      printf("\nTable pour %u\n", i);
      printf("=============\n\n");

      for (j = 1; j <= 9; j++)
        printf("%u x %u = %u\n", j, i, i * j);
    }
}
```

FIGURE 2.10 – Générateur de tables de multiplication

L'instruction `break` ne peut être employée que dans le corps d'une instruction `switch`, `while`, `do ... while` ou `for`. Dans le cas de plusieurs instructions imbriquées, l'instruction `break` se rapporte uniquement à la plus interne d'entre elles.

Examinons par exemple le fragment de code suivant.

```
for (i = 0;; i++)
  if (i > 100)
    break;
```

On a ici une boucle dont le gardien est toujours vrai. Si son corps était vide, elle effectuerait donc indéfiniment des itérations, en incrémentant le compteur de boucle `i` à chaque étape. Cependant, le corps de cette boucle contient une instruction de choix conditionnel qui provoque la terminaison de la boucle dès que la condition `i > 100` devient vraie. Au total, ce code exécutera donc 102 itérations de la boucle, pour les valeurs de `i` allant de 0 à 101.

Il existe une autre instruction de rupture de séquence appelée `continue`. Cette instruction ne peut être utilisée que dans le corps d'une instruction de boucle `while`, `do ... while` ou `for` (à la différence de `break`, il n'est pas permis de l'employer dans le corps d'une instruction `switch`). L'effet de `continue` est de terminer l'itération courante de la boucle et de passer immédiatement à l'itération suivante. Dans le cas d'une boucle

```
for (expr1; expr2; expr3)
    instr;
```

cela signifie que l'exécution de `continue` au sein de *instr* provoque la terminaison immédiate de *instr*, suivie par une évaluation de *expr3*, et ensuite d'une évaluation du gardien *expr2* afin de déterminer s'il faut à nouveau effectuer une itération.

Tout comme pour `break`, une instruction `continue` placée dans plusieurs boucles imbriquées se rapporte uniquement à celle qui est la plus interne.

Il ne faut pas abuser des instructions `break` et `continue`, mais leur utilisation permet dans certains cas de simplifier la structure d'un programme et donc de le rendre plus lisible. Par exemple, le morceau de code suivant montre comment programmer une opération qui doit être effectuée pour toutes les paires (`i`, `j`) telles que $0 \leq$ `i` < 100, $1 \leq$ `j` $<$ `i` et `j` ne divise pas `i`.

```
for (i = 0; i < 100; i++)
  for (j = 1; j < i; j++)
    {
      if (!(i % j))
        continue;
      ⋮
    }
```

On aurait bien sûr pu ajouter une branche `else` à l'instruction `if`, contenant l'opération à effectuer, ou bien placer cette opération dans le corps d'une instruction `if` évaluant une condition contraire, mais l'utilisation de `continue` permet ici d'éviter un niveau supplémentaire d'indentation, ce qui améliore la lisibilité du code.

2.5 Les commentaires

Les programmes que l'on rédige ne sont pas uniquement destinés à être traités par un compilateur ; il est également important qu'ils soient lisibles et facilement compréhensibles pour les programmeurs amenés à les mettre au point ou y apporter des modifications. Une première étape dans ce sens consiste bien sûr à choisir des identificateurs, tant pour les variables que pour d'autres éléments de programmes que nous étudierons plus tard, qui possèdent un nom clair, cohérent, et qui décrit précisément l'usage qui en est fait.

En addition à cela, on peut également insérer des *commentaires* dans les programmes, qui sont des fragments de texte ignorés par le compilateur. Nous avons notamment déjà évoqué à la section 2.4.2 la pertinence d'écrire un tel commentaire lorsqu'on omet une instruction `break` entre deux parties du corps d'une instruction `switch`.

En langage C, les commentaires sont délimités par "`/*`" et "`*/`". Le texte situé entre ces marqueurs n'est pas lu par le compilateur. Ces commentaires ne peuvent pas être imbriqués. Il existe également une autre forme de commentaire admise par les versions modernes du langage, inspirée du langage de programmation C++ : un commentaire peut aussi commencer par "`//`", et se terminer à la fin de la ligne courante. L'exemple suivant montre un morceau de code contenant les deux formes de commentaires.

```
/* Position de l'objet */

double x, y;  // Coordonnées
double h;     // Hauteur
```

2.6 Exercices

1. En supposant que les variables `x`, `y` et `z` sont entières et initialisées à zéro, quelle sera leur valeur après l'exécution de chacune des instructions suivantes ?

   (a) `z = x++ + --y;`
   (b) `z -= x++ && y++ ? 1 : 2;`
   (c) `z += ++x && ++y ? 1 : 2;`
   (d) `z = x > 0 || !++y ? x++ : ++x;`
   (e) `z = (x++, x > 0) ? !x : 1 - !x;`

2. Écrire des fragments de code équivalents à ceux donnés ci-après, mais utilisant l'instruction de contrôle `while`.

   (a)
   ```
   for (i = 0; i < 1000; i += 10)
     j += i;
   ```

   (b)
   ```
   i = j = 0;
   do
     {
       j += i;
       i += 2;
     }
   while (j < k);
   ```

   (c)
   ```
   for (i = 0, j = 0; i < 1000; i++)
     {
       if (f(i) > f(j))
          continue;
       j += i;
     }
   ```

   (d)
   ```
   for (;;);
   ```

3. Écrire des fragments de code équivalents à ceux donnés ci-après, mais utilisant l'instruction de contrôle `for`.

(a)
```
i = 2;
while (i < 100000)
  {
    j += i;
    i *= 2;
  }
```

(b)
```
p = premier();
while (!est_dernier(p))
  {
    traiter(p);
    p = suivant(p);
  }
```

(c)
```
while (attendre());
```

(d)
```
i = 0;
j = 0;
while (1)
  {
    j += i++;
    if (j >= k)
      break;
  }
```

4. Écrire des programmes implémentant les algorithmes obtenus pour les exercices de la section 1.5

   Note : Nous donnons ici quelques éléments relatifs à la manipulation des nombres réels, qui fait l'objet du premier exercice. Pour les fonctions `printf` et `scanf`, le marqueur de format correspondant au type `double` est "`%lf`". Pour calculer la racine carrée et la valeur absolue d'un nombre, on peut employer (respectivement) les fonctions `sqrt` et `fabs` de la bibliothèque standard. Pour cela, la directive `#include <math.h>` doit figurer au début du programme. Enfin, il faut explicitement indiquer au compilateur que la bibliothèque de fonctions mathématiques doit être liée au programme ; pour le compilateur `GCC`, cela se fait en ajoutant l'option "`-lm`" en fin de la ligne de commande.

5. Les *nombres de Fibonacci* $F_1, F_2, F_3, \ldots$ sont définis par les règles suivantes :

$$\begin{aligned} F_1 &= 1 \\ F_2 &= 1 \\ F_n &= F_{n-1} + F_{n-2} \text{ pour tout } n > 2. \end{aligned}$$

Écrire un programme capable de calculer efficacement la valeur de F_n pour un indice n donné.

6. Une *suite de Collatz* s'obtient en calculant, pour un nombre entier $z_0 > 1$ donné, les nombres $z_1, z_2, z_3, \ldots$ de la façon suivante. Pour tout $i \geq 0$:

$$\begin{cases} \text{Si } z_i \textit{ est pair}, \text{ alors } z_{i+1} = \dfrac{z_i}{2}. \\ \text{Si } z_i \textit{ est impair}, \text{ alors } z_{i+1} = 3z_i + 1. \end{cases}$$

Écrire un programme capable de calculer, pour un nombre z_0 entré au clavier, le nombre d'étapes nécessaires pour atteindre le nombre 1 (c'est-à-dire la plus petite valeur de i pour laquelle on a $z_i = 1$). Par exemple, pour $z_0 = 3$, le programme doit afficher 7, puisque l'on a $z_1 = 10$, $z_2 = 5$, $z_3 = 16$, $z_4 = 8$, $z_5 = 4$, $z_6 = 2$ et $z_7 = 1$.

Note : La question de savoir si la valeur 1 est toujours atteinte à partir d'une valeur quelconque de z_0 est un problème ouvert en mathématiques.

7. Écrire un programme permettant de calculer efficacement la valeur d'un *coefficient binomial*

$$C_n^m = \frac{n!}{m!(n-m)!},$$

où n et m sont des entiers donnés tels que $0 \leq m \leq n$, et où $p! = p.(p-1)\ldots 2.1$.

8. Écrire un programme capable de factoriser un nombre entier entré au clavier. Le résultat doit être affiché sous la forme

facteur_premier^puissance facteur_premier^puissance ...

où les facteurs premiers apparaissent dans l'ordre croissant.

Par exemple, pour le nombre 147015, le programme doit afficher "`3^5 5 11^2`".

Chapitre 3

Quelques notions d'algorithmique

Le chapitre 2 a introduit suffisamment d'éléments du langage C pour permettre de rédiger des programmes simples, c'est-à-dire exprimer des algorithmes dans un formalisme qui permet à un ordinateur de les exécuter.

Nous nous intéressons à présent à la façon de concevoir de tels algorithmes, c'est-à-dire aux techniques qui permettent de passer de l'énoncé d'un problème informatique à un algorithme résolvant ce problème. Il n'existe cependant pas de méthode systématique permettant de faire cela ; l'algorithmique ne peut s'apprendre qu'en se construisant une expérience basée sur la résolution d'exercices variés. Dans ce chapitre, nous allons introduire un problème particulier, la recherche de nombres parfaits, et en développer plusieurs solutions de plus en plus élaborées. Nous étudierons ensuite des mécanismes permettant de comparer les performances de ces solutions, et de s'assurer qu'elles sont correctes.

3.1 La recherche de nombres parfaits

Un nombre entier $n \geq 1$ est dit *parfait* s'il est égal à la somme de ses diviseurs, excepté lui-même. Par exemple, le nombre 28 est parfait, car on a

$$28 = 1 + 2 + 4 + 7 + 14.$$

Le problème que nous allons chercher à résoudre est celui qui consiste à trouver tous les nombres parfaits qui appartiennent à un intervalle donné, par exemple ceux qui sont inférieurs à 10^6.

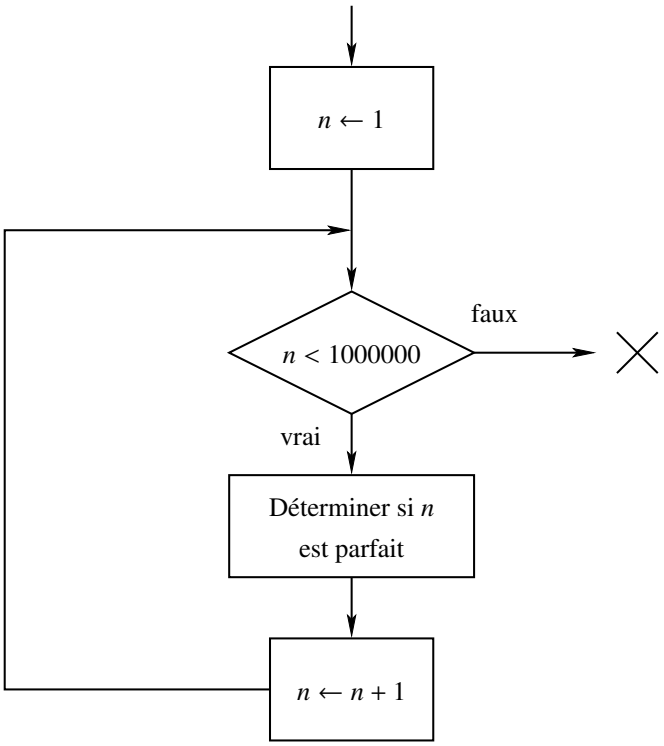


FIGURE 3.1 – Énumération des nombres à tester

3.1.1 Première solution

Lorsqu'on est confronté à un problème algorithmique, une bonne stratégie consiste à essayer de le décomposer en une combinaison de sous-problèmes plus simples. Dans le cas présent, le problème de rechercher tous les nombres parfaits inférieurs à 10^6 peut se décomposer en deux sous-problèmes :

— Énumérer tous les entiers dans l'intervalle $[1,\ 10^6 - 1]$.

— Pour chacun de ces entiers, déterminer s'il est parfait ou non.

Le premier de ces sous-problèmes ne présente aucune difficulté ; nous avons vu au chapitre 2 comment programmer une boucle qui énumère toutes les valeurs dans un intervalle donné. Une solution est donnée par l'organigramme de la figure 3.1 : on initialise une variable n à 1, et on l'incrémente tant que le gardien de boucle $n < 10^6$ reste vrai.

Nous avons donc réduit le problème à celui de déterminer si un nombre $n \geq 1$ donné est parfait ou non.

Une façon simple de résoudre ce dernier problème consiste à se baser sur la définition d'un nombre parfait : pour déterminer si un nombre $n \geq 1$ est parfait, il suffit de calculer la somme

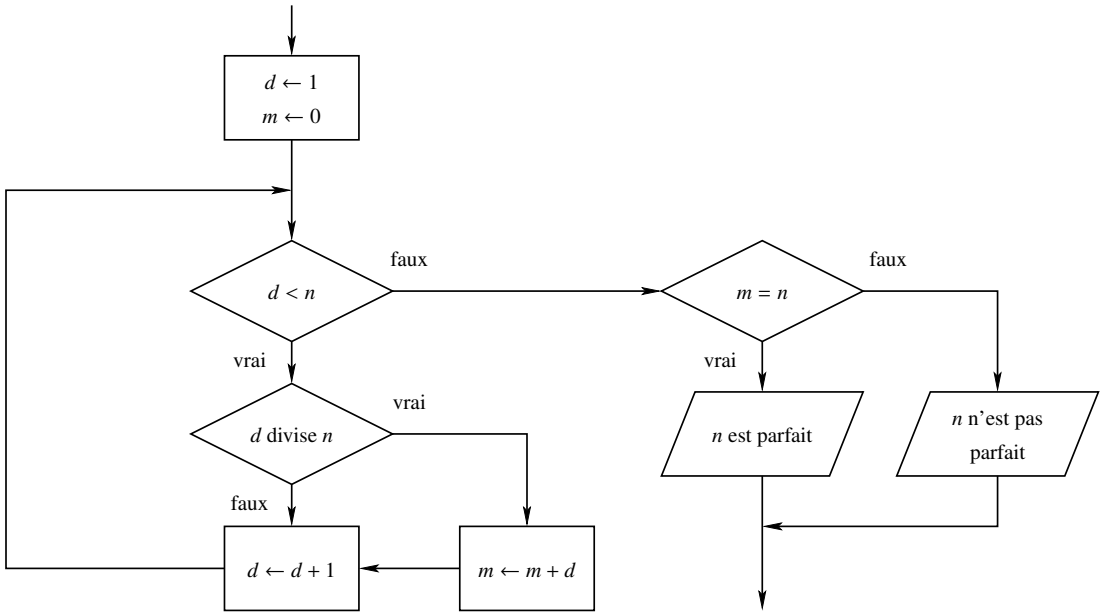


FIGURE 3.2 – Organigramme de la première solution

m de tous les diviseurs de n (sauf n lui-même), et de déterminer si m est égal à n. Pour calculer m, on peut énumérer tous les diviseurs potentiels d de n différents de n, qui appartiennent nécessairement à l'intervalle $[1, n - 1]$, tester pour chacun d'entre eux s'il divise effectivement n, et additionner ceux qui satisfont cette propriété.

Un organigramme formalisant cette solution est donné à la figure 3.2. Sa traduction en langage C est donnée à la figure 3.3.

Remarquons que dans ce programme, la borne supérieure des entiers à énumérer est représentée par une constante `n_max`, au lieu d'être directement encodée dans l'expression du gardien de boucle. Il s'agit d'une bonne habitude de programmation, qui permet de facilement modifier la valeur de cette borne supérieure, en garantissant que cette modification sera correctement répercutée vers tous les endroits du programme qui en dépendent.

3.1.2 Deuxième solution

Il est clair que l'algorithme de la figure 3.2 n'est pas optimal. En effet, cet algorithme effectue un certain nombre d'opérations qui sont inutiles. Par exemple, il n'est pas nécessaire de tester si 1 divise n, car cette propriété est toujours vraie. On pourrait donc gagner une étape en énumérant les diviseurs potentiels d de n à partir de 2 plutôt que de 1. Cela nécessiterait d'initialiser la somme m des diviseurs à 1 plutôt qu'à 0, ce qui revient à considérer que le diviseur 1 est systématiquement présent. Cela n'est cependant pas le cas pour le nombre $n = 1$, car on se limite aux diviseurs qui sont strictement inférieurs à n. Ce problème peut être résolu en commençant l'énumération des

```
#include <stdio.h>

int main()
{
  const unsigned n_max = 999999;

  unsigned n, m, d;

  for (n = 1; n <= n_max; n++)
    {
      for (d = 1, m = 0; d < n; d++)
        if (!(n % d))
          m += d;

      if (m == n)
        printf("%u\n", n);
    }
}
```

FIGURE 3.3 – Implémentation de la première solution

nombres n à considérer à partir de 2 plutôt que de 1, ce qui est correct car 1 n'est pas parfait.

Une autre amélioration est de réaliser que le plus grand diviseur de n inférieur à n est au plus égal à $n/2$, ce qui permet de réduire l'intervalle dans lequel on recherche les diviseurs potentiels à $[2, \lfloor n/2 \rfloor]$. Enfin, lorsqu'on additionne les diviseurs qui ont été trouvés, si l'on obtient une valeur intermédiaire m qui est strictement supérieure à n, alors il n'est plus nécessaire de continuer à chercher d'autres diviseurs de n. On sait en effet déjà que n n'est pas parfait, car la somme de tous ses diviseurs (sauf lui-même) est alors forcément supérieure à n.

Une implémentation de toutes ces améliorations est donnée à la figure 3.4. Dans ce programme, il y a deux façons de sortir de la boucle sur `d` : soit on a $\texttt{d} > \lfloor \texttt{n}/2 \rfloor$ et `m` contient la somme de tous les diviseurs de `n` inférieurs à `n`, soit on a $\texttt{m} > \texttt{n}$ et `m` contient la somme d'un sous-ensemble des diviseurs de `n`. Dans les deux cas, la propriété exprimant que `n` est un nombre parfait si et seulement si $\texttt{m} = \texttt{n}$ est vraie.

3.1.3 Troisième solution

Pour encore améliorer la solution fournie par le programme de la figure 3.4, il faut étudier plus en profondeur le problème que l'on cherche à résoudre. On peut remarquer que pour tout nombre $n \geq 1$ et diviseur d de n, le nombre $d' = n/d$ est également un diviseur de n. En effet, les diviseurs de n peuvent être regroupés en paires, les diviseurs d et d' étant appariés si $d.d' = n$.

```c
#include <stdio.h>

int main()
{
  const unsigned n_max = 999999;

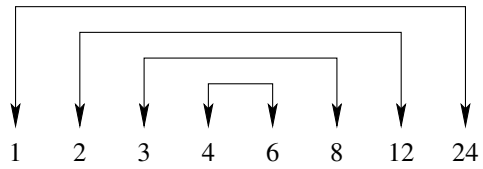
  unsigned n, m, d;

  for (n = 2; n <= n_max; n++)
    {
      for (d = 2, m = 1; d <= n / 2 && m <= n; d++)
        if (!(n % d))
          m += d;

      if (m == n)
        printf("%u\n", n);
    }
}
```

FIGURE 3.4 – Implémentation de la deuxième solution

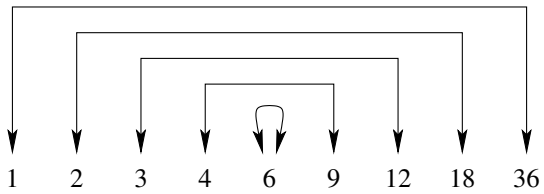
Ce mécanisme est illustré par le diagramme suivant pour $n = 24$:



On peut exploiter cette propriété pour énumérer plus efficacement les diviseurs de n qui sont supérieurs à 1 et inférieurs à n : au lieu de balayer l'intervalle $[2, \lfloor n/2 \rfloor]$, il suffit d'énumérer toutes les valeurs d dans l'intervalle $[2, \lfloor \sqrt{n} \rfloor]$, et pour chacune d'entre elles, de considérer les deux candidats diviseurs d et n/d. L'avantage est que le nombre d'étapes nécessaires pour trouver l'ensemble des diviseurs de n est alors considérablement réduit pour de grandes valeurs de n.

Il y a cependant un cas particulier qu'il faut veiller à traiter correctement : si n est un carré parfait, c'est-à-dire, s'il existe $k \in \mathbb{N}$ tel que $n = k^2$, alors le diviseur $d = k$ est tel que $d' = n/d = d$, en d'autres termes ce diviseur se retrouve apparié avec lui-même.

Cette situation est illustrée par le diagramme suivant, pour $n = 36$:



Dans ce cas particulier, il faut s'assurer que le diviseur $d = d'$ n'est compté qu'une seule fois dans le calcul de la somme des diviseurs de n.

Un programme qui implémente cette troisième solution est donné à la figure 3.5. Par rapport à celui de la figure 3.4, le gardien de la boucle sur `d` a été modifié de façon à terminer l'énumération des diviseurs à $\lfloor \sqrt{\texttt{n}} \rfloor$ plutôt qu'à $\lfloor \texttt{n}/2 \rfloor$.

Le langage C ne possède pas d'opérateur permettant de calculer une racine carrée. La *bibliothèque mathématique* fournit une fonction `sqrt` calculant la racine carrée d'un nombre réel, mais il ne s'agit que d'une solution approximative [1] qui n'est pas appropriée ici. Dans le programme de la figure 3.5, ce problème est résolu en remplaçant la condition $\texttt{d} \leq \sqrt{\texttt{n}}$ par $\texttt{d.d} \leq \texttt{n}$, qui peut être calculée en arithmétique entière.

1. Rappelons qu'en toute généralité, la manipulation de nombres réels par un ordinateur est sujette à des imprécisions.

```c
#include <stdio.h>

int main()
{
  const unsigned n_max = 999999;

  unsigned n, m, d, d2;

  for (n = 2; n <= n_max; n++)
    {
      for (d = 2, m = 1; d * d <= n && m <= n; d++)
        if (!(n % d))
          {
            m += d;
            d2 = n / d;
            if (d2 != d)
              m += d2;
          }

      if (m == n)
        printf("%u\n", n);
    }
}
```

FIGURE 3.5 – Implémentation de la troisième solution

	Temps d'exécution		
`n_max`	solution 1	solution 2	solution 3
999	0,95 ms	0,47 ms	58 μs
9999	93 ms	45 ms	1,4 ms
99999	9,2 s	4,4 s	37 ms
999999	930 s	447 s	1,1 s
9999999	> 1 h	> 1 h	33 s
99999999	> 1 h	> 1 h	1061 s

FIGURE 3.6 – Temps d'exécution des trois solutions au problème des nombres parfaits

Il faut cependant faire attention, quand on écrit des expressions telles que `d * d <= n`, à garantir qu'un dépassement arithmétique ne se produira pas lors de leur évaluation. Dans le cas présent, la plus grande valeur de `d` qui sera potentiellement considérée est égale à $\lfloor \sqrt{n} \rfloor + 1$. Le produit `d * d` fournira donc une valeur inférieure ou égale à $n + 2\sqrt{n} + 1$. Sur une architecture représentant les entiers sur 32 bits, ce nombre reste contenu dans l'intervalle des valeurs représentables, pour la valeur de `n_max` figurant dans le programme.

Dans la suite du programme, pour chaque diviseur `d` de `n` qui est considéré, on calcule le diviseur `d2` qui lui est apparié, et l'on ajoute ce dernier diviseur à la somme courante `m` uniquement s'il est différent de `d`.

3.1.4 Comparaison des performances

Nous avons développé aux sections 3.1.1, 3.1.2 et 3.1.3 trois solutions au problème de recherche des nombres parfaits appartenant à un intervalle donné, de la plus simple à la plus élaborée. Nous souhaitons maintenant déterminer laquelle de ces solutions est la plus performante. Une première façon de le faire consiste à mesurer expérimentalement leur temps d'exécution. Pour caractériser le mieux possible le comportement des programmes, nous allons effectuer ces mesures pour différentes valeurs de la constante `n_max`.

Bien sûr, le temps d'exécution d'un programme dépend de l'ordinateur et du compilateur utilisés. Les résultats fournis à la figure 3.6 ont été obtenus avec un ordinateur doté d'un processeur Intel Core i7-9750H tournant à 2,60 GHz. Le compilateur employé est GCC dans sa version 10.3.1, avec un niveau d'optimisation [2] égal à 3. Le temps d'exécution a été limité à une heure.

2. Les compilateurs sont capables d'*optimiser* le code machine qu'ils produisent, notamment dans le but de le rendre plus efficace. Pour le compilateur GCC, le niveau d'optimisation le plus élevé s'obtient en ajoutant l'option "`-O3`" à la ligne de commande.

On voit dans le tableau de la figure 3.6 que le temps d'exécution de notre première solution augmente approximativement d'un facteur 100 quand `n_max` augmente d'un facteur 10. Cela conduit à ce que cette version du programme dépasse la limite de temps imposée pour des valeurs de `n_max` supérieures ou égales à $10^7 - 1$.

Les performances de la deuxième solution sont similaires, le temps d'exécution augmentant aussi d'un facteur environ égal à 100 lorsque `n_max` augmente d'un facteur 10. Sur une échelle absolue, cette deuxième solution est cependant à peu près deux fois plus rapide que la première. Ce n'est pas surprenant, étant donné que la principale différence entre les deux programmes est que le deuxième examine environ deux fois moins de diviseurs potentiels que le premier.

La troisième solution est quant à elle bien meilleure que les deux premières, et permet d'obtenir un résultat dans un temps raisonnable pour des valeurs de `n_max` allant jusqu'à $10^8 - 1$. Cette situation est représentative de beaucoup de problèmes algorithmiques : il est souvent possible d'améliorer considérablement les performances d'une solution naïve en exploitant à bon escient les propriétés que l'on découvre en étudiant plus en profondeur [3] le problème à résoudre.

3.2 La complexité en temps

À la section 3.1.4, nous avons comparé les performances des trois solutions obtenues pour le problème de recherche des nombres parfaits, en implémentant ces solutions et en mesurant expérimentalement leur temps de calcul pour différentes valeurs de `n_max`.

Il serait utile de disposer d'un outil permettant de raisonner sur les performances d'un algorithme sans devoir l'implémenter explicitement, et en restant le plus indépendant possible des caractéristiques des ordinateurs qui pourraient l'exécuter.

3.2.1 Principes

Nous avons déjà évoqué à la section 1.4.1 la *complexité en temps* [4], qui est une mesure du temps nécessaire à l'exécution d'un algorithme ou d'un programme, exprimée en fonction de la taille d'un ou de plusieurs *paramètres* du problème. On ne s'intéresse à cette complexité que pour des paramètres de grande taille, et à une constante de proportionnalité près. Lorsque plusieurs exécutions sont possibles pour une taille donnée des paramètres, on ne tient compte que de celle

3. Signalons que notre troisième programme est loin de fournir la meilleure solution possible au problème de recherche de nombres parfaits. En exploitant des propriétés plus avancées du problème, des mathématiciens ont réussi à développer des algorithmes beaucoup plus efficaces.

4. Il existe aussi une notion de *complexité en espace*, que nous étudierons plus tard.

qui présente le temps de calcul le plus élevé (complexité dans le cas le plus défavorable, *worst-case complexity*).

Pour pouvoir calculer le temps d'exécution d'un programme, on compte le nombre d'opérations qu'il effectue. Cela nécessite bien sûr de définir ce qui constitue une opération élémentaire. Par exemple, on pourrait considérer que l'évaluation d'une expression telle que `n++` constitue une seule opération, ou bien au contraire estimer qu'elle nécessite de lire en mémoire la valeur de `n`, de l'incrémenter et d'écrire le résultat en mémoire, ce qui représente trois opérations. Il serait même possible de pousser le raisonnement plus loin en examinant le nombre d'instructions du processeur générées par le compilateur pour cette instruction, pour une architecture particulière.

On souhaite que la complexité en temps d'un programme reste insensible à ce genre de détail. C'est la raison pour laquelle cette complexité s'exprimera sous une forme qui ne tient pas compte des constantes de proportionnalité qui affectent le nombre total d'opérations effectuées. On peut ainsi considérer que l'évaluation d'une expression telle que `n++` représente 1, 3, 10 ou même k opérations élémentaires, où k est borné par une valeur indépendante des paramètres du problème, sans que cela ne change la complexité de l'algorithme étudié. Ce mécanisme permet en particulier de s'affranchir des détails matériels de l'environnement d'exécution.

3.2.2 La notation "grand-O"

La notation "grand-O" permet de décrire le comportement asymptotique d'une fonction (c'est-à-dire dans notre contexte, pour de grandes valeurs de ses arguments), d'une façon indépendante des constantes de proportionnalité qui l'affectent. Elle est donc bien adaptée pour exprimer la complexité d'un algorithme ou d'un programme.

Définition

Soient deux fonctions $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. Pour bien fixer les idées, on peut considérer qu'elles décrivent le temps d'exécution d'un algorithme en fonction de la taille d'un paramètre d'entrée n. On écrit

$$f \in O(g),$$

ou, par abus d'écriture,

$$f(n) \in O(g(n)),$$

s'il existe un seuil $n_0 \in \mathbb{N}$ et une constante de proportionnalité $c \in \mathbb{R}_{\geq 0}$ tel que pour toutes les valeurs de n situées au delà de n_0, la valeur de $f(n)$ est bornée par celle de $c.g(n)$. Formellement, on a

$$f \in O(g) \text{ ssi } \exists n_0 \in \mathbb{N},\, c \in \mathbb{R}_{\geq 0} :\ \forall n > n_0 :\ f(n) \leq c.g(n).$$

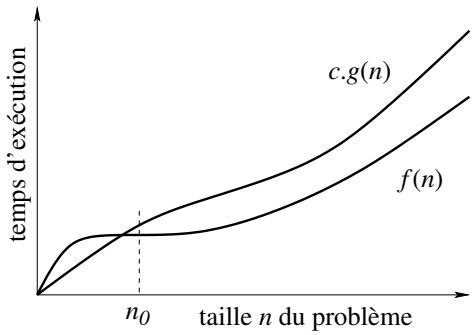


FIGURE 3.7 – Notation "grand-O"

Le cas de deux fonctions f et g telles que $f \in O(g)$ est illustré à la figure 3.7. On voit que ces fonctions ne sont comparées que pour des grandes valeurs de leur argument, les valeurs inférieures ou égales au seuil n_0 n'étant pas prises en compte. De plus, les fonctions f et g peuvent être multipliées par n'importe quel facteur constant strictement positif sans que cela ne change la situation, puisqu'il suffit d'adapter la valeur du coefficient c en fonction de ce facteur. (Nous allons démontrer formellement cette propriété un peu plus loin.)

La notation "grand-O" se généralise directement aux fonctions admettant n'importe quel nombre d'arguments. Pour deux fonctions $f, g : \mathbb{N}^k \to \mathbb{R}_{\geq 0}$, avec $k > 0$, on a $f \in O(g)$ ssi

$$\exists n_{0,1}, n_{0,2}, \ldots, n_{0,k} \in \mathbb{N}, c \in \mathbb{R}_{\geq 0} : \forall n_1 > n_{0,1}, n_2 > n_{0,2}, \ldots, n_k > n_{0,k}, : \\ f(n_1, n_2, \ldots, n_k) \leq c.g(n_1, n_2, \ldots n_k).$$

Propriétés

Nous allons à présent étudier quelques propriétés utiles de la notation "grand-O". Par souci de simplicité, nous nous limitons au cas des fonctions à un seul argument, mais la généralisation de ces propriétés à des fonctions d'*arité* quelconque est immédiate.

Pour commencer, si deux fonctions f et g sont telles que $f \in O(g)$, alors pour tout $k \in \mathbb{R}_{\geq 0}$, on a aussi

$$k.f \in O(g).$$

En effet, si $f \in O(g)$, alors il existe $n_0 \in \mathbb{N}$ et $c \in \mathbb{R}_{\geq 0}$ tels que

$$\forall n > n_0 : f(n) \leq c.g(n).$$

On a alors

$$\forall n > n_0 : k.f(n) \leq (ck).g(n),$$

avec $ck \in \mathbb{R}_{\geq 0}$, qui entraîne $k.f \in O(g)$. Intuitivement, cette propriété établit que les facteurs constants strictement [5] positifs n'interviennent pas dans le calcul d'une complexité.

Parmi les corollaires de cette propriété, on a

$$a^{n+b} \in O\left(a^n\right)$$

pour $a \in \mathbb{R}_{\geq 0}$ et $b \in \mathbb{R}$, en suivant la convention que n est le paramètre qui varie. En effet, on a $a^{n+b} = a^b.a^n$, avec $a^b \geq 0$.

De même, on a

$$\log_a n \in O\left(\log_b n\right)$$

pour tous $a,\ b \in \mathbb{R}_{>1}$, car $\log_a n = \dfrac{1}{\log_b a} \log_b n$ et $\dfrac{1}{\log_b a} \geq 0$. Cela montre qu'il n'est pas important de fixer la base utilisée pour les logarithmes quand on écrit une complexité ; en pratique, on écrira souvent $O(\log n)$ pour spécifier une complexité logarithmique, sans mentionner de base.

Une autre propriété intéressante est que si deux fonctions e et f sont telles que $e \in O(g)$ et $f \in O(g)$ vis-à-vis d'une fonction commune g, alors on a

$$e + f \in O(g).$$

Cette propriété peut se démontrer de la façon suivante. Si $e \in O(g)$, alors il existe $n_0 \in \mathbb{N}$ et $c \in \mathbb{R}_{\geq 0}$ tels que

$$\forall n > n_0 :\ e(n) \leq c.g(n).$$

De même, si $f \in O(g)$, alors il existe $n'_0 \in \mathbb{N}$ et $c' \in \mathbb{R}_{\geq 0}$ tels que

$$\forall n > n'_0 :\ f(n) \leq c'.g(n).$$

On en déduit

$$\forall n > \max(n_0, n'_0) :\ e(n) + f(n) \leq (c + c').g(n),$$

qui entraîne $e + f \in O(g)$ étant donné que l'on a $c + c' \geq 0$.

Cette propriété possède le corollaire important suivant. Si $a_0, a_1, \ldots, a_m \in \mathbb{R}_{\geq 0}$, pour $m \geq 0$, alors

$$\sum_{i=0}^{m} a_i\, n^i = O\left(n^m\right).$$

En effet, pour tout $i \in [0, m]$, on a $a_i\, n^i \in O\left(n^m\right)$, car $n^i \leq n^m$ pour tout $n \geq 1$, et le facteur constant $a_i \geq 0$ peut-être négligé. Enfin, si tous les termes du polynôme sont $O\left(n^m\right)$, alors leur somme l'est également d'après la propriété d'addition établie précédemment. En résumé, si un calcul de complexité produit un polynôme à coefficients non négatifs, alors seul son terme de degré le

5. Un facteur égal à zéro réduit évidemment la complexité à $O(0)$.

plus élevé est significatif. Cette propriété reste valide si certains coefficients du polynôme sont négatifs (à l'exception de celui du terme de degré le plus élevé qui doit être positif). En effet, dans ce cas, supprimer les termes de coefficient négatif fournit une majoration du polynôme en question, pour les valeurs positives de n. Si $a_0, a_1, \ldots, a_{m-1} \in \mathbb{R}$ et $a_m \in \mathbb{R}_{>0}$, pour $m \geq 0$, on a donc

$$\sum_{i=0}^{m} a_i \, n^i = O\left(n^m\right).$$

La dernière propriété que nous présentons concerne les produits de fonctions. Si deux fonctions e et f sont telles que $e \in O(g)$ et $f \in O(h)$ vis-à-vis de deux autres fonctions g et h, alors on a

$$e.f \in O(g.h).$$

En effet, si $e \in O(g)$, alors il existe $n_0 \in \mathbb{N}$ et $c \in \mathbb{R}_{\geq 0}$ tels que

$$\forall n > n_0 : \; e(n) \leq c.g(n).$$

De même, si $f \in O(h)$, alors il existe $n'_0 \in \mathbb{N}$ et $c' \in \mathbb{R}_{\geq 0}$ tels que

$$\forall n > n'_0 : \; f(n) \leq c'.h(n).$$

On a donc

$$\forall n > \max(n_0, n'_0) : \; e(n).f(n) \leq (c.c').g(n).h(n),$$

avec $c.c' \geq 0$, qui entraîne $e.f \in O(g.h)$.

3.2.3 Les classes de complexité

La notion de complexité en temps permet de comparer l'efficacité des algorithmes et des programmes, en regroupant au sein de *classes* ceux dont la complexité partage la même notation "grand-O" [6]. Dans le cas où il n'y a qu'un seul paramètre n qui varie, une liste non exhaustive de telles classes est donnée ici, par ordre décroissant d'efficacité :

$$\begin{aligned} O(1) \;&\subset\; O(\log\log n) \subset O(\log n) \subset O(n) \subset O(n \log n) \\ &\subset\; O\left(n^2\right) \subset O\left(n^3\right) \subset O\left(n^4\right) \subset \cdots \\ &\subset\; O\left(2^n\right) \subset O\left(2^{2^n}\right) \subset O\left(2^{2^{2^n}}\right) \subset \cdots \\ &\subset\; O\left(2^{2^{2^{\cdots}}} \Big\} n\right) \subset \cdots \end{aligned}$$

6. Rigoureusement parlant, la notation "grand-O" que nous avons introduite ne décrit qu'une borne supérieure de complexité. Il existe une autre notation, que nous n'étudierons pas dans ce cours, qui permet de décrire précisément le niveau de complexité d'un algorithme ou d'un programme, en combinant des bornes inférieure et supérieure.

La classe $O(1)$ est celle des programmes qui s'exécutent en temps borné. En d'autres termes, cela signifie que leur temps d'exécution possède une borne supérieure constante qui n'est jamais atteinte, quelle que soit la valeur de leur paramètre n. Dans la classe $O(n)$, on trouve les programmes dont le temps de calcul croît proportionnellement à la taille de leurs données d'entrée ; on dit alors que leur complexité est *linéaire*. Ces programmes ainsi que ceux des classes inférieures sont considérés comme étant très efficaces. La classe $O(n \log n)$ contient des programmes qui présentent toujours une efficacité élevée, ce qui signifie qu'il restent utilisables pour de grandes valeurs de leur paramètre, bien qu'ils ne soient pas de complexité linéaire. Les classes des programmes *quadratiques* $O\left(n^2\right)$, *cubiques* $O\left(n^3\right)$, et ainsi de suite, correspondent aux complexités *polynomiales*. On trouve ensuite la classe *exponentielle* $O\left(2^n\right)$, puis les classes *super-exponentielles* $O\left(2^{2^n}\right)$, $O\left(2^{2^{2^n}}\right)$, …, la classe $O\left(2^{2^{2^{\cdots}}\}n}\right)$ qui est la première à être *non élémentaire*, et puis d'autres classes correspondant à des programmes encore moins efficaces.

Certains auteurs estiment que la frontière entre les algorithmes suffisamment efficaces, c'est-à-dire utilisables en pratique, et les autres se situe entre les complexités polynomiale et exponentielle. On constate cependant que les performances des algorithmes quadratiques sont parfois déjà insuffisantes pour certaines applications. Il faut également garder à l'esprit que la complexité que nous calculons correspond toujours au cas le plus défavorable. Il existe des algorithmes qui possèdent une complexité en temps élevée, mais dont le temps de calcul reste acceptable dans une très grande majorité des cas où on les utilise.

3.2.4 Application aux programmes de recherche de nombres parfaits

Pour calculer la complexité des trois programmes de recherche de nombres parfaits que nous avons obtenus, nous allons compter le nombre d'instructions élémentaires qu'ils exécutent, en exprimant ce nombre en fonction du ou des paramètres de ces programmes. Dans le cas présent, il y a un seul paramètre, qui correspond à la valeur de `n_max`.

Premier programme

Commençons par le programme de la figure 3.3. Certaines instructions de ce programme ne sont exécutées qu'une seule fois, notamment la déclaration des variables `n_max`, `n`, `m` et `d`, ainsi que l'expression d'affectation `n = 1`. Le nombre total d'instructions exécutées par le programme qui figurent dans cette catégorie est donc borné indépendamment de `n_max`. En utilisant la notation "grand-O", on peut donc décrire ce nombre par

$$O(1).$$

Le programme comprend aussi des opérations qui sont effectuées

— n_max fois : `d = 1`, `m = 0`, `if (m == n)`.

— au plus n_max fois : `printf("%u\n", n)`.

— n_max + 1 fois : `n <= n_max`, `n++`.

Le nombre total de ces instructions s'élève à

$$O(\texttt{n_max}).$$

Il reste à compter les opérations effectuées dans la boucle interne du programme, c'est-à-dire `d < n`, `d++`, `if (!(n % d))` et `m += d`. Pour une valeur donnée de n, ces opérations sont exécutées au plus n fois. Étant donné que n prend successivement pour valeur 1, 2, ..., n_max, on arrive à un total égal à

$$\begin{aligned} O(1 + 2 + \cdots + \texttt{n_max}) &= O\left(\frac{\texttt{n_max}(\texttt{n_max} + 1)}{2}\right) \\ &= O\left(\texttt{n_max}^2\right). \end{aligned}$$

En assemblant les totaux obtenus pour les trois catégories d'instructions que nous avons examinées, on obtient finalement que la complexité du programme vaut

$$O\left(1 + \texttt{n_max} + \texttt{n_max}^2\right) = O\left(\texttt{n_max}^2\right).$$

En d'autres termes, ce programme possède une complexité quadratique. Ce comportement quadratique a pu être observé dans l'étude expérimentale des performances réalisée à la section 3.1.4 : le temps d'exécution du programme augmentait environ d'un facteur 100 quand la valeur de n_max augmentait d'un facteur 10.

Deuxième programme

La complexité en temps du programme de la figure 3.4 se calcule de façon similaire à celle du premier programme. La seule différence importante est que la boucle interne du programme effectue maintenant un nombre d'itérations borné par $\texttt{n}/2$ plutôt que par n. De plus, l'énumération des valeurs de n commence dans ce programme à 2 plutôt qu'à 1. On a donc :

— Opérations effectuées une seule fois :

$$O(1).$$

— Opérations effectuées $\texttt{n_max} - 1$ fois, au plus $\texttt{n_max} - 1$ fois, ou $\texttt{n_max}$ fois :

$$O(\texttt{n_max}).$$

— Opérations de la boucle interne :

$$\begin{aligned} O\left(\frac{2}{2} + \frac{3}{2} + \cdots + \frac{\texttt{n_max}}{2}\right) &= O\left(\frac{1}{4}\texttt{n_max}^2 + \frac{1}{4}\texttt{n_max} - \frac{1}{2}\right) \\ &= O\left(\frac{1}{4}\texttt{n_max}^2 + \frac{1}{4}\texttt{n_max}\right) \\ &= O\left(\texttt{n_max}^2\right). \end{aligned}$$

La complexité de ce deuxième programme vaut donc

$$O\left(1 + \texttt{n_max} + \texttt{n_max}^2\right) = O\left(\texttt{n_max}^2\right),$$

en d'autres termes, elle est identique à celle du premier programme. Cela corrobore les observations expérimentales de la section 3.1.4, qui ont montré que ce programme possédait également un comportement quadratique.

Troisième programme

Pour le calcul de la complexité en temps du programme de la figure 3.5, il faut maintenant tenir compte du fait que le nombre d'itérations de la boucle interne est borné par $\lfloor \sqrt{\texttt{n}} \rfloor - 1$ pour une valeur de $\texttt{n}$ donnée. Cela entraîne que le nombre d'opérations effectuées dans cette boucle vaut

$$O\left(\sqrt{\texttt{n}}\right).$$

On a donc pour le programme dans son ensemble :

— Opérations effectuées une seule fois :

$$O(1).$$

— Opérations effectuées $\texttt{n_max} - 1$ fois, au plus $\texttt{n_max} - 1$ fois, ou $\texttt{n_max}$ fois :

$$O(\texttt{n_max}).$$

— Opérations de la boucle interne [7] :

$$O\left(\sqrt{2} + \sqrt{3} + \cdots + \sqrt{\texttt{n_max}}\right) \subseteq O\left((\texttt{n_max} - 1)\sqrt{\texttt{n_max}}\right) = O\left(\texttt{n_max}\sqrt{\texttt{n_max}}\right).$$

La complexité en temps du troisième programme vaut donc en définitive

$$O\left(1 + \texttt{n_max} + \texttt{n_max}\sqrt{\texttt{n_max}}\right) = O\left(\texttt{n_max}^{1,5}\right).$$

Cette complexité est donc meilleure que celle des deux premiers programmes, car la fonction $\texttt{n_max}^{1,5}$ croît considérablement moins vite que la fonction $\texttt{n_max}^2$. Cette différence suffit à rendre cette version du programme utilisable pour de beaucoup plus grandes valeurs de son paramètre. C'est ce que nous avons observé expérimentalement à la section 3.1.4.

3.3 L'analyse d'un programme

À ce stade du cours, nous avons étudié comment résoudre des problèmes algorithmiques, comment implémenter les algorithmes obtenus sous la forme de programmes pouvant concrètement être exécutés par un ordinateur, et comment mesurer l'efficacité de ces algorithmes et de ces programmes. Nous abordons à présent une autre question, qui est celle de garantir qu'un algorithme ou un programme est correct, c'est-à-dire de prouver que son fonctionnement reste toujours conforme à ses spécifications. Il s'agit d'une question importante, car certains systèmes informatiques sont utilisés pour des applications critiques (par exemple, des véhicules autonomes), où leur défaillance peut avoir des conséquences graves.

7. La majoration $\sqrt{\texttt{n}} \leq \sqrt{\texttt{n_max}}$ utilisée dans ce calcul peut sembler être grossière, et l'on pourrait chercher à la remplacer par une borne plus précise. Étant donné que la fonction $\texttt{n} \mapsto \sqrt{\texttt{n}}$ est strictement croissante, on a

$$\sqrt{\texttt{n}} < \int_{\texttt{n}}^{\texttt{n}+1} \sqrt{x}\,dx$$

qui donne

$$\begin{aligned}\sqrt{2} + \sqrt{3} + \cdots + \sqrt{\texttt{n_max}} &< \int_2^{\texttt{n_max}+1} \sqrt{x}\,dx \\ &= \frac{2}{3}\left((\texttt{n_max} + 1)\sqrt{\texttt{n_max} + 1} - 2\sqrt{2}\right) \\ &< \frac{2}{3}\left((\texttt{n_max} + 1)\left(\sqrt{\texttt{n_max}} + 1\right) - 2\sqrt{2}\right).\end{aligned}$$

Cette dernière expression conduit aussi à une complexité appartenant à $O\left(\texttt{n_max}\sqrt{\texttt{n_max}}\right)$.

Lorsqu'on développe un programme, une façon simple de détecter des erreurs de principe ou d'implémentation consiste à le *tester*, c'est-à-dire à en examiner les exécutions pour un ensemble de scénarios soigneusement choisis pour être les plus représentatifs possibles des différentes situations qui peuvent se produire. Le problème de cette approche est que le test n'est généralement pas exhaustif ; le fait qu'un programme fonctionne correctement pour un ensemble donné de scénarios ne garantit pas que ce sera toujours le cas pour n'importe quelle exécution. Contrairement à cette approche, nous visons dans cette section à obtenir une couverture complète des comportements d'un programme. Notre objectif consiste donc à pouvoir prouver mathématiquement qu'un programme est correct pour toutes ses exécutions possibles.

3.3.1 Les triplets de Hoare

La méthode que nous allons introduire pour établir qu'un programme est correct est basée sur la notion de *triplet de Hoare* [8]. Il s'agit d'un élément d'une logique permettant de raisonner formellement sur le comportement de programmes informatiques. Nous n'allons pas dans ce cours étudier tous les détails de cette logique, mais seulement en présenter les principes élémentaires à un niveau d'abstraction élevé.

Un triplet de Hoare est une formule

$$\{P\}\ S\ \{Q\},$$

où

— S est un fragment de code dont on analyse les exécutions.

— P et Q sont des *assertions*, c'est-à-dire des formules de logique s'évaluant en une valeur vraie ou fausse. Ces formules peuvent faire intervenir la valeur des variables et des constantes utilisées par S, ainsi que celles de ses données d'entrée et de sortie. L'assertion P est appelée la *précondition* et Q la *postcondition* du triplet.

La précondition P représente une condition que l'on suppose être toujours vraie avant d'exécuter S ; en d'autres termes, on ne s'intéresse qu'aux exécutions de S pour lesquelles P est initialement satisfaite. La postcondition Q représente une condition qui doit être vraie à l'issue de l'exécution de S pour que ce fragment de code soit considéré correct. On dit que le triplet $\{P\}\ S\ \{Q\}$ est *valide* si dans tous les cas où P est initialement vraie, alors après avoir exécuté S, Q est également vraie.

Par exemple, si `x` est une variable de type `int`, alors le triplet

$$\{\ 10 \leq \texttt{x} \leq 20\ \}\ \ \texttt{x++;}\ \ \{\ 11 \leq \texttt{x} \leq 21\ \}$$

8. C.A.R. Hoare, 1969. *An axiomatic basis for computer programming*. Communications of the ACM, 12(10):576–580.

est valide. En effet, si `x` possède une valeur entre 10 et 20, alors après avoir exécuté l'instruction `x++`, cette valeur sera nécessairement comprise entre 11 et 21.

Les détails sont très importants quand on analyse un programme. Si à la place du triplet précédent, on avait écrit

$$\{\ \mathtt{x} \geq 10\ \}\ \ \mathtt{x{+}{+};}\ \ \{\ \mathtt{x} \geq 11\ \},$$

alors la situation aurait été différente. Bien sûr, l'évaluation de `x++` a pour effet d'incrémenter `x`, et si une valeur est supérieure ou égale à 10, alors elle devient supérieure ou égale à 11 après avoir été incrémentée. Il ne faut cependant pas oublier que l'arithmétique des valeurs de type `int` ne suit pas exactement celle des nombres entiers : pour l'architecture x86-64, les valeurs représentables sont limitées à l'intervalle $[-2^{31}, 2^{31}-1]$, et incrémenter la borne supérieure $2^{31}-1$ de cet intervalle fournit le résultat -2^{31}. Pour une variable `x` de type `int`, ce triplet est donc invalide.

On est souvent amené quand on écrit un triplet à devoir faire référence dans la postcondition à la valeur des variables du programme avant et après avoir exécuté le fragment de code analysé. On emploie alors une notation différente pour les valeurs initiales et finales. Par exemple, on peut considérer que le nom d'une variable désigne sa valeur initiale, et que ce nom muni d'un symbole prime représente sa valeur finale. Par exemple, le triplet

$$\{\ \mathbf{V}\ \}\ \ \mathtt{x{+}{+};}\ \ \left\{ \begin{array}{ll} \mathtt{x}' = \mathtt{x} + 1 & \text{si } \mathtt{x} < 2^{31} - 1 \\ \mathtt{x}' = -2^{31} & \text{si } \mathtt{x} = 2^{31} - 1 \end{array} \right\}$$

décrit précisément l'effet de l'instruction `x++` lorsque `x` est de type `int`. Notons que ce triplet, possède une précondition $\{\ \mathbf{V}\ \}$ qui est toujours vraie. Avec cette convention, le premier triplet que nous avons étudié dans cette section se réécrit

$$\{\ 10 \leq \mathtt{x} \leq 20\ \}\ \ \mathtt{x{+}{+};}\ \ \{\ 11 \leq \mathtt{x}' \leq 21\ \}.$$

D'autres exemples de triplets valides sont donnés ici :

— $\left\{\ \mathtt{x} \in [-2^{31}, 2^{31}-1]\ \right\}$ `if (x < 0) x = 0;` $\left\{\ \mathtt{x}' \in [0, 2^{31}-1]\ \right\}$.

En effet, en supposant à nouveau que `x` est de type `int`, ce fragment de programme laisse la valeur de cette variable inchangée si elle est positive ou nulle, et la remplace par zéro sinon. Un triplet valide décrivant plus précisément l'effet de ce code est le suivant.

$$\left\{\ \mathtt{x} \in [-2^{31}, 2^{31}-1]\ \right\}\ \ \mathtt{if\ (x < 0)\ x = 0;}\ \ \left\{ \begin{array}{ll} \mathtt{x}' = \mathtt{x} & \text{si } \mathtt{x} \geq 0 \\ \mathtt{x}' = 0 & \text{si } \mathtt{x} < 0 \end{array} \right\}$$

— { (n'importe quelle assertion) } (n'importe quel code) { **V** }.

Dans ce triplet, quel que soit le comportement du code analysé, la postcondition sera trivialement satisfaite après son exécution, puisque cette postcondition est identiquement vraie.

— { (n'importe quelle assertion) } `for (;;);` { (n'importe quelle assertion) }.

Ce triplet contient une instruction qui implémente une boucle infinie, en d'autres termes, dont l'exécution ne se termine pas. Pour que ce triplet soit valide, il faut qu'après chaque exécution satisfaisant initialement la précondition, la postcondition soit vraie. Étant donné qu'aucune exécution ne se termine jamais, il n'existe aucune situation où la postcondition sera évaluée. Ce triplet est donc valide.

— { **F** } (n'importe quel code) { (n'importe quelle assertion) }.

Pour cet exemple, la précondition ne peut jamais être satisfaite. Il n'existe donc pas d'exécution pour laquelle la précondition est initialement vraie. Ce triplet est donc valide.

Pour terminer cette présentation des triplets de Hoare, nous montrons ici comment les utiliser pour prouver qu'une séquence d'instructions est correcte. Notre objectif consiste à démontrer que la séquence

```
c = a;
a = b;
b = c;
```

qui figure dans le programme de la figure 1.2 permute correctement la valeur des deux variables `a` et `b`.

Le raisonnement que nous allons tenir demande de pouvoir faire référence aux valeurs de `a`, `b` et `c` à chaque moment de l'exécution de la séquence, c'est-à-dire initialement, après la première instruction, après la deuxième, et à la fin de la séquence. Nous utiliserons respectivement les indices 0, 1, 2 et 3 pour ces trois situations. Par exemple, a_0 dénote la valeur initiale de `a`, b_3 la valeur finale de `b`, et c_2 la valeur de `c` après la deuxième instruction.

Avec cette notation, nous pouvons écrire les triplets valides suivants qui caractérisent l'effet de chaque instruction [9] :

$$\{ \mathbf{V} \}\ \texttt{c = a;}\ \{ a_1 = a_0,\ b_1 = b_0,\ c_1 = a_0 \}$$
$$\{ \mathbf{V} \}\ \texttt{a = b;}\ \{ a_2 = b_1,\ b_2 = b_1,\ c_2 = c_1 \}$$
$$\{ \mathbf{V} \}\ \texttt{b = c;}\ \{ a_3 = a_2,\ b_3 = c_2,\ c_3 = c_2 \}$$

9. En logique de Hoare, on dispose d'un tel triplet pour l'ensemble des instructions du langage de programmation utilisé.

(En effet, chaque instruction d'affectation modifie la valeur d'une variable et laisse les autres inchangées.)

En composant ces trois triplets, on obtient le triplet valide

$$
\begin{array}{c}
\{\ \mathbf{V}\ \} \\
\mathtt{c = a;} \\
\mathtt{a = b;} \\
\mathtt{b = c;} \\
\{\ \exists \mathtt{a}_1, \mathtt{b}_1, \mathtt{c}_1, \mathtt{a}_2, \mathtt{b}_2, \mathtt{c}_2 : \mathtt{a}_1 = \mathtt{a}_0, \mathtt{b}_1 = \mathtt{b}_0, \mathtt{c}_1 = \mathtt{a}_0, \\
\mathtt{a}_2 = \mathtt{b}_1, \mathtt{b}_2 = \mathtt{b}_1, \mathtt{c}_2 = \mathtt{c}_1, \\
\mathtt{a}_3 = \mathtt{a}_2, \mathtt{b}_3 = \mathtt{c}_2, \mathtt{c}_3 = \mathtt{c}_2\ \}.
\end{array}
$$

L'élimination des variables intermédiaires $\mathtt{a}_1$, $\mathtt{b}_1$, $\mathtt{c}_1$, $\mathtt{a}_2$, $\mathtt{b}_2$ et $\mathtt{c}_2$ fournit enfin le triplet valide

$$
\begin{array}{lll}
 & \mathtt{c = a;} & \\
\{\ \mathbf{V}\ \} & \mathtt{a = b;} & \{\ \mathtt{a}_3 = \mathtt{b}_0,\ \mathtt{b}_3 = \mathtt{a}_0,\ \mathtt{c}_3 = \mathtt{a}_0\ \}, \\
 & \mathtt{b = c;} &
\end{array}
$$

qui exprime que la séquence a bien pour effet de permuter correctement a et b (et aussi, accessoirement, de recopier la valeur initiale de a dans c).

Pour prouver qu'un programme S est correct, on démontre donc que le triplet

$$\{P\}\ S\ \{Q\}$$

est valide, où P représente les conditions initiales du programme (que l'on présuppose), et Q implique la propriété que l'on souhaite établir. Notons que cette démarche ne permet de prouver que la *correction partielle* de S, en d'autres termes, le fait que ce programme est correct chaque fois que son exécution se termine. En effet, nous avons vu que si S ne se termine pas, alors n'importe quel triplet $\{P\}\ S\ \{Q\}$ est valide. On prouve la *correction totale* d'un programme en démontrant à la fois sa correction partielle et la propriété exprimant que les exécutions de ce programme se terminent. Nous aborderons ce dernier point à la section 3.3.4.

3.3.2 Les invariants de boucle

Nous avons vu à la section précédente que pour prouver la validité d'un triplet

$$\{P\}\ S\ \{Q\}$$

lorsque S est une séquence d'instructions, il suffit de composer les triplets correspondant à chacune de ces instructions. Le problème devient plus difficile lorsque S est une instruction de boucle. Dans ce cas, on utilise un *invariant de boucle*, qui est une assertion I satisfaisant les trois propriétés suivantes :

— I est une conséquence de la précondition P, en d'autres termes, I est vrai chaque fois que P l'est.

— Si I est vrai avant une itération de la boucle, alors I est également vrai après cette itération.

— En sortie de boucle, I implique la postcondition Q. En d'autres termes, si I est vrai en sortie de boucle, alors Q l'est aussi.

Formellement, si S est une boucle de la forme

`while (`*condition*`)`
iteration ;

alors ces trois propriétés peuvent respectivement s'écrire

— *Propriété* 1 *:* $P \Rightarrow I$,

— *Propriété* 2 *:* $\{ I \wedge condition \}$ *iteration ;* $\{ I \}$,

— *Propriété* 3 *:* $(I \wedge \neg condition) \Rightarrow Q$,

où "$\Rightarrow$", "$\wedge$" et "$\neg$" dénotent respectivement l'implication, la conjonction et la négation logiques [10].

Si l'on réussit à trouver une assertion I pour laquelle on peut démontrer qu'elle satisfait ces trois propriétés, alors on dispose d'une preuve que le triplet

$$\{P\}\ S\ \{Q\}$$

est valide, car pour toute exécution de la boucle S pour laquelle la précondition P est initialement satisfaite :

— L'invariant I est vrai avant d'effectuer une première itération, grâce à la première propriété.

— Si la boucle effectue une première itération, alors puisque I est vrai avant cette itération, il l'est également après celle-ci, par la deuxième propriété.

— Le même raisonnement permet d'établir pour toutes les itérations successives de la boucle que I est vrai avant et après celles-ci. En particulier, I est vrai après la dernière itération (éventuelle) de la boucle,

10. Nous utiliserons aussi le symbole de disjonction logique "$\vee$" par la suite.

— Après la dernière itération (éventuelle) de la boucle, le gardien *condition* de cette dernière devient faux. Étant donné que l'invariant I est vrai, la troisième propriété entraîne que la postcondition Q est satisfaite.

En pratique, la principale difficulté que l'on rencontre pour mettre en œuvre cette méthode des invariants est celle de trouver un invariant de boucle convenable, c'est-à-dire satisfaisant les trois propriétés. Il n'existe aucune méthode générale [11] permettant de synthétiser automatiquement un tel invariant. La meilleure approche consiste à écrire une assertion qui caractérise le plus précisément possible le travail effectué par la boucle jusqu'à l'itération courante. Intuitivement, l'invariant peut alors être vu comme une sorte de documentation formelle du principe de fonctionnement de cette boucle.

3.3.3 Illustration

Pour illustrer la méthode des invariants, nous considérons la deuxième variante de notre programme de recherche de nombres parfaits, dont le code source est donné à la figure 3.4. Ce programme comprend deux boucles imbriquées : une boucle dont le but est d'énumérer toutes les valeurs de `n` entre 2 et `n_max`, et une autre sur `d` chargée de déterminer si une valeur particulière de `n` correspond ou non à un nombre parfait.

Commençons par examiner cette deuxième boucle, qui constitue la partie la plus difficile du programme. Établir la correction partielle de cette boucle revient à démontrer le triplet suivant.

$$
\begin{array}{l}
\{\, \texttt{n} \geq 2 \,\} \\
\texttt{for (d = 2, m = 1; d <= n / 2 \&\& m <= n; d++)} \\
\quad \texttt{if (!(n \% d))} \\
\qquad \texttt{m += d;} \\
\{\, \texttt{m} = \texttt{n} \Leftrightarrow \texttt{n} \text{ est un nombre parfait} \,\}
\end{array}
\tag{3.1}
$$

Une première étape consiste à sortir l'expression d'initialisation `d = 2, m = 1` de l'instruction `for`. Le triplet à valider se réécrit

$$
\begin{array}{l}
\{\, \texttt{n} \geq 2 \,\} \\
\texttt{d = 2, m = 1;} \\
\texttt{for (; d <= n / 2 \&\& m <= n; d++)} \\
\quad \texttt{if (!(n \% d))} \\
\qquad \texttt{m += d;} \\
\{\, \texttt{m} = \texttt{n} \Leftrightarrow \texttt{n} \text{ est un nombre parfait} \,\}.
\end{array}
$$

11. Il s'agit d'un problème indécidable, tout comme ceux mentionnés à la section 1.4.2.

L'effet de l'expression d'initialisation est capturé par le triplet valide

$$\{\, \mathtt{n} \geq 2 \,\}$$

```
d = 2, m = 1;
```

$$\{\, \mathtt{n} \geq 2,\ \mathtt{d} = 2,\ \mathtt{m} = 1 \,\}.$$

Il reste donc a établir la validité du triplet

$$\{\, \mathtt{n} \geq 2,\ \mathtt{d} = 2,\ \mathtt{m} = 1 \,\}$$

```
for (; d <= n / 2 && m <= n; d++)
    if (!(n % d))
        m += d;
```

$$\{\, \mathtt{m} = \mathtt{n} \ \Leftrightarrow\ \mathtt{n} \text{ est un nombre parfait} \,\}.$$

Après avoir transformé la boucle `for` en une boucle `while` équivalente, ce triplet devient

$$\{\, \mathtt{n} \geq 2,\ \mathtt{d} = 2,\ \mathtt{m} = 1 \,\}$$

```
while (d <= n / 2 && m <= n)
    {
        if (!(n % d))
            m += d;
        d++;
    }
```

$$\{\, \mathtt{m} = \mathtt{n} \ \Leftrightarrow\ \mathtt{n} \text{ est un nombre parfait} \,\}. \tag{3.2}$$

Cherchons un invariant de boucle permettant de démontrer la validité de ce triplet. Pour rappel, il doit s'agir d'une assertion I satisfaisant les trois propriétés suivantes :

— *Propriété* 1 *:* $(\mathtt{n} \geq 2 \ \wedge\ \mathtt{d} = 2 \ \wedge\ \mathtt{m} = 1) \ \Rightarrow\ I$.

— *Propriété* 2 *:*

$$\left\{\, I \ \wedge\ \mathtt{d} \leq \left\lfloor \frac{\mathtt{n}}{2} \right\rfloor \ \wedge\ \mathtt{m} \leq \mathtt{n} \,\right\}$$

```
if (!(n % d))
    m += d;
d++;
```

$$\{\, I \,\}.$$

— *Propriété* 3 *:* $\left(I \wedge \left(\mathtt{d} > \left\lfloor \frac{\mathtt{n}}{2} \right\rfloor \vee \mathtt{m} > \mathtt{n}\right)\right) \Rightarrow (\mathtt{m} = \mathtt{n} \Leftrightarrow \mathtt{n}$ est un nombre parfait).

Notre approche consiste à d'abord proposer un candidat invariant I, et à ensuite démontrer qu'il satisfait ces trois propriétés. Pour trouver I, on essaie de caractériser le plus précisément possible l'effet de la boucle sur la valeur des variables, jusqu'à une itération donnée :

— La valeur de $\mathtt{n}$ n'est pas modifiée, et continuera toujours à satisfaire la condition

$$\mathtt{n} \geq 2$$

extraite de la précondition.

— Avant n'importe quelle itération de la boucle, la valeur de $\mathtt{d}$ satisfait $\mathtt{d} \leq \lfloor \mathtt{n}/2 \rfloor$, et cette valeur est incrémentée par la boucle. La condition satisfaite avant et après chaque itération est donc

$$\mathtt{d} \leq \left\lfloor \frac{\mathtt{n}}{2} \right\rfloor + 1.$$

— La variable $\mathtt{m}$ sert à retenir la somme de tous les diviseurs de $\mathtt{n}$ jusqu'à un certain seuil. L'effet de chaque itération est de déterminer s'il faut ou non ajouter la valeur courante de $\mathtt{d}$ à cette somme. Avant et après chaque itération, la condition satisfaite par la valeur de $\mathtt{m}$ est donc

$$\mathtt{m} = \text{somme des diviseurs de } \mathtt{n} \text{ inférieurs à } \mathtt{d}.$$

On obtient donc en résumé le candidat invariant I suivant.

$$I : \mathtt{n} \geq 2 \ \wedge \ \mathtt{d} \leq \left\lfloor \frac{\mathtt{n}}{2} \right\rfloor + 1 \ \wedge \ \mathtt{m} = \text{somme des diviseurs de } \mathtt{n} \text{ inférieurs à } \mathtt{d}. \tag{3.3}$$

Démontrons à présent que cet invariant est valide, c'est-à-dire qu'il satisfait les trois propriétés d'un invariant de boucle.

— *Propriété* 1 *:* Il faut prouver

$$(\mathtt{n} \geq 2 \ \wedge \ \mathtt{d} = 2 \ \wedge \ \mathtt{m} = 1) \ \Rightarrow \ I.$$

Clairement, si $\mathtt{n} \geq 2$ et $\mathtt{d} = 2$, alors on a $\mathtt{n} \geq 2$ et $\mathtt{d} \leq \lfloor \mathtt{n}/2 \rfloor + 1$. De plus, le seul diviseur de $\mathtt{n}$ strictement inférieur à 2 est 1, et l'on a bien $\mathtt{m} = 1$.

— *Propriété* 2 *:* Il faut maintenant démontrer que le triplet

$$\left\{ I \ \wedge \ \mathtt{d} \leq \left\lfloor \frac{\mathtt{n}}{2} \right\rfloor \ \wedge \ \mathtt{m} \leq \mathtt{n} \right\}$$

```
if (!(n % d))
    m += d;
d++;
```

$\{ I \}$

est valide. Remarquons premièrement que la valeur de `n` n'est pas modifiée par ce fragment de code. En suivant la convention que `x` et `x'` dénotent respectivement la valeur d'une variable `x` avant et après l'exécution du code, on a donc ici $\mathtt{n}' = \mathtt{n}$ et $\mathtt{n} \geq 2$, ce qui fournit $\mathtt{n}' \geq 2$.

La valeur de `d` est quant à elle incrémentée par l'instruction `d++`, en d'autres termes on a $\mathtt{d}' = \mathtt{d} + 1$. Étant donné que la précondition impose $\mathtt{d} \leq \lfloor \mathtt{n}/2 \rfloor$, on obtient [12] $\mathtt{d}' \leq \lfloor \mathtt{n}'/2 \rfloor + 1$.

Pour raisonner maintenant sur la valeur de `m`, il faut tenir compte de deux cas possibles, correspondant au fait que l'expression évaluée par l'instruction `if` est vraie ou fausse :

— *Si `d` divise `n` :* Il faut dans ce cas démontrer la validité du triplet

$$\left\{ I \wedge \mathtt{d} \leq \left\lfloor \frac{\mathtt{n}}{2} \right\rfloor \wedge \mathtt{m} \leq \mathtt{n} \ \wedge \mathtt{d} \text{ divise } \mathtt{n} \right\}$$

```
m += d;
d++;
```

$\{ I \}$.

On a alors

$$\mathtt{m} = \text{(somme des diviseurs de } \mathtt{n} \text{ inférieurs à } \mathtt{d})$$

$$\mathtt{n}' = \mathtt{n}$$

$$\mathtt{d}' = \mathtt{d} + 1$$

$$\mathtt{m}' = \mathtt{m} + \mathtt{d}$$

$$\text{(somme des diviseurs de } \mathtt{n} \text{ inférieurs à d')} = \text{(somme des diviseurs de } \mathtt{n} \text{ inférieurs à } \mathtt{d}) + \mathtt{d}$$

qui entraîne

$$\mathtt{m}' = \text{(somme des diviseurs de } \mathtt{n}' \text{ inférieurs à d')}.$$

12. À ce stade, il est intéressant de réaliser que si l'on s'était trompé en écrivant dans l'invariant, par exemple, $\mathtt{d} \leq \lfloor \mathtt{n}/2 \rfloor$ au lieu de $\mathtt{d} \leq \lfloor \mathtt{n}/2 \rfloor + 1$, c'est à cette étape de la preuve que l'on s'en serait rendu compte.

— *Si* $\mathtt{d}$ *ne divise pas* $\mathtt{n}$ *:* Le triplet à démontrer est dans ce cas

$$\left\{ I \wedge \mathtt{d} \leq \left\lfloor \frac{\mathtt{n}}{2} \right\rfloor \wedge \mathtt{m} \leq \mathtt{n} \ \wedge \ \mathtt{d} \text{ ne divise pas } \mathtt{n} \right\}$$

$$\mathtt{d{+}{+};}$$

$$\{ I \}.$$

On a alors

$$\mathtt{m} = \text{(somme des diviseurs de } \mathtt{n} \text{ inférieurs à } \mathtt{d})$$

$$\mathtt{n}' = \mathtt{n}$$

$$\mathtt{d}' = \mathtt{d} + 1$$

$$\mathtt{m}' = \mathtt{m}$$

$$\text{(somme des diviseurs de } \mathtt{n} \text{ inférieurs à } \mathtt{d'}) = \text{(somme des diviseurs de } \mathtt{n} \text{ inférieurs à } \mathtt{d})$$

qui fournit

$$\mathtt{m}' = \text{(somme des diviseurs de } \mathtt{n}' \text{ inférieurs à } \mathtt{d'}).$$

En résumé, nous avons à l'issue de l'exécution du fragment de code la propriété

$$\mathtt{n}' \geq 2 \ \wedge \ \mathtt{d}' \leq \left\lfloor \frac{\mathtt{n}'}{2} \right\rfloor + 1 \ \wedge \ \mathtt{m}' = \text{somme des diviseurs de } \mathtt{n'} \text{ inférieurs à } \mathtt{d'},$$

qui correspond bien à notre candidat invariant I évalué sur la valeur des variables après l'exécution.

— *Propriété* 3 *:* Il reste à prouver que l'on a

$$\left(I \wedge \left(\mathtt{d} > \left\lfloor \frac{\mathtt{n}}{2} \right\rfloor \vee \mathtt{m} > \mathtt{n} \right) \right) \Rightarrow (\mathtt{m} = \mathtt{n} \ \Leftrightarrow \ \mathtt{n} \text{ est un nombre parfait}).$$

Nous pouvons traiter séparément deux cas :

— *Si* $I \wedge \mathtt{d} > \left\lfloor \frac{\mathtt{n}}{2} \right\rfloor$ *:* Dans ce cas, on a

$$\mathtt{d} \leq \left\lfloor \frac{\mathtt{n}}{2} \right\rfloor + 1$$

et

$$\mathtt{d} > \left\lfloor \frac{\mathtt{n}}{2} \right\rfloor,$$

qui entraînent

$$\mathtt{d} = \left\lfloor \frac{\mathtt{n}}{2} \right\rfloor + 1.$$

L'assertion I implique que la valeur de `m` est égale à la somme des diviseurs de `n` inférieurs à `d`, c'est-à-dire, d'après le résultat précédent, à la somme des diviseurs de `n` inférieurs ou égaux à $\lfloor \mathtt{n}/2 \rfloor$. On sait que `n` n'admet aucun diviseur supérieur à $\lfloor \mathtt{n}/2 \rfloor$ et inférieur à `n`, donc la valeur de `m` est égale à la somme des diviseurs de `n` inférieurs à `n`. On en déduit que l'on a `m = n` ssi `n` est un nombre parfait.

— *Si* $I \wedge \mathtt{m} > \mathtt{n}$ *:* Dans ce cas, l'assertion I implique également que la valeur de `m` est égale à la somme des diviseurs de `n` inférieurs à `d`. On a aussi

$$\mathtt{d} \leq \left\lfloor \frac{\mathtt{n}}{2} \right\rfloor + 1$$

et

$$\mathtt{n} \geq 2,$$

qui entraînent

$$\mathtt{d} \leq \mathtt{n}.$$

On en déduit que la valeur de `m` correspond à la somme d'un certain nombre de diviseurs de `n` inférieurs à `n`. Étant donné que l'on a `m > n`, et que les diviseurs de `n` considérés sont positifs, on a que la somme de tous les diviseurs de `n` inférieurs à `n` est nécessairement supérieure à `n`, et dès lors `n` ne peut pas être un nombre parfait. La condition `m = n` ssi `n` est un nombre parfait est donc vérifiée, puisque les deux propositions qui la composent sont fausses.

Nous avons donc, en résumé, prouvé que l'invariant (3.3) est valide, ce qui démontre la validité des triplets (3.2) et (3.1), et donc la correction partielle du fragment de code

```
for (d = 2, m = 1; d <= n / 2 && m <= n; d++)
  if (!(n % d))
    m += d;

if (m == n)
  printf("%u\n", n);
```

qui forme le corps de la boucle principale du programme de la figure 3.4.

Il reste maintenant à analyser cette boucle principale, c'est-à-dire à démontrer que le code

```
for (n = 2; n <= n_max; n++)
  {
    /*  Traitement de n  */
  }
```

traite toutes les valeurs de n situées dans l'intervalle [2, n_max]. On voit immédiatement que cette propriété, qui est beaucoup plus simple que celle que nous avons démontrée pour la boucle interne du programme, est correcte. Afin d'être complet, nous allons cependant montrer comment prouver cette propriété à l'aide de la méthode des invariants. Nous supposerons que la valeur de la constante n_max est au moins égale à 2, et que l'opération de traitement d'une valeur ne modifie ni n ni n_max.

Pour écrire une précondition, une postcondition et un invariant pour cette boucle, nous avons besoin de pouvoir faire référence aux valeurs de n qui font l'objet d'un traitement par l'opération effectuée dans la boucle. Une solution simple à ce problème consiste à considérer une variable fictive T dont le contenu représente à chaque instant l'ensemble des valeurs de n déjà traitées. Le triplet dont on doit démontrer la validité s'écrit alors de la façon suivante.

$$\{\ \texttt{n_max} \geq 2 \ \wedge\ T = \{\ \}\ \}$$

```
for (n = 2; n <= n_max; n++)
  {
    /*  Traitement de n  */
  }
```

$$\{\ T = [2, \texttt{n_max}]\ \}$$

En effet, initialement aucune valeur de n n'a été traitée, et à l'issue de l'exécution de ce fragment de code, on souhaite que toutes les valeurs de n situées dans l'intervalle [2, n_max] l'aient été.

On commence par extraire l'expression d'initialisation `n = 2` et à transformer la boucle for en une boucle while, ce qui réduit ce triplet à

$$\{\ \texttt{n_max} \geq 2 \ \wedge\ \texttt{n} = 2 \ \wedge\ T = \{\ \}\ \}$$

```
while (n <= n_max)
  {
    /*  Traitement de n  */
    n++;
  }
```

$$\{\ T = [2, \texttt{n_max}]\ \}.$$

On peut écrire un invariant de boucle I qui capture le fait qu'avant et après chaque itération, les valeurs déjà traitées sont celles qui sont inférieures à la valeur courante de n. On obtient

$$I : \texttt{n} \leq \texttt{n_max} + 1 \ \wedge\ T = [2, \texttt{n} - 1].$$

Montrons que cet invariant est correct :

— *Propriété* 1 *:* Il faut prouver

$$(\texttt{n_max} \geq 2 \ \wedge \ \texttt{n} = 2 \ \wedge \ T = \{\,\}) \ \Rightarrow \ I,$$

ce qui est immédiat car l'intervalle [2, 1] est vide.

— *Propriété* 2 *:* Le triplet à démontrer est

$$\{\, I \ \wedge \ \texttt{n} \leq \texttt{n_max} \,\}$$

```
/*  Traitement de n  */
n++;
```

$$\{\, I \,\}.$$

On a

$$\begin{aligned} \texttt{n_max}' &= \texttt{n_max} \\ \texttt{n} &\leq \texttt{n_max} \\ T &= [2, \texttt{n} - 1] \\ \texttt{n}' &= \texttt{n} + 1 \\ T' &= T \ \cup \ \{\texttt{n}\}, \end{aligned}$$

dont on déduit

$$\begin{aligned} \texttt{n}' &\leq \texttt{n_max}' + 1 \\ T' &= [2, \texttt{n}' - 1]. \end{aligned}$$

Ces deux contraintes correspondent bien à notre candidat invariant I évalué sur les valeurs de n et de T considérées après l'exécution du fragment de code.

— *Propriété* 3 *:* Il reste à démontrer

$$(I \ \wedge \ \texttt{n} > \texttt{n_max}) \ \Rightarrow T = [2, \texttt{n_max}].$$

On a

$$\texttt{n} \leq \texttt{n_max} + 1$$

et

$$\texttt{n} > \texttt{n_max},$$

qui donnent

$$\texttt{n} = \texttt{n_max} + 1.$$

En combinant cette propriété avec

$$T = [2, \mathtt{n} - 1]$$

qui fait partie de l'invariant, on obtient bien

$$T = [2, \mathtt{n_max}].$$

En pratique, la méthode des invariants est employée dans le cadre du développement d'applications critiques, pour lesquelles il est indispensable de s'assurer que les logiciels produits sont exempts d'erreurs. Le processus de preuve des invariants peut être grandement facilité par l'utilisation d'outils de démonstration automatique ou assistée tels que *Isabelle/HOL* [13]. Il existe un compilateur C qui a été entièrement vérifié de façon formelle par ce type d'approche [14].

3.3.4 La terminaison d'un programme

Démontrer qu'un triplet

$$\{P\}\ S\ \{Q\}$$

est valide garantit que la propriété exprimée par la postcondition Q sera satisfaite après toute exécution de S pour laquelle la précondition P est initialement vraie. Comme nous l'avons déjà mentionné, cela ne permet d'établir que la *correction partielle* de S, c'est-à-dire le fait que ce programme est correct à condition qu'il se termine. Si l'exécution de S ne se termine pas, alors le triplet est valide quelles que soient les assertions P et Q.

Pour démontrer la correction totale d'un programme, il faut donc aussi prouver que celui-ci se termine. Nous avons cependant déjà mentionné que ce *problème de l'arrêt* est en toute généralité impossible à résoudre. Nous allons dès lors nous limiter à présenter une condition suffisante pour prouver qu'un programme se termine, en gardant à l'esprit qu'il existera nécessairement des cas que cette méthode sera incapable de traiter.

On peut prouver qu'un programme se termine en montrant que ses exécutions finissent toujours par sortir de toutes les boucles qui le composent. Un procédé simple pour démontrer qu'une boucle se termine toujours consiste à munir celle-ci d'un *variant de boucle*, aussi appelé *fonction de terminaison*. Il s'agit d'une expression qui peut faire intervenir la valeur courante des variables et des constantes du programme, et dont l'évaluation retourne un entier positif ou nul. Pour être valide, un variant de boucle doit satisfaire la propriété suivante : chaque itération complète de la

13. T. Nipkow, M. Wenzel, L. C. Paulson, 2002. *Isabelle/HOL : a proof assistant for higher-order logic*, Springer-Verlag. Voir également `https://isabelle.in.tum.de`.

14. Xavier Leroy, 2009. *Formal verification of a realistic compiler*. Communications of the ACM, 52(7):107–115. Voir également `https://compcert.org`.

boucle diminue toujours sa valeur. Étant donné qu'il n'existe pas de séquence infinie strictement décroissante d'entiers positifs ou nuls, cette propriété entraîne que la boucle se termine.

Si l'on parvient à trouver un variant valide pour une boucle donnée, alors cela suffit à démontrer que les exécutions de cette boucle se terminent toujours. La réciproque n'est pas vraie : il existe des boucles qui se terminent toujours, mais qui n'admettent pas de variant sous la forme où nous l'avons défini.

Nous illustrons maintenant la notion de variant de boucle en démontrant que les exécutions de la boucle interne

```
for (d = 2, m = 1; d <= n / 2 && m <= n; d++)
  if (!(n % d))
    m += d;
```

du programme de la figure 3.4 se terminent toujours. Pour cela, on peut par exemple utiliser le variant

$$v = \left\lfloor \frac{\mathtt{n}}{2} \right\rfloor + 1 - \mathtt{d}.$$

En effet, nous avons montré à la section 3.3.3 que cette boucle admet un invariant qui implique au début et à la fin de chaque itération

$$\mathtt{d} \leq \left\lfloor \frac{\mathtt{n}}{2} \right\rfloor + 1.$$

On en déduit que la valeur de v est un entier positif ou nul. De plus, chaque itération de la boucle incrémente `d` et laisse la valeur de `n` inchangée, ce qui entraîne que v diminue à chaque itération. Cela prouve que v est un variant de boucle valide, ce qui suffit à établir que la boucle se termine toujours.

La même approche permet de prouver la terminaison de la boucle externe

```
for (n = 2; n <= n_max; n++)
  {
    /*  Traitement de n  */
  }
```

du programme. Dans ce cas, on peut utiliser par exemple le variant de boucle

$$v = \mathtt{n_max} + 1 - \mathtt{n}.$$

3.4 Exercices

1. Déterminer si les triplets suivants sont valides ou non, en supposant que la variable `x` a préalablement été déclarée de type `int`.

   (a) $\{\mathtt{x} > 0\}$ `x--;` $\{\mathtt{x} \geq 0\}$.
   (b) $\{\mathtt{x} > 0\}$ `x++;` $\{\mathtt{x} > 0\}$.
   (c) $\{\mathbf{T}\}$ `x++;` $\{\mathtt{x} > 0\}$.
   (d) $\{\mathbf{F}\}$ `x++;` $\{\mathtt{x} > 0\}$.
   (e) $\{\mathtt{x} > 0\}$ `for (; !(x % 2); x /= 2);` {x est impair}.

2. Le fragment de code suivant calcule la factorielle `fact` d'un nombre `n`.

```
int i, fact;
for (i = 2, fact = 1; i <= n; i++)
  fact *= i;
```

   En supposant que la valeur initiale de `n` est strictement positive, et est telle qu'aucun dépassement arithmétique ne se produit lors de l'exécution de ce fragment de code :

   (a) Démontrer que
   $$2 \leq \mathtt{i} \leq \mathtt{n} + 1 \ \wedge \ \mathtt{fact} = \prod_{2 \leq \mathtt{j} < \mathtt{i}} \mathtt{j}$$
   est un invariant de la boucle contenue dans ce code.
   (b) À l'aide d'un variant, prouver que l'exécution de cette boucle se termine toujours.

3. Le fragment de code suivant calcule la somme des chiffres `s` d'un nombre non signé `n`.

```
unsigned m, s;
for (m = n, s = 0; m; m /= 10)
  s += m % 10;
```

   (a) À l'aide d'un invariant de boucle, démontrer que ce fragment de code fournit un résultat correct.
   (b) À l'aide d'un variant, prouver que l'exécution de ce code se termine toujours.

4. (a) Démontrer que les programmes obtenus comme solutions des problèmes 2, 3, 5 et 6 de la section 1.5 et 5 et 7 de la section 2.6 sont corrects.
   (b) Déterminer la complexité en temps de ces programmes.

Chapitre 4

Les fonctions et les procédures

Dans ce chapitre, nous introduisons les notions de *fonction* et de *procédure*, qui sont des mécanismes permettant de commander l'exécution d'un même fragment de code à partir de différents endroits d'un programme.

4.1 Introduction

Il est très fréquent lorsqu'on développe un programme de devoir effectuer une opération identique à plusieurs endroits de celui-ci. On pourrait pour cela employer la fonctionnalité de *copier-coller* de l'éditeur, afin de recopier à plusieurs endroits du code source le fragment de programme réalisant l'opération. Il ne s'agit cependant pas d'une bonne approche, car elle mène à un programme redondant. La redondance est problématique en programmation, car il est difficile au cours de l'évolution d'un programme de garantir que les modifications qui lui seront apportées maintiendront toujours parfaitement la cohérence entre les différentes parties censées rester identiques. Il faut notamment s'assurer que la correction d'une erreur détectée dans une de ces parties sera toujours très exactement effectuée de la même façon dans les autres.

On évite d'obtenir un code source redondant en veillant à ce que chaque opération réalisée par le programme ne soit programmée qu'une seule fois. On utilise alors un mécanisme permettant de déclencher l'exécution de cette opération à partir de n'importe quel endroit du code source.

En programmation, on appelle *fonction* un morceau de code dont l'exécution peut être *invoquée*, ou *appelée*, à partir d'autres endroits d'un programme.

Nous avons déjà rencontré des fonctions dans les programmes étudiés jusqu'ici. Par exemple, nous avons vu que des instructions telles que `printf(...)` et `scanf(...)` ne correspondent pas à des mots-clés du langage C, mais à des opérations dont l'implémentation est disponible

dans une bibliothèque standard. Ces instructions sont en fait des expressions dont l'évaluation invoque des fonctions nommées `printf` et `scanf`. Un autre exemple est donné par la fonction `sqrt` évoquée à l'exercice 4 de la section 2.6. Une expression telle que

```
x = sqrt(2.0)
```

invoque la fonction `sqrt` de la bibliothèque standard afin de calculer une approximation de $\sqrt{2}$, et écrit ensuite le résultat obtenu dans la variable `x`, supposée de type `double`.

Lorsqu'une instruction invoque une fonction, son exécution est suspendue pendant tout le temps que dure l'exécution de cette fonction. Dans l'exemple précédent, cela signifie que l'évaluation de la sous-expression `sqrt(2.0)` conduit à exécuter la fonction `sqrt`, et que cette évaluation est suspendue pendant que cette fonction s'exécute. Elle reprend dès que la fonction a terminé son exécution, en produisant un résultat qui est ensuite affecté à `x`.

Comme on le voit dans cet exemple, on peut fournir certaines données d'entrée à une fonction que l'on invoque. Formellement, une fonction accepte un certain nombre de *paramètres*, et les valeurs attribuées à ces paramètres quand la fonction est invoquée sont appelés les *arguments*. Dans notre exemple, la fonction `sqrt` accepte un paramètre de type `double`, et l'expression `sqrt(2.0)` effectue un appel à cette fonction avec l'argument 2.0.

À la fin de son exécution, une fonction peut renvoyer une *valeur de retour* au code qui l'a invoquée. Il s'agit de la valeur que prendra l'expression d'invocation de la fonction quand elle aura terminé son évaluation. Dans notre exemple, l'évaluation de `sqrt(2.0)` fournira comme valeur l'approximation de $\sqrt{2}$ calculée par la fonction `sqrt`. Il arrive parfois que l'on programme des fonctions qui ne retournent pas de valeur ; celles-ci sont alors appelées des *procédures*.

4.2 La programmation des fonctions

Pour programmer des fonctions, nous avons besoin de deux mécanismes : le premier pour définir des fragments de programme constituant des fonctions, et le second pour invoquer ces fonctions à partir d'autres endroits du programme.

4.2.1 La définition d'une fonction

En langage C, la définition d'une fonction prend l'une des formes suivantes, selon que la fonction admette ou non des paramètres.

```
type-retour id ([ type-par1 id1 [, type-par2 id2 [, ...]]])
{
  ⋮
}
```

```
type-retour id (void)
{
  ⋮
}
```

Comme toujours, les éléments entourés de crochets en italique "*[*" et "*]*" sont optionnels. L'identificateur *id* indique le nom que l'on donne à la fonction ; celui-ci ne peut pas correspondre à un mot-clé du langage. Le type de la valeur retournée par la fonction est spécifié par *type-retour*. Dans le cas d'une procédure, on le remplace par le type fictif `void`. Les parenthèses qui suivent l'identificateur de la fonction font partie de la syntaxe de la définition, et sont obligatoires. On définit à l'intérieur de ces parenthèses la liste des paramètres de la fonction, chacun de ces paramètres étant respectivement caractérisé par un identificateur *id1*, *id2*, ... et un type *type-par1*, *type-par2*, ... Dans le cas particulier d'une fonction qui ne possède aucun paramètre, on écrit `void` entre les deux parenthèses [1]. Enfin, le bloc qui figure à la fin de la définition constitue le *corps* de la fonction ; il contient les instructions qui s'exécuteront chaque fois que la fonction sera invoquée.

En langage C standard [2], il est interdit de définir une fonction dans le corps d'une autre fonction. Cela signifie qu'un programme C est essentiellement composé d'une suite de définitions de fonctions. Ce mécanisme permet de développer des programmes de grande taille, en décomposant leurs instructions en un certain nombre de fonctions et de procédures implémentant chacune une opération simple. Les fonctions et les procédures qui composent un programme peuvent être réparties dans des fichiers sources multiples. Une règle de bonne pratique est de veiller à ce que le code de chaque fonction ou procédure reste simple, par exemple en s'assurant qu'il ne dépasse pas la taille d'une page de texte. Quand un programmeur est amené à rédiger une fonction de plus grande taille, il s'agit souvent d'un symptôme indiquant qu'il serait judicieux de la décomposer en sous-fonctions plus simples.

1. Il est également permis de spécifier une liste de paramètres vide, c'est-à-dire ne rien écrire entre les parenthèses. Dans ce cas, le compilateur acceptera que la fonction soit invoquée avec un ou plusieurs arguments, même si ceux-ci ne sont pas utilisés. Cette pratique est découragée.

2. Il existe cependant des extensions du langage, supportées par certains compilateurs, qui autorisent de définir des fonctions au sein d'autres fonctions.

Nous avons déjà rencontré une définition de fonction dans les programmes rédigés jusqu'ici. La construction

```
int main()
{
  ⋮
}
```

définit une fonction appelée `main`, qui ne prend pas de paramètre [3] et retourne une valeur de type `int`. Cette fonction constitue le *point d'entrée* du programme ; il s'agit de celle qui est invoquée par l'environnement d'exécution dès que le programme est lancé. La valeur entière retournée par cette fonction à la fin de son exécution est un *code de diagnostic* qui sert à indiquer si cette exécution s'est terminée normalement ou bien à la suite d'une erreur. La convention habituelle est de considérer qu'une valeur de retour égale à zéro indique une terminaison normale, et une valeur non nulle une condition d'erreur.

Dans le corps d'une fonction, on dispose d'une instruction de contrôle permettant de terminer immédiatement l'exécution de cette fonction, et de retourner une valeur. Sa syntaxe est la suivante.

```
return [ expression ];
```

Si cette instruction est utilisée dans le corps d'une fonction définie avec un type de retour T différent de `void`, alors le mot-clé `return` doit être suivi d'une expression de type T. L'évaluation de cette expression fournit la valeur qui sera retournée par la fonction quand cette instruction sera exécutée. Toutes les exécutions possibles du corps de la fonction doivent obligatoirement [4] se terminer par une telle instruction `return`.

Dans le cas d'une procédure (c'est-à-dire, d'une fonction définie avec le type de retour `void`), le mot-clé `return` ne peut pas être suivi d'une expression, et le seul effet de l'instruction est de terminer l'exécution de la procédure. Son utilisation à la fin de cette dernière est optionnelle.

On emploie souvent l'instruction `return` pour simplifier la logique d'une fonction ou d'une procédure. Par exemple, le fragment de programme suivant définit une fonction `racine` acceptant un argument réel et retournant une valeur réelle, avec la convention que cette fonction doit

3. Nous verrons cependant au chapitre 6 que la fonction `main` reçoit des arguments lors de son invocation. C'est pour cette raison que nous spécifions une liste de paramètres vide, plutôt que d'écrire `void` entre les parenthèses.

4. Nous n'avons pas respecté cette règle pour la fonction `main` des programmes développés jusqu'ici. Les compilateurs C traitent d'une façon particulière cette fonction représentant le point d'entrée du programme, et ajoutent automatiquement une instruction `return 0;` à la fin de son corps si une telle instruction n'est pas déjà présente.

retourner la valeur spéciale -1.0 chaque fois qu'elle est invoquée avec un argument négatif, afin de signaler une condition d'erreur.

```
double racine(double x)
{
  if (x < 0.0)
    return -1.0;
   ⋮
  return ...;
}
```

Bien sûr, il aurait été possible de traiter le cas où l'argument est positif ou nul dans une partie `else` de l'instruction `if`. Employer l'instruction `return` pour terminer immédiatement l'exécution de la fonction après avoir traité le cas particulier d'un argument négatif conduit cependant à un programme plus lisible.

Pour donner un exemple complet de définition de fonction, nous reprenons l'implémentation de l'algorithme d'Euclide donnée à la figure 1.2. Un programme pourrait être amené à calculer le plus grand commun diviseur de deux nombres à plusieurs endroits de son code source, ce qui nous conduit à transformer cette implémentation en une fonction. Le résultat est donné à la figure 4.1. Cette fonction accepte deux paramètres entiers `a` et `b`, et retourne une valeur entière [5]. Notons que l'affichage du résultat, qui était effectué par un appel à `printf` dans le programme de la figure 1.2, est remplacé dans la fonction par une instruction `return`.

4.2.2 Les paramètres d'une fonction

Les paramètres d'une fonction peuvent être vus comme un cas particulier de variables dont la portée correspond au corps de la fonction. Ces variables sont donc allouées au moment où la fonction est invoquée, et sont libérées quand la fonction termine son exécution. La valeur initiale des paramètres est égale à celle des arguments fournis lors de l'invocation de la fonction.

Par exemple, nous verrons à la section 4.2.4 que la fonction `pgcd` définie à la figure 4.1 peut être invoquée par une expression telle que `pgcd(24, 18)`. L'évaluation de cette expression provoque l'exécution du corps de la fonction `pgcd`, après avoir initialisé `a` avec la valeur 24 et `b` avec la valeur 18. Dans le corps de la fonction, les variables correspondant aux paramètres peuvent être manipulées de la même façon que toute autre variable, en particulier, il est permis

5. Le type `int` a été utilisé pour les paramètres et la valeur de retour de cette fonction afin de rester cohérent avec le programme de la figure 1.2, mais on aurait pu employer ici le type `unsigned`, étant donné que les entiers manipulés sont toujours positifs ou nuls.

```
int pgcd(int a, int b)
{
  int c;

  if (a > b)
    {
      c = a;
      a = b;
      b = c;
    }

  while (a)
    {
      c = a;
      a = b % a;
      b = c;
    }

  return b;
}
```

FIGURE 4.1 – Fonction implémentant l'algorithme d'Euclide

d'en modifier librement la valeur. L'effet d'une telle modification reste entièrement local au corps de la fonction, et n'influence pas le code qui invoque la fonction. On dit qu'en langage C, le passage des arguments d'une fonction s'effectue *par valeur*.

4.2.3 La déclaration d'une fonction

Lorsqu'on développe un programme de grande taille, il arrive fréquemment de devoir invoquer à partir d'une partie de ce programme une fonction définie ailleurs, en particulier dans un autre fichier source. Chaque fichier source est cependant compilé séparément. Pour que le compilateur traite correctement l'instruction d'invocation de la fonction, il est alors nécessaire qu'il dispose d'un certain nombre de caractéristiques de cette fonction, en particulier son identificateur, le nombre et le type de ses paramètres, et le type de sa valeur de retour. Ces informations sont fournies par une *déclaration* de la fonction, aussi appelée le *prototype* de la fonction. On distingue donc les notions de *déclaration* et de *définition* d'une fonction : une déclaration spécifie le nom de la fonction, le nombre et le type de ses paramètres, et le type de sa valeur de retour. Une définition de la fonction fournit en outre le corps de la fonction, c'est-à-dire le détail des instructions qui sont exécutées chaque fois que la fonction est invoquée.

En langage C, quand on souhaite déclarer une fonction sans la définir entièrement, on utilise une des deux syntaxes suivantes, selon que la fonction admette ou non des paramètres.

```
type-retour id (type-par1 [ id1 ] [, type-par2 [ id2 ] [, ...]]);
```

```
type-retour id (void);
```

L'identificateur de la fonction est donné par *id* et son type de retour par *type-retour*. Le type des paramètres successifs de la fonction est donné par *type-par1*, *type-par2*, ... Il est autorisé, mais pas obligatoire, de faire suivre le type d'un paramètre par l'identificateur de ce dernier, afin de rester cohérent avec la syntaxe de la définition d'une fonction. Dans une déclaration de fonction, ces identificateurs de paramètres n'ont cependant aucune utilité, et peuvent différer de ceux qui figurent dans la définition de cette fonction.

Par exemple, la fonction `pgcd` de la figure 4.1 se déclare de la façon suivante.

```
int pgcd(int, int);
```

Une telle déclaration n'est utile que si l'on souhaite invoquer la fonction `pgcd` à partir d'un fichier source où cette fonction n'a pas été précédemment définie. En d'autres termes, la définition d'une fonction fait aussi office de déclaration pour la suite du fichier source dans laquelle elle apparaît.

Traditionnellement, si un programme définit un ensemble de fonctions et de procédures destinées à être utilisées par d'autres parties du programme, ou bien par d'autres programmes, on regroupe les déclarations de ces fonctions et de ces procédures dans un *fichier d'en-tête (header file)* auquel on donne l'extension `.h`. Ce fichier peut alors être incorporé aux fichiers sources qui nécessitent ces déclarations [6], par le biais de la *directive de prétraitement (preprocessing directive)* `#include`. Celle-ci admet les deux formes suivantes.

```
#include <source.h>
```

```
#include "source.h"
```

Dans les deux cas, cette directive a pour effet d'insérer le contenu du fichier `source.h` à l'endroit où elle figure. La différence entre les deux formes concerne la nature du fichier `source.h`. Dans le premier cas, il s'agit d'un fichier d'en-tête *standard*, mis à la disposition du programmeur par

6. Par convention, ces fichiers possèdent l'extension `.c`.

l'environnement de développement utilisé. Dans le second cas, ce fichier fait partie du projet en cours de développement et est rédigé par le programmeur. Cela signifie que le compilateur recherchera alors ce fichier aux mêmes endroits que ceux contenant le reste du code source du programme.

Il est important que les fichiers qui font l'objet d'une directive `#include` ne contiennent que des déclarations de fonctions et de procédures, et non leurs définitions. Fournir l'implémentation de fonctions ou de procédures à l'intérieur d'un fichier d'en-tête est une mauvaise pratique de programmation, qui peut notamment conduire à ce qu'une telle implémentation soit compilée plusieurs fois au sein d'un même projet.

Pour terminer, remarquons que ce mécanisme de fichiers d'en-tête a déjà été utilisé dans les programmes rédigés jusqu'ici. Ainsi, la directive

```
#include <stdio.h>
```

présente dans la plupart de ces programmes indique au compilateur qu'il faut incorporer au fichier source en cours de traitement le contenu du fichier d'en-tête standard `stdio.h`. Celui-ci contient, entre autres choses, les prototypes d'un certain nombre de fonctions de la bibliothèque standard permettant de réaliser des opérations d'entrée/sortie *(*st*and*ard i*nput/*o*utput)*. Parmi ces fonctions se trouvent `printf` et `scanf` que nous avons utilisées pour afficher des valeurs à l'écran et effectuer des saisies au clavier. Cette directive fournit donc au compilateur les informations dont il a besoin pour pouvoir invoquer correctement ces fonctions.

4.2.4 L'invocation d'une fonction

On invoque une fonction ou une procédure en évaluant une expression de la forme

`f(`*expr1*`, `*expr2*`, ...)`,

où `f` est l'identificateur de la fonction invoquée, et *expr1*, *expr2*, ... sont des expressions dont l'évaluation fournit les arguments de l'appel. Le nombre de ces expressions doit correspondre au nombre de paramètres de `f`. Dans le cas particulier où ce nombre est nul, la parenthèse fermante suit immédiatement la parenthèse ouvrante dans l'expression.

L'évaluation de cette expression d'invocation de fonction s'effectue en évaluant d'abord *expr1*, *expr2*, ..., et puis en invoquant `f` avec les arguments ainsi obtenus. L'évaluation de l'expression est suspendue pendant le temps nécessaire à l'exécution de `f`. Dès que cette exécution est terminée, cette évaluation reprend et fournit comme résultat final la valeur de retour produite

```
int pgcd(int, int);

int ppcm(int a, int b)
{
  int g;

  g = pgcd(a, b);

  return (a / g) * b;
}
```

FIGURE 4.2 – Fonction calculant le PPCM de deux nombres

par `f`. Dans le cas particulier où `f` est une procédure (en d'autres termes, si le type de retour de `f` est `void`), on considère que l'évaluation de l'expression globale ne produit pas de valeur.

Pour illustrer le mécanisme d'invocation de fonction, nous considérons le programme de la figure 4.2, qui définit une fonction `ppcm` capable de calculer le *Plus Grand Commun Multiple (PPCM)* de deux nombres entiers strictement positifs.

Cette fonction calcule le PPCM de `a` et `b` en divisant [7] leur produit par leur PGCD. Le calcul du PGCD de `a` et `b` est réalisé par la fonction `pgcd` de la figure 4.1, que l'on suppose être placée dans un fichier source séparé. Pour que cette fonction puisse être invoquée dans le programme de la figure 4.2, il est nécessaire de disposer de son prototype, qui figure à la première ligne de ce programme.

Pour pouvoir compiler et exécuter concrètement le programme de la figure 4.2, il faut y ajouter une fonction `main` qui invoque la fonction `ppcm`. Un exemple de tel programme de test est donné à la figure 4.3. Si l'on suppose que les programmes des figures 4.1, 4.2 et 4.3 sont respectivement placés dans des fichiers sources appelés `pgcd.c`, `ppcm.c` et `test.c`, alors la ligne de commande suivante permet de les compiler en générant un programme exécutable appelé `test`.

```
% gcc -O3 -o test pgcd.c ppcm.c test.c
```

4.2.5 Les notions d'interface et d'implémentation

Le concept de fonction est essentiel pour le développement de programmes de grande taille. Il existe en effet des programmes C composés de plusieurs centaines de millions de lignes de

7. Remarquons que nous effectuons l'opération de division avant la multiplication, afin d'éviter un dépassement arithmétique.

```c
#include <stdio.h>

int ppcm(int, int);

int main()
{
  int a, b;

  printf("Entrez deux entiers strictement positifs: ");
  scanf("%d %d", &a, &b);

  printf("Le PPCM est égal à %d.\n", ppcm(a, b));

  return 0;
}
```

FIGURE 4.3 – Programme de test du PPCM

code source, et il est illusoire de penser que chaque développeur collaborant à l'élaboration de tels programmes en comprend tous les détails. La notion de fonction permet d'introduire de la *modularité* dans le développement : l'utilisateur d'une fonction peut invoquer celle-ci dans le but d'effectuer l'opération qu'elle implémente sans devoir connaître ses détails internes d'implémentation. Réciproquement, le programmeur d'une fonction peut implémenter celle-ci sans devoir se familiariser avec les détails précis de la façon dont elle intervient dans le reste du programme.

On appelle l'*interface* d'une fonction ou d'une procédure l'ensemble des informations permettant de l'utiliser. Il s'agit d'une documentation de cette fonction ou procédure, précisant

— son nom,

— le nombre, le type et la signification de ses paramètres,

— le type de la signification de sa valeur de retour éventuelle,

— une description de l'opération qu'elle effectue, et

— un ensemble de contraintes d'utilisation éventuelles.

L'idée est que l'interface d'une fonction ou d'une procédure est beaucoup plus simple que son *implémentation*, qui contient quant à elle tous ses détails internes de programmation. La modularité est obtenue par le fait qu'il est suffisant de connaître l'interface d'une fonction pour pouvoir l'exploiter à partir d'autres parties du programme. On dispose également de la liberté de remplacer l'implémentation d'une fonction ou d'une procédure par une autre, par exemple parce qu'on vient d'en créer une nouvelle version plus efficace, sans que cela n'affecte l'utilisation de

```
int pgcd(int a, int b)
{
  int c;

  if (a > b)
    return pgcd(b, a);

  while (a)
    {
      c = a;
      a = b % a;
      b = c;
    }

  return b;
}
```

FIGURE 4.4 – Fonction PGCD récursive (version 1)

cette fonction ou procédure, à condition que cette modification en préserve l'interface.

Il s'agit d'un mécanisme que nous avons déjà utilisé dans les programmes rédigés jusqu'ici ; par exemple, nous avons employé la fonction `printf` de la bibliothèque standard en nous basant uniquement sur sa documentation, sans connaître ses détails d'implémentation internes.

4.3 La récursivité

4.3.1 Principes

La notion de *récursivité* correspond à la possibilité pour une fonction de s'appeler elle-même, c'est-à-dire de s'invoquer à partir de son propre code. On parle aussi de récursivité quand un ensemble de fonctions s'invoquent mutuellement, par exemple quand une fonction `f` invoque une fonction `g` qui invoque à son tour `f`.

La récursivité permet de résoudre simplement certains problèmes algorithmiques. Par exemple, dans l'implémentation de la fonction `pgcd` fournie à la figure 4.1, on traite le cas `a > b` en permutant les valeurs de `a` et de `b` grâce à une séquence de trois instructions d'affectation. Une approche plus élégante consiste dans ce cas à calculer `pgcd(b, a)` et à retourner le résultat obtenu à l'issue de ce calcul. Une nouvelle implémentation de la fonction `pgcd` basée sur ce principe est donnée à la figure 4.4.

```
int pgcd(int a, int b)
{
  if (a > b)
    return pgcd(b, a);

  if (a)
    return pgcd(b % a, a);

  return b;
}
```

FIGURE 4.5 – Fonction PGCD récursive (version 2)

Si l'on considère par exemple l'exécution de cette fonction pour `a` = 24 et `b` = 18, alors la condition `a > b` est satisfaite, et le programme se réduit à évaluer `pgcd(18, 24)` et à retourner le résultat de cette évaluation. Cette dernière consiste à exécuter la fonction `pgcd` avec `a` = 18 et `b` = 24. Cette fois, la condition `a > b` n'est pas satisfaite, et le programme effectue la boucle `while` qui suit l'instruction `if`. Ce scénario conduit donc a exécuter deux fois la fonction `pgcd`, avec à chaque fois des valeurs différentes des paramètres `a` et `b`.

Le fait que ces paramètres `a` et `b` prennent des valeurs différentes pour chacun des deux appels à `pgcd` n'est pas problématique. Nous avons vu à la section 4.2.2 que les paramètres d'une fonction peuvent être considérés comme étant un cas particulier de variables, allouées au moment où la fonction est invoquée et libérées à la fin de l'exécution de cette fonction. Dans notre exemple, il y a donc un exemplaire des variables `a` et `b` qui est alloué pour l'appel `pgcd(24, 18)`, et un autre pour l'appel `pgcd(18, 24)`. Il n'y a pas de conflit entre ces deux paires de variables, qui occupent des endroits distincts de la mémoire de l'ordinateur.

On peut exploiter le mécanisme de récursivité pour transformer encore plus en profondeur le programme de la figure 4.4. Dans ce programme, l'opération effectuée par la boucle `while` consiste à remplacer répétitivement, tant que la valeur de `a` est non nulle, la paire d'entiers (`a`, `b`) par ($b \bmod a$, `a`). Il est tout à fait possible de remplacer cette boucle par un appel récursif à `pgcd(b % a, a)`. Le résultat est donné à la figure 4.5.

Comme on le voit, ce programme est plus simple que celui de la figure 4.1. En particulier, il ne comprend plus d'instruction de boucle, car le mécanisme permettant de répéter l'exécution de certaines opérations est maintenant entièrement assuré par la récursivité. Il existe des langages de programmation, par exemple *Lisp* et *Scheme*, qui encouragent cette approche.

Pour le programme de la figure 4.5, une invocation de `pgcd(24, 18)` conduit à invoquer `pgcd(18, 24)`, qui invoque à son tour `pgcd(6, 18)`, qui invoque `pgcd(0, 6)`. Ce der-

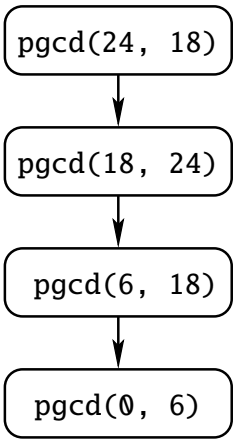


FIGURE 4.6 – Schéma d'appels récursifs pour `pgcd(24, 18)`

nier appel à la fonction retourne la valeur 6, qui devient celle de toutes les expressions d'invocation que nous avons mentionnées. Une représentation graphique du schéma d'appels de cet exemple est donnée à la figure 4.6. Chaque nœud de cette structure arborescente correspond à une invocation de la fonction ; un nœud n_1 est le parent d'un autre n_2 si l'appel représenté par n_2 est effectué à l'intérieur de l'appel associé à n_1.

4.3.2 La complexité en espace

Nous avons vu que la version récursive d'un programme peut être plus simple et plus élégante que sa version non-récursive (ou *itérative*). Cette version récursive n'est cependant pas nécessairement plus efficace. En effet, chaque appel de fonction nécessite d'allouer une structure de données dans la mémoire de l'ordinateur, chargée de retenir

— les arguments de cet appel,

— les variables créées par la fonction invoquée, et

— l'endroit où le code qui invoque la fonction doit reprendre son exécution dès que cette fonction se termine.

Pour un programme donné, l'ensemble des structures à allouer pour les invocations de fonc-

```
unsigned fact(unsigned n)
{
  if (n <= 1)
    return 1;

  return n * fact(n - 1);
}
```

FIGURE 4.7 – Fonction récursive calculant une factorielle

tions croît linéairement [8] avec la *profondeur de récursion*, qui correspond au plus grand nombre d'appels de fonction en cours à n'importe quel moment de l'exécution de ce programme. Pour l'exemple de la figure 4.6, cette profondeur de récursion est égale à 4.

Nous avons défini à la section 3.2 une notion de complexité en temps, représentant le temps nécessaire à l'exécution d'un programme. De façon similaire, nous introduisons la *complexité en espace*, qui mesure la quantité de mémoire consommée par l'exécution d'un programme. Cette complexité en espace se note aussi à l'aide de la notation "grand-O".

Nous illustrons cette notion de complexité en espace à l'aide du programme de la figure 4.7, qui définit une fonction capable de calculer la factorielle $\mathtt{n}!$ d'un nombre entier $\mathtt{n} \geq 1$.

Pour une valeur donnée de `n`, un appel à `fact(n)` provoque un appel à `fact(n - 1)`, qui invoque `fact(n - 2)`, et ainsi de suite jusqu'à `fact(1)`. Ce schéma d'appels récursifs est illustré à la figure 4.8 pour le cas particulier $\mathtt{n} = 6$. On a donc en résumé une profondeur de récursion égale à `n`, ce qui signifie que la quantité de mémoire requise pour gérer les appels récursifs admet une borne supérieure proportionnelle à `n`. Étant donné que le programme ne consomme pas d'autres emplacements de mémoire, on en déduit que sa complexité en espace vaut

$$O(\mathtt{n}).$$

Notons que pour ce problème, il est facile d'écrire une fonction itérative équivalente ; celle-ci est donnée à la figure 4.9. Cette fonction n'effectue pas d'appels récursifs, en d'autres termes, sa profondeur de récursion est bornée [9]. La complexité en espace de ce programme vaut donc

$$O(1),$$

ce qui le rend donc meilleur que celui de la figure 4.7.

8. Signalons cependant que pour un programme tel que celui de la figure 4.5, les compilateurs modernes sont capables de déterminer qu'aucun traitement ne doit plus être effectué à la suite de chaque appel récursif, et d'exploiter cette propriété pour réduire la quantité de données à mémoriser. Ce mécanisme de *récursivité terminale (tail recursion)* est également mis en œuvre par les langages de programmation dits *fonctionnels*, tels que *Lisp*, *Scheme* ou *Scala*.

9. Selon notre définition, cette profondeur est égale à 1 puisqu'un seul appel à `fact` est en cours à chaque instant de l'exécution du programme.

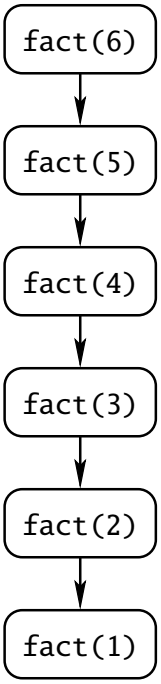


FIGURE 4.8 – Schéma d'appels récursifs pour `fact(6)`

```
unsigned fact(unsigned n)
{
  unsigned r;

  for (r = 1; n > 1; n--)
    r *= n;

  return r;
}
```

FIGURE 4.9 – Fonction itérative calculant une factorielle

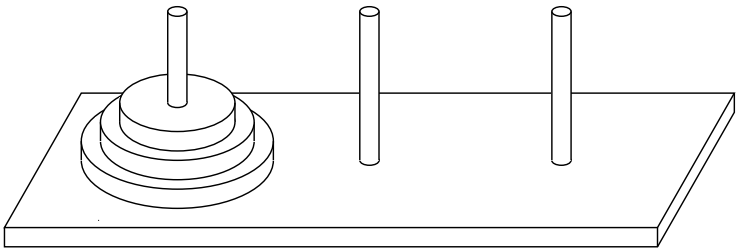


FIGURE 4.10 – Les tours de Hanoï

4.3.3 La terminaison d'une fonction récursive

Pour qu'une fonction récursive se termine, il est nécessaire que sa profondeur de récursion reste toujours finie. Cela signifie qu'il doit exister un ou plusieurs *cas de base* que n'importe quelle chaîne d'appels récursifs de la fonction finit toujours par atteindre, et qui n'effectuent quant à eux plus d'appels. Par exemple, l'exécution du programme de la figure 4.7 finit toujours par atteindre le cas de base `fact(1)` qui retourne le résultat 1 sans invoquer la fonction `fact`.

On peut démontrer la terminaison d'une telle fonction en employant un *variant* tout comme pour les boucles des programmes itératifs. Dans ce contexte, un variant est une fonction prenant en entrée les arguments de la fonction récursive, et retournant un entier naturel strictement décroissant à chaque appel récursif. L'existence d'un tel variant prouve que la profondeur de récursion reste finie, puisqu'il n'existe pas de séquence infinie d'entiers positifs ou nuls strictement décroissants. Pour le programme de la figure 4.7, on voit immédiatement que le variant

$$v = \mathtt{n}$$

convient.

4.3.4 Application : les tours de Hanoï

L'intérêt de la récursivité est qu'elle permet de résoudre simplement et élégamment certains problèmes algorithmiques. Nous illustrons maintenant cela à l'aide du problème des *tours de Hanoï*.

Il s'agit d'un jeu composé de trois tiges fixées sur une planchette, sur lesquelles sont empilés un certain nombre n de disques possédant chacun un diamètre différent des autres. Initialement, tous les disques sont placés sur la première tige, par ordre décroissant de diamètre. Cette configuration initiale est illustrée à la figure 4.10, pour $n = 3$.

Le but du jeu est d'amener tous les disques sur la deuxième tige, en respectant les deux règles suivantes : on ne peut déplacer qu'un disque à la fois, et l'on ne peut placer un disque

que sur un disque de diamètre supérieur ou bien directement sur la planchette. Nous souhaitons écrire un programme qui, pour une valeur donnée de n, est capable de générer une séquence de mouvements permettant de résoudre le jeu.

Pour pouvoir exploiter la récursivité, on fait l'hypothèse que l'on dispose déjà d'une solution pour déplacer d'une tige à une autre un nombre de disques strictement inférieur à n, et l'on utilise cette solution pour en construire une pour le problème à n disques. Il s'agit de la même approche que celle mise en œuvre dans le programme de la figure 4.7 : pour calculer la factorielle $n!$ d'un nombre $n > 1$, on fait l'hypothèse que l'on dispose du résultat du calcul de $(n-1)!$, et il suffit alors de multiplier ce résultat par n pour obtenir la valeur de $n!$.

Le problème de déplacer n disques de la tige a à la tige b en s'aidant de la tige c, avec $n > 1$ et $a, b, c \in \{1, 2, 3\}$, peut se résoudre en trois étapes :

1. Déplacer $n - 1$ disques de la tige a à la tige c.

2. Déplacer un disque de la tige a à la tige b.

3. Déplacer $n - 1$ disques de la tige c à la tige b.

Cette procédure est illustrée à la figure 4.11. Le cas de base de cet algorithme récursif est obtenu pour $n = 1$; dans ce cas, la solution consiste à déplacer directement l'unique disque en jeu de la tige de départ a vers la tige d'arrivée b.

Un programme C implémentant cette solution est donné à la figure 4.12. Ce programme est composé de deux fonctions : une fonction `main` demandant à l'utilisateur d'introduire un nombre `n`, en recommençant jusqu'à avoir $\texttt{n} \geq 1$, et une procédure `hanoi` résolvant le problème pour cette valeur de `n` et pour des numéros de tiges de départ et de destination donnés. Cette procédure admet comme quatrième argument le numéro de la troisième tige. Bien sûr, ce numéro pourrait facilement être calculé à partir de ceux des tiges de départ et de destination, mais comme il est toujours connu au moment d'invoquer la procédure `hanoi`, cette solution présente l'avantage d'être plus simple. Un exemple de résultat produit par ce programme est fourni à la figure 4.13.

Après avoir obtenu ce programme, on peut chercher à estimer le coût de son exécution, c'est-à-dire déterminer ses complexités en temps et en espace. Il est pour cela nécessaire de caractériser la façon dont les appels à la procédure `hanoi` sont organisés. La figure 4.14 montre le schéma des appels à cette procédure pour le cas $\texttt{n} = 4$. Pour chacun de ces appels, cette figure précise le nombre des disques concernés et si cet appel correspond à la première (#1), deuxième (#2) ou troisième (#3) étape de la procédure récursive. Un diagramme similaire peut être dessiné pour n'importe quelle valeur de `n` supérieure à 1.

Chaque appel à la procédure `hanoi` effectue un nombre borné d'opérations, en plus d'éventuellement invoquer récursivement cette même procédure. On peut donc calculer la complexité en temps du programme, en supposant que l'utilisateur fournit une valeur convenable de `n` à son

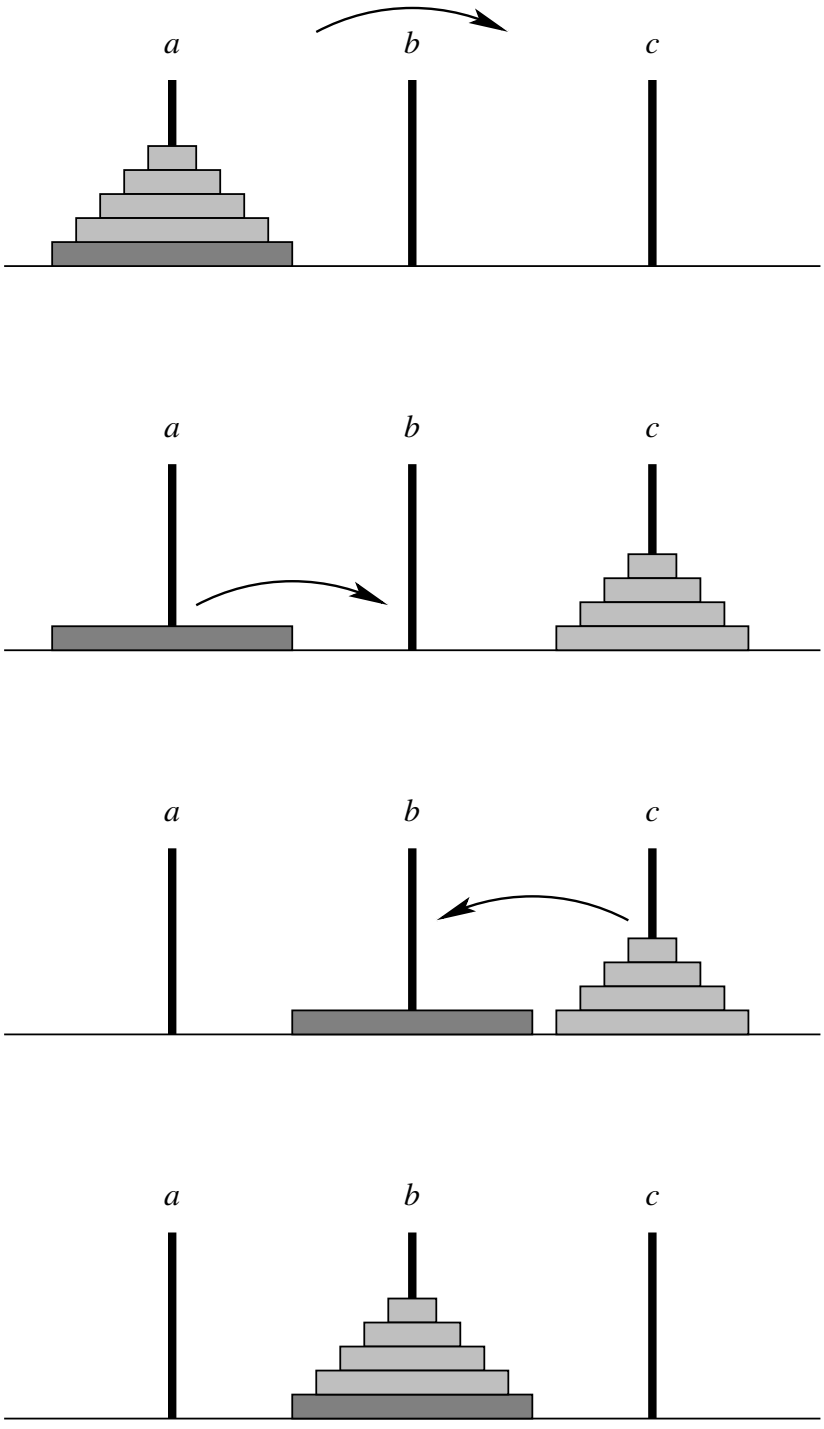


FIGURE 4.11 – Les tours de Hanoï : procédure récursive

```c
#include <stdio.h>

/* Déplace <n> disques de la tige <origine> vers la tige
   <destination>, en s'aidant de la tige <autre>. */

void hanoi(int n, int origine, int destination, int autre)
{
  if (n <= 1)
    {
      printf("Déplacer un disque de la tige %d vers la tige %d.\n",
             origine, destination);
      return;
    }

  hanoi(n - 1, origine, autre, destination);
  hanoi(1, origine, destination, autre);
  hanoi(n - 1, autre, destination, origine);
}

int main()
{
  int n;

  do
    {
      printf("Entrez un nombre de disques: ");
      scanf("%d", &n);
    }
  while (n < 1);

  hanoi(n, 1, 2, 3);

  return 0;
}
```

FIGURE 4.12 – Les tours de Hanoï : solution

```
Entrez un nombre de disques: 5
Déplacer un disque de la tige 1 vers la tige 2.
Déplacer un disque de la tige 1 vers la tige 3.
Déplacer un disque de la tige 2 vers la tige 3.
Déplacer un disque de la tige 1 vers la tige 2.
Déplacer un disque de la tige 3 vers la tige 1.
Déplacer un disque de la tige 3 vers la tige 2.
Déplacer un disque de la tige 1 vers la tige 2.
Déplacer un disque de la tige 1 vers la tige 3.
Déplacer un disque de la tige 2 vers la tige 3.
Déplacer un disque de la tige 2 vers la tige 1.
Déplacer un disque de la tige 3 vers la tige 1.
Déplacer un disque de la tige 2 vers la tige 3.
Déplacer un disque de la tige 1 vers la tige 2.
Déplacer un disque de la tige 1 vers la tige 3.
Déplacer un disque de la tige 2 vers la tige 3.
Déplacer un disque de la tige 1 vers la tige 2.
Déplacer un disque de la tige 3 vers la tige 1.
Déplacer un disque de la tige 3 vers la tige 2.
Déplacer un disque de la tige 1 vers la tige 2.
Déplacer un disque de la tige 3 vers la tige 1.
Déplacer un disque de la tige 2 vers la tige 3.
Déplacer un disque de la tige 2 vers la tige 1.
Déplacer un disque de la tige 3 vers la tige 1.
Déplacer un disque de la tige 3 vers la tige 2.
Déplacer un disque de la tige 1 vers la tige 2.
Déplacer un disque de la tige 1 vers la tige 3.
Déplacer un disque de la tige 2 vers la tige 3.
Déplacer un disque de la tige 1 vers la tige 2.
Déplacer un disque de la tige 3 vers la tige 1.
Déplacer un disque de la tige 3 vers la tige 2.
Déplacer un disque de la tige 1 vers la tige 2.
```

FIGURE 4.13 – Les tours de Hanoï : résultat pour $n = 5$

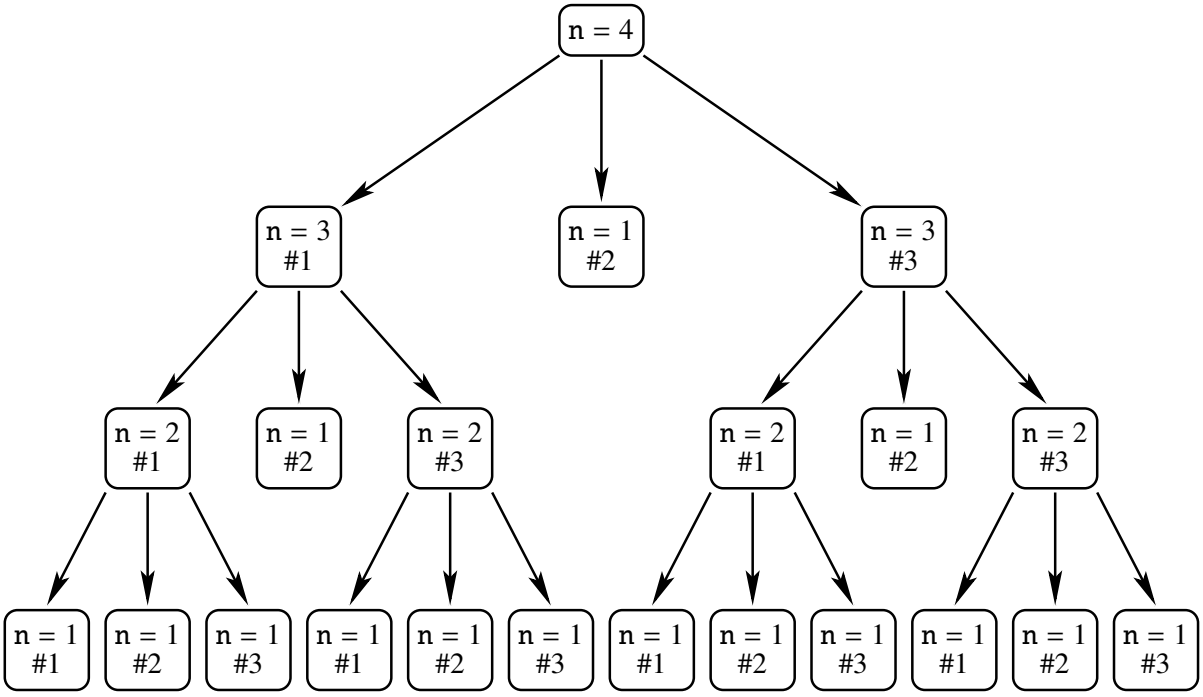


FIGURE 4.14 – Les tours de Hanoï : schéma d'appels récursifs

premier essai, en comptant le nombre d'invocations de la procédure `hanoi`.

Appelons n_0 la valeur de `n` fournie par l'utilisateur, c'est-à-dire celle correspondant au nombre total de disques du problème. Comme on le voit en généralisant le diagramme de la figure 4.14 à cette valeur de `n`, les étapes 1 et 3 de l'algorithme nécessitent

— 2 appels à la procédure `hanoi` avec `n` $= n_0 - 1$.

— 4 appels avec `n` $= n_0 - 2$

— 8 appels avec `n` $= n_0 - 3$, et ainsi de suite jusqu'à

— 2^{n_0-1} appels avec `n` $= 1$.

Le nombre total d'appels à la procédure `hanoi` pour effectuer les étapes 1 et 3 vaut donc

$$2 + 4 + 8 + \cdots + 2^{n_0-1} = 2^{n_0} - 2.$$

Pour chaque valeur de `n`, on observe que le nombre d'appels à la procédure `hanoi` nécessaires pour réaliser l'étape 2 de l'algorithme est égal à la moitié du nombre d'appels à cette procédure pour les étapes 1 et 3. L'étape 2 nécessite donc au total

$$2^{n_0-1} - 1$$

appels à la procédure `hanoi`. En ajoutant l'appel initial à cette procédure effectué par la fonction `main`, on obtient que le nombre total d'appels à la procédure `hanoi` vaut

$$(2^{n_0} - 2) + \left(2^{n_0-1} - 1\right) + 1 = O(2^{n_0}).$$

La complexité en temps du programme de la figure 4.12 est donc égale à

$$O(2^n),$$

où n est le nombre de disques du problème.

Calculons à présent la complexité en espace de notre solution. Celle-ci correspond à la profondeur de récursion. On voit immédiatement que celle-ci est égale à n pour un nombre de disques n donné : chaque appel récursif à la procédure `hanoi` en vue d'effectuer les étapes 1 ou 3 diminue d'une unité le nombre de disques, jusqu'à atteindre le cas de base correspondant au cas d'un seul disque. La complexité en espace du programme de la figure 4.12 est donc égale à

$$O(n).$$

4.4 Les variables globales

Jusqu'à ce stade, les seules variables que nous avons rencontrées étaient définies à l'intérieur d'un bloc, ou bien correspondaient à un paramètre d'une fonction ou d'une procédure. De telles variables sont allouées lorsque leur définition est exécutée, ou lorsque la fonction ou la procédure dont elles sont un paramètre est invoquée. Elles sont libérées à la fin du bloc qui contient leur définition, ou à l'issue de l'exécution de cette fonction ou procédure.

On appelle cette catégorie de variables, qui n'ont qu'une existence temporaire au cours de l'exécution d'un programme et dont la portée est limitée à un bloc d'instructions exécutables, les *variables locales*. Il existe également une autre sorte de variables, les *variables globales*, qui restent allouées en permanence au cours de l'exécution d'un programme et possèdent une portée plus étendue.

4.4.1 Définition et déclaration

En langage C, une définition de variable globale possède la syntaxe suivante. Une telle définition se place en dehors du corps de toute fonction.

[`static` *] [* `const` *] type identificateur [= valeur]*
[`,` *identificateur [= valeur]]*
⋮
[`,` *identificateur [= valeur]]* `;`

Comme pour les définitions de variables locales, le mot-clé `const` est employé pour définir des constantes, c'est-à-dire interdire que la valeur de la variable concernée puisse être modifiée. La définition offre la possibilité d'attribuer une valeur initiale aux variables. Pour une variable globale, cette valeur est celle que possède la variable au début de chaque exécution du programme. On peut également définir plusieurs variables partageant le même type au sein d'une seule définition.

La portée d'une variable globale s'étend à l'entièreté du programme, même si celui-ci comprend plusieurs fichiers sources. Il y a cependant moyen de limiter cette portée au seul fichier source qui contient la définition de cette variable, en préfixant cette définition du mot-clé `static`. Pour des projets de grande taille, ce mécanisme permet d'éviter des conflits de noms entre les variables définies par des programmeurs responsables de parties différentes du programme.

Les définitions de variables globales se placent au même endroit que les définitions de fonctions. En d'autres termes, un programme C prend essentiellement la forme d'une suite de définitions de fonctions et de définitions de variables globales. Dans un but de lisibilité, il est conseillé

de placer dans chaque fichier source les définitions de variables avant les définitions de fonctions, mais il ne s'agit pas d'une contrainte imposée par le langage.

Si une variable globale est définie sans employer le mot-clé `static`, alors elle peut être utilisée dans un fichier source différent de celui qui contient sa définition. Il est cependant nécessaire que le compilateur dispose du nom et du type de cette variable pour pouvoir traiter correctement les instructions où elle intervient. On fournit alors ces informations au compilateur par le biais d'une *déclaration* de variable globale, qui possède la syntaxe suivante.

`extern` *type identificateur [*, *identificateur [*, *...]]* ;

Un exemple de programme définissant une variable globale dans un fichier source, et accédant à cette variable dans un autre, sera donné à la section 4.4.2.

Lorsqu'on rédige un programme, il faut être conscient qu'employer des variables globales présente plusieurs inconvénients. Premièrement, si une fonction ou une procédure effectue des opérations qui consultent ou modifient des variables globales, alors le travail effectué par cette fonction ou cette procédure n'est pas entièrement caractérisé par la valeur des arguments qui lui sont passés. Cela a pour conséquence qu'appeler plusieurs fois la fonction ou la procédure avec les mêmes arguments peut conduire à des comportements différents, ce qui nuit à la modularité du programme. On dit qu'une fonction est *pure* si l'opération qu'elle effectue ne dépend que des valeurs qu'elle reçoit en arguments, et si elle ne modifie pas de variable globale ni d'autre partie de la mémoire de l'ordinateur utilisée par le reste du programme. Par opposition, on dit qu'une fonction produit des *effets de bord* si son exécution peut conduire à modifier des données extérieures à cette fonction. Une bonne pratique de programmation consiste à viser dans la mesure du possible à n'écrire que des fonctions pures.

Un autre inconvénient des variables globales est qu'elles n'existent qu'en un seul exemplaire. Si l'on place, par exemple, les champs individuels d'une structure de données dans des variables globales, cela empêche d'utiliser plusieurs copies simultanées de cette structure dans le programme. Enfin, comme nous l'avons vu, définir des variables globales sans utiliser le mot-clé `static` peut provoquer des conflits de noms avec les variables définies dans d'autres parties du programme.

Signalons qu'il est aussi permis d'employer le mot-clé `static` pour définir des variables locales à un bloc de code, par exemple les variables locales à une fonction. La valeur de ces variables est alors préservée d'une exécution à l'autre du bloc contenant cette définition. Ce mécanisme entraîne donc un effet de bord. Si une telle définition comprend une expression d'initialisation, alors celle-ci n'est appliquée qu'une seule fois, à la première exécution de l'instruction.

Enfin, le mot-clé `static` peut aussi préfixer des définitions de fonctions ou de procédures. Celles-ci ne peuvent alors être invoquées qu'à partir du même fichier source que celui qui contient

ces définitions. Pour des programmes de grande taille, ce mécanisme permet d'éviter des conflits de noms avec les fonctions ou procédures définies dans d'autres fichiers sources.

4.4.2 Exemples

Compteur simple

Le programme donné à la figure 4.15 implémente une structure de données simple représentant un compteur. Cette structure retient une valeur entière initialisée à zéro, que l'on peut incrémenter ou décrémenter ainsi que tester si elle est ou non égale à zéro. Cette structure permet par exemple de compter le nombre d'utilisations simultanées d'une ressource, en incrémentant le compteur à chaque nouvel accès à cette ressource et en le décrémentant à la fin de cet accès.

Le code source de ce programme est réparti en deux fichiers : le premier (`compteur.c`) contient l'implémentation des fonctions `compteur_init`, `compteur_plus`, `compteur_moins` et `compteur_est_zero`, permettant respectivement d'initialiser, d'incrémenter, de décrémenter et de tester la valeur du compteur. Le fichier d'en-tête `compteur.h` contient quant à lui les déclarations de ces fonctions.

Cette structure de données utilise une variable globale `compteur_valeur` pour retenir la valeur courante du compteur. La définition de cette variable utilise le mot-clé `static`, afin de limiter sa portée au fichier `compteur.c`. Cela signifie que d'autres parties du programme sont libres de réutiliser ce nom pour leurs propres variables, sans que cela ne produise de conflit. En revanche, les quatre définitions de fonctions qui figurent dans `compteur.c` n'emploient pas le mot-clé `static`, ce qui signifie que ces fonctions peuvent être invoquées depuis n'importe quel fichier source du programme.

Les opérations effectuées par les quatre fonctions implémentées dans `compteur.c` dépendent de la variable globale `compteur_valeur`, en particulier les trois premières en modifient la valeur par effet de bord. Cela présente l'inconvénient, comme nous l'avons vu, de ne pas permettre de manipuler simultanément plusieurs compteurs distincts.

Un exemple de programme de test de cette structure de données est fourni à la figure 4.16. Le code source de ce programme commence par inclure `compteur.h` afin de disposer du prototype des quatre fonctions implémentées par `compteur.c`. Il effectue une séquence particulière d'opérations consistant à initialiser le compteur, à en incrémenter et décrémenter quelques fois la valeur, et à afficher à deux moments si celle-ci est nulle ou non. Notons que pour obtenir un programme exécutable complet, il est bien sûr nécessaire de compiler à la fois `compteur.c` et le programme de test de la figure 4.16.

compteur.c:

```
#include "compteur.h"

static unsigned compteur_valeur = 0;

void compteur_init(void)
{
  compteur_valeur = 0;
}

void compteur_plus(void)
{
  compteur_valeur++;
}

void compteur_moins(void)
{
  if (compteur_valeur)
    compteur_valeur--;
}

int compteur_est_zero(void)
{
  return !compteur_valeur;
}
```

compteur.h:

```
void compteur_init(void);
void compteur_plus(void);
void compteur_moins(void);
int  compteur_est_zero(void);
```

FIGURE 4.15 – Compteur simple

```
#include <stdio.h>
#include "compteur.h"

static void affiche(void)
{
  printf(compteur_est_zero() ? "= 0\n" : "!= 0\n");
}

int main()
{
  compteur_init();
  compteur_plus();
  compteur_plus();
  compteur_moins();

  affiche();

  compteur_moins();

  affiche();

  return 0;
}
```

FIGURE 4.16 – Programme de test du compteur

nombre-appels.c:

```
#include "nombre-appels.h"

unsigned nombre_appels = 0;

void f(void)
{
  nombre_appels++;

  /* Autres opérations */
}
```

nombre-appels.h:

```
extern unsigned nombre_appels;

void f(void);
```

FIGURE 4.17 – Mesure du nombre d'invocations d'une fonction

Mesure du nombre d'invocations d'une fonction

Ce deuxième exemple vise à illustrer la manipulation d'une variable globale à partir d'un autre fichier source que celui dans lequel elle est définie. Le but consiste à mesurer le nombre de fois qu'une fonction particulière est invoquée au cours de l'exécution d'un programme, par exemple en vue de calculer des statistiques permettant d'estimer la complexité en temps d'un programme.

Le programme est donné à la figure 4.17. Le fichier source `nombre-appels.c` définit une variable globale `nombre_appels` et une procédure `f` qui incrémente cette variable. Le fichier d'en-tête `nombre-appels.h` contient la déclaration de `nombre_appels` et le prototype de `f`. Il suffit donc d'inclure ce fichier d'en-tête au début d'un fichier source pour que celui-ci puisse accéder à `nombre_appels` et invoquer `f`. Un exemple de programme de test est donné à la figure 4.18 ; ce programme invoque `f` trois fois et affiche ensuite la valeur de `nombre_appels`.

4.5 Les macros

Nous avons vu au début de ce chapitre que le mécanisme de définition de fonction permet d'éviter de programmer des opérations redondantes : si l'on est amené à effectuer un même travail à plusieurs endroits d'un programme, il suffit de placer les opérations correspondantes dans une fonction ou une procédure, et d'invoquer celle-ci à partir d'autres parties du programme.

```
#include <stdio.h>
#include "nombre-appels.h"

int main()
{
  f();
  f();
  f();

  printf("nombre d'appels de f(): %u\n", nombre_appels);

  return 0;
}
```

FIGURE 4.18 – Programme de test pour la mesure du nombre d'invocations

Le mécanisme d'appel de fonctions possède cependant un coût. Lorsqu'on invoque une fonction, il est indispensable de garantir que le code qui effectue cette invocation, qui se retrouve suspendu pendant l'exécution de la fonction, pourra reprendre correctement son exécution quand la fonction sera terminée. En pratique, le compilateur implémente cela en générant des instructions du processeur qui sauvegardent certaines données quand la fonction est invoquée et les restaurent ensuite. La gestion des arguments et de la valeur de retour d'une fonction nécessitent aussi d'effectuer certaines opérations.

En général, le coût de ces opérations additionnelles réalisées lors de l'invocation d'une fonction reste négligeable comparé à celui des opérations utiles effectuées par cette fonction. Ce n'est cependant pas le cas lorsque le travail effectué par une fonction est très simple, par exemple comme dans le cas de la procédure `compteur_plus` du programme de la figure 4.15. Il existe alors un autre mécanisme, les *macros*, permettant de programmer une opération une seule fois et de la déployer ensuite à plusieurs endroits d'un programme.

Les macros ressemblent superficiellement aux fonctions, mais leur principe de fonctionnement est très différent. Il s'agit d'un mécanisme purement syntaxique, qui est mis en œuvre avant la compilation proprement dite du programme [10]. Une macro se définit grâce à une directive de prétraitement qui prend la forme suivante.

```
#define nom[([ id1 [, id2 [, ... ]]])] texte
```

10. Nous avons vu à la section 4.2.3 que la directive de prétraitement `#include` produit également ses effets avant la compilation du programme. Les macros sont traitées de la même façon, par le même composant du compilateur appelé le *préprocesseur*.

Comme on le voit en examinant les crochets en italique qui indiquent des éléments optionnels, on a le choix de faire suivre *nom* d'une suite d'identifiants de paramètres entre parenthèses, séparés par une virgule, ou de ne rien écrire. L'effet de cette directive consiste à substituer syntaxiquement, dans la suite du fichier source courant, toutes les occurrences de *nom* par le texte de substitution *texte*. Si des identificateurs *id1*, *id2*, ... sont présents, alors *nom* doit être suivi du nombre approprié d'arguments entre parenthèses pour que cette substitution soit effectuée. Ces identificateurs sont alors remplacés dans *texte* par l'argument qui leur correspond. Le texte de substitution *texte* se termine à la fin de la ligne qui le contient.

Des exemples de définitions de macros sont donnés ici.

```
#define PI 3.14159265358979323846
#define carre(x) ((x) * (x))
#define max(a, b) ((a) > (b) ? (a) : (b))
```

La première de ces macros ne possède pas de paramètre. Son effet consiste à remplacer, dans la suite du fichier source où elle apparaît, toutes les occurrences de `PI` par le texte de substitution `3.14159265358979323846`.

La deuxième macro remplace toutes les occurrences de `carre(x)` dans la suite du fichier par `((x) * (x))`, où `x` est n'importe quel texte. Par exemple, `carre(a + b)` sera remplacé par `((a + b) * (a + b))`. On voit qu'il est judicieux d'employer des parenthèses autour des arguments et de l'expression globale dans les définitions de macros. Si l'on écrivait, par exemple,

```
#define carre(x) x * x
```

,

alors cela conduirait à remplacer `carre(a + b)` par `a + b * a + b`, ce qui ne correspond probablement pas à l'intention du programmeur.

Le troisième exemple montre comment programmer une macro qui détermine laquelle parmi deux valeurs est la plus grande.

Il est important de réaliser que le mécanisme des macros, qui repose sur une substitution syntaxique, reste très différent de celui des fonctions. Une expression telle que

```
carre(n++)
```

ne s'évalue pas de la même façon selon que `carre` est une fonction ou une macro : dans le premier cas, la variable `n` n'est incrémentée qu'une seule fois. Dans le second cas, en supposant que la définition de `carre` est la première que nous avons vue, cette expression devient

```
((n++) * (n++))
```

dont l'évaluation incrémente `n` deux fois.

L'utilisation des macros est à réserver à des applications pour lesquelles le texte de substitution reste court. Il s'agit de la raison pour laquelle ce texte se termine à la fin de la ligne courante. Signalons qu'il est néanmoins possible d'écrire des textes de substitution qui s'étendent sur plusieurs lignes de code source, en terminant chacune d'entre elles sauf la dernière par le *caractère de continuation* "\".

4.6 Exercices

4.6.1 Principes

Cette série d'exercices porte sur l'écriture de programmes C graphiques. On dispose pour cela d'une bibliothèque de fonctions permettant de générer des images au format[11] SVG *(Scalable Vector Graphics)*. Cette bibliothèque peut être téléchargée depuis la page WWW du cours. Elle fournit les trois fonctions suivantes.

— `draw_init(nom_fichier, largeur, hauteur)` initialise un nouveau fichier au format SVG appelé `nom_fichier`, destiné à représenter une image de dimension `(largeur, hauteur)`. Par exemple, on écrira `draw_init("fichier.svg", 800, 600);` pour créer une image de 800×600 pixels qui sera générée dans le fichier `fichier.svg` du répertoire courant. Cette fonction doit être appelée préalablement à toute autre opération de la bibliothèque.

— `draw_line(x1, y1, x2, y2, trait, r, g, b)` trace une ligne entre les points de coordonnées `(x1, y1)` et `(x2, y2)`, d'épaisseur `trait` et de couleur `(r, g, b)`. Les coordonnées `x1`, `y1`, `x2`, `y2` ainsi que la valeur de `trait` sont exprimées en pixels. L'origine du système d'axes est situé en haut et à gauche de l'image ; l'axe horizontal va de gauche à droite et l'axe vertical de haut en bas. Une couleur `(r, g, b)` est caractérisée par ses composantes respectivement rouge (`r`), verte (`g`) et bleue (`b`), comprises entre 0 et 255.

— `draw_close()` finalise le fichier courant. Cette fonction doit toujours être appelée, et doit l'être en tant que dernière opération de la bibliothèque.

11. Les images représentées dans ce format peuvent être visualisées par la plupart des logiciels de manipulation d'images, ainsi que par les navigateurs WWW.

Chacune de ces fonctions retourne 0 en cas de succès, et un entier négatif en cas d'erreur.

Pour certains exercices, il est également nécessaire d'utiliser les fonctions mathématiques de la bibliothèque standard :

— Le fichier d'en-tête standard à inclure est `math.h`.

— Dans le cas d'une compilation en ligne de commande à l'aide de `GCC`, l'option "`-lm`" doit être ajoutée à celles du compilateur.

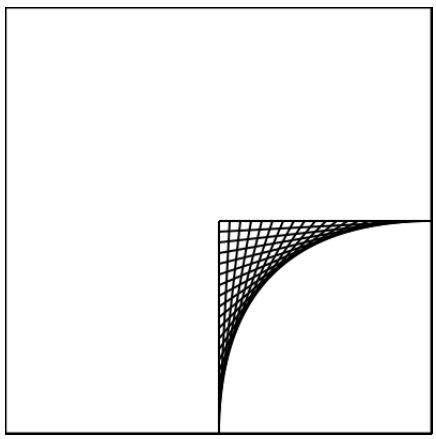
— Les fonctions `double sin(double)` et `double cos(double)` implémentent respectivement les fonctions trigonométriques sinus et cosinus, pour des angles exprimés en radians.

— La constante `M_PI` fournit la valeur approximée de π.

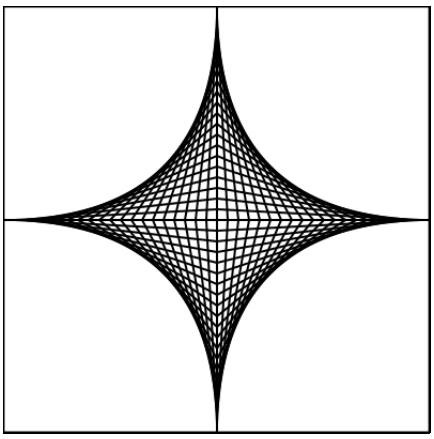
4.6.2 Énoncés

1. (a) La première étape de cet exercice consiste à écrire un programme qui crée une image de 400×400 pixels, et y simule le mouvement d'une latte droite de longueur initiale égale à 200 pixels, dont les extrémités se déplacent sur deux segments horizontal et vertical issus du centre de l'image. Lorsqu'une extrémité se déplace vers le haut sur le segment vertical, l'autre se décale vers la droite du même nombre de pixels sur le segment horizontal.

   L'objectif est d'arriver à une image similaire à la suivante. Pour obtenir cette image, 21 positions de la latte sont simulées. L'épaisseur du trait est de 2 pixels.

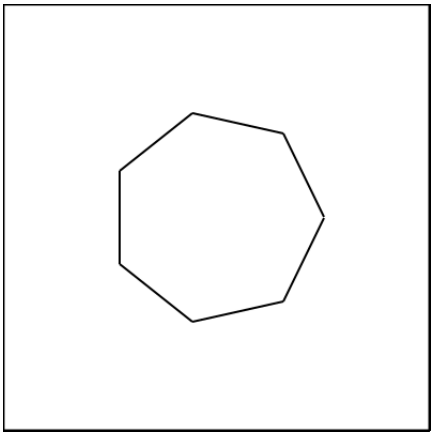


(b) L'objectif consiste maintenant à compléter ce programme de façon à arriver à une image similaire à la suivante, en faisant à présent se déplacer la latte sur deux droites horizontale et verticale se croisant au centre de l'image.

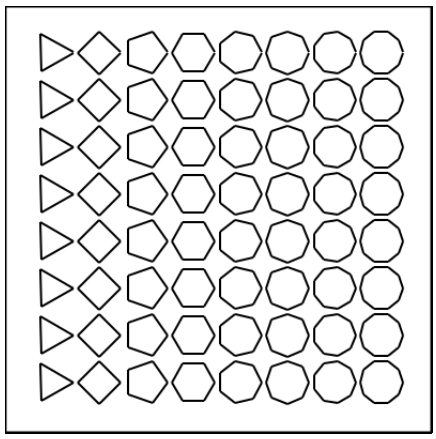


2. (a) La première étape de ce problème consiste à écrire une fonction capable de dessiner un polygone régulier. Le nombre de côtés de ce polygone, les coordonnées de son centre, le rayon de son cercle circonscrit, l'épaisseur du trait et la couleur de celui-ci doivent être des paramètres de cette fonction.

Par exemple, un appel à cette fonction pour un nombre de côtés égal à 7, un rayon de 100 pixels, et un polygone centré sur le milieu d'une image de 400×400 pixels devrait produire un résultat similaire au suivant.



(b) La deuxième étape consiste à écrire un programme exploitant la fonction obtenue au point 1, capable de générer une image similaire à la suivante.

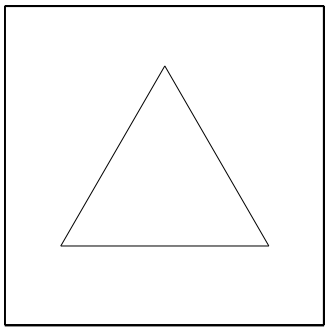


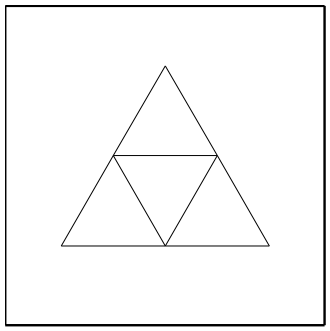
Dans cette image, le nombre de côtés des polygones varie de 3 à 10 lorsqu'on se déplace de gauche à droite. On peut également envisager de créer une version colorisée de cette image, par exemple en faisant varier graduellement la couleur des polygones du rouge au vert lorsqu'on se déplace de haut en bas.

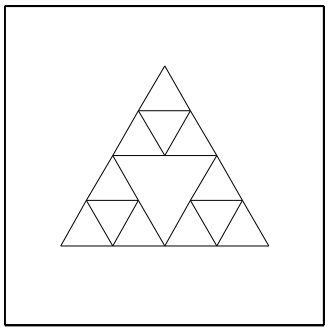
3. Le but de cet exercice est d'écrire un programme capable de générer une image de 400×400 pixels approximant le *triangle de Sierpinski*. Ce triangle se construit grâce à la procédure suivante.

   (a) On dessine un triangle équilatéral.
   (b) On découpe ce triangle en quatre triangles plus petits en joignant les milieux de ses côtés.
   (c) On répète la même procédure dans chacun de ces petits triangles, à l'exception de celui situé au centre.

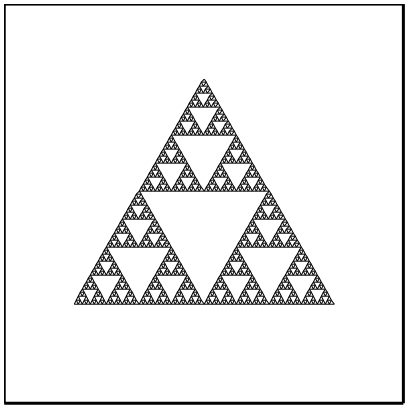
Par exemple, les trois premières applications de cette procédure produisent les images suivantes.





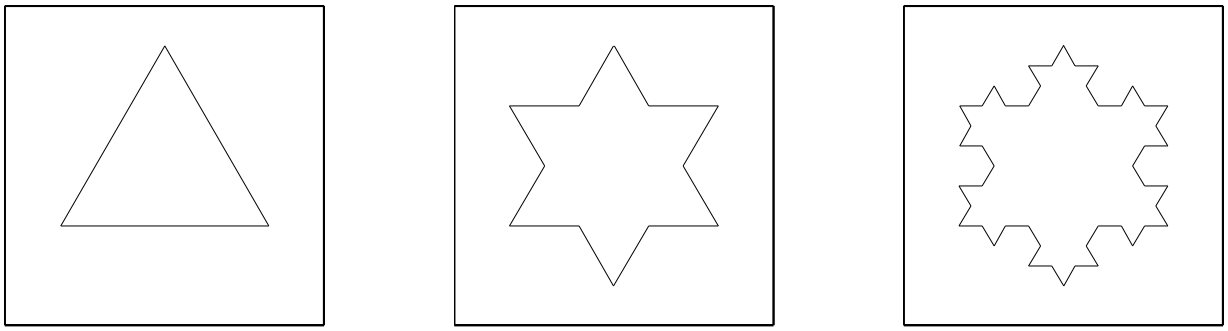


Le résultat obtenu après 6 applications est le suivant.

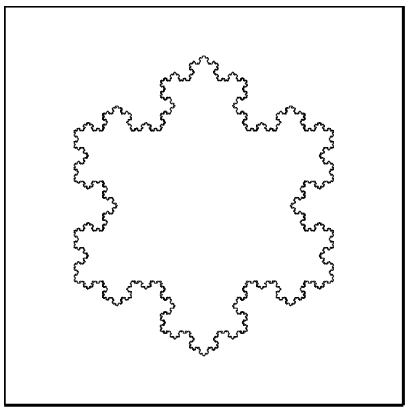


4. Cet exercice porte sur l'écriture d'un programme capable de dessiner une approximation du *Flocon de Koch*. Celui-ci s'obtient en appliquant plusieurs fois de suite une transformation particulière à un triangle équilatéral. Cette transformation consiste à remplacer chaque segment de droite de la figure par une ligne brisée, produite en découpant le segment en trois parties égales, en construisant un triangle équilatéral extérieur à la figure sur la partie centrale, et en retirant finalement cette partie centrale.

Les trois premières étapes de cette procédure produisent les résultats suivants.



L'image obtenue après 6 étapes est la suivante.



4.7 Exercices supplémentaires

1. *(Examen de juin 2019)*

   (a) Écrire une fonction prenant en arguments deux entiers strictement positifs n et f, et calculant la multiplicité du facteur f dans n, c'est-à-dire le plus grand nombre entier m tel que f^m divise n. Par exemple, si $n = 324$ et $f = 9$, on a $m = 2$ car $324 = 9^2.4$.

   (b) Par la méthode des invariants, démontrer que la valeur retournée par cette fonction est correcte.

2. *(Examen de janvier 2021)*

   (a) Écrire une fonction prenant en argument un nombre entier strictement positif n, et retournant le nombre de diviseurs de n. (Par exemple, cette fonction doit retourner 1 pour $n = 1$, 2 pour $n = 2$, et 8 pour $n = 24$.) On demande que l'implémentation de cette fonction soit raisonnablement efficace.

   (b) Déterminer les complexités en temps et en espace de la fonction obtenue au point (a).

   (c) Par la méthode des invariants, démontrer que la valeur retournée par la fonction obtenue au point (a) est correcte. Démontrer également que cette fonction se termine.

3. *(Examen de janvier 2022)*

   (a) Écrire une fonction prenant en argument un entier $n > 1$, et retournant le plus grand diviseur d de n tel que $d < n$. Par exemple, cette fonction appliquée à $n = 21$ doit retourner 7. On souhaite que l'implémentation de cette fonction soit raisonnablement efficace.

   (b) Déterminer les complexités en temps et en espace de la fonction obtenue au point (a).

4. *(Examen de juin 2022)*

   (a) Écrire une fonction prenant en argument un nombre entier non signé, et retournant la somme de tous les chiffres de son écriture décimale. Par exemple, pour le nombre 2022, cette fonction doit retourner 6.

   (b) Par la méthode des invariants, démontrer que la valeur retournée par cette fonction est correcte.

5. *(Examen d'août 2022)*

   (a) Écrire une fonction prenant en argument un nombre entier non signé et non nul, et retournant le nombre de zéros situés à la fin de son écriture décimale. Par exemple, pour le nombre 42000, cette fonction doit retourner 3. Pour le nombre 2022, elle doit retourner 0.

   (b) Par la méthode des invariants, démontrer que la valeur retournée par cette fonction est correcte.

6. *(Examen de janvier 2023)*

   L'*indicatrice d'Euler* est une fonction ϕ telle que pour tout nombre entier strictement positif n, $\phi(n)$ a pour valeur le nombre d'entiers m contenus dans l'intervalle $[1, n]$ tels que $\text{pgcd}(m, n) = 1$, où $\text{pgcd}(m, n)$ est le plus grand commun diviseur de m et de n.

   (a) Donner une implémentation de cette fonction, en supposant que l'on dispose d'une bibliothèque qui implémente déjà la fonction pgcd.

   (b) Calculer la complexité en temps de la fonction obtenue au point (a), en sachant que la complexité en temps de $\text{pgcd}(m, n)$, avec $m \leq n$, est $O(\log m)$.

7. *(Examen d'août 2023)*

   (a) Écrire une fonction calculant le plus petit diviseur strictement supérieur à 1 d'un nombre $n > 1$ donné. Par exemple, pour $n = 51$ et $n = 11$, la fonction doit respectivement retourner 3 et 11. On demande que l'implémentation de cette fonction soit raisonnablement efficace, c'est-à-dire que sa complexité en temps soit meilleure que $O(n)$.

   (b) Calculer la complexité en temps de la fonction obtenue au point (a).

8. *(Examen de janvier 2024)*

   (a) Écrire une fonction `nfp` calculant le nombre de diviseurs premiers d'un nombre $n > 0$ donné. (*Rappel :* Un nombre $d > 0$ est dit *premier* s'il possède exactement deux diviseurs.) Par exemple, on doit avoir $\texttt{nfp}(150) = 3$, $\texttt{nfp}(7) = 1$ et $\texttt{nfp}(1) = 0$.

   Pour résoudre ce problème, on suppose que l'on dispose d'une fonction déjà programmée `premier(`m`)` qui retourne une valeur booléenne indiquant si son argument $m > 0$ est premier. La complexité en temps de la fonction `premier(`m`)` est $O(1)$. On souhaite que l'implémentation de la fonction `nfp(`n`)` soit raisonnablement efficace, c'est-à-dire que sa complexité en temps soit meilleure que $O(n)$.

   (b) Calculer la complexité en temps de la fonction obtenue au point (a).

9. *(Examen de juin 2026)*

   (a) Écrire une fonction prenant en arguments deux entiers strictement positifs, et retournant le nombre de chiffres identiques situés à la même position dans leur écriture décimale, en considérant les positions de la droite vers la gauche. Par exemple, pour les nombres 2026 et 1206, cette fonction doit retourner 1, car seul le chiffre des unités 6 leur est commun. Pour 1234444 et 274444, la fonction doit retourner 5.

(b) Par la méthode des invariants, démontrer que la valeur retournée par cette fonction est correcte.

10. *(Examen de juin 2019)*

(a) Décrire, le plus simplement possible, l'opération effectuée par la fonction suivante.

```
unsigned f(unsigned v)
{
  int n;

  if (!v)
    return 0;

  n = v % 10 == 9 ? 1 : 0;

  return f(v / 10) + n;
}
```

(b) Quelle est la complexité en temps de cette fonction ?

(c) Écrire une fonction réalisant exactement la même opération, mais sans effectuer d'appel récursif.

11. *(Examen d'août 2022)*

(a) Calculer les complexités en temps et en espace de la fonction suivante, en expliquant votre raisonnement.

```
unsigned f(unsigned n)
{
  if (n <= 1)
    return n;

  return f(n / 2) + f(n % 2);
}
```

(b) Écrire une fonction qui calcule exactement la même opération, mais sans effectuer d'appel récursif.

12. *(Examen d'août 2023)*

(a) Expliquer, le plus simplement possible, ce que calcule la fonction suivante :

```
double f(double x, unsigned n)
{
  double a;

  if (!n)
    return x;

  a = f(x, n - 1);

  return a * a;
}
```

(b) Écrire une fonction réalisant exactement la même opération, mais sans effectuer d'appel récursif. *Note :* Il n'est pas permis d'utiliser des fonctions issues de la bibliothèque standard.

13. *(Examen de juin 2024)*

(a) Décrire le plus simplement possible ce que calcule la fonction suivante :

```
unsigned f(unsigned long n)
{
    if (n == 0 || n % 2 != 0)
      return 0;

    return 1 + f(n / 2);
}
```

(b) Quelle est la complexité en espace de cette fonction ?

(c) Écrire une fonction réalisant exactement la même opération que la fonction du point (a), mais sans effectuer d'appel récursif.

14. *(Examen d'août 2024)*

(a) Décrire le plus simplement possible ce que calcule la fonction suivante :

```
unsigned f(unsigned i, unsigned j)
{
    if (i < j)
      return f(j, i);

    if (j == 0)
      return 0;

    return i + f(i, j - 1);
}
```

(b) Quelle est la complexité en espace de cette fonction ?

(c) Écrire une fonction réalisant exactement la même opération que la fonction du point (a), mais sans effectuer d'appel récursif.

15. *(Examen de janvier 2025)*

(a) Décrire le plus simplement possible ce que calcule la fonction suivante :

```
unsigned f(unsigned a, unsigned b)
{
  if (a < b || b == 0)
    return 0;

  return 1 + f(a - b, b);
}
```

(b) Quelle est la complexité en espace de cette fonction ? (Justifier votre réponse.)

(c) Écrire une fonction réalisant exactement la même opération, mais sans effectuer d'appel récursif ni d'itération de boucle.

16. *(Examen de janvier 2026)*

(a) Décrire le plus simplement possible ce que calcule la fonction suivante :

```
unsigned f(unsigned a, unsigned b)
{
  if (!b)
    return 0;

  if (a < b)
     return a;

  return f(a - b, b);
}
```

(b) Quelle est la complexité en espace de la fonction du point (a) ?

(c) Rédiger une fonction réalisant exactement la même opération que la fonction du point (a), mais avec des complexités en temps et en espace toutes deux égales à $O(1)$.

Chapitre 5

Les tableaux et les chaînes de caractères

Nous avons vu aux sections 2.2 et 4.4 que les variables correspondent à des emplacements individuels réservés au sein de la mémoire de l'ordinateur. Nous généralisons à présent cette notion de variable en introduisant la possibilité d'allouer un nombre déterminé d'emplacements de mémoire, accessibles de façon uniforme. Cela permet, par exemple, de représenter un texte en plaçant les caractères qui le composent dans des cases successives de la mémoire. Pour manipuler une telle représentation, il est alors utile de disposer d'un mécanisme permettant d'accéder à n'importe quel caractère sur base de sa position dans le texte. Ce procédé de représentation des textes sera approfondi à la section 5.4.

5.1 Les vecteurs

Un *vecteur*, aussi appelé *tableau unidimensionnel,* ou simplement *tableau* quand il n'y a pas d'ambiguïté, est une collection

$$[x_0 x_1 \cdots x_{n-1}]$$

de n variables x_0, x_1, …, x_{n-1}, possédant chacune le même type. Le nombre $n > 0$ de ces variables est appelé la *taille* du vecteur. Au sein du vecteur, chaque *élément* x_i est caractérisé par son *indice* $i \in [0, n-1]$, et est directement accessible sur base de cet indice. Notons que les indices commencent toujours à 0, et sont consécutifs.

5.1.1 La définition et l'utilisation des vecteurs

En langage C, la construction suivante permet de définir un ou plusieurs vecteurs.

```
[ static ] [ const ] type id[taille] [ = { val1, val2, ... } ]
                     [ , id[taille] [ = { val1, val2, ... } ] ]
                                ⋮
                     [ , id[taille] [ = { val1, val2, ... } ] ] ;
```

Dans cette définition, l'identificateur de chaque vecteur est donné par *id* et sa taille par *taille*. (Dans le cas où plusieurs vecteurs sont définis, ils doivent bien sûr posséder des identificateurs distincts, et leurs tailles peuvent être différentes.) En langage C de base [1], la taille des vecteurs définis de cette façon doit être une constante connue à la compilation du programme. Le type de chaque élément du ou des vecteurs est spécifié par *type*. Notons que les crochets qui entourent *taille* font partie de la syntaxe de la définition et sont obligatoires. Il est permis, mais pas obligatoire, de fournir des valeurs initiales *val1*, *val2*, ... pour les éléments d'un vecteur, entourées d'accolades "{" et "}" et separées par des virgules. Si ces valeurs sont présentes dans la définition, alors on peut omettre d'écrire *taille*, la taille du vecteur étant alors déterminée sur base du nombre de valeurs fournies.

Par exemple, la définition

```
int t[100];
```

est celle d'un vecteur appelé `t`, contenant 100 éléments entiers dont l'indice est numéroté de 0 à 99. Cette définition n'attribue pas de valeur initiale particulière aux éléments de ce vecteur. La définition

```
double g[] = { -0.618034, 1.618034 };
```

est celle d'un vecteur `g` contenant 2 éléments réels.

Les définitions de vecteurs se placent au même endroit que les définitions de variables que nous avons rencontrées jusqu'ici. Il est donc possible de définir des vecteurs locaux ou globaux en écrivant leur définition respectivement à l'intérieur d'un bloc ou à l'extérieur de toute fonction. Les règles de portée et la signification des mots-clés optionnels `static` et `const` sont identiques à celles des définitions de variables, le second signifiant que les éléments du vecteur ne peuvent pas être modifiés.

1. Il existe des extensions du langage, supportées par certains compilateurs, qui permettent de spécifier des vecteurs de taille variable. Nous introduirons au chapitre 6 un procédé plus général permettant d'allouer une quantité de mémoire qui n'est connue qu'au cours de l'exécution du programme.

Après avoir défini un vecteur, il est possible de manipuler ses éléments grâce à des expressions de la forme

$$\boxed{id\texttt{[}expr\texttt{]}},$$

où *id* est l'identificateur du vecteur, et l'évaluation de *expr* fournit l'indice de l'élément concerné. Cette expression est également une valeur à gauche, et peut donc être utilisée pour modifier la valeur de l'élément dans une expression d'affectation.

Par exemple, le fragment de code suivant définit un tableau de 100 éléments entiers, et leur attribue les valeurs 2, 4, 6, …, 200.

```
int t[100];
unsigned i;

for (i = 0; i < 100; i++)
  t[i] = 2 * (i + 1);
```

En langage C, lorsqu'on évalue une expression de la forme $\boxed{id\texttt{[}expr\texttt{]}}$ ou quand on l'utilise comme valeur à gauche, il est important de garantir que le résultat de l'évaluation de *expr* correspond toujours à un indice valide, c'est-à-dire appartenant à l'intervalle $[0, n-1]$ où n est la taille du tableau *id*. C'est le programmeur qui est responsable de cela ; aucune vérification de l'indice n'est effectuée lors des accès au tableau qui ont lieu pendant l'exécution du programme. Ne pas respecter cette règle est une erreur de programmation fréquente. Ses conséquences peuvent être de corrompre le contenu de la mémoire de l'ordinateur, ou bien provoquer l'arrêt de l'exécution du programme par le système d'exploitation suite à la détection d'un accès invalide à la mémoire (*erreur de segmentation, segmentation fault*).

5.1.2 Application : le crible d'Ératosthène

Pour illustrer l'utilisation des vecteurs, nous considérons le problème qui consiste à rechercher tous les nombres premiers inférieurs ou égaux à une borne n donnée, par exemple $n = 10^6$. Un nombre entier est dit premier s'il possède exactement deux diviseurs, c'est-à-dire 1 et lui-même.

Pour rechercher des nombres premiers, il n'est pas nécessaire d'utiliser des vecteurs ; on peut par exemple suivre une approche similaire à celle du calcul des nombres parfaits mise en œuvre au chapitre 3, et énumérer tous les nombres de 1 à n en déterminant explicitement quels sont leurs diviseurs.

On peut cependant obtenir une solution plus efficace en construisant un tableau contenant tous les nombres de l'intervalle considéré qui sont potentiellement premiers, et en enlevant de ce

tableau tous les multiples d'un nombre p dès que l'on a déterminé que celui-ci est effectivement premier. Il s'agit du principe du *crible d'Ératosthène*, qui est un algorithme de recherche de nombres premiers connu depuis plus de 2000 ans.

Plus précisément, cet algorithme effectue les opérations suivantes, pour une valeur donnée (supérieure ou égale à 2) de la borne n.

1. On initialise un ensemble P avec l'ensemble de tous les nombres entiers appartenant à l'intervalle $[2, n]$. Cet ensemble contient tous les nombres potentiellement premiers de l'intervalle. Notons que nous avons exclu d'emblée le nombre 1, que nous savons ne pas être premier.
2. On sait que le plus petit nombre p contenu dans P est premier. En effet, s'il était divisible par un autre nombre que 1 et lui-même, alors il existerait un nombre premier d tel que $1 < d < p$, et p aurait été éliminé de l'ensemble lors de l'étape qui aurait détecté d.

   On affiche donc p, et l'on retire de l'ensemble P tous ses multiples p, $2p$, $3p$, ...
3. Tant qu'il reste des nombres dans P, on recommence l'étape 2.

L'étape 2 de cet algorithme peut être améliorée : pour une valeur donnée de p, on sait que les multiples $2p$, $3p$, ..., $(p-1)p$ de ce nombre ont nécessairement déjà été enlevés de l'ensemble P aux étapes précédentes. En effet, les nombres 2, 3, ..., $p-1$ sont tous des multiples de nombres premiers strictement inférieurs à p. On peut donc se limiter, à l'étape 2 de l'algorithme, à n'enlever de l'ensemble P que le nombre p et ses multiples p^2, $(p+1)p$, $(p+2)p$, ...

Pour implémenter cet algorithme en C, la seule difficulté est celle de représenter l'ensemble P. On remarque que pour chaque entier situé dans l'intervalle $[2, n]$, il suffit de retenir une valeur booléenne indiquant si cet entier appartient ou non à P. Une solution simple consiste donc à définir un vecteur `B` de valeurs booléennes, tel que pour tout $i \in [2, n]$, on a `B[`i`]` $= 1$ si $i \in$ `P`, et `B[`i`]` $= 0$ si $i \notin$ `P`.

L'étape 2 de l'algorithme nécessite de rechercher la plus petite valeur contenue dans P, ce qui revient à balayer le vecteur `B` jusqu'à trouver le premier élément non nul. On remarque que les valeurs de p traitées par l'algorithme sont strictement croissantes ; on peut donc implémenter ce balayage en le commençant juste après la valeur précédente de p plutôt qu'au début du vecteur. Cette solution permet également de ne pas être obligé de retirer p du tableau à l'étape 2.

Une implémentation en C du crible d'Ératosthène basée sur ces principes est donnée à la figure 5.1. Notons que dans ce programme, les deux premiers éléments `B[0]` et `B[1]` du vecteur `B` sont inutilisés. Il y aurait eu moyen de définir `B` avec une taille inférieure de deux unités, de telle façon que pour tout $i \in [2, n]$, on ait `B[`$i-2$`]` $= 1$ si $i \in$ `P`, et `B[`$i-2$`]` $= 0$ si $i \notin$ `P`. Cette solution n'a pas été retenue car elle n'apporte qu'un gain en espace marginal, et elle complique inutilement la lisibilité du programme.

```c
#include <stdio.h>

#define N_MAX 999999

static unsigned char b[N_MAX + 1];

int main()
{
  unsigned i, p;

  for (i = 2; i <= N_MAX; i++)
    b[i] = 1;

  for (p = 2; p <= N_MAX; p++)
    {
      if (!b[p])
        continue;

      printf("%u\n", p);

      for (i = p; (unsigned long long) i * p <= N_MAX; i++)
        b[i * p] = 0;
    }

  return 0;
}
```

FIGURE 5.1 – Crible d'Ératosthène

Le type `unsigned char` employé pour représenter les valeurs booléennes est un compromis entre l'efficacité de cette représentation et sa facilité d'utilisation. Si l'on souhaitait employer ce programme pour de très grandes valeurs [2] de n, il serait utile de définir une structure de données qui ne nécessiterait de mémoriser qu'un seul bit d'information pour chaque élément de P, mais une telle structure est un peu plus difficile à mettre en œuvre.

Enfin, la raison pour laquelle une conversion explicite de type figure dans l'expression évaluant le gardien de boucle `i * p <= N_MAX` est que le calcul du produit `i * p` peut conduire à un dépassement arithmétique pour de grandes valeurs de la variable `p`. Afin d'éviter que cela ne se produise, on force ce calcul à être effectué avec des valeurs de type `unsigned long long`.

5.2 Le passage de tableaux à une fonction

5.2.1 Principes

Il est tout à fait possible, comme pour tous les autres types de données que nous avons étudiés, de passer un ou plusieurs arguments de type vecteur à une fonction. Dans la définition ou la déclaration de cette fonction, on utilise alors la forme

type id`[]`

pour déclarer le ou les paramètres correspondants, où *id* est le nom du paramètre (permettant d'y faire référence dans le corps de la fonction), et *type* est le type des éléments du vecteur. Signalons qu'il est permis de spécifier la taille du vecteur entre les crochets, mais étant donné que, comme nous allons le voir dans un instant, cette taille n'est pas utilisée, il est plus simple de l'omettre.

La discussion du mécanisme de passage d'un tableau en argument à une fonction permet d'introduire le concept important de *pointeur*, qui sera étudié en détail au chapitre 6. Un pointeur correspond à l'*endroit* où se trouve une donnée dans la mémoire de l'ordinateur. Lorsqu'un tableau est passé en argument à une fonction, la donnée transmise à cette fonction est un pointeur vers ce tableau, et non le contenu de celui-ci.

Par conséquent, si la fonction modifie les éléments du tableau, alors cette modification aura un effet qui ne sera pas seulement local au corps de cette fonction, mais pourra aussi être observé par le code qui invoque celle-ci.

2. Signalons qu'il existe de bien meilleurs algorithmes pour tester la primalité de grands nombres.

```c
#include <stdio.h>

static void initialiser_vecteur(int v[], unsigned n)
{
  unsigned i;

  for (i = 0; i < n; i++)
    v[i] = i + 1;
}

static void afficher_vecteur(const int v[], unsigned n)
{
  unsigned i;

  for (i = 0; i < n; i++)
    printf(" %d", v[i]);

  printf("\n");
}

#define NB 20

int main()
{
  int t[NB];

  initialiser_vecteur(t, NB);
  afficher_vecteur(t, NB);

  return 0;
}
```

FIGURE 5.2 – Mécanisme de passage d'un tableau à une fonction

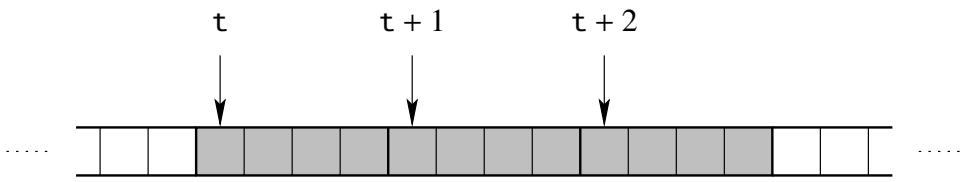


FIGURE 5.3 – Représentation d'un vecteur en mémoire

5.2.2 Illustration

Ce mécanisme est illustré par le programme de la figure 5.2. Celui-ci définit deux procédures `initialiser_vecteur` et `afficher_vecteur` chargées respectivement d'initialiser et d'afficher les éléments d'un vecteur d'entiers de taille donnée. Ces procédures sont invoquées depuis la fonction `main`. La procédure d'initialisation place dans les éléments successifs du vecteur les entiers 1, 2, ..., n, où n est la taille du vecteur. Étant donné que cette fonction reçoit comme premier argument un pointeur vers le vecteur `t` alloué par la fonction `main`, son effet consiste à modifier les éléments de ce vecteur, qui sont ensuite affichés par `afficher_vecteur`. Cela montre que l'effet des modifications effectuées par `initialiser_vecteur` n'est pas limité au corps de cette fonction. Il s'agit en fait d'un cas particulier de *passage par variable*, qui permet à une fonction de transmettre des données au code qui l'invoque par l'intermédiaire de ses arguments. Nous verrons au chapitre 6 un mécanisme permettant d'implémenter ce passage par variable dans un cadre plus général.

5.2.3 L'arithmétique sur les pointeurs

En langage C, les éléments d'un vecteur sont toujours placés dans des cases consécutives de la mémoire de l'ordinateur. Un pointeur vers un tableau représente l'endroit ou se trouve le *premier élément* de celui-ci.

De plus, si `p` est un pointeur vers un tableau et i une valeur entière, alors l'expression

`p + i`

fournit un pointeur vers l'élément d'indice i du tableau, c'est-à-dire son $(i + 1)$-ème étant donné que les éléments d'un vecteur sont indicés à partir de 0.

Par exemple, la figure 5.3 montre comment le vecteur défini par l'instruction

```
int t[3];
```

est représenté dans un ordinateur basé sur l'architecture x86-64. Chaque élément de ce vecteur est constitué de 32 bits d'information, correspondant à 4 octets de la mémoire de l'ordinateur.

```
static void permuter_consecutifs(int v[])
{
  int aux;

  aux = v[0];
  v[0] = v[1];
  v[1] = aux;
}

int main()
{
  int t[100];

    ⋮

  permuter_consecutifs(t + 10);

    ⋮

}
```

FIGURE 5.4 – Arithmétique de pointeurs

Le vecteur occupe donc dans son ensemble 12 octets. Les expressions `t`, `t + 1` et `t + 2` représentent des pointeurs vers respectivement le premier, le cinquième et le neuvième de ces octets.

Ce mécanisme d'arithmétique de pointeurs est illustré par le programme de la figure 5.4. Ce programme définit une procédure `permuter_consecutifs` qui accepte en argument un vecteur d'entiers, et a pour effet d'en permuter les deux premiers éléments. Cela est équivalent à dire que cette procédure reçoit en argument un pointeur vers un entier, et permute la valeur de cet entier avec celle de celui qui le suit immédiatement en mémoire. Dans la fonction `main`, cette procédure `permuter_consecutifs` est invoquée avec l'argument `t + 10`, dont l'évaluation fournit un pointeur vers le onzième élément du vecteur `t`. Cet appel à la procédure aura donc pour effet de permuter la valeur des onzième et douzième éléments `t[10]` et `t[11]` de ce vecteur.

5.3 Le tri d'un vecteur

Les constructions introduites aux sections 5.1 et 5.2 permettent maintenant d'aborder des problèmes algorithmiques un peu plus complexes que ceux que nous avons traités jusqu'ici. Nous nous intéressons ici au problème de *trier* un vecteur, c'est-à-dire d'en réordonner les éléments par rapport à une relation d'ordre donnée. Par simplicité, nous considérons dans cette section

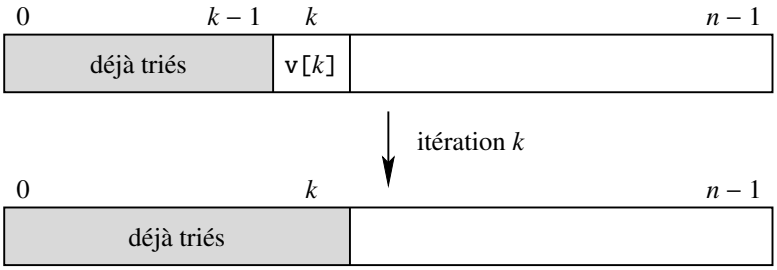


FIGURE 5.5 – Itération du tri par insertion

uniquement le cas de vecteurs d'entiers que l'on souhaite trier par ordre croissant de valeur. Les solutions que nous allons développer ont cependant une portée plus générale et s'appliquent à n'importe quel type de données à condition de disposer d'une fonction de comparaison implémentant une relation d'ordre total sur ces données.

Le problème de trier un vecteur possède un très grand nombre de solutions qui ont fait l'objet de travaux approfondis. Dans le cadre de ce cours, nous nous limitons à en présenter deux solutions, la première étant simple et applicable au cas de vecteurs de petite taille, et la seconde plus élaborée mais présentant une meilleure efficacité. Le lecteur intéressé trouvera également dans les exercices de la section 5.6 la description d'autres algorithmes de tri.

5.3.1 Le tri par insertion

Il existe de nombreux algorithmes de tri simples : le *tri à bulles (bubble sort)*, le *tri par sélection (selection sort)*, le *tri à peigne (comb sort)*, ... Parmi ceux-ci, le *tri par insertion (insertion sort)* présente plusieurs avantages : il est simple à implémenter, et il est efficace lorsque les données d'entrée sont déjà presque entièrement triées.

Pour trier un vecteur `v` de taille n, l'algorithme de tri par insertion effectue $n-1$ itérations successives. Celles-ci sont respectivement chargées de trier les 2, 3, ... jusqu'aux n premiers éléments du vecteur.

Examinons la k-ème itération de cet algorithme, pour $k \in [1, n-1]$. Avant d'effectuer cette itération, les k premiers éléments du vecteur sont triés [3]. Le but de l'itération est alors d'y insérer le $(k+1)$-ème élément `v[k]`, de façon à ce que les $k+1$ premiers éléments du vecteur deviennent triés. Une représentation graphique de cette opération est donnée à la figure 5.5.

3. Remarquons que dans le cas de la première itération ($k = 1$), ces k premiers éléments forment un ensemble de taille 1, qui est donc naturellement déjà trié.

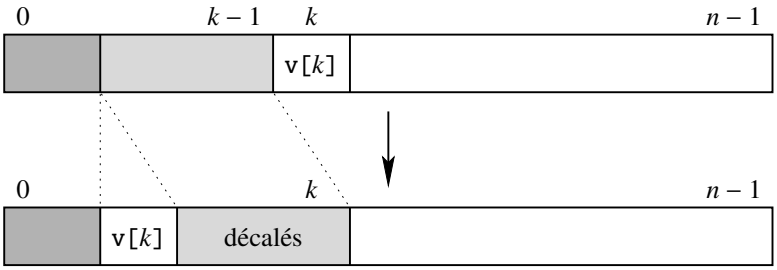


FIGURE 5.6 – Insertion d'une valeur

```
/* Tri du vecteur v de taille n, par ordre croissant */

void tri_insertion(int v[], int n)
{
  int k, i, a;

  for (k = 1; k < n; k++)
    {
      a = v[k];
      for (i = k - 1; i >= 0 && v[i] > a; i--)
        v[i + 1] = v[i];

      v[i + 1] = a;
    }
}
```

FIGURE 5.7 – Tri par insertion

Pour insérer `v[`k`]` dans `v[0 :` $k-1$`]`, c'est-à-dire le sous-vecteur de `v` commençant à l'indice 0 et se terminant à l'indice $k-1$, qui est supposé déjà trié, on procède de la façon suivante.

1. On recopie la valeur de `v[`k`]` dans une variable auxiliaire a.
2. On balaie de la droite vers la gauche les éléments du sous-vecteur `v[0 :` $k-1$`]` qui sont strictement supérieurs à a, et on les décale chacun d'une position vers la droite.
3. On insère la valeur sauvegardée dans a immédiatement à gauche du dernier élément décalé (ou à la position d'indice k si aucun décalage n'a été effectué).

Le principe de cet algorithme est illustré à la figure 5.6. L'implémentation complète en langage C du tri par insertion est donnée à la figure 5.7. Notons que cette implémentation figure dans une fonction, ce qui permet de l'utiliser dans n'importe quel programme nécessitant de trier

un vecteur d'entiers [4].

La fonction `tri_insertion` prend en arguments un pointeur `v` vers un vecteur d'entiers, et le nombre `n` d'éléments de ce vecteur. Elle est composée de deux boucles imbriquées : la boucle externe, sur `k`, compte les $n-1$ itérations à effectuer. La boucle interne, sur `i`, effectue l'opération de décalage dans le sous-vecteur `v[0 : k-1]`. Cette opération se termine soit lorsqu'on rencontre une valeur `v[i]` qui est inférieure ou égale à `a`, soit après avoir décalé la première case `v[0]`. Remarquons que dans ce dernier cas, la valeur de `i` devient négative au moment d'évaluer pour la dernière fois le gardien de la boucle interne. Il s'agit de la raison pour laquelle les variables `i`, `k` et le paramètre `n` sont déclarés de type `int` et non `unsigned`. Remarquons pour terminer que dans le cas particulier où aucun décalage ne doit être effectué, l'instruction `v[i + 1] = a` replace la valeur de `v[k]` à l'endroit précis où elle a été lue, et n'a donc aucun effet. Cela n'est pas problématique, et constitue une solution plus simple que de traiter séparément ce cas particulier.

On peut essayer de caractériser l'efficacité de l'algorithme de tri par insertion en calculant sa complexité en temps et en espace. Pour une valeur donnée de la taille `n` du vecteur, le programme de la figure 5.7 effectue $n-1$ itérations de la boucle sur `k`. Pour chaque valeur de `k`, la boucle interne peut être amenée à décaler dans le pire des cas la totalité des `k` valeurs qui composent le sous-vecteur `v[0 : k - 1]`. Cela conduit à effectuer un maximum de $k+1$ fois chaque opération qui figure dans cette boucle interne. On obtient donc une complexité en temps égale à

$$\begin{aligned} O(2+3+\cdots+n) &= O\left(\frac{n(n+1)}{2}-1\right) \\ &= O\left(n^2\right). \end{aligned}$$

Ce comportement quadratique est atteint dans la situation la plus défavorable, qui correspond au cas où l'opération de décalage doit à chaque fois se poursuivre jusqu'au début du vecteur. Cela se produit lorsque les éléments du vecteur sont initialement triés par ordre décroissant de valeur. En revanche, le comportement de l'algorithme de tri par insertion est meilleur lorsqu'on l'applique à des vecteurs qui sont déjà presque entièrement triés.

Déterminer la complexité en espace de l'algorithme revient à mesurer la quantité de mémoire nécessaire à son exécution. Il existe plusieurs notions de complexité en espace, selon que l'on compte ou non la place occupée par les données d'entrée fournies au programme. Dans le cas présent, la manipulation des éléments du vecteur est essentielle au fonctionnement de l'algorithme, et cela a donc un sens de considérer que la quantité de mémoire occupée par ce vecteur doit être comptée. Cette quantité de mémoire est proportionnelle à la taille `n` du vecteur. Il faut lui ajouter la place occupée par les variables du programme dans la mémoire de l'ordinateur, qui est bornée indépendamment de `n`. Enfin, ce programme n'effectue pas d'appel récursif. On

4. Nous n'avons pas encore vu les mécanismes permettant de rendre un tel code capable de trier des données de n'importe quel type.

obtient donc au total une complexité en espace égale à

$$O(\mathrm{n}).$$

Si, en revanche, on ne compte pas l'espace occupé par les données d'entrée, mais uniquement la mémoire additionnelle utilisée par l'algorithme, la complexité en espace de ce dernier vaut

$$O(1).$$

5.3.2 Le tri par fusion

La complexité en temps quadratique de l'algorithme de tri par insertion signifie en pratique qu'il n'est utilisable que pour traiter des vecteurs de petite taille. Nous cherchons maintenant à développer une solution plus efficace.

L'algorithme que nous allons présenter ici est un exemple d'application d'une stratégie générale permettant de résoudre certains types de problèmes algorithmiques, appelée l'approche *diviser pour régner (divide and conquer)*. Cette approche consiste à décomposer une instance du problème en plusieurs instances de plus petite taille, appliquer récursivement la même procédure de résolution du problème à chacune d'entre elles, et combiner ensuite les solutions ainsi obtenues de façon à construire une solution au problème de départ.

Pour le problème consistant à trier les éléments d'un vecteur de taille n par ordre croissant, cette approche mène à l'algorithme suivant :

1. Si $n = 1$, alors il n'y a rien à faire, car un vecteur qui ne contient qu'un seul élément est déjà naturellement trié.

2. Si $n > 1$, alors :
   2.1 On divise le vecteur en deux sous-vecteurs de tailles égales, ou égales à une unité près si n est impair.

   2.2 On trie récursivement, à l'aide du même algorithme, chacun de ces deux sous-vecteurs.

   2.3 On fusionne les deux sous-vecteurs triés de façon à obtenir une version triée du vecteur complet.

La figure 5.8 illustre les opérations de division et de fusion de vecteurs réalisées par cet algorithme lors du tri d'un vecteur contenant initialement [3, 5, 1, 4, 2].

Le cœur de cet algorithme réside essentiellement dans l'opération de fusion de deux sous-vecteurs effectuée à l'étape 2.3, d'où son nom de *tri par fusion (merge sort)*. Cette opération

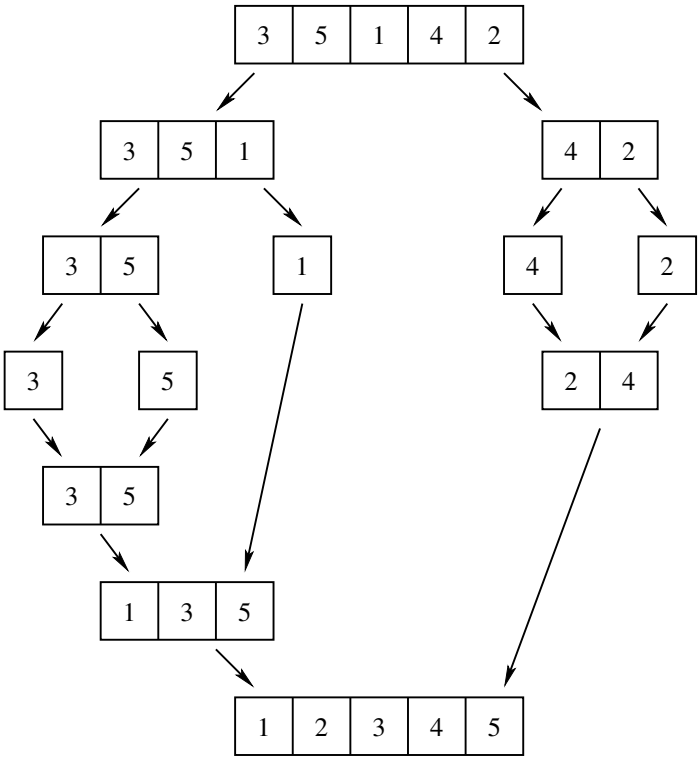


FIGURE 5.8 – Exemple de tri par fusion

est réalisée de la façon suivante. Soient v_1 and v_2 deux vecteurs que l'on souhaite fusionner. On suppose que les éléments de v_1 et de v_2 sont déjà triés. L'objectif consiste à construire un vecteur v trié dans lequel figurent une et une seule fois chaque élément de v_1 et chaque élément de v_2.

Le premier élément de v, c'est-à-dire celui de plus petite valeur, correspond nécessairement au premier élément de v_1 ou de v_2. On peut donc comparer ces deux éléments, et transférer le plus petit d'entre eux dans le vecteur v. Il suffit ensuite de recommencer la même opération avec les éléments restants de v_1 et de v_2, jusqu'à ce que ces deux vecteurs soient vides. Par ce procédé, la fusion de v_1 et v_2 s'effectue en un nombre d'opérations qui est proportionnel au nombre total d'éléments contenus dans ces deux vecteurs.

L'implémentation pratique de cet algorithme de fusion se heurte à deux obstacles. Premièrement, on ne dispose pas d'un mécanisme permettant de retirer un élément d'un vecteur. Ensuite, lorsqu'on fusionne deux sous-vecteurs v_1 et v_2 extraits d'un vecteur v afin d'obtenir une version triée de v, il faut pouvoir écrire le résultat de cette opération quelque part en mémoire sans que cela n'affecte v_1 ni v_2. Ces problèmes peuvent être résolus [5] de la façon suivante.

— Avant de fusionner v_1 et v_2, on recopie le contenu de ces vecteurs vers un espace mémoire préalloué, à partir duquel on lira ensuite les données à fusionner.

— Pendant l'opération de fusion, on munit chacun des vecteurs v_1 et v_2 d'un *curseur*, qui est un nombre entier représentant l'indice du prochain élément qui en sera extrait. L'opération consistant à retirer un élément d'un vecteur se réduit alors à incrémenter le curseur correspondant.

— On munit également d'un curseur l'emplacement de mémoire où le résultat de la fusion doit être placé. Ce curseur représente à chaque étape de l'algorithme l'indice où le prochain élément du résultat doit être écrit.

L'algorithme de fusion d'un vecteur v_1 de taille n_1 et d'un vecteur v_2 de taille n_2 en un vecteur v_3 est alors le suivant. Les curseurs associés à v_1, v_2 et v_3 sont respectivement notés i_1, i_2 et i_3.

1. On initialise i_1, i_2 et i_3 à 0.

2. Tant que $i_1 < n_1$ ou $i_2 < n_2$ (c'est-à-dire, tant qu'il reste des éléments dans au moins un des vecteurs v_1 et v_2) :

   — Si $i_1 \geq n_1$ (c'est-à-dire, s'il ne reste des éléments que dans v_2), alors on recopie l'élément $v_2[i_2]$ dans $v_3[i_3]$, et l'on incrémente i_2 et i_3.

5. La solution présentée ici a l'avantage de la simplicité. Il existe des solutions plus élaborées, qui ne nécessitent pas de mémoire supplémentaire ou qui minimisent le nombre d'opérations de transfert de valeurs entre vecteurs.

— Si $i_2 \geq n_2$ (c'est-à-dire, s'il ne reste des éléments que dans v_1), alors on recopie l'élément $v_1[i_1]$ dans $v_3[i_3]$, et l'on incrémente i_1 et i_3.

— Si $i_1 < n_1$ et $i_2 < n_2$ (c'est-à-dire, s'il reste des éléments à la fois dans v_1 et v_2), alors on recopie la plus petite valeur parmi $v_1[i_1]$ et $v_2[i_2]$ dans $v_3[i_3]$. On incrémente alors i_3, ainsi que i_1 ou i_2 selon que cette plus petite valeur provenait de v_1 ou v_2.

Une implémentation en langage C de l'algorithme de tri par fusion est donnée aux figures 5.9 et 5.10. Chacune des trois procédures qui composent ce programme est accompagnée d'un commentaire qui spécifie l'opération qu'elle effectue et ses modalités d'utilisation.

La procédure `tri_fusion` est la fonction principale de ce programme ; elle peut être invoquée pour trier n'importe quel vecteur `v` d'entiers qui lui est passé en argument. Cette procédure a besoin de connaître la taille `n` de ce vecteur, et nécessite un espace auxiliaire `v_aux` de même taille. Celui-ci sera utilisé pour y recopier le contenu des vecteurs à fusionner avant chaque opération de fusion, comme expliqué précédemment. L'opération effectuée par la procédure `tri_fusion` est très simple, et se limite à invoquer la procédure `tri_sousvecteur` avec des bornes correspondant à l'entièreté de `v`.

La procédure `tri_sousvecteur` est celle qui est appelée récursivement. Elle est chargée de trier un sous-vecteur donné du vecteur `v` qui lui est passé en argument, correspondant aux éléments situés entre les indices `a` et `b`. Tout comme `tri_fusion`, cette procédure nécessite un argument supplémentaire `v_aux` correspondant à un espace de travail qu'elle peut utiliser pour copier des résultats intermédiaires. Dans cet espace de travail, la procédure n'utilise que les emplacements situés entre les indices `a` et `b`.

Les premières instructions de la procédure `tri_sousvecteur` commencent par calculer le nombre `n` d'éléments à trier. Si celui-ci est plus petit que 2, alors le sous-vecteur est déjà trié et la procédure se termine. Sinon, on calcule un indice *pivot* `c` situé au milieu du sous-vecteur à trier. Ce calcul est effectué de façon à ce que les sous-vecteurs `v[a:c - 1]` et `v[c:b]` soient de même taille si `n` est pair, et que le premier contienne un élément de plus que le second si `n` est impair. Ensuite, la procédure s'invoque récursivement pour chacun de ces deux sous-vecteurs.

À ce stade, le vecteur `v` contient un sous-vecteur trié entre les indices `a` et `c - 1`, et un autre entre les indices `c` et `b`. Il reste maintenant à fusionner ces sous-vecteurs. Cette opération est réalisée en recopiant préalablement les éléments de ces sous-vecteurs vers l'espace auxiliaire `v_aux` grâce à une boucle `for`, et en invoquant ensuite la procédure `fusion`.

Cette procédure `fusion` accepte 7 arguments : deux vecteurs `v1` et `v2` à fusionner, ainsi que les bornes sur les indices des éléments de ces vecteurs à prendre en compte, et un vecteur `v3` destiné à accueillir le résultat de la fusion. Remarquons que pour l'appel à `fusion` effectué par la procédure `tri_sousvecteur`, ce dernier argument prend pour valeur `v + a`. Cela signifie que le résultat de l'opération de fusion doit être écrit dans le vecteur `v` à partir de l'indice `a`. La procédure `fusion` implémente directement l'algorithme de fusion précédemment décrit.

```
/*
   Fusionne les vecteurs v1[a1:b1] et v2[a2:b2], supposés triés,
   et place le résultat dans le vecteur v3[], supposé distinct
   de v1[] et v2[] ainsi que de taille suffisante
*/

static void fusion(const int v1[], unsigned a1, unsigned b1,
                   const int v2[], unsigned a2, unsigned b2,
                   int v3[])
{
  unsigned i1, i2, i3;

  for (i1 = a1, i2 = a2, i3 = 0; i1 <= b1 || i2 <= b2;)
    if (i1 > b1)
      v3[i3++] = v2[i2++];
    else
      if (i2 > b2)
        v3[i3++] = v1[i1++];
      else
        v3[i3++] = v1[i1] < v2[i2] ? v1[i1++] : v2[i2++];
}

/*
   Trie le vecteur v[a:b], par ordre croissant, en utilisant
   le vecteur v_aux[a:b] comme espace de travail auxiliaire
*/

static void tri_sousvecteur(int v[], unsigned a, unsigned b, int v_aux[])
{
  unsigned n, c, i;

  n = b - a + 1;
  if (n < 2)
    return;

  c = a + (n + 1) / 2;
  tri_sousvecteur(v, a, c - 1, v_aux);
  tri_sousvecteur(v, c, b, v_aux);

  for (i = a; i <= b; i++)
    v_aux[i] = v[i];

  fusion(v_aux, a, c - 1,
         v_aux, c, b,
         v + a);
}
```

FIGURE 5.9 – Tri par fusion (1/2)

```
/*
   Trie le vecteur v de taille n > 0, par ordre croissant, en utilisant
   le vecteur v_aux, de même taille, comme espace de travail auxiliaire
 */

void tri_fusion(int v[], unsigned n, int v_aux[])
{
  tri_sousvecteur(v, 0, n - 1, v_aux);
}
```

FIGURE 5.10 – Tri par fusion (2/2)

On peut maintenant chercher à déterminer la complexité en temps et en espace de l'algorithme de tri par fusion, dans le but d'estimer si celui-ci constitue une meilleure solution que le tri par insertion.

Chaque appel à la procédure `tri_sousvecteur` est amené à recopier et à fusionner un nombre d'éléments correspondant à ceux du sous-vecteur à trier. Ces opérations présentent un coût en temps qui est linéaire en ce nombre d'éléments. Les autres opérations effectuées par `tri_sousvecteur`, à l'exception des appels récursifs, représentent un coût borné.

Pour trier un vecteur dont la taille n est une puissance de 2, c'est-à-dire telle que $n = 2^k$ avec $k \in \mathbb{N}$, les opérations de copie et de fusion doivent traiter

— 2 vecteurs de taille $\frac{n}{2}$,

— 4 vecteurs de taille $\frac{n}{4}$,

— 8 vecteurs de taille $\frac{n}{8}$,

$\vdots$

— n vecteurs de taille 1.

Le coût total de ces opérations vaut donc·

$$O(kn) = O(n \log n).$$

Pour trier un vecteur dont la taille n n'est pas une puissance de 2, le nombre d'opérations nécessaires est borné par celui du tri d'un nombre d'éléments égal à la plus petite puissance de deux supérieure ou égale à n. On a donc en définitive une complexité en temps égale à

$$O(n \log n),$$

qui est bien meilleure que la complexité quadratique de l'algorithme de tri par insertion.

La complexité en espace de l'algorithme de tri par fusion se détermine en remarquant que l'espace mémoire nécessaire au tri d'un vecteur de taille n correspond à celui de $2n$ éléments : ceux du vecteur à trier, et ceux de l'espace de travail auxiliaire. Il faut ajouter à cela l'espace occupé par les variables du programme, qui croît linéairement avec la profondeur de récursion de la procédure `tri_sousvecteur`. Cette profondeur est égale à

$$\lceil \log_2 n \rceil + 1,$$

où $\lceil x \rceil$ dénote le plus petit entier supérieur ou égal à x, ce qui conduit à une complexité en espace égale à

$$O(n + \log n) = O(n).$$

5.4 Les chaînes de caractères

Beaucoup de programmes sont amenés à manipuler des textes. C'est bien sûr le cas des logiciels de traitement de texte ou de messagerie électronique, mais on peut également prendre comme exemple un navigateur WWW, qui reçoit d'un serveur des données à traiter qui sont principalement textuelles, ou encore un compilateur, qui traite des fichiers sources composés de texte.

Dans la mémoire de l'ordinateur, un texte est représenté par une *chaîne de caractères (character string)*. Il s'agit essentiellement d'un vecteur dont chaque élément correspond à un caractère du texte. Les caractères qui peuvent être employés font partie d'un *alphabet* dont les détails dépendent de l'environnement utilisé. Dans ce cours, nous nous limiterons au procédé le plus simple permettant de représenter des textes. Des techniques plus évoluées ont cependant été développées, notamment pour gérer efficacement des alphabets de grande taille tels que ceux employés pour l'écriture du chinois, du japonais, et d'autres langues. Leur étude est abordée dans le cours d'*Organisation des ordinateurs*.

5.4.1 Principes

En langage C, une chaîne de caractères est représentée par un vecteur dont chaque élément est de type `char`. Comme expliqué à la section 2.2.1, une valeur de ce type occupe un octet dans la mémoire de l'ordinateur, et le caractère qu'elle représente appartient donc à un alphabet comprenant au plus 256 symboles. Le jeu de caractères disponibles dépend de l'environnement, mais nous faisons l'hypothèse qu'il contient tous les symboles dont nous avons besoin pour programmer ainsi que pour écrire des textes en français ou en anglais. Un exemple de jeu de caractères appelé le *code ASCII (American Standard for Information Interchange)*, sur lequel

32		48	0	64	@	80	P	96	‘	112	p
33	!	49	1	65	A	81	Q	97	a	113	q
34	"	50	2	66	B	82	R	98	b	114	r
35	#	51	3	67	C	83	S	99	c	115	s
36	$	52	4	68	D	84	T	100	d	116	t
37	%	53	5	69	E	85	U	101	e	117	u
38	&	54	6	70	F	86	V	102	f	118	v
39	’	55	7	71	G	87	W	103	g	119	w
40	(	56	8	72	H	88	X	104	h	120	x
41	)	57	9	73	I	89	Y	105	i	121	y
42	*	58	:	74	J	90	Z	106	j	122	z
43	+	59	;	75	K	91	[	107	k	123	{
44	,	60	<	76	L	92	\	108	l	124	|
45	-	61	=	77	M	93	]	109	m	125	}
46	.	62	>	78	N	94	^	110	n	126	~
47	/	63	?	79	O	95	_	111	o		

FIGURE 5.11 – Le code ASCII

se basent la grande majorité des procédés d’encodage de caractères modernes, est donné à la figure 5.11.

Nous avons vu à la section 5.1.1 que la définition d’un vecteur requiert d’en préciser la taille. On souhaite cependant pouvoir définir une variable capable de contenir une chaîne de caractères sans devoir fixer précisément la longueur de celle-ci. Par exemple, pour une application destinée à traiter le fichier du personnel d’une entreprise, il doit être possible de définir des variables correspondant à un nom et un prénom, dont la longueur n’est clairement pas la même d’une personne à une autre. On résout ce problème en allouant un vecteur d’une taille donnée, et en autorisant d’y placer des chaînes de taille variable, en veillant bien sûr à ne pas dépasser la capacité allouée. Cela entraîne que la représentation d’une chaîne doit contenir, en plus des caractères qui la composent, une information sur sa longueur.

La solution mise en œuvre par le langage C consiste à faire suivre le dernier caractère d’une chaîne par un symbole spécial appelé *terminateur*, qui indique la fin de la chaîne. Ce terminateur correspond au caractère fourni par l’expression constante `'\0'`. Sa valeur numérique est nulle.

Il est important de tenir compte de la présence du terminateur quand on alloue une chaîne de caractères. Par exemple, la définition

```
char message[10];
```

alloue un vecteur `message` capable de retenir une chaîne composée d’au plus 9 caractères.

Le contenu d’une chaîne de caractères peut être initialisé dans la définition de celle-ci grâce à la syntaxe particulière suivante.

```
char id[] = "texte";
```

Celle-ci définit un vecteur d'identificateur *id*, contenant initialement les caractères qui composent *texte*, suivis du caractère terminateur. Le nombre d'octets alloués correspond donc à la longueur de *texte* plus un.

Cela implique, par exemple, que les trois définitions suivantes sont équivalentes.

```
char message[] = "Hello";
```

```
char message[] = { 'H', 'e', 'l', 'l', 'o', '\0' };
```

```
char message[] = { 72, 101, 108, 108, 111, 0 };
```

Dans cet exemple, il faut bien comprendre que le symbole "=" qui apparaît dans les trois définitions fait partie de la syntaxe de ces définitions, et ne désigne pas l'opérateur d'affectation. Après avoir défini le vecteur `message` d'une de ces trois façons, il n'est notamment pas permis dans la suite du programme d'écrire une instruction telle que celle-ci.

```
message = "Bonjour"; // Invalide!
```

Il n'existe en effet pas en langage C d'opérateur permettant de modifier en une seule opération les éléments successifs d'un vecteur. Pour effectuer une telle modification, il faut écrire des instructions qui affectent individuellement une valeur à ces éléments, ou appeler une fonction qui réalise cette opération.

5.4.2 Exemples

Copie d'une chaîne de caractères

Afin d'illustrer le mécanisme de modification du contenu d'une chaîne de caractères, nous donnons à la figure 5.12 une procédure capable de recopier le contenu d'une chaîne `src` dans une chaîne `dest`. Cette procédure admet un argument supplémentaire `n`, spécifiant un nombre maximum de caractères (incluant le terminateur) qui peuvent être écrits dans `dest`. La présence de cet argument permet de prévenir des situations où l'opération de copie serait amenée à dépasser le nombre d'octets alloués pour la chaîne `dest`, ce qui constituerait une erreur grave. Notons

```
/*
   Recopie la chaîne <src> dans la chaîne <dest>, en transférant
   au plus n symboles (incluant le terminateur éventuel).
*/

void strncpy(char dest[], const char src[], unsigned n)
{
  unsigned i;

  for (i = 0; i < n; i++)
    if (!((dest[i] = src[i])))
      break;
}
```

FIGURE 5.12 – Procédure de copie d'une chaîne de caractères

que la procédure `strncpy` de la figure 5.12 est en fait déjà disponible sous une forme similaire [6] dans la bibliothèque standard.

Dans ce programme, il y a deux façons de sortir de la boucle `for` : soit après avoir effectué `n` itérations, soit en exécutant l'instruction `break`. Ce dernier cas se produit si l'évaluation de l'expression

```
!((dest[i] = src[i]))
```

retourne une valeur vraie, c'est-à-dire si l'évaluation de

```
dest[i] = src[i]
```

retourne une valeur nulle. Cette dernière expression est une application de l'opérateur d'affectation "=", qui recopie la valeur de `src[i]` dans `dest[i]`. Cette valeur recopiée correspond à celle produite par l'évaluation de l'expression ; elle devient donc nulle après avoir recopié le caractère terminateur. La boucle se termine donc après avoir recopié ce caractère, ou bien après avoir recopié le nombre maximum `n` d'octets pouvant être écrits dans `dest`.

Signalons pour terminer la discussion relative à cet exemple qu'il n'est pas habituel d'utiliser l'opérateur d'affectation "=" dans l'expression qui contrôle une instruction `if`. Le plus souvent, cela correspond à une erreur de programmation résultant d'une confusion avec l'opérateur de comparaison "==". Dans le cas présent, le fait d'avoir entouré la sous-expression

6. La principale différence est que la version de la bibliothèque standard écrit toujours exactement `n` octets dans `dest`, en répétant le caractère terminateur autant de fois que nécessaire. Elle définit aussi une valeur de retour, égale à `dest`. Le prototype de cette fonction est disponible dans le fichier d'en-tête `string.h`.

```
#define TAILLE_MAX_MESSAGE 200

char message[TAILLE_MAX_MESSAGE + 1];

    ⋮

strncpy(message, "Bonjour", TAILLE_MAX_MESSAGE + 1);
```

FIGURE 5.13 – Utilisation de strncpy

```
unsigned nb_chiffres(const char s[])
{
  unsigned i, n;

  for (n = i = 0; s[i]; i++)
    if (s[i] >= '0' && s[i] <= '9')
      n++;

  return n;
}
```

FIGURE 5.14 – Comptage du nombre de chiffres dans une chaîne

```
dest[i] = src[i]
```

de deux paires de parenthèses est une convention indiquant que cet emploi de l'opérateur "=" est intentionnel.

Un exemple de fragment de code exécutable utilisant la procédure `strncpy` pour recopier une chaîne de caractères est donné à la figure 5.13. Dans ce code, l'expression `"Bonjour"` alloue une chaîne constante contenant les caractères qui composent le texte "Bonjour" ainsi que le caractère terminateur, et s'évalue en un pointeur vers cette chaîne. L'appel à la procédure `strncpy` a pour effet de recopier cette chaîne dans le vecteur `message`.

Comptage du nombre de chiffres dans une chaîne

Le programme de la figure 5.14 implémente une procédure capable de compter le nombre d'occurrences des caractères "0", "1", ... et "9" dans une chaîne de caractères qui lui est passée en argument. Ce programme est basé sur une boucle qui inspecte un à un les caractères qui composent la chaîne, jusqu'à atteindre le caractère terminateur. L'évaluation du gardien `s[i]` de cette boucle retourne en effet une valeur qui n'est égale à zéro que pour ce terminateur. La

procédure détermine ensuite si le code du caractère courant appartient à l'intervalle

['0','9'].

Cette solution n'est correcte que si le jeu de caractères utilisé attribue des codes consécutifs aux chiffres de 0 et 9. C'est le cas pour tous les systèmes de représentation actuellement utilisés, tel que celui de la figure 5.11.

5.5 Les tableaux multidimensionnels

Le mécanisme de définition de vecteurs introduit à la section 5.1.1 peut être généralisé à des tableaux de n'importe quelle dimension.

5.5.1 Définition et utilisation

La définition d'un tableau multidimensionnel possède la syntaxe suivante.

[`static` *] [* `const` *] type id*`[`*taille*$_1$`]` `[`*taille*$_2$`]` $\cdots$ `[`*taille*$_n$`]`
[`=` `{ { ` $\cdots$ `{` *val*`,` *val*`,` `...` `},`
$\vdots$
`{` *val*`,` *val*`,` `...` `}` $\cdots$ `} } ]`;

Par rapport à la forme de définition de tableau présentée à la section 5.1.1, la principale différence réside dans l'utilisation de plusieurs paires de crochets "`[`" et "`]`" pour spécifier le nombre approprié de dimensions et la taille correspondante du tableau vis-à-vis de chacune de ces dimensions. Quand des données permettant d'initialiser le tableau sont fournies, celles-ci sont structurées à l'aide d'accolades imbriquées. La signification des mot-clés optionnels `static` et `const` reste inchangée. Tout comme pour les tableaux unidimensionnels, cette syntaxe peut s'étendre à la définition de plusieurs tableaux dont les éléments partagent le même type.

Par exemple, le code suivant définit un tableau d'entiers à deux dimensions possédant 3 lignes et 4 colonnes, et attribue des valeurs initiales croissantes à ses éléments successifs.

```
int t[3][4] = { { 1, 2, 3, 4 },
                { 5, 6, 7, 8 },
                { 9, 10, 11, 12 } };
```

Chaque élément d'un tableau de dimension n est identifié par un tuple d'indices

$$(i_1, i_2, \ldots, i_n),$$

avec $\forall k \in [1, n] : 0 \leq i_k < taille_k$, où $taille_1$, $taille_2$, …, $taille_n$ caractérisent la taille du tableau par rapport aux n dimensions. Pour accéder à un tel élément, on écrit une expression de la forme

$$id\texttt{[}expr_1\texttt{]}\ \texttt{[}expr_2\texttt{]}\ \cdots\ \texttt{[}expr_n\texttt{]}$$

qui généralise celle utilisée pour les vecteurs. L'identificateur *id* est celui du tableau concerné, et l'évaluation des expressions $expr_1$, $expr_2$, …, $expr_n$ fournit les indices i_1, i_2, …, i_n de l'élément visé. Cette construction peut aussi être utilisée comme valeur à gauche. Par exemple, l'expression

```
t[1][2]
```

désigne l'élément situé à la deuxième ligne et la troisième colonne du tableau `t` défini dans l'exemple précédent.

5.5.2 Le passage d'un tableau multidimensionnel à une fonction

Pour passer un tableau multidimensionnel en argument à une fonction, on déclare le paramètre correspondant de cette fonction de la façon suivante.

$$type\ \ id\texttt{[}[taille_1]\texttt{]}\texttt{[}taille_2\texttt{]}\ \cdots\ \texttt{[}taille_n\texttt{]}$$

Cette fois encore, il est important de bien distinguer dans cette spécification les crochets en italique "*[*" et "*]*", qui indiquent les éléments optionnels, des crochets droits "`[`" et "`]`" qui font partie de la syntaxe de la déclaration. Dans le cas présent, on voit que la taille du tableau par rapport à chacune de ses dimensions doit être indiquée, sauf la première qui reste optionnelle.

La raison de cette obligation d'indiquer toutes les tailles sauf la première est technique. Tout comme pour le cas d'un vecteur, quand on passe un tableau multidimensionnel en argument à une fonction, seul un pointeur correspondant à l'emplacement du premier élément de ce tableau est transmis. Les éléments du tableau sont placés consécutivement dans la mémoire de l'ordinateur, dans un ordre qui explore récursivement chaque dimension. Par exemple, le tableau `t` à deux dimensions défini à la section précédente occupe 12 emplacements de la mémoire correspondant chacun à une valeur entière. Ceux-ci sont structurés en trois groupes successifs de quatre emplacements représentant chacun une ligne du tableau. En d'autres termes, les valeurs initiales fournies dans l'exemple de la section 5.5.1 pour les éléments du tableau correspondent à l'ordre de ces éléments en mémoire.

Avec cet arrangement, si l'on numérote à partir de 0 les emplacements de mémoire qui contiennent les éléments d'un tableau à n dimensions, alors le numéro associé à l'élément caractérisé par le tuple d'indices

$$(i_1, i_2, \ldots, i_n),$$

avec $\forall k \in [1, n] : \ 0 \leq i_k < taille_k$, est égal à

$$\begin{aligned} & i_1\, taille_2\, taille_3\, \ldots\, taille_n \\ + \ & i_2\, taille_3\, \ldots\, taille_n \\ & \vdots \\ + \ & i_{n-1}\, taille_n \\ + \ & i_n. \end{aligned}$$

Comme on le voit, la taille de toutes les dimensions intervient dans cette expression, sauf celle de la première. C'est pour cette raison que celle-ci n'est pas requise dans la déclaration.

5.5.3 Application : bibliothèque de transformations géométriques

Une application importante des tableaux multidimensionnels en ingénierie concerne bien sûr le calcul matriciel. Nous montrons ici comment construire une bibliothèque de fonctions permettant de manipuler des transformations géométriques dans un espace à deux dimensions.

Les transformations qui nous intéressent sont celles que l'on peut représenter à l'aide de *transformations affines*. Une transformation affine est une fonction

$$\mathbb{R}^2 \ \rightarrow \ \mathbb{R}^2 : \ \vec{x} \mapsto \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \vec{x} + \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}$$

caractérisée par six paramètres a_{11}, a_{12}, a_{21}, a_{22}, b_1, $b_2 \in \mathbb{R}$. Elle s'applique aux coordonnées cartésiennes de points du plan. En choisissant judicieusement la valeur des paramètres d'une telle transformation, on peut représenter des transformations géométriques diverses telles que des translations, des rotations, des mises à l'échelle, ...

Notre objectif consiste à développer une bibliothèque de fonctions permettant de créer une transformation affine représentant une translation, une rotation ou une mise à l'échelle donnée, de composer deux transformations (c'est-à-dire, calculer une transformation équivalente à une séquence de deux transformations données), et de calculer l'image d'un point par une transformation. Pour ce faire, nous choisissons de représenter les paramètres d'une transformation affine par un tableau de réels

$$\begin{bmatrix} a_{11} & a_{12} & b_1 \\ a_{21} & a_{22} & b_2 \end{bmatrix}$$

à deux lignes et trois colonnes.

geometrie-2d.c:

```
#include <math.h>
#include "geometrie-2d.h"

/*
   Place dans la transformation <t> une rotation d'angle <angle> autour
   de l'origine des axes.
*/

void geo2d_creer_rotation(double angle, double t[2][3])
{
  t[0][0] = t[1][1] = cos(angle);
  t[1][0] = sin(angle);
  t[0][1] = -t[1][0];
  t[0][2] = t[1][2] = 0.0;
}

/*
   Place dans la transformation <t> une mise à l'échelle de facteur
   <facteur> par rapport à l'origine des axes.
*/

void geo2d_creer_echelle(double facteur, double t[2][3])
{
  t[0][0] = t[1][1] = facteur;
  t[0][1] = t[1][0] = t[0][2] = t[1][2] = 0.0;
}

/*
   Place dans la transformation <t> une translation de vecteur (<x>, <y>).
*/

void geo2d_creer_translation(double x, double y, double t[2][3])
{
  t[0][0] = t[1][1] = 1.0;
  t[0][1] = t[1][0] = 0.0;
  t[0][2] = x;
  t[1][2] = y;
}
```

FIGURE 5.15 – Bibliothèque de transformations géométriques 2D (1/2)

```
/*
   Compose deux transformations <t1> et <t2>, effectuées
   dans cet ordre, et place le résultat dans la transformation <t>.
*/

void geo2d_composer(const double t1[2][3], const double t2[2][3],
                    double t[2][3])
{
  unsigned i, j;

  for (i = 0; i < 2; i++)
    for (j = 0; j < 3; j++)
      t[i][j] = t2[i][0] * t1[0][j] + t2[i][1] * t1[1][j];

  t[0][2] += t2[0][2];
  t[1][2] += t2[1][2];
}

/*
   Applique la transformation <t> aux coordonnées <v> d'un point dans
   le plan.
 */

void geo2d_appliquer(const double t[2][3], double v[2])
{
  double tv[2][3], tr[2][3];

  geo2d_creer_translation(v[0], v[1], tv);
  geo2d_composer(tv, t, tr);

  v[0] = tr[0][2];
  v[1] = tr[1][2];
}
```

geometrie-2d.h:

```
void geo2d_creer_rotation(double, double[2][3]);
void geo2d_creer_echelle(double, double[2][3]);
void geo2d_creer_translation(double, double, double[2][3]);
void geo2d_composer(const double[2][3], const double[2][3], double[2][3]);
void geo2d_appliquer(const double[2][3], double[2]);
```

FIGURE 5.16 – Bibliothèque de transformations géométriques 2D (2/2)

Un exemple de solution est fourni aux figures 5.15 et 5.16. Ce programme est composé d'un fichier d'en-tête `geometrie-2d.h` fournissant le prototype de cinq fonctions, et d'un fichier `geometrie-2d.c` contenant l'implémentation de ces fonctions. Ce fichier inclut le fichier d'en-tête standard `math.h` afin de disposer du prototype des fonctions trigonométriques [7] `sin` et `cos` de la bibliothèque standard, qui sont employées dans la procédure `geo2d_creer_rotation` pour construire une matrice de rotation.

Dans ce programme, les procédures `geo2d_creer_rotation`, `geo2d_creer_echelle` et `geo2d_creer_translation` permettent de créer des transformations élémentaires. Elles acceptent comme arguments les paramètres de ces transformations, c'est-à-dire l'angle de rotation autour de l'origine pour une rotation, exprimé en radians, le facteur d'échelle pour une mise à l'échelle, et les composantes du vecteur de déplacement pour une translation. Ces procédures ne définissent pas de valeur de retour, mais écrivent leur résultat dans le tableau qui leur est passé comme dernier argument.

La procédure `geo2d_composer` accepte deux transformations en arguments. Elle effectue des opérations matricielles visant à calculer leur composition, et écrit le résultat de ce calcul dans le tableau qui lui est fourni comme troisième argument. La procédure `geo2d_appliquer` applique une transformation donnée à un point. Le deuxième argument de cette procédure sert à la fois à fournir les coordonnées du point qui subit la transformation et à recueillir le résultat de cette transformation. Cette procédure aurait pu être programmée de la même façon que `geo2d_composer`, mais la solution mise en œuvre dans le programme est plus élégante : pour calculer l'image d'un point $\vec{p}$ par une transformation θ, on effectue les opérations suivantes.

1. Créer une translation $\theta_{\vec{p}}$ qui applique l'origine des axes $(0, 0)$ sur le point $\vec{p}$.

2. Calculer la composition σ des deux transformations $\theta_{\vec{p}}$ et θ, effectuées dans cet ordre.

3. Extraire les deux termes constants de la représentation de σ. Ceux-ci correspondent aux coordonnées de l'image par σ du point $(0, 0)$, qui est égale à l'image par θ de $\vec{p}$.

5.6 Exercices

1. Écrire une fonction capable de calculer la moyenne arithmétique d'un ensemble de nombres réels (de type `double`) contenus dans un tableau donné.

7. Comme nous l'avons déjà vu aux chapitres 2 et 4, il faut alors indiquer au compilateur que le programme exécutable doit être lié à l'implémentation de la partie mathématique de la bibliothèque standard. Pour le compilateur GCC, cela implique d'ajouter "`-lm`" aux options de compilation.

2. Écrire un programme capable d'afficher la valeur des *coefficients binomiaux*

$$C_n^m = \frac{n!}{m!\,(n-m)!}$$

pour n et m croissants, avec $n \geq 0$ et $0 \leq m \leq n$ (c'est-à-dire, C_0^0, C_1^0, C_1^1, C_2^0, C_2^1, C_2^2, C_3^0, C_3^1, C_3^2, ...).

Suggestion : Exploiter la propriété

$$C_{n+1}^{m+1} = C_n^m + C_n^{m+1}.$$

3. Écrire une fonction capable de déterminer la longueur de la plus grande séquence d'entiers croissants consécutifs contenue dans un tableau d'entiers donné.

   Par exemple, pour un tableau contenant $[1, 2, 6, 4, 5, 6, 6, 3, 4, 1]$, cette fonction doit retourner 3, correspondant à la longueur de la séquence $[4, 5, 6]$.

4. Écrire une fonction capable de comparer deux chaînes de caractères données, et de retourner la position du premier caractère qui diffère entre ces deux chaînes (1 s'il s'agit du premier, 2 pour le deuxième, et ainsi de suite), ou bien 0 si les deux chaînes possèdent le même contenu.

5. Écrire une fonction capable de déterminer si une chaîne de caractères donnée est un palindrome, ce qui signifie que sa lecture de la gauche vers la droite et de la droite vers la gauche fournit le même résultat.

6. Écrire une fonction capable de calculer la longueur du plus grand suffixe commun entre deux chaînes de caractères données.

7. Écrire une fonction capable de déterminer si deux chaînes de caractères sont égales aux blancs près, c'est-à-dire sans tenir compte des symboles d'espacement `' '` qu'elles peuvent contenir.

8. *(Examen de janvier 2019)*
   (a) Écrire une fonction prenant en arguments deux tableaux d'entiers ainsi que la longueur (supposée commune) de ces tableaux, retournant 1 si ces tableaux possèdent le même contenu, et 0 sinon.

   (b) Par la méthode des invariants, démontrer que la valeur retournée par cette fonction est correcte.

9. *(Examen d'août 2019)*

   (a) Écrire une fonction prenant en arguments deux tableaux d'entiers t_1 et t_2 ainsi que leur taille (supposée commune), et retournant la longueur de leur plus grand préfixe commun, c'est-à-dire le plus grand entier n tel que $t_1[i] = t_2[i]$ pour tout i tel que $0 \leq i < n$.

   (b) Par la méthode des invariants, démontrer que la valeur retournée par cette fonction est correcte. Démontrer également que cette fonction se termine.

10. *(Examen de juin 2021)*

    (a) Écrire une procédure prenant en arguments

    — un tableau `t` d'octets non signés,

    — le nombre `n` ≥ 0 d'octets contenu dans `t`,

    — un tableau `h` de 256 entiers de type `int`.

    L'opération effectuée par cette procédure consiste à modifier le contenu du tableau `h` de la façon suivante : après l'exécution de la procédure, pour tout $i \in [0, 255]$, `h[`i`]` doit contenir le nombre d'octets de `t` égaux à i. Par exemple, si le tableau `t` contient $[7, 0, 42, 7, 0, 7]$, alors on doit avoir `h[0]` $= 2$, `h[7]` $= 3$, `h[42]` $= 1$, et `h[`i`]` $= 0$ pour les autres valeurs de i.

    (b) Déterminer la complexité en temps de la procédure obtenue au point (a).

    (c) Par la méthode des invariants, démontrer que l'opération effectuée par la procédure obtenue au point (a) est correcte.

11. *(Examen d'août 2021)*

    (a) Écrire une procédure prenant en arguments

    — un tableau `t` de nombres réels,

    — le nombre `n` ≥ 0 d'éléments de `t`.

    L'opération effectuée par cette procédure consiste à modifier le contenu du tableau `t` de la façon suivante : si t_0, t_1, t_2, … sont (respectivement) les contenus initiaux des

cases `t[0]`, `t[1]`, `t[2]`, ... de `t`, alors après l'exécution de la procédure, la première case de `t` doit contenir t_0, la deuxième $t_0 + t_1$, la troisième $t_0 + t_1 + t_2$, et ainsi de suite.

(b) Déterminer la complexité en temps de la procédure obtenue au point (a).

(c) Par la méthode des invariants, démontrer que l'opération effectuée par la procédure obtenue au point (a) est correcte.

12. *(Examen de janvier 2022)*

(a) Écrire une fonction prenant en argument une chaîne de caractères, et retournant la somme de tous les chiffres qu'elle contient. Par exemple, pour la chaîne `"Le 24 janvier 2022"`, cette fonction doit retourner 12. Dans le cas particulier d'une chaîne qui ne contient aucun chiffre, la fonction doit retourner 0.

(b) Par la méthode des invariants, démontrer que la valeur retournée par cette fonction est correcte.

13. *(Examen de juin 2022)*

(a) Écrire une fonction prenant en arguments un tableau `t` d'entiers non signés et le nombre d'éléments `n` de ce tableau, et retournant le plus grand entier présent dans au moins deux cases consécutives de `t`, ou 0 si `t` ne contient pas de valeurs consécutives identiques.Par exemple, pour le tableau $[10, 1, 4, 4, 4, 0, 7, 7, 10]$, cette fonction doit retourner 7. Pour le tableau $[4, 7, 2, 7, 4, 2]$, elle doit retourner 0.

(b) Déterminer les complexités en temps et en espace de la fonction obtenue au point (a).

14. *(Examen d'août 2022)*

Écrire une procédure prenant en arguments un tableau `t` de chaînes de caractères, un tableau `b` d'octets, et la taille `n` (commune) de ces deux tableaux. L'objectif de cette procédure consiste à écrire dans `b`[i], pour chaque indice i, une valeur booléenne qui est vraie si la chaîne contenue dans `t`[i] est vide, et fausse sinon.

15. *(Examen de janvier 2023)*

(a) Écrire une fonction acceptant en arguments deux tableaux de nombres entiers `t1` et `t2` ainsi que le nombre (identique) `n` d'éléments de ces tableaux, et retournant le plus grand indice i pour lequel `t1`[i] = `t2`[i]. Par exemple, pour les tableaux $[0, -1, 0, 10, 10]$ et $[10, -1, -10, 10, 0]$, cette fonction doit retourner 3.

(b) Par la méthode des invariants, démontrer que la valeur retournée par cette fonction est correcte.

16. *(Examen de juin 2023)*

(a) Écrire une fonction capable de calculer la longueur d'une chaîne de caractères reçue en argument, sans tenir compte des caractères blancs (`' '`) éventuels situés au début de cette chaîne. Par exemple, pour les chaînes `"ZZZZ"`, `"  abcd"` et `" 1 2 "`, cette fonction doit retourner 4. Dans le cas d'une chaîne vide ou ne contenant que des caractères blancs, la fonction doit retourner 0.

(b) Par la méthode des invariants, démontrer que la valeur retournée par cette fonction est correcte. Démontrer également que cette fonction se termine.

17. *(Examen d'août 2023)*

(a) Écrire une fonction prenant en arguments un pointeur *s* vers une chaîne de caractères, et un caractère *c*, et retournant le nombre de copies de *c* situées au début de la chaîne. Par exemple, pour la chaîne `"aaabbaba"` et le caractère `'a'`, la fonction doit retourner 3. Pour cette même chaîne et le caractère `'b'` ou `'c'`, la fonction doit retourner 0.

(b) Par la méthode des invariants, démontrer que la valeur retournée par cette fonction est correcte. Démontrer également que cette fonction se termine.

18. *(Examen de janvier 2024)*

(a) Écrire une fonction prenant en arguments un tableau d'entiers `t1`, la taille `n` de ce tableau, et un tableau d'entiers `t2` de taille supérieure ou égale à `n`. Cette fonction doit recopier dans le tableau `t2` tous les éléments de `t1` qui sont strictement positifs, dans le même ordre où ils apparaissent dans `t1`. La fonction doit ensuite retourner le nombre d'éléments recopiés.

Par exemple, si `t1` contient $[-1, 0, 1, 1, 4, 0, 2]$, alors les éléments 1, 1, 4 et 2 doivent être recopiés dans le tableau `t2`, dans cet ordre, et la fonction doit retourner 4.

(b) Par la méthode des invariants, démontrer que l'opération effectuée par la fonction développée au point (a) est correcte. Démontrer également que cette opération se termine.

19. *(Examen de juin 2024)*

(a) Écrire une fonction prenant en argument un pointeur vers une chaîne de caractères, et retournant une valeur booléenne indiquant si cette chaîne est entièrement composée de chiffres, à l'exclusion de tout autre type de caractère.

Par exemple, pour les chaînes `"2024"`, `"007"` et `""`, la fonction doit retourner une valeur vraie. Pour les chaînes `"1 2 3"`, `"INFO2009"` et `"a4"`, elle doit retourner une valeur fausse.

(b) Par la méthode des invariants, démontrer que la valeur retournée par cette fonction est correcte.

20. *(Examen d'août 2024)*

(a) Écrire une fonction prenant en arguments un tableau t d'entiers signés supposés triés par ordre croissant, ainsi que la taille n de ce tableau, et retournant le nombre de valeurs différentes parmi les éléments de t. Par exemple, si t contient $[-1, -1, 4, 5, 5]$, alors la fonction doit retourner 3. Si $n = 0$, la fonction doit retourner 0.

(b) Par la méthode des invariants, démontrer que la valeur retournée par cette fonction est correcte.

(c) Calculer la complexité en temps de cette fonction.

21. *(Examen de janvier 2025)*

(a) Un tableau est dit *monotone* si ses éléments sont triés par ordre croissant ou décroissant. Par exemple, les tableaux $[\,2, 0, 0, -1\,]$, $[\,42\,]$ et $[\,10, 10, 20, 30\,]$ sont monotones, mais les tableaux $[\,23, 1, 2025\,]$ et $[\,-1, 1, 1, -1\,]$ ne le sont pas.

Écrire une fonction qui prend en arguments un tableau d'entiers `t` et sa taille `n`, et qui retourne 1 si `t` est monotone et 0 sinon.

(b) Par la méthode des invariants, démontrer que la valeur calculée par cette fonction est correcte.

22. *(Examen de juin 2025)*

(a) Écrire une fonction prenant en arguments un tableau d'entiers `t` et sa taille `n`, et qui retourne le nombre d'occurrences de la plus grande valeur contenue dans `t`, c'est-à-dire le nombre de fois où cette valeur apparaît dans le tableau. Par exemple, si le

tableau contient [20, 0, 20, 5], alors la fonction doit retourner 2. Si le tableau contient [1, −2, 3], elle doit retourner 1. Si le tableau est vide, elle doit retourner 0.

Note importante : On demande de programmer cette fonction à l'aide d'une seule boucle.

(b) Par la méthode des invariants, démontrer que la valeur calculée par cette fonction est correcte.

23. *(Examen d'août 2025)*

(a) Écrire une fonction prenant en arguments un tableau d'entiers signés `t` ainsi que sa taille strictement positive `n`, et qui retourne l'écart entre les deux valeurs extrêmes du tableau. Par exemple, si le tableau contient $[2, -2, 10, 5, 10]$, alors la fonction doit retourner 12. Si le tableau contient $[5, 5]$ ou $[3]$, elle doit retourner 0.

(b) Par la méthode des invariants, démontrer que la valeur calculée par cette fonction est correcte.

24. *(Examen de janvier 2026)*

Un tableau d'entiers `t` est à *préfixes positifs dominants* si chacun de ses préfixes contient un nombre d'éléments strictement positifs au moins égal à son nombre d'éléments strictement négatifs. En d'autres termes, si $\texttt{t}_{<i}$ désigne le sous-tableau de `t` contenant tous les éléments d'indice strictement inférieur à i, le tableau `t` est à préfixe positif dominant si pour tout i, le nombre d'éléments strictement positifs de $\texttt{t}_{<i}$ est supérieur ou égal au nombre d'éléments strictement négatifs de $\texttt{t}_{<i}$. Par exemple, le tableau $[1, -2, 4, -5]$ et le tableau vide sont à préfixes positifs dominants alors que les tableaux $[-1, 4, 5, 8]$ et $[0, -3, 4]$ ne le sont pas.

(a) Écrire une fonction prenant en arguments un tableau d'entiers `t` ainsi que sa taille `n`, et qui retourne 1 si `t` est à préfixes positifs dominants et 0 sinon.

(b) Par la méthode des invariants, démontrer que la valeur calculée par cette fonction est correcte.

25. *(Examen de janvier 2019)*

(a) Décrire, le plus simplement possible, l'opération effectuée par la fonction suivante.

```
unsigned f(char s[])
{
  unsigned n;

  if (!s[0])
    return 0;

  if (s[0] >= '0' && s[0] <= '9')
    n = 1;
  else
    n = 0;

  return n + f(s + 1);
}
```

(b) Quelle est la complexité en temps de cette fonction ?

(c) Écrire une fonction réalisant exactement la même opération, mais sans effectuer d'appel récursif.

26. *(Examen d'août 2019)*

(a) Décrire, le plus simplement possible, l'opération effectuée par la fonction suivante.

```
long f(int t[], int nb)
{
  if (nb <= 0)
    return 0;

  return t[0] + f(t + 2, nb - 2);
}
```

(b) Quelle est la complexité en temps de cette fonction ?

(c) Écrire une fonction réalisant exactement la même opération, mais sans effectuer d'appel récursif.

27. *(Examen de janvier 2022)*

(a) Décrire, le plus simplement possible, l'opération effectuée par la fonction suivante.

```
double f(double t[], double a, unsigned n)
{
  if (a < 0.0)
    return f(t, -a, n);

  if (n)
   return f(t + 1, a, --n);

  return a * t[0];
}
```

(b) Écrire une fonction réalisant exactement la même opération, mais n'effectuant aucun appel récursif et ne comprenant aucune instruction de boucle.

28. *(Examen de juin 2026)*

(a) Décrire le plus simplement possible ce que calcule la fonction suivante :

```
unsigned f(int t1[], int t2[], unsigned n)
{
  if (n-- && t1[n] == t2[n])
    return f(t1, t2, n) + 1;
  else
    return 0;
}
```

(b) Quelle est la complexité en espace de la fonction du point (a), en ne tenant pas compte de la mémoire occupée par les données d'entrée ?

(c) Écrire une fonction réalisant exactement la même opération que la fonction du point (a), mais sans effectuer d'appel récursif.

29. Le *tri à bulles (bubble sort)* est un algorithme de tri d'un vecteur basé sur le principe suivant. On parcourt le vecteur du début vers la fin, en comparant à chaque étape deux éléments consécutifs. (En d'autres termes, on commence par comparer les deux premiers éléments, puis le deuxième avec le troisième, puis le troisième avec le quatrième, et ainsi de suite.) Lorsqu'une comparaison révèle que les deux éléments concernés ne sont pas

dans l'ordre souhaité, on les permute. À l'issue de cette procédure, le dernier élément du vecteur a atteint sa place définitive : il s'agit de l'élément le plus grand ou le plus petit, selon que le tri soit respectivement par ordre croissant ou décroissant. Ensuite, on répète cette procédure de façon à déterminer l'avant-dernier élément du vecteur trié, puis l'élément précédent, et ainsi de suite jusqu'à arriver à un vecteur entièrement trié.

(a) Décrire cet algorithme en pseudo-code.

(b) Déterminer la complexité en temps de cet algorithme, en fonction de la taille du vecteur à trier.

(c) Écrire une fonction implémentant cet algorithme, dans le cas d'un tableau d'entiers à trier par ordre croissant.

30. Le *tri par sélection (selection sort)* est un autre algorithme de tri d'un vecteur, fonctionnant de la façon suivante. On balaie le vecteur de façon à y repérer le plus petit élément (dans le cas d'un tri par ordre croissant) ou le plus grand (pour un ordre décroissant). On permute ensuite cet élément avec le premier élément du vecteur, qui prend alors sa valeur finale. On effectue ensuite la même opération en commençant le balayage à partir du deuxième élément du vecteur, puis du troisième, et ainsi de suite.

    (a) Décrire cet algorithme en pseudo-code.

    (b) Déterminer la complexité en temps de cet algorithme, en fonction de la taille du vecteur à trier.

    (c) Écrire une fonction implémentant cet algorithme, dans le cas d'un tableau d'entiers à trier par ordre croissant.

31. En adaptant une des fonctions obtenues aux deux exercices précédents (au choix), écrire un programme capable de lire un ensemble de chaînes de caractères entrées au clavier, et de les trier par ordre lexicographique (c'est-à-dire, l'ordre du dictionnaire).

    Notes :
    — La première étape consiste à écrire une fonction capable de comparer deux chaînes de caractères données par ordre lexicographique.

    — Pour lire une chaîne au clavier, on peut utiliser la fonction `fgets` de la bibliothèque standard, comme dans l'exemple suivant.

```
#include <stdio.h>
#define TAILLE_MAX  100

  ...
  char chaine[TAILLE_MAX];

  fgets(chaine, TAILLE_MAX, stdin);
  ...
```

Attention, si la saisie se termine par un retour chariot, alors le caractère correspondant (`'\n'`) fait partie de la chaîne écrite par `fgets`.

32. Le *tri par comptage (counting sort)* est un algorithme de tri d'un vecteur utilisable lorsque les éléments de ce vecteur ne prennent qu'un petit nombre de valeurs possibles. Son principe consiste à parcourir le vecteur une seule fois, en comptant le nombre d'occurrences de chaque valeur. Par exemple, à l'issue du parcours du vecteur $[1, 0, 1, 2, 1, 0]$, on obtient que ce vecteur contient 2 occurrences de la valeur 0, 3 occurrences de 1 et 1 occurrence de 2. À partir de ce résultat, il est ensuite facile de générer le vecteur trié en produisant le nombre de copies attendues de chaque occurrence, c'est-à-dire $[0, 0, 1, 1, 1, 2]$ pour notre exemple.

    (a) Décrire cet algorithme en pseudo-code.

    (b) Déterminer la complexité en temps et en espace de cet algorithme, en fonction de la taille du vecteur à trier et du nombre de valeurs pouvant être prises par les éléments du vecteur.

    (c) Écrire une fonction appliquant cet algorithme au tri par ordre croissant des caractères contenus dans une chaîne donnée. (Par exemple, le tri de la chaîne de caractères `"algorithmique"` doit produire `"aeghiilmoqrtu"`.)

33. (a) Écrire en pseudo-code un algorithme capable de déterminer efficacement si deux chaînes de caractères données sont des anagrammes l'une de l'autre, c'est-à-dire si elles sont composées des mêmes caractères, dans un ordre qui peut être différent. (Par exemple, les chaînes `"logarithme"` et `"lithomarge"` sont des anagrammes d'`"algorithme"`.) *Suggestion :* Exploiter le résultat de l'exercice précédent.

    (b) Déterminer la complexité en temps de cet algorithme.

    (c) Traduire cet algorithme en une fonction acceptant comme arguments les deux chaînes de caractères à comparer.

34. Le *tri rapide (quicksort)* est un algorithme de tri d'un vecteur basé sur l'approche diviser pour régner. Son principe consiste à d'abord choisir un *pivot*, qui est un élément quelconque du vecteur. (Une stratégie simple consiste à systématiquement choisir le premier élément du vecteur comme pivot, mais d'autres choix sont possibles.) Ensuite, on réordonne les éléments du vecteur de façon à avoir un préfixe contenant tous les éléments de valeur inférieure à celle du pivot, puis le pivot, puis un suffixe contenant les éléments restants. Enfin, on applique récursivement le même algorithme de tri aux sous-vecteurs correspondant au préfixe et au suffixe.

    (a) Décrire cet algorithme en pseudo-code.

    (b) Déterminer la complexité en temps de cet algorithme, en fonction de la taille du vecteur à trier.

    Indice : Le pire cas du point de vue du temps d'exécution correspond à la plus grande profondeur possible de récursion.

    (c) Écrire une fonction implémentant cet algorithme, dans le cas d'un tableau d'entiers à trier par ordre croissant.

35. Écrire une fonction implémentant l'algorithme de tri par fusion étudié à la section 5.3.2 de façon itérative, c'est-à-dire sans effectuer d'appels récursifs.

36. *(Examen de janvier 2023)*

    On dispose d'un vecteur d'entiers triés par ordre croissant, dans lequel on souhaite rechercher s'il contient ou non un nombre x donné. Pour résoudre ce problème, on utilise l'algorithme suivant :

    — Si le vecteur est vide, alors la réponse est non.

    — Si le vecteur ne contient qu'un seul élément, alors on l'examine afin de déterminer s'il est égal à x.

    — Si le vecteur contient plus d'un élément, alors on le sépare en deux parties plus petites, correspondant à un préfixe et un suffixe du vecteur situés de part et d'autre d'un élément situé approximativement au milieu de celui-ci. Si cet élément n'est pas égal à x, alors on poursuit récursivement la recherche dans une seule de ces deux parties (devant être choisie judicieusement) à l'aide du même algorithme.

(a) Donner une implémentation (récursive) de cet algorithme, sous la forme d'une fonction retournant une valeur booléenne vraie si le nombre recherché appartient au vecteur, et fausse sinon.

(b) Déterminer les complexités en temps et en espace de la fonction obtenue au point (a).

37. *(Examen d'août 2024)*

Le *triangle d'Euler* est une construction mathématique constituée d'une séquence de lignes de longueur croissante : pour $i = 1, 2, \ldots$, la ligne d'indice i contient i nombres entiers $A_{i,0}, A_{i,1}, \ldots, A_{i,i-1}$ définis de la façon suivante :

— Le premier et le dernier élément de chaque ligne sont égaux à 1 :

$$\forall i > 0 : A_{i,0} = A_{i,i-1} = 1.$$

— Les autres éléments sont déterminés par l'équation

$$\forall 0 < j < i - 1 : A_{i,j} = (i - j)A_{i-1,j-1} + (j + 1)A_{i-1,j}.$$

(a) Écrire un programme qui affiche à l'écran les N premières lignes du triangle d'Euler, où $N \geq 1$ est une constante définie au début de ce programme.

Par exemple, pour $N = 6$, le programme doit produire l'affichage suivant :

```
1
1 1
1 4 1
1 11 11 1
1 26 66 26 1
1 57 302 302 57 1
```

(b) Calculer la complexité en temps et en espace du programme obtenu au point (a).

38. *(Examen de janvier 2025)*

(a) Écrire une procédure prenant en arguments un tableau d'entiers non signés `t`, sa taille `n`, un tableau `u`, et la taille `m` de ce dernier. On suppose que les éléments de `t` sont tous différents et que l'on a $\texttt{m} \leq \texttt{n}$. La procédure demandée doit écrire dans `u` les `m` plus petits éléments de `t`, dans un ordre quelconque. Le contenu de `t` doit rester inchangé.

Par exemple, si `t` contient $[\,1, 5, 3, 0, 4, 7\,]$ et $\texttt{m} = 3$, alors la procédure doit écrire dans `u` les valeurs 0, 1 et 3.

Note : On ne demande pas la solution la plus efficace possible ; un algorithme polynomial suffit.

(b) Calculer la complexité en temps de la procédure obtenue au point (a).

39. *(Examen de juin 2025)*

(a) Écrire une procédure prenant en arguments deux tableaux d'octets non signés `t` et `u`, leur taille commune `n`, et un tableau `b` de 256 entiers signés. Pour tout $i \in [0, 255]$, la procédure doit placer dans `b[`i`]` une valeur booléenne vraie si `t` et `u` contiennent tous les deux le même nombre d'octets égaux à i, et fausse sinon. On demande que cette procédure soit la plus efficace possible.

(b) Calculer la complexité en temps de la procédure obtenue au point (a). Expliquer pourquoi cette complexité est optimale (c'est-à-dire, pourquoi il n'existe pas de solution présentant une meilleure complexité).

40. *(Examen d'août 2025)*

(a) Écrire une procédure qui inverse l'ordre des caractères dans une chaîne qu'elle reçoit en argument. Par exemple, si on fournit à cette procédure la chaîne de caractères `"Informatique"`, celle-ci doit être modifiée en `"euqitamrofnI"`.

Note importante : Pour résoudre ce problème, il n'est pas permis d'utiliser des fonctions issues de la bibliothèque standard. Vous pouvez en revanche définir des fonctions supplémentaires de votre choix.

(b) Calculer la complexité en temps de la procédure obtenue au point (a).

41. *(Examen de janvier 2026)*

(a) Écrire une fonction prenant en argument une chaîne de caractères `s`, et qui calcule efficacement le nombre de segments de `s` composés d'un et un seul caractère. Un segment est défini comme étant une sous-chaîne non vide qui commence et se termine à des positions déterminées.

Par exemple, la chaîne `"aaabbca"` contient 11 segments composés d'un seul caractère : 4 correspondent à la sous-chaîne `"a"`, 2 à `"aa"`, 1 à `"aaa"`, 2 à `"b"`, 1 à `"bb"` et 1 à `"c"`. Pour cette chaîne, la fonction doit donc retourner 11.

Astuce : Le nombre de segments contenus dans une sous-chaîne composée de m caractères vaut $\frac{m(m+1)}{2}$.

Note : Si nécessaire, vous êtes libre de définir des fonctions auxiliaires de votre choix. Il n'est pas permis en revanche d'utiliser des fonctions issues de la bibliothèque standard.

(b) Calculer la complexité en temps de la fonction obtenue au point (a).

42. *(Examen de juin 2026)*

(a) Écrire une fonction prenant en arguments deux chaînes de caractères `s1` et `s2`, et qui retourne :

- -1 si `s1` apparaît avant `s2` dans l'ordre du dictionnaire,
- 0 si les deux chaînes sont identiques,
- $+1$ si `s1` apparaît après `s2`.

On suppose que `s1` et `s2` sont non vides et ne contiennent que des lettres minuscules encodées en ASCII.

Par exemple pour `s1 = "introduction"` et `s2 = "informatique"`, la fonction doit retourner $+1$, puisque le premier mot figure après le second dans l'ordre du dictionnaire.

(b) Calculer la complexité en temps de la fonction obtenue au point (a).

Chapitre 6

Les structures et les pointeurs

Ce chapitre est consacré à l'étude de deux mécanismes de programmation essentiels à la plupart des développements en langage C : la définition de types de données structurés, et la notion de pointeur, que nous avons déjà évoquée au chapitre 5 dans le cadre de la manipulation de tableaux.

6.1 Les données structurées

Nous avons vu aux sections 2.2 et 4.4 comment définir une *variable*, qui correspond à un emplacement de la mémoire de l'ordinateur capable de retenir une valeur d'un type donné. Nous avons ensuite au chapitre 5 généralisé cette notion de variable en montrant comment représenter des tableaux, qui constituent des ensembles d'emplacements de mémoire numérotés, retenant des valeurs partageant le même type.

La notion de *structure* généralise ces concepts. Une structure peut être vue comme un ensemble d'emplacements de la mémoire qui ne sont pas identifiés par un numéro mais par un nom appelé *champ*. Pour chaque champ, une structure peut retenir une donnée d'un type déterminé, ce type pouvant être différent d'un champ à un autre.

À l'aide d'une telle structure, on peut par exemple représenter la fiche signalétique d'un étudiant au sein d'un logiciel gérant les inscriptions à l'Université. Une telle structure possédera des champs correspondant respectivement au nom, au prénom, à la date de naissance, aux données d'inscription, à une photographie, ..., de l'étudiant concerné. L'intérêt du mécanisme de structure est de permettre de manipuler l'ensemble de ces données en tant qu'une seule entité, même si certaines d'entre elles possèdent des types différents.

En programmation, on est ainsi fréquemment amené à traiter des assemblages de données

de types hétérogènes. Un autre exemple concerne la représentation informatique d'une image. Celle-ci peut prendre la forme d'une structure dont les champs correspondent

— aux dimensions horizontale et verticale de l'image, qui sont des nombres entiers,

— à son contenu, représenté par un tableau bidimensionnel de pixels,

— à un commentaire, prenant la forme d'une chaîne de caractères,

— ...

Dans une telle structure, chaque pixel peut à son tour être représenté par une structure regroupant ses composantes de couleur et son opacité.

On peut également employer des structures pour représenter des données dont les composantes partagent le même type, mais pour lesquelles on préfère nommer ces composantes plutôt que de les numéroter comme on le ferait pour les éléments d'un tableau. Par exemple, on peut représenter un nombre complexe par une structure définissant deux champs réels pour ses parties réelle et imaginaire, et appeler ces deux champs `re` et `im`. On pourrait aussi utiliser une structure pour représenter les transformations géométriques introduites à la section 5.5.3, en définissant des champs différents pour la matrice de transformation et pour le vecteur de translation.

6.1.1 La définition d'un type de données structuré

En langage C, la construction suivante permet de définir un nouveau type structuré.

```
struct id
{
    type1 id1 [, id1b [, ... ]];
    type2 id2 [, id2b [, ... ]];
        ⋮
};
```

Ce nouveau type a pour nom `struct` *id*, où *id* est l'identificateur qui figure dans la définition. Chaque variable de ce type possédera les champs *id1*, *id2*, ..., dont le type respectif est donné par *type1*, *type2*, ... Il est aussi permis de définir plusieurs champs qui partagent le même type en les séparant par des virgules.

Par exemple, la définition

```
struct complexe
{
  double re, im;
};
```

est celle d'un type structuré appelé `struct complexe`, que l'on peut ensuite utiliser dans des définitions et déclarations de variables. On peut par exemple écrire

```
struct complexe x, y, z;
```

pour définir trois variables `x`, `y` et `z` de ce type. Chacune de ces variables contient alors deux champs `re` et `im` capables de retenir une valeur réelle de type `double`.

On accède aux champs d'une variable possédant un type structuré grâce à l'opérateur "`.`" : on place à gauche de cet opérateur le nom de la variable concernée, et à droite le nom du champ auquel on souhaite accéder. La forme obtenue constitue à la fois une expression, dont l'évaluation fournit la valeur du champ en question, mais aussi une valeur à gauche qui peut servir à modifier cette valeur. Dans l'exemple précédent, les parties réelle et imaginaire des variables `x`, `y` et `z` se notent ainsi `x.re`, `x.im`, `y.re`, `y.im`, `z.re` et `z.im`.

Les définitions de types structurés sont habituellement placées en dehors des fonctions ; dans ce cas, ces types sont utilisables dans la suite du programme, jusqu'à la fin du fichier source courant. Il est également autorisé de placer une telle définition à l'intérieur d'un bloc d'instructions exécutables. Dans ce cas, la portée de cette définition suit les mêmes règles que pour les déclarations de variables locales, et s'étend jusqu'à la fin du plus petit bloc contenant cette définition.

Notons qu'il est permis de fusionner la définition d'un type structuré avec celle d'une ou de plusieurs variables de ce type ; il est alors autorisé (mais pas obligatoire) d'omettre l'identificateur du type après le mot-clé `struct`. Par exemple, on aurait pu définir de la façon suivante les variables `x`, `y` et `z` de l'exemple précédent.

```
struct
{
  double re, im;
} x, y, z;
```

Cette forme de définition n'est utile que si le type concerné n'est utilisé qu'une seule fois dans le programme. En pratique, il est fréquent qu'un type intervienne dans plusieurs définitions

de variables, ou dans les paramètres d'une ou plusieurs fonctions. Dans un tel cas, il vaut mieux nommer le type afin de pas devoir le redéfinir de façon redondante.

Lorsqu'on utilise des types structurés, le fait de devoir systématiquement préfixer leur nom du mot-clé `struct` nuit parfois à la lisibilité des programmes. Cela peut être évité grâce à la construction

`typedef` *type id*`;` ,

qui permet dans la suite du programme de faire référence au type *type* par le nom abrégé *id*. Le type *type* peut correspondre à n'importe quel type étudié jusqu'ici : primitif, tableau ou structuré. Dans le cas d'un type structuré, il est permis de ne pas le nommer en omettant d'écrire un identificateur à la suite du le mot-clé `struct`. Par exemple, la construction

```
typedef struct
{
  double re, im;
} complexe;
```

définit un nouveau type appelé `complexe`, et non `struct complexe` comme précédemment. Pour définir trois variables `x`, `y` et `z` de ce type, on écrit alors

```
complexe x, y, z;
```

Signalons pour terminer qu'il est permis qu'un type structuré possède des champs de n'importe quel type : primitif, tableau, ou même structuré. L'exemple suivant est celui d'une structure représentant la fiche signalétique d'un étudiant ; il n'est valide que si le type `struct date` a été précédemment défini.

```
#define LONGUEUR_MAX_NOM 50

struct etudiant
{
  char nom[LONGUEUR_MAX_NOM + 1],
       prenom[LONGUEUR_MAX_NOM + 1];
  struct date date_naissance;
     ⋮
};
```

6.1.2 L'utilisation des données structurées

Les paramètres d'une fonction peuvent être de type structuré. Le mécanisme qui est alors mis en œuvre est celui d'un *passage par valeur*, similaire au cas des types primitifs étudié à la section 4.2.2. Cela signifie que lorsqu'une donnée structurée est passée en argument à une fonction ou une procédure, cette donnée est recopiée dans le paramètre correspondant au moment de l'appel. Si la fonction ou la procédure modifie ensuite la valeur de ce paramètre, cette modification aura un effet purement local et ne sera pas répercutée vers le code appelant.

Il est important de bien réaliser que ce mécanisme diffère fondamentalement de celui qui régit le passage d'un tableau en argument à une fonction, étudié aux sections 5.2 et 5.5.2. Pour rappel, dans le cas d'un tableau, seul un pointeur vers son contenu est passé à la fonction ; cela entraîne qu'une modification des éléments du tableau effectuée par la fonction devient visible pour le code qui invoque celle-ci.

Pour illustrer cette différence, nous considérons le programme de la figure 6.1. Celui-ci définit un type structuré `struct s` possédant un champ `t` qui prend la forme d'un vecteur de deux entiers. La fonction principale de ce programme alloue une variable `x` de ce type, et place les valeurs 10 et 20 dans les deux cases du tableau. Elle invoque ensuite la procédure `permuter` en lui passant en argument la donnée structurée contenue dans `x`, ce qui revient à recopier le contenu de `x` dans le paramètre `a` de cette procédure. L'opération de permutation des deux éléments du vecteur effectuée par la procédure affecte alors uniquement le contenu de `a` et pas celui de `x`, qui reste inchangé. Après cette opération, la fonction `main` affiche donc "`10 20`".

On peut maintenant comparer ce programme à celui de la figure 6.2, dans lequel le type structuré a été remplacé par un vecteur de deux entiers. Dans ce deuxième programme, la procédure `permuter` reçoit en argument un pointeur vers le vecteur `x` alloué par la fonction `main`. Cela entraîne que les modifications réalisées par `permuter` sont effectuées dans ce vecteur `x` et non dans une copie de ce vecteur comme pour le programme de la figure 6.1. Ces modifications sont donc maintenant visibles pour le code qui invoque `permuter`. Contrairement au précédent, ce programme affiche donc "`20 10`".

Pour terminer cette discussion portant sur l'utilisation des types structurés par les fonctions, signalons qu'il est permis qu'une fonction retourne une valeur de type structuré. Il est également possible d'appliquer l'opérateur d'affectation "=" à une variable de type structuré ; par exemple, si `x` et `y` sont des variables partageant le même type structuré, alors l'expression `x = y` recopie l'entièreté du contenu de `y` dans `x`. En revanche, l'opérateur de comparaison "==" ne peut pas être employé avec des valeurs de type structuré.

```c
#include <stdio.h>

struct s
{
  int t[2];
};

static void permuter(struct s a)
{
  int aux;

  aux = a.t[0];
  a.t[0] = a.t[1];
  a.t[1] = aux;
}

int main()
{
  struct s x;

  x.t[0] = 10;
  x.t[1] = 20;

  permuter(x);

  printf("%d %d\n", x.t[0], x.t[1]);

  return 0;
}
```

FIGURE 6.1 – Passage d'une structure à une fonction

```c
#include <stdio.h>

static void permuter(int a[2])
{
  int aux;

  aux = a[0];
  a[0] = a[1];
  a[1] = aux;
}

int main()
{
  int x[2];

  x[0] = 10;
  x[1] = 20;

  permuter(x);

  printf("%d %d\n", x[0], x[1]);

  return 0;
}
```

FIGURE 6.2 – Passage d'un tableau à une fonction

6.1.3 Application : manipulation de nombres complexes

Un exemple de programme exploitant les types structurés est donné aux figures 6.3 et 6.4. Ce programme implémente une bibliothèque de fonctions permettant de manipuler des nombres complexes [1]. Son fichier d'en-tête `complexe.h` contient les prototypes de ces fonctions, mais également la définition du type structuré `complexe`. Cela permet de faire intervenir ce type dans le prototype des fonctions, mais aussi de l'utiliser dans n'importe quel programme qui inclut ce fichier d'en-tête.

1. Mentionnons que les versions modernes du langage C disposent déjà d'une représentation de nombres complexes et d'opérations associées, que nous n'étudions pas dans ce cours.

complexe.c:

```
#include <math.h>
#include "complexe.h"

complexe complexe_init(double re, double im)
{
  complexe c;

  c.re = re;
  c.im = im;

  return c;
}

double complexe_re(complexe c)
{
  return c.re;
}

double complexe_im(complexe c)
{
  return c.im;
}

complexe complexe_somme(complexe a, complexe b)
{
  complexe c;

  c.re = a.re + b.re;
  c.im = a.im + b.im;

  return c;
}

complexe complexe_difference(complexe a, complexe b)
{
  complexe c;

  c.re = a.re - b.re;
  c.im = a.im - b.im;

  return c;
}
```

FIGURE 6.3 – Bibliothèque de manipulation de nombres complexes (1/2)

```
complexe complexe_produit(complexe a, complexe b)
{
  complexe c;

  c.re = a.re * b.re - a.im * b.im;
  c.im = a.re * b.im + a.im * b.re;

  return c;
}

double complexe_module(complexe a)
{
  return hypot(a.re, a.im);
}

double complexe_distance(complexe a, complexe b)
{
  return complexe_module(complexe_difference(a, b));
}
```

complexe.h:

```
typedef struct
{
  double re, im;
} complexe;

complexe complexe_init(double, double);
double   complexe_re(complexe);
double   complexe_im(complexe);
complexe complexe_somme(complexe, complexe);
complexe complexe_difference(complexe, complexe);
complexe complexe_produit(complexe, complexe);
double   complexe_module(complexe);
double   complexe_distance(complexe, complexe);
```

FIGURE 6.4 – Bibliothèque de manipulation de nombres complexes (2/2)

6.2 Les pointeurs

La notion de pointeur a déjà été rencontrée à la section 5.2, dans le cadre du passage de tableaux en argument à une fonction ou une procédure. Nous généralisons à présent ce concept à des données de type quelconque.

6.2.1 Le passage d'arguments par variable

À la section 6.1.2, nous avons vu que le passage d'une donnée structurée en argument à une fonction s'effectue *par valeur*, ce qui signifie que le contenu de cet argument est entièrement recopié dans le paramètre correspondant au moment de l'appel. Ce mécanisme présente plusieurs inconvénients. Premièrement, dans le cas d'une donnée de grande taille comme par exemple une image, cette opération de copie est inefficace : elle prend du temps, et elle nécessite de disposer d'un double espace mémoire, pour la donnée originale et pour la copie. Un autre inconvénient est que la fonction n'est pas capable de retourner des valeurs au code qui l'appelle par l'intermédiaire de ses arguments ; cette possibilité est notamment utile quand on souhaite retourner plusieurs valeurs à la fois.

En revanche, quand une fonction ou une procédure reçoit un tableau en argument, nous savons que le mécanisme employé, appelé *passage par variable*, est différent et consiste à transmettre un *pointeur* vers le tableau. Ces notions de pointeur et de passage par variable peuvent être généralisées à d'autres données que des tableaux : un pointeur vers une donnée représente l'endroit où celle-ci se trouve dans la mémoire de l'ordinateur. Lorsqu'on passe un pointeur vers une donnée à une fonction, cette donnée n'est pas recopiée, ce qui signifie qu'elle ne reste présente qu'en un seul exemplaire dans la mémoire de l'ordinateur. Si la fonction appelée modifie la donnée désignée par le pointeur, alors cette modification devient visible pour le code appelant, comme nous l'avons vu avec le programme de la figure 6.2. Ce mécanisme permet donc à une fonction ou une procédure de retourner des valeurs au code qui l'invoque par l'intermédiaire de ses arguments.

6.2.2 Les pointeurs en langage C

En langage C, si `T` est un type, alors `T *` désigne le type d'un pointeur vers une valeur de type `T`. Par exemple, la définition

```
int *p;
```

est celle d'une variable `p` contenant un pointeur vers une valeur entière. Il est aussi possible

```c
#include <stdio.h>

int main()
{
  int  a;
  int *p;

  p = &a;
 *p = 42;

  printf("%d\n", a);

  return 0;
}
```

FIGURE 6.5 – Exemple de programme manipulant un pointeur

de définir des pointeurs vers des données arbitraires, c'est-à-dire de n'importe quel type, en employant le type [2] `void *`.

Le langage C définit deux opérateurs importants liés aux pointeurs. L'opérateur "&" s'applique à une valeur à gauche, comme le nom d'une variable, un champ d'une donnée structurée ou un élément d'un tableau. L'expression obtenue fournit un pointeur vers la donnée correspondante. Par exemple, si `v` est une variable de type `int`, alors l'évaluation de l'expression `&v` retourne un pointeur vers `v`. La valeur de ce pointeur représente l'endroit où se trouve la variable `v` dans la mémoire de l'ordinateur ; on dit alors que ce pointeur *pointe* vers `v`. Le type de ce pointeur se note `int *`.

Le deuxième opérateur, qui peut être vu comme le dual de "&", est l'opérateur [3] unaire "*". Lorsque cet opérateur est appliqué à un pointeur, il *déréférence* ce pointeur, ce qui signifie qu'il accède à la donnée pointée par celui-ci. En d'autres termes, l'évaluation de l'expression `*`*expr*, où *expr* s'évalue en un pointeur p, retourne la donnée pointée par p. Cette expression constitue également une valeur à gauche, et peut donc être placée avant l'opérateur d'affectation "=" pour modifier la donnée pointée. Lorsqu'on utilise ce mécanisme, il est important de garantir que le pointeur pointe bien vers un emplacement valide de la mémoire de l'ordinateur ; c'est au programmeur qu'incombe cette responsabilité.

Un exemple de programme manipulant un pointeur à l'aide de ces opérateurs est donné à la figure 6.5. Ce programme définit une variable entière `a` et un pointeur `p` vers un entier.

2. Il s'agit de la troisième application du mot-clé `void` que nous rencontrons, après son utilisation pour spécifier qu'une fonction ne possède pas de valeur de retour, ou n'accepte pas d'argument.

3. Il s'agit aussi de la troisième utilisation du symbole "*", qui dénote également l'opérateur binaire de multiplication, et qui apparaît dans le type des pointeurs.

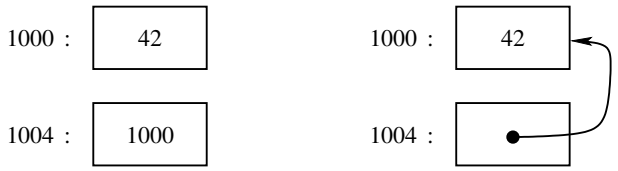


FIGURE 6.6 – Représentation graphique d'un pointeur

Signalons qu'il aurait été possible de fusionner les déclarations de ces deux variables en écrivant

```
int a, *p;
```

.

Le programme attribue ensuite à `p` la valeur retournée par l'évaluation de `&a`, ce qui a pour effet de faire pointer `p` vers `a`. L'instruction suivante affecte la valeur 42 à la donnée pointée par `p`, ce qui est équivalent à attribuer cette valeur à `a`. Ce programme affiche donc "42".

Nous voyons dans cet exemple qu'il est autorisé d'employer l'opérateur d'affectation "=" pour attribuer une valeur à un pointeur. Il est aussi permis d'appliquer l'opérateur de comparaison "==" à des pointeurs : deux pointeurs sont considérés comme étant égaux si et seulement s'ils pointent vers la même donnée, c'est-à-dire s'ils désignent le même endroit dans la mémoire de l'ordinateur.

D'un point de vue technique, on peut considérer que les cases qui composent la mémoire de l'ordinateur sont numérotées, le numéro attribué à une case étant appelé son *adresse*. La valeur d'un pointeur vers une donnée est alors égale à l'adresse de celle-ci. Cette situation est illustrée à la figure 6.6. Dans cet exemple, on suppose que les variables `a` et `p` de l'exemple précédent sont respectivement placées aux adresses 1000 et 1004 de la mémoire. La partie gauche de la figure montre le contenu de ces variables lorsque `a` vaut 42 et `p` pointe vers `a`. En pratique, les adresses précises des variables sont déterminées par le compilateur et l'environnement d'exécution, et ne sont pas explicitement connues par le programmeur. On s'abstrait alors de ces détails en représentant graphiquement à l'aide d'une flèche le fait que `p` pointe vers `a`, comme sur la partie droite de la figure.

Il existe une valeur spéciale de pointeur, appelée le *pointeur vide*. Ce pointeur ne pointe pas vers un emplacement valide de la mémoire, mais n'importe quel pointeur peut lui être comparé pour déterminer s'il lui est égal ou non. Le pointeur vide peut être utilisé pour signaler une situation particulière, comme par exemple le constat qu'une opération censée retourner un pointeur vers une donnée a échoué. Le pointeur vide ne peut pas être déréférencé. En langage C, le pointeur vide se note `NULL`, qui est une macro définie notamment dans les fichiers d'en-tête `stdlib.h` et `stdio.h`. Lorsqu'on la convertit en un entier grâce à une conversion de type expli-

```c
#include <stdio.h>

/*  Permute les entiers pointés par p1 et p2.  */

static void permuter(int *p1, int *p2)
{
  int aux;

  aux = *p1;
  *p1 = *p2;
  *p2 = aux;
}

int main()
{
  int a, b;

  a = 42;
  b = -7;

  printf("a = %d, b = %d\n", a, b);
  permuter(&a, &b);
  printf("a = %d, b = %d\n", a, b);

  return 0;
}
```

FIGURE 6.7 – Exemple de passage d'arguments par variable

cite ou implicite, `NULL` prend la valeur 0. Par conséquent, si l'on interprète un pointeur comme une valeur booléenne, un pointeur non vide est vrai et le pointeur vide est faux.

Un exemple de programme utilisant des pointeurs pour implémenter un passage d'arguments par variable à une procédure est donné à la figure 6.7. Dans ce programme, la procédure `permuter` reçoit en arguments deux pointeurs vers des valeurs entières, et a pour effet de permuter ces deux valeurs. On peut donc l'employer dans le reste du programme pour permuter n'importe quelle paire de valeurs entières, à condition de disposer de pointeurs vers ces valeurs. Dans la fonction `main`, cette procédure est invoquée en lui passant un pointeur vers la variable `a` et un autre vers `b` ; à l'issue de cet appel, les valeurs de `a` et de `b` auront donc été permutées. Le programme affiche donc "`a = 42, b = -7`" avant l'appel, et "`a = -7, b = 42`" après celui-ci.

L'instruction `permuter(&a, &b);` qui figure dans ce programme ressemble à l'instruction `scanf("%d %d", &a, &b);` présente dans celui de la figure 1.2, qui est le premier programme

que nous avons examiné dans ce cours. Nous sommes maintenant à même d'expliquer les principes de la fonction `scanf` de la bibliothèque standard, qui permet de lire des valeurs entrées par l'utilisateur. Cette fonction admet comme arguments une chaîne de caractères suivie par un certain nombre de pointeurs de type `void *`. Chaque fois que la fonction détecte un *marqueur de format* commençant par "`%`" dans la chaîne de caractères, elle lit une valeur du type indiqué par ce marqueur, et l'écrit à l'endroit spécifié par un des pointeurs, en considérant ceux-ci dans l'ordre où ils apparaissent. Les marqueurs de format qui peuvent être employés sont similaires à ceux de la fonction `printf` et incluent

— "`%d`" pour les entiers signés (de type `int`),

— "`%u`" pour les entiers non signés (de type `unsigned`),

— "`%ld`", "`%lu`", "`%lld`" et "`%llu`" pour les types (respectifs) `long int`, `long unsigned`, `long long int` et `long long unsigned`.

— "`%f`" et "`%lf`" pour (respectivement) les types `float` et `double`,

— ...

L'instruction `scanf("%d %d", &a, &b);` a donc pour effet de lire deux valeurs entières signées et de les écrire successivement dans les variables `a` et `b`. Mentionnons pour terminer que la fonction `scanf` retourne un entier égal au nombre de valeurs qui ont été correctement lues.

6.2.3 Les pointeurs vers des données structurées

Il est assez fréquent d'utiliser des pointeurs pour accéder à des données structurées. Comme nous l'avons vu en discutant des mécanismes de passage d'arguments à une fonction, cela peut permettre d'éviter de recopier ces données, ce qui présente un coût prohibitif quand elles sont de grande taille.

Le langage C définit un opérateur dédié à l'accès aux champs d'une donnée structurée accessible par le biais d'un pointeur. Si `p` est un pointeur vers une donnée structurée possédant un champ `c`, alors l'expression

```
p -> c
```

est un raccourci d'écriture pour

```
(*p).c
```

.

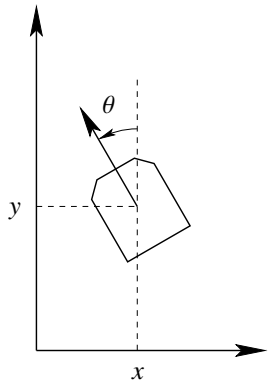


FIGURE 6.8 – Configuration spatiale d'un mobile

En d'autres termes, cette expression déréférence le pointeur `p` et accède au champ `c` de la valeur structurée pointée par celui-ci. Une expression de cette forme constitue également une valeur à gauche.

Nous illustrons l'utilisation de pointeurs vers des données structurées à l'aide du programme des figures 6.9 et 6.10. Celui-ci fournit une bibliothèque permettant de manipuler la *configuration spatiale* d'un mobile évoluant dans un espace à deux dimensions, c'est-à-dire la combinaison de sa position et de son orientation. La figure 6.8 montre comment cette configuration s'exprime : la position du mobile correspond aux coordonnées cartésiennes (x, y) de son point de référence, et son orientation est représentée par l'angle θ que fait son axe géométrique avec l'axe des ordonnées.

Les angles manipulés par les fonctions trigonométriques de la bibliothèque standard doivent être exprimés en radians. La fonction `normaliser_angle` a pour but de normaliser la représentation d'un angle en ramenant sa valeur dans l'intervalle fermé à gauche et ouvert à droite $[-\pi, \pi)$. Son implémentation repose sur la fonction `fmod` de la bibliothèque standard, qui calcule le reste de la division réelle de son premier argument par le second.

La procédure `init_configuration` sert à initialiser une configuration à l'aide de valeurs données de x, y et θ. Le premier argument de cette procédure est passé par variable, ce qui permet à la procédure d'en modifier le contenu. Ce mécanisme de passage par variable est également mis en œuvre par les procédures `deplacer_configuration` et `tourner_configuration` ; celles-ci utilisent leur premier argument comme donnée d'entrée, représentant la configuration du mobile avant le mouvement qu'elles calculent, mais aussi comme donnée de sortie pour recevoir le résultat de ce calcul.

configuration-2d.c:

```
#include <math.h>
#include "configuration-2d.h"

/*
    Retourne un angle équivalent à <angle>, mais appartenant à
    l'intervalle [-PI, PI).
*/

static double normaliser_angle(double angle)
{
  double r;

  r = fmod(angle, 2.0 * M_PI);
  if (r < -M_PI)
    r += 2.0 * M_PI;
  else
    if (r >= M_PI)
      r -= 2.0 * M_PI;

  return r;
}

/*
    Initialise la configuration *<c> avec la position (<x>, y>) et
    l'orientation <theta>.
 */

void init_configuration(configuration *c, double x, double y, double theta)
{
  c -> x = x;
  c -> y = y;
  c -> theta = normaliser_angle(theta);
}

/*
    Met à jour la configuration *<c> après un déplacement de longueur <l>
    dans la direction courante.
 */

void deplacer_configuration(configuration *c, double l)
{
  c -> x -= l * sin(c -> theta);
  c -> y += l * cos(c -> theta);
}
```

FIGURE 6.9 – Bibliothèque de manipulation de configurations 2D (1/2)

```
/*
    Met à jour la configuration *<c> après une rotation d'angle <angle>.
 */

void tourner_configuration(configuration *c, double angle)
{
  c -> theta = normaliser_angle(c -> theta + angle);
}
```

configuration-2d.h:

```
typedef struct
{
  double x, y;   /* position */
  double theta;  /* orientation */
} configuration;

void init_configuration(configuration *, double, double, double);
void deplacer_configuration(configuration *, double);
void tourner_configuration(configuration *, double);
```

FIGURE 6.10 – Bibliothèque de manipulation de configurations 2D (2/2)

6.2.4 Les pointeurs et les tableaux

Nous avons déjà évoqué à la section 5.2.3 le lien qui existe entre les pointeurs et les tableaux : si `v` est l'identificateur d'un vecteur, par exemple s'il a été déclaré par l'instruction

`int v[100];`,

alors l'expression `v` s'évalue en un pointeur vers le premier élément de `v`, et une expression de la forme `v + i` fournit un pointeur vers le $(i + 1)$-ème élément de `v`.

On en déduit que les expressions `v` et `&v[0]` sont équivalentes ; en effet, elles s'évaluent toutes les deux en un pointeur vers le premier élément de `v`. De même, les expressions `v + i` et `&v[i]` sont équivalentes pour n'importe quelle valeur de i.

Nous savons que quand un tableau est passé en argument à une fonction, le paramètre correspondant prend pour valeur un pointeur vers le premier élément de ce tableau. On en déduit que pour un type `T` quelconque, les déclarations `T a[]` et `T *a` d'un paramètre `a` d'une fonction ou d'une procédure sont équivalentes. Ces deux formes de déclarations de paramètres peuvent être utilisées indifféremment.

```
/*
    Retourne la longueur de la chaîne de caractères <s>.
 */

unsigned strlen(const char *s)
{
  unsigned l = 0;

  while (*s++)
    l++;

  return l;
}
```

FIGURE 6.11 – Calcul de la longueur d'une chaîne de caractères

À titre d'exemple, nous donnons le programme de la figure 6.11, qui définit une fonction[4] `strlen` capable de calculer la longueur d'une chaîne de caractères fournie en argument.

Comme nous venons de l'expliquer, le paramètre `s` de cette fonction aurait pu être déclaré `char s[]` sans que cela n'affecte le programme. Ce dernier énumère tous les caractères qui composent la chaîne pointée par `s`, jusqu'à atteindre le caractère terminateur. À chaque étape, le gardien de boucle prend pour valeur celle du caractère pointé par `s`, et ce pointeur est ensuite incrémenté. L'itération incrémente une variable `l` qui compte le nombre de caractères lus. La boucle se termine dès que le caractère pointé par `s` prend une valeur numérique nulle, ce qui ne se produit que pour le caractère terminateur. Dans le cas particulier d'une *chaîne vide*, c'est-à-dire dont le premier caractère est déjà le terminateur, la fonction se termine sans avoir effectué d'itération de la boucle, et retourne alors 0 qui est bien la valeur attendue.

Pour terminer, remarquons que dans l'expression

`*s++`,

l'opérateur "++" possède une priorité supérieure à celle de l'opérateur "*" : c'est le pointeur `s` qui est incrémenté, et non la donnée pointée par `s`.

4. Une fonction similaire est implémentée par la bibliothèque standard. Son prototype se trouve dans le fichier d'en-tête standard `string.h`.

6.3 L'allocation dynamique de mémoire

Lorsqu'on définit un tableau en utilisant les instructions étudiées au chapitre 5, il est nécessaire d'en préciser la taille, ce qui oblige que celle-ci soit connue à la compilation du programme. Il est cependant fréquent de devoir mémoriser et traiter des données dont la taille ne devient connue qu'au moment de l'exécution, notamment parce que ces données sont fournies au programme par l'utilisateur ou l'environnement, ou parce qu'elles résultent d'un calcul. Par exemple, quand un navigateur WWW contacte un serveur afin de lui demander de transmettre le contenu d'une page, la taille des données reçues en réponse est a priori inconnue et peut être très variable d'une page à une autre.

Dans ce chapitre, nous introduisons des mécanismes permettant au cours de l'exécution d'un programme d'allouer des blocs de mémoire de taille arbitraire, de les redimensionner, et de les libérer lorsqu'ils ne sont plus utiles.

6.3.1 Principes

L'allocation dynamique de mémoire est implémentée par les fonctions suivantes, disponibles dans la bibliothèque standard. Leur prototype se trouve dans le fichier d'en-tête standard `stdlib.h`.

— La fonction [5]

```
void *malloc(unsigned n)
```

alloue un nouveau bloc de mémoire composé de `n` octets consécutifs, et retourne un pointeur vers le premier de ces octets. Ce bloc peut être vu comme un tableau d'octets de taille `n`, mais peut être utilisé pour y placer des valeurs de n'importe que type, à condition qu'il soit de taille suffisante. Le fait d'avoir été alloué signifie que ce bloc est maintenant réservé, et que les emplacements de mémoire qu'il occupe resteront bien séparés de ceux des variables et des autres blocs alloués par le programme.

L'opération d'allocation d'un bloc peut échouer, par exemple parce qu'il ne reste plus suffisamment d'emplacements disponibles dans la mémoire de l'ordinateur. Dans ce cas, la fonction `malloc` retourne le pointeur vide pour signaler cette situation d'erreur.

— La fonction

```
void *realloc(void *p, unsigned n)
```

5. Remarquons qu'il ne s'agit pas d'une procédure : son type de retour est `void *`, c'est-à-dire un pointeur vers une donnée de n'importe quel type, et non `void`.

redimensionne le bloc de mémoire pointé par `p` en lui attribuant la nouvelle taille `n`, exprimée en octets. Cette opération n'est valide que si `p` pointe vers un bloc de mémoire précédemment retourné par un appel à `malloc` ou `realloc`. Le redimensionnement peut conduire à augmenter ou à diminuer la taille du bloc. Dans le cas d'une augmentation, les données situées dans le bloc sont préservées. Dans le cas d'une diminution, celles situées dans le préfixe du bloc qui n'est pas affecté par le redimensionnement le sont également. Dans les deux cas, l'opération peut conduire à déplacer le bloc vers un autre endroit de la mémoire de l'ordinateur. La fonction retourne un pointeur vers le bloc redimensionné, qui peut donc potentiellement avoir changé d'adresse, ou bien le pointeur vide dans le cas d'une erreur.

— La procédure

```
void free(void *p)
```

libère le bloc pointé par `p`. Tout comme pour la fonction `realloc`, cette opération n'est valide que si `p` correspond à un pointeur précédemment retourné [6] par une des fonctions `malloc` ou `realloc`. Après avoir libéré un bloc, les ressources attribuées à celui-ci sont récupérées par le système d'allocation dynamique de mémoire. Cela signifie que ce bloc ne peut plus être utilisé par le programme.

La taille des blocs alloués par les fonctions `malloc` et `realloc` est exprimée en octets. Pour réserver un bloc destiné à contenir des données d'un type `T` donné, il est alors nécessaire de calculer le nombre d'octets occupés par une instance de `T`. On peut pour cela employer l'opérateur `sizeof`, applicable à une variable ou à un type, qui retourne le nombre d'octets occupés en mémoire par cette variable ou par une valeur de ce type. Par exemple, pour une architecture `x86-64`, l'évaluation des expressions

```
sizeof(int)
```

et

```
sizeof(int *)
```

retourne respectivement 4 et 8, car un entier est représenté sur 32 bits et un pointeur sur 64.

Un exemple de programme employant l'allocation dynamique de mémoire est donné à la figure 6.12. Celui-ci définit une fonction qui alloue un vecteur de `n` entiers non signés, la valeur de `n` étant passée en argument à la fonction. La fonction initialise les éléments de ce vecteur

6. Il est cependant permis de passer le pointeur vide à `free`, auquel cas cette fonction n'a aucun effet.

```
#include <stdlib.h>

unsigned *creer_vecteur(unsigned n)
{
  unsigned *p, i;

  p = malloc(n * sizeof(unsigned));
  if (!p)
    return NULL;

  for (i = 0; i < n; i++)
    p[i] = i + 1;

  return p;
}
```

FIGURE 6.12 – Allocation dynamique d'un vecteur

avec les valeurs de 1 à `n`, et retourne un pointeur vers le premier d'entre eux. Dans le cas où l'allocation du vecteur échoue, la fonction retourne le pointeur vide.

6.3.2 La copie de blocs en mémoire

Pour manipuler des blocs de mémoire, il est utile de connaître les deux fonctions suivantes de la bibliothèque standard. Leur prototype se trouve dans le fichier d'en-tête standard `string.h`.

— La fonction

```
void *memcpy(void *dest, const void *src, unsigned n)
```

recopie `n` octets consécutifs à partir de celui pointé par `src`, vers l'emplacement de la mémoire désigné par `dest`. Cette fonction retourne le pointeur `dest`. Cette fonction ne peut être utilisée que si le bloc de `n` octets pointé par `src` et le bloc de `n` octets pointé par `dest` ne se chevauchent pas, c'est-à-dire s'ils occupent des emplacements entièrement distincts de la mémoire de l'ordinateur.

— La fonction

```
void *memmove(void *dest, const void *src, unsigned n)
```

effectue la même opération que `memcpy`, mais à la différence de cette dernière, elle peut

```
#include <stdio.h>
#include <string.h>

int main()
{
  double t[] = { 1.0, 2.0, 3.0, 4.0 };
  double u[4];

  unsigned i;

  memcpy(u, t, 4 * sizeof(double));

  for (i = 0; i < 4; i++)
    printf(" %lf", u[i]);

  printf("\n");

  return 0;
}
```

FIGURE 6.13 – Copie du contenu d'un vecteur

être employée même si les blocs de `n` octets pointés par `src` et `dest` possèdent une intersection commune non vide.

Ces fonctions permettent, en particulier, de recopier le contenu d'un tableau dans un autre. Cette opération est illustrée par le programme de la figure 6.13, qui définit deux vecteurs et recopie le contenu du premier dans le second.

6.3.3 Application : retournement d'une séquence de valeurs

On emploie l'allocation dynamique de mémoire quand il est impossible ou difficile de prévoir au moment de la compilation d'un programme la quantité de mémoire qui sera nécessaire à son exécution. Nous montrons ici un exemple d'une telle situation. Le problème consiste à lire une séquence de valeurs entières entrées au clavier, de taille arbitraire, et à afficher ensuite ces valeurs en ordre inverse de leur occurrence dans la séquence. Par exemple, si l'utilisateur introduit successivement les valeurs 3, 5, 1, 4 et 2, le programme doit afficher 2, 4, 1, 5 et 3.

Pour résoudre ce problème, il est nécessaire de retenir toutes les valeurs qui ont été entrées, afin de pouvoir les restituer plus tard. On pourrait pour cela employer un vecteur, mais il est impossible de déterminer a priori la taille qui devrait être allouée pour celui-ci : si l'on surestime le nombre de valeurs qui seront effectivement entrées par l'utilisateur, alors on monopolise

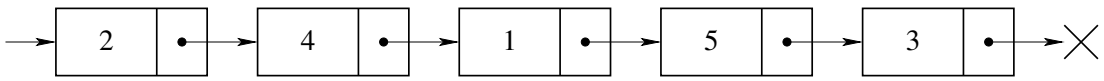


FIGURE 6.14 – Exemple de liste simplement liée

inutilement une partie de la mémoire de l'ordinateur ; si l'on sous-estime ce nombre, alors le programme ne sera pas capable de produire un résultat correct.

Nous présentons ici une autre solution, qui consiste à mémoriser la séquence de valeurs entrée par l'utilisateur à l'aide d'une *liste simplement liée*. Une liste simplement liée est une structure de données composée d'un certain nombre p d'éléments $e_1, e_2, \ldots, e_p$. Chaque élément e_i prend la forme d'une donnée structurée possédant un premier champ qui retient une valeur, et un deuxième qui contient un pointeur vers l'élément suivant e_{i+1}. Le dernier élément e_p de la liste ne possède pas de successeur, ce qui est représenté par le fait que son pointeur est vide. On accède à la liste liée par l'intermédiaire d'un pointeur vers son premier élément e_1, qui constitue la *tête* de la liste.

Un exemple de liste simplement liée représentant la séquence de valeurs 2, 4, 1, 5 et 3 est donné à la figure 6.14. Chaque élément de la liste est représenté par un rectangle dont les deux compartiments correspondent aux champs contenant la valeur associée à cet élément, et le pointeur vers son successeur. Le pointeur vide est représenté par une flèche menant à une croix.

L'intérêt de cette approche est que les éléments d'une liste liée peuvent être alloués dynamiquement, au fur et à mesure de la lecture des valeurs à retenir. On obtient l'algorithme suivant pour retourner la séquence de valeurs entrée par l'utilisateur.

1. On commence avec une liste vide, c'est-à-dire ne contenant aucun élément. Une telle liste est représentée par un pointeur ℓ vide.

2. Pour chaque nouvelle valeur v fournie par l'utilisateur :

   2.1 On alloue un nouvel élément e et on attribue la valeur v à son premier champ.

   2.2 On fait pointer le second champ de e vers la tête courante de la liste, c'est-à-dire qu'on le rend égal à ℓ.

   2.3 On modifie ℓ pour le faire pointer vers e, en d'autres termes, l'élément e devient la nouvelle tête de la liste.

3. À ce stade, la liste contient toutes les valeurs entrées par l'utilisateur, dans l'ordre inverse

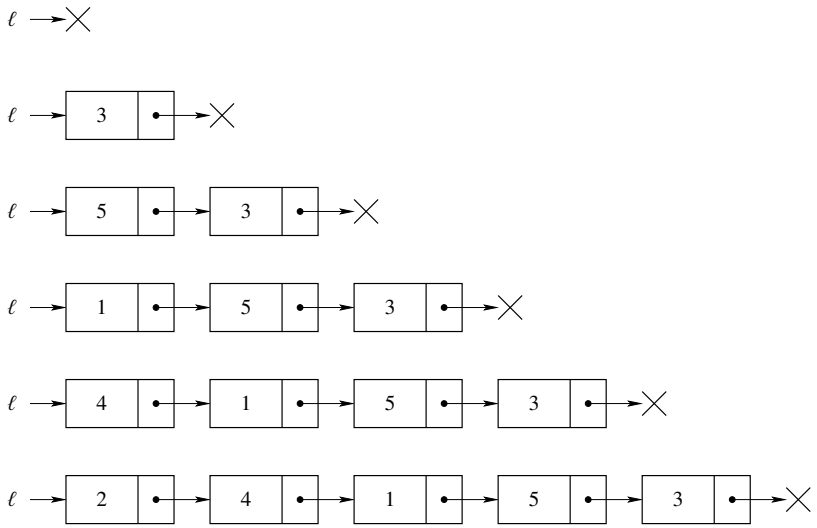


FIGURE 6.15 – Exécution de l'algorithme de retournement

où elles ont été fournies. Il reste à parcourir cette liste élément par élément. Tant que ℓ n'est pas vide :

3.1 On affiche la valeur du premier champ de l'élément pointé par ℓ.

3.2 On remplace ℓ par le contenu du deuxième champ de l'élément pointé par ℓ, ce qui a pour effet de retirer de la liste courante l'élément situé à sa tête.

La figure 6.15 montre le contenu successif de la liste liée au cours de l'exécution des étapes 1 et 2 de cet algorithme, pour la séquence de valeurs 3, 5, 1, 4 et 2.

Un programme C implémentant cet algorithme est donné à la figure 6.16. Ce programme commence par définir un type structuré `struct element_liste` pour les éléments de la liste liée. Ce type est composé d'un champ `valeur` pour la valeur retenue dans l'élément, et d'un pointeur `suivant` vers l'élément suivant dans la liste. Remarquons qu'il est permis d'utiliser le type `struct element_liste` au sein de sa propre définition.

Le programme définit deux fonctions `creer_liste` et `afficher_et_liberer_liste`, dédiées respectivement à la construction d'une liste simplement liée à partir de valeurs fournies par l'utilisateur (correspondant aux étapes 1 et 2 de l'algorithme), et au parcours de cette liste (étape 3). Ces deux fonctions sont successivement invoquées par la fonction `main` du programme.

Pour disposer d'un moyen de détecter la fin de la séquence entrée par l'utilisateur, la fonction `creer_liste` impose que les valeurs qui font partie de cette séquence soient positives ou nulles ;

```c
#include <stdio.h>
#include <stdlib.h>

struct element_liste
{
  int valeur;
  struct element_liste *suivant;
};

/*
   Retourne une liste simplement liée contenant des valeurs lues au clavier, par ordre inverse de
   lecture. En cas d'erreur, interrompt l'exécution du programme.
 */

static struct element_liste *creer_liste(void)
{
  struct element_liste *liste, *nouvel_element;
  int valeur;

  for (liste = NULL;;)
    {
      printf("Entrez une valeur positive (ou négative pour terminer): ");
      if (scanf("%d", &valeur) != 1 || valeur < 0)
        return liste;

      nouvel_element = malloc(sizeof(struct element_liste));
      if (!nouvel_element)
        exit(-1);

      nouvel_element -> valeur = valeur;
      nouvel_element -> suivant = liste;
      liste = nouvel_element;
    }
}

/*
   Affiche à l'écran les valeurs contenues dans la liste simplement liée <l> et libère cette liste.
 */

static void afficher_et_liberer_liste(struct element_liste *l)
{
  struct element_liste *e, *e_suivant;

  for (e = l; e; e = e_suivant)
    {
      printf(" %d", e -> valeur);
      e_suivant = e -> suivant;
      free(e);
    }
}

int main()
{
  struct element_liste *liste;

  liste = creer_liste();

  printf("Liste retournée:");
  afficher_et_liberer_liste(liste);
  printf("\n");

  return 0;
}
```

FIGURE 6.16 – Programme de retournement d'une séquence de valeurs

de cette façon, la saisie d'une valeur négative termine la séquence. Il est également possible d'indiquer la fin de la séquence en n'introduisant aucune valeur lorsque le programme le demande, ce qui conduit alors la fonction `scanf` à retourner 0.

Pour chaque nouvelle valeur lue, le programme invoque la fonction `malloc` pour allouer un élément de la liste, de type `struct element_liste`. Cette opération échouera si la quantité de mémoire disponible est insuffisante. Dans ce cas, l'invocation de `malloc` retourne le pointeur vide, ce qui conduit le programme à appeler la fonction `exit` de la bibliothèque standard. Celle-ci termine immédiatement l'exécution du programme. Elle admet un argument entier qui spécifie le code de diagnostic [7] retourné par le programme ; on le fixe ici à -1 pour signaler qu'une erreur s'est produite. Dans le cas où l'allocation dynamique des éléments s'effectue sans problème pour la totalité de la séquence, la fonction `creer_liste` retourne un pointeur vers la tête de la liste simplement liée qu'elle a créée.

La fonction `afficher_et_liberer_liste` effectue un parcours de la liste liée qui lui est passée en argument. Pour chaque élément visité, la fonction affiche la valeur qu'il contient, et libère ensuite la mémoire correspondant à cet élément. Cette dernière opération n'est pas à strictement parler nécessaire ici, car tous les emplacements de mémoire alloués dynamiquement par un programme sont automatiquement libérés quand celui-ci se termine. Libérer explicitement les données allouées dynamiquement lorsqu'elles ne sont plus utiles est cependant une bonne pratique de programmation.

Notons que le code responsable du parcours de la liste recopie la valeur du champ `suivant` d'un élément dans une variable auxiliaire `e_suivant` avant de libérer cet élément ; il n'est en effet plus permis d'accéder à un bloc mémoire alloué dynamiquement après sa libération.

6.3.4 L'allocation dynamique de tableaux

Il est facile d'allouer dynamiquement un tableau unidimensionnel, en invoquant la fonction `malloc` avec un argument égal au nombre total d'octets nécessaire pour en mémoriser les éléments. Par exemple, le fragment de code

```
int n;
double *v;
   ⋮
/* Calcul de la taille n à donner au vecteur v. */
   ⋮
v = malloc(n * sizeof(double));
```

7. Celui-ci possède la meme signification que le code de retour de la fonction `main`.

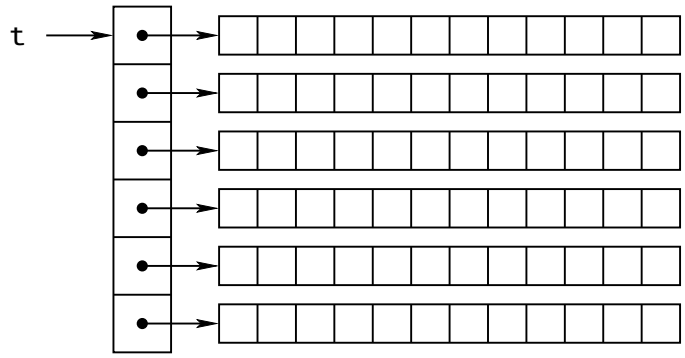


FIGURE 6.17 – Tableau bidimensionnel

alloue un vecteur `v` contenant `n` éléments de type `double`, où la valeur de `n` n'est connue qu'à l'exécution du programme. Rappelons qu'un vecteur est représenté par un pointeur vers son premier élément ; le fait d'avoir défini la variable `v` comme un pointeur permet d'utiliser des expressions telles que `v[0]`, `v[1]`, ... dans la suite du programme pour accéder aux éléments du vecteur.

Nous abordons maintenant le problème consistant à allouer dynamiquement un tableau de dimension d supérieure à 1. Si `t` désigne un tel tableau, alors il peut être vu comme un vecteur dont les éléments `t[0]`, `t[1]`, ... contiennent des pointeurs vers des tableaux de dimension $d-1$ représentant les lignes de `t`. Cette situation est illustrée à la figure 6.17 dans le cas d'un tableau bidimensionnel comprenant 6 lignes et 12 colonnes.

La procédure suivante permet donc d'allouer un tableau `t` à d dimensions, avec $d > 1$.

1. On alloue un vecteur destiné à contenir `t[0]`, `t[1]`, ... La taille de ce vecteur correspond à la taille n_1 de la première dimension de `t`, et chacun de ses élément est destiné à contenir un pointeur vers un sous-tableau.

2. On alloue récursivement n_1 sous-tableaux de dimension $d-1$, correspondant aux lignes de `t`, et l'on place dans les éléments `t[0]`, `t[1]`, ... de `t` des pointeurs vers ces sous-tableaux.

Un exemple de programme allouant dynamiquement un tableau bidimensionnel d'entiers est donné à la figure 6.18. La structure créée par ce programme est particulière : il s'agit d'un tableau *triangulaire*, dont la première ligne contient 1 élément, la deuxième ligne 2 éléments, et ainsi de suite. Un tableau de cette forme est illustré à la figure 6.19. Le nombre de lignes souhaité pour le tableau, correspondant aussi au nombre de colonnes de sa dernière ligne, est passé en argument à la fonction `creer_tableau_triangulaire`.

```
#include <stdlib.h>

/*  Alloue un tableau triangulaire t d'entiers, possédant un
 *  élément t[i][j]  pour tout i dans l'intervalle [0, n-1]
 *  et j tel que 0 <= j <= i. Retourne un pointeur vers le
 *  tableau créé, ou NULL en cas d'erreur. */

int **creer_tableau_triangulaire(unsigned n)
{
  int **t;
  unsigned i, k;

  if (!(t = malloc(n * sizeof(int *))))
    return NULL;

  for (i = 0; i < n; i++)
    if (!(t[i] = malloc((i + 1) * sizeof(int))))
      {
        for (k = 0; k < i; k++)
          free(t[k]);

        free(t);
        return NULL;
      }

  return t;
}
```

FIGURE 6.18 – Création d'un tableau triangulaire

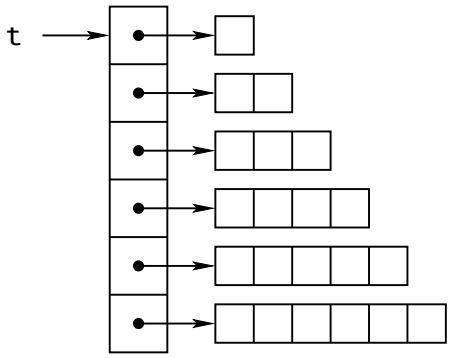


FIGURE 6.19 – Tableau triangulaire

Dans ce programme, le type employé pour définir la variable `t` ainsi que pour la valeur de retour de la fonction `creer_tableau_triangulaire` est `int **` : il désigne un pointeur vers un pointeur vers un entier. Remarquons aussi que le premier appel à la fonction `malloc` a pour but d'allouer un pointeur vers un vecteur dont les éléments sont des pointeurs vers des entiers. La taille des chaque élément de ce vecteur est donc donnée par

```
sizeof(int *)
```

.

Enfin, il ne faut pas oublier qu'un appel à `malloc` peut toujours échouer, dans le cas d'une insuffisance de mémoire. Si la première invocation de cette fonction, chargée d'allouer le vecteur de lignes, échoue, alors la fonction `creer_tableau_triangulaire` se termine en retournant elle-aussi simplement le pointeur vide. Si un appel suivant à `malloc` échoue, la procédure à suivre est plus compliquée car il devient alors nécessaire de libérer toutes les lignes précédemment allouées, ainsi que le vecteur de lignes.

Un deuxième exemple de programme mettant en œuvre les mécanismes d'allocation de tableaux multidimensionnels est donné aux figures 6.20, 6.21 et 6.22. Il s'agit d'une ébauche de bibliothèque permettant de manipuler des matrices rectangulaires, en implémentant des fonctions pour créer et libérer une matrice, ainsi que pour calculer la somme et le produit de deux matrices données. À la différence de l'exemple de la section 5.5.3, les matrices considérées peuvent ici être de n'importe quelle dimension. Cette bibliothèque pourrait être complétée par des fonctions permettant de consulter et de modifier les éléments des matrices, de créer des matrices spéciales telles que les matrices d'identité, d'effectuer des opérations comme la réduction vers une forme échelonnée ou la diagonalisation d'une matrice, ...

Comme d'habitude, le code source de cette bibliothèque est réparti en deux fichiers sources `matrice.h` et `matrice.c`. Le premier contient la définition du type structuré permettant de représenter une matrice, et le prototype des quatre fonctions de la bibliothèque. Le second fi-

matrice.c:

```
#include <stdio.h>
#include <stdlib.h>
#include "matrice.h"

/*
   Crée une nouvelle matrice de n lignes et m colonnes, dont les éléments
   sont initialement nuls.
   Retourne un pointeur vers la matrice créée, ou NULL en cas d'erreur.
*/

matrice *creer_matrice(unsigned n, unsigned m)
{
  matrice *r;
  unsigned i, j;

  r = malloc(sizeof(matrice));
  if (!r)
    return NULL;

  r -> el = malloc(n * sizeof(double *));
  if (!(r -> el))
    {
      free(r);
      return NULL;
    }

  for (i = 0; i < n; i++)
    {
      r -> el[i] = malloc(m * sizeof(double));
      if (!(r -> el[i]))
        {
          for (j = 0; j < i; j++)
            free(r -> el[j]);
          free(r -> el);
          free(r);
          return NULL;
        }

      for (j = 0; j < m; j++)
        r -> el[i][j] = 0.0;
    }

  r -> n = n;
  r -> m = m;

  return r;
}
```

FIGURE 6.20 – Bibliothèque de manipulation de matrices (1/3)

```
/*
   Libère l'espace mémoire occupé par la matrice m.
*/

void liberer_matrice(matrice *m)
{
  unsigned i;

  for (i = 0; i < m -> n; i++)
    free(m -> el[i]);

  free(m -> el);
  free(m);
}

/*
   Calcule la somme de deux matrices a et b. Retourne un pointeur vers
   une nouvelle matrice contenant cette somme, ou le pointeur vide en
   cas d'erreur.
*/

matrice *somme_matrices(const matrice *a, const matrice *b)
{
  matrice *s;
  unsigned i, j;

  if (a -> n != b -> n || a -> m != b -> m)
    return NULL;

  s = creer_matrice(a -> n, a -> m);
  if (!s)
    return NULL;

  for (i = 0; i < a -> n; i++)
    for (j = 0; j < a -> m; j++)
      s -> el[i][j] = a -> el[i][j] + b -> el[i][j];

  return s;
}
```

FIGURE 6.21 – Bibliothèque de manipulation de matrices (2/3)

```
/*
   Calcule le produit de deux matrices a et b. Retourne un pointeur
   vers une nouvelle matrice contenant ce produit, ou le pointeur
   vide en cas d'erreur.
*/

matrice *produit_matrices(const matrice *a, const matrice *b)
{
  matrice *p;
  unsigned i, j, k;

  if (a -> m != b -> n)
    return NULL;

  p = creer_matrice(a -> n, b -> m);
  if (!p)
    return NULL;

  for (i = 0; i < a -> n; i++)
    for (j = 0; j < b -> m; j++)
      for (k = 0; k < a -> m; k++)
        p -> el[i][j] += a -> el[i][k] * b -> el[k][j];

  return p;
}
```

matrice.h:

```
/*  Représente une matrice réelle de n lignes et m colonnes  */

typedef struct
{
  double **el;
  unsigned n, m;
} matrice;

matrice *creer_matrice(unsigned, unsigned);
void     liberer_matrice(matrice *);
matrice *somme_matrices(const matrice *, const matrice *);
matrice *produit_matrices(const matrice *, const matrice *);
```

FIGURE 6.22 – Bibliothèque de manipulation de matrices (3/3)

chier fournit l'implémentation de ces fonctions. Comme toujours, le code de chaque fonction est précédé par un commentaire qui en précise les modalités d'utilisation.

Le type `matrice` est composé d'une structure qui comprend trois champs. Les champs `n` et `m` contiennent respectivement le nombre de lignes et le nombre de colonnes de la matrice représentée. Le champ `el` est un pointeur vers un tableau bidimensionnel contenant les éléments de la matrice, de type `double`. De façon similaire à l'exemple du tableau triangulaire, on placera dans `el` un pointeur vers un vecteur de lignes, dont chaque élément pointera vers un vecteur de réels. Le type du champ `el` doit donc correspondre à un pointeur vers un pointeur vers une valeur réelle, et s'écrit donc `double **`.

La fonction `creer_matrice` est basée sur les mêmes principes que ceux mis en œuvre dans la fonction `creer_tableau_triangulaire` de l'exemple précédent ; la difficulté principale est de ne pas oublier de libérer les données déjà allouées dans le cas où un appel à `malloc` échoue. Des opérations de libération similaires sont effectuées dans la fonction `liberer_matrice`.

L'implémentation des fonctions `somme_matrices` et `produit_matrices` appelle peu de commentaires. Ces fonctions commencent par vérifier que les dimensions des deux matrices qui leur sont fournies en arguments sont compatibles : elles doivent posséder le même nombre de lignes et le même nombre de colonnes pour une somme, et le nombre de colonnes de la première matrice doit correspondre au nombre de lignes de la deuxième pour un produit. Si cette condition n'est pas remplie, les fonctions retournent le pointeur vide afin de signaler une situation d'erreur. Sinon, elles créent une nouvelle matrice de taille appropriée, et calculent la valeur à placer dans chacun de ses éléments en suivant les règles du calcul matriciel.

6.4 Les paramètres de `main`

Lorsque nous avons introduit la fonction `main` au chapitre 4, nous avons expliqué que cette fonction constitue le point d'entrée d'un programme, et qu'elle retourne un code de diagnostic entier qui sert à indiquer à la fin de son exécution si ce programme s'est terminé normalement ou bien à la suite d'une erreur.

Il se trouve que cette fonction `main` possède des paramètres ; ceux-ci lui permettent de recevoir des données transmises par le mécanisme qui lance l'exécution d'un programme. On appelle ces données les *arguments d'exécution* du programme. Nous étudions ici les deux premiers [8] de ces paramètres.

— Le premier paramètre de `main`, appelé par convention `argc`, est de type `int` ; il fournit le

8. Certains environnements de programmation en définissent aussi un troisième, que nous n'étudions pas dans ce cours.

nombre d'arguments d'exécution transmis au programme par l'environnement.

— Le deuxième paramètre, habituellement appelé `argv`, est défini par `char *argv[]`; il s'agit donc d'un tableau de chaînes de caractères. La taille de ce tableau est égale à la valeur de `argc`. Chacune des chaînes de caractères qu'il contient correspond à un argument d'exécution.

Quand on lance l'exécution d'un programme en ligne de commande, les arguments d'exécution prennent pour valeur les données qui sont écrites après le nom du programme. Dans la plupart des environnements, ces données sont précédées par un premier argument qui contient le nom du programme. Par exemple, dans un environnement Unix, la ligne de commande

```
% ./test abc 123456
```

lance le programme contenu dans le fichier exécutable `test` situé dans le répertoire courant, en lui fournissant les deux arguments d'exécution "abc" et "123456". La fonction `main` du programme sera donc invoquée avec `argc` égal à 3, et `argv` égal à un pointeur vers un tableau dont les éléments pointent vers trois chaînes de caractères contenant respectivement "./test", "abc" et "123456". Cette situation est illustrée à la figure 6.23. Signalons pour être complet que les éléments du tableau pointé par `argv` qui correspondent aux arguments d'exécution sont suivis par un élément supplémentaire égal au pointeur vide.

Un programme capable d'afficher les arguments d'exécution qui lui sont transmis quand il est lancé est donné à la figure 6.24. Comme on le voit, la fonction `printf` de la bibliothèque standard est capable d'afficher des chaînes de caractères, en employant le marqueur de format dédié "`%s`".

6.5 Exercices

1. Déterminer ce qui sera affiché lors de l'exécution des trois fragments de code suivants.

   (a)
   ```
   int  x[2] = { 0, 0 };
   int *y;

   y = x;
   (++y)[0] = 1;
   printf("%d %d\n", x[0], x[1]);
   ```

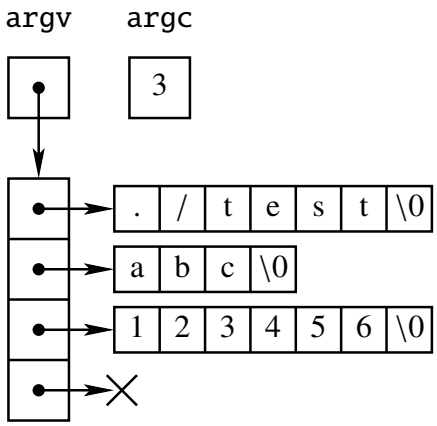


FIGURE 6.23 – Arguments de `main`

```
#include <stdio.h>

int main(int argc, char *argv[])
{
  int i;

  printf("Les arguments du programme sont");

  for (i = 0; i < argc; i++)
    printf(" [%s]", argv[i]);

  printf(".\n");

  return 0;
}
```

FIGURE 6.24 – Programme affichant ses arguments d'exécution

(b)
```
int a, b, *c;

a = 1;
b = 2;
c = &a;
a += *c;
c = &b;
*c-- = 3;
printf("%d %d\n", a, b);
```

(c)
```
struct
{
  int val, *adr;
} x, *y;

x.adr = &x.val;
y = &x;
x.val = 41;
y -> val = 42;
printf("%d\n", *(y -> adr));
```

2. *(Examen de janvier 2019)*

   Quelle est la valeur affichée lors de l'exécution du fragment de code suivant ? (Justifier votre réponse.)

```
int a[] = { 1, 0 };
int *b[2];
int **c;

b[*a] = a + 1;
c = b;

printf("%d\n", **++c);
```

3. *(Examen de juin 2019)*

   Quelle est la valeur affichée lors de l'exécution du fragment de code suivant ? (Justifier votre réponse.)

```
int t[2] = { 1, 2 };
int *a, **b;
```

```
a = t + *t;
b = &a;

printf("%d\n", *--*b);
```

4. *(Examen d'août 2019)*

   Quelle est la valeur affichée lors de l'exécution du fragment de code suivant ? (Justifier votre réponse.)

```
int t[2] = { -2, 0 };
int *a, b;

b = *t ? ++*t : --*t;
a = &b;

printf("%d\n", a[*(t + 1)]);
```

5. *(Examen de janvier 2020)*

   Qu'affiche l'exécution du fragment de code suivant ? (Justifier votre réponse.)

```
char s[] = "42", *t = "1", *u;

u = s;
*++u -= (*t) + 1;

printf("[%s]\n", s);
```

6. *(Examen de janvier 2020)*

   (a) Décrire, le plus simplement possible, l'opération effectuée par la fonction suivante.

```
long f(int *t)
{
  if (!t[0])
    return 1;

  return t[0] * f(t + 1);
}
```

   (b) Quelle est la complexité en temps de cette fonction ?

(c) Écrire une fonction réalisant exactement la même opération, mais sans effectuer d'appel récursif.

7. *(Examen de janvier 2021)*

Expliquer le plus simplement possible (une phrase suffit) l'opération réalisée par cette fonction :

```
int f(char *s)
{
  int r;

  for (r = 0; *s; s++)
    if (*s >= '0' && *s <= '9')
      r = (r + 1) % 2;

  return r;
}
```

8. *(Examen de juin 2021)*

Expliquer le plus simplement possible (une phrase suffit) l'opération réalisée par cette fonction :

```
unsigned f(char *s, char *t)
{
  unsigned n;

  for (n = 0; *s; s++, t++, n++)
    if (*t != *s)
      break;

  return n;
}
```

9. *(Examen d'août 2021)*

Expliquer le plus simplement possible (une phrase suffit) l'opération réalisée par cette fonction :

```
unsigned f(unsigned v[], unsigned nb, unsigned m)
{
 int b;
```

```
  if (!nb)
    return 0;

  b = (*v % m) ? 0 : 1;

  return f(v + 1, nb - 1, m) + b;
}
```

10. *(Examen de juin 2022)*

    (a) Décrire, le plus simplement possible, l'opération effectuée par la fonction suivante.

```
float f(float t[], unsigned n)
{
  unsigned m;

  if (n == 0)
    return 1.0;

  if (n == 1)
    return *t;

  m = n / 2;

  return f(t, m) * f(t + m, n - m);
}
```

    (b) Écrire une fonction réalisant exactement la même opération, mais n'effectuant aucun appel récursif.

11. *(Examen de juin 2023)*

    (a) Expliquer, le plus simplement possible, ce que calcule la fonction suivante :

```
int p(char *s1, char *s2)
{
  if (!*s1)
    return 1;

  if (*s1 != *s2)
    return 0;
```

```
  return p(++s1, ++s2);
}
```

(b) Écrire une fonction réalisant exactement la même opération, mais sans effectuer d'appel récursif.

12. *(Examen de janvier 2024)*

(a) Décrire le plus simplement possible ce que calcule la fonction suivante.

```
unsigned f(char *s, char c)
{
  unsigned r;

  if (!*s)
    return 0;

  r = f(s + 1, c);

  if (*s == c)
    ++r;

  return r;
}
```

(b) Quelle est la complexité en espace de cette fonction ?

(c) Écrire une fonction réalisant exactement la même opération que la fonction donnée au point (a), mais sans effectuer d'appel récursif. Pour résoudre ce problème, il n'est pas permis de faire appel à des fonctions issues de la bibliothèque standard.

13. *(Examen de juin 2025)*

(a) Décrire le plus simplement possible ce que calcule la fonction suivante :

```
unsigned f(char *s)
{
  if (!*s)
    return 0;

  return f(s + 1) + (*s == ' ' || *s == '\n') ? 0 : 1;
}
```

(b) Quelle est la complexité en espace de cette fonction ? (Justifier votre réponse.)

(c) Écrire une fonction réalisant exactement la même opération, mais sans effectuer d'appel récursif. Il n'est pas permis d'utiliser les fonctions de la librairie standard.

14. *(Examen d'août 2025)*

(a) Décrire le plus simplement possible (une phrase suffit) ce que calcule la fonction suivante :

```
int f(int *a, unsigned b)
{
  if (b <= 1)
    return 1;

  return (a[b - 2] <= a[b - 1]) && f(a, b - 1);
}
```

(b) Quelle est la complexité en espace de la fonction du point (a) ?

(c) Écrire une fonction réalisant exactement la même opération que la fonction du point (a), mais sans effectuer d'appel récursif.

15. Soit une fonction `int rand(int min, int max)` qui retourne un nombre entier aléatoire compris dans l'intervalle [`min`, `max`]. On demande d'écrire une fonction qui alloue un tableau d'entiers de taille donnée et qui remplit celui-ci avec des entiers aléatoires compris dans un intervalle donné.

La fonction demandée est déclarée de la façon suivante :

```
int rand_tab(unsigned int n, int min, int max, int **dest);
```

où

— `n` est la taille du tableau,

— `min` et `max` sont les bornes inférieure et supérieure des nombres aléatoires à insérer dans le tableau,

— `dest` la destination du tableau généré.

La fonction retourne une valeur différente de 0 si le tableau a pu être généré et 0 si ce n'est pas le cas (en particulier, si `min > max` ou `dest == NULL`).

16. Soit une fonction `int f(int *x)` donnée. Cette fonction est capable de produire un nombre limité et a priori inconnu d'entiers. Lorsque `f` est appelée, si un entier est produit, il est stocké dans l'espace pointé par `x` et `f` retourne une valeur différente de 0. Sinon, `f` retourne 0 et aucun appel suivant ne produira d'entier.

    On demande d'écrire un programme qui remplit, au moyen de `f`, un tableau d'entiers t de taille `n` donnée ($n \geq 0$) à partir de l'indice 0 et tant que `f` produit un nombre. À la fin de l'exécution du programme, la variable `nb` contiendra le nombre d'entiers que `f` a produit (avec $nb \leq n$).

17. *(Examen de juin 2023)*

    (a) Écrire une procédure prenant en arguments un pointeur vers un vecteur `v` d'octets non signés, le nombre `n` d'éléments de ce vecteur, et un pointeur vers un vecteur `b` de 256 entiers. L'opération devant être effectuée par cette procédure consiste, pour chaque $i \in [0, 255]$, à placer dans `b[`i`]` une valeur booléenne vraie si le vecteur `v` contient la valeur i (à n'importe quelle position), et fausse sinon. On demande que l'implémentation de cette fonction soit la plus efficace possible.

    (b) Calculer la complexité en temps de la procédure obtenue au point (a).

18. *(Examen de juin 2023)*

    Écrire une fonction prenant en arguments deux chaînes de caractères, et retournant un pointeur vers une chaîne nouvellement allouée égale à leur concaténation, c'est-à-dire, au contenu de la première chaîne immédiatement suivi par celui de la seconde. Par exemple, pour les chaînes `"infor"` et `"matique"`, le résultat doit contenir `"informatique"`.

    Notes :
    — Vous pouvez programmer des fonctions ou des types de données supplémentaires si votre solution le nécessite.

    — Il est permis d'employer des fonctions de la bibliothèque standard, en particulier la fonction `strlen(str)` qui retourne la longueur de la chaîne de caractères pointée par `str`. Le fichier d'en-tête correspondant est `string.h`.

19. *(Examen d'août 2023)*

Écrire une fonction prenant en arguments deux tableaux d'entiers `t1` et `t2` ainsi que la taille (commune) `n`, supposée non nulle, de ces tableaux. Cette fonction doit retourner un pointeur vers un tableau nouvellement alloué de taille 2`n`, contenant l'entrelacement des éléments de `t1` et `t2` : si `t1` contient les éléments `t1[0]`, `t1[1]`, …, `t1[n − 1]` et `t2` les éléments `t2[0]`, `t2[1]`, …, `t2[n − 1]`, alors le tableau construit par la fonction doit contenir les éléments `t1[0]`, `t2[0]`, `t1[1]`, `t2[1]`, …, `t1[n − 1]`, `t2[n − 1]`, dans cet ordre. En cas d'erreur, la fonction doit retourner le pointeur vide.

20. *(Examen de juin 2024)*

Le *triangle de Pascal* est une construction mathématique constituée d'une séquence de lignes de longueur croissante : pour $i = 0, 1, 2, \ldots$, la ligne d'indice i contient $i+1$ nombres entiers $C_i^0, C_i^1, \ldots, C_i^i$, définis de la façon suivante.

— Le premier et le dernier élément de chaque ligne sont égaux à 1 :

$$\forall i \geq 0 : \; C_i^0 = C_i^i = 1.$$

— Si les lignes sont centrées les unes sur les autres, alors chaque élément autre que le premier et le dernier est égal à la somme des deux éléments de la ligne précédente qui sont situés immédiatement à sa gauche et à sa droite :

$$\forall 0 < j < i : \; C_i^j = C_{i-1}^{j-1} + C_{i-1}^j.$$

À titre d'exemple, les six premières lignes du triangle de Pascal (correspondant aux indices de 0 à 5) sont les suivantes.

$$\begin{array}{ccccccccccc}
 & & & & & 1 & & & & & \\
 & & & & 1 & & 1 & & & & \\
 & & & 1 & & 2 & & 1 & & & \\
 & & 1 & & 3 & & 3 & & 1 & & \\
 & 1 & & 4 & & 6 & & 4 & & 1 & \\
1 & & 5 & & 10 & & 10 & & 5 & & 1
\end{array}$$

(a) Écrire une fonction *pascal* prenant en argument l'indice i d'une ligne du triangle de Pascal, et retournant un pointeur vers un vecteur nouvellement alloué contenant les éléments de cette ligne.

Suggestion : Allouer un vecteur de taille appropriée, et y écrire successivement les lignes d'indice 0, 1, …, i du triangle.

(b) Calculer la complexité en temps et en espace de la fonction obtenue au point (a).

21. *(Examen de janvier 2019)*

(a) Écrire un fragment de code définissant les deux types structurés suivants :

— un type `point` représentant les coordonnées cartésiennes (x, y) d'un point dans le plan, où x et y sont des valeurs réelles.

— un type `segment` représentant un segment de droite caractérisé par son origine et sa destination (qui sont des points).

(b) Écrire une fonction prenant en arguments quatre réels x_1, y_1, x_2 et y_2 définissant les extrémités (x_1, y_1) et (x_2, y_2) d'un segment de droite, et retournant un pointeur vers une représentation de ce segment, nouvellement allouée.

(c) Écrire une fonction permettant de libérer une représentation d'un segment créée par la fonction obtenue au point (b).

22. *(Examen de juin 2019)*

(a) Écrire un fragment de code définissant un type structuré capable de retenir le nom et le prénom d'une personne, tous deux représentés sur au plus 20 caractères.

(b) Écrire une fonction prenant en argument des pointeurs vers deux instances du type obtenu à la question précédente, et retournant une valeur booléenne qui indique si les deux personnes concernées possèdent le même nom et le même prénom. La comparaison doit tenir compte des caractères non alphabétiques, mais pas de la casse (majuscule ou minuscule) des lettres ; par exemple, les noms "le petit" et "Le Petit" sont considérés égaux, mais différents de "Lepetit". Il est permis de définir des fonctions auxiliaires de votre choix, mais pas de faire appel à des fonctions issues d'une bibliothèque.

23. *(Examen d'août 2019)*

(a) Écrire un fragment de code définissant un type structuré capable de représenter un intervalle $[a, b]$ de nombres entiers, avec $a \in \mathbb{Z} \cup \{-\infty\}$ et $b \in \mathbb{Z} \cup \{+\infty\}$ tels que $a \leq b$.

(b) Écrire une fonction prenant en argument des pointeurs vers deux instances du type obtenu à la question précédente, et retournant une valeur booléenne qui indique si le premier intervalle est entièrement inclus dans le second. (Par exemple l'intervalle $[1, 4]$ est inclus dans $[0, +\infty]$, mais pas dans $[-2, 3]$.) Il est permis de définir des fonctions auxiliaires de votre choix.

24. *(Examen de janvier 2020)*

(a) Écrire un fragment de code définissant une liste simplement liée d'entiers, c'est-à-dire une structure de données composée d'un ensemble ordonné d'éléments, tels que chacun mémorise une valeur entière et contient un pointeur vers son successeur.

(b) Écrire une fonction allouant une liste liée vide.

(c) Écrire une fonction capable d'insérer un entier donné en tête d'une liste liée donnée.

(d) Écrire une fonction capable de calculer la somme des valeurs entières contenues dans une liste liée donnée.

Note : Pour résoudre ces problèmes, vous êtes libre de définir des structures de données et des fonctions auxiliaires de votre choix.

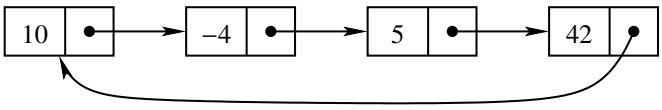
25. *(Examen de janvier 2021)*

Un *tampon circulaire* est une structure de données composée d'un certain nombre $k > 0$ d'éléments ordonnés $e_1, e_2, \ldots e_k$, telle que chaque élément e_i contient

— une valeur (qu'on suppose être un nombre entier par souci de simplicité), et

— un pointeur vers l'élément e_{i+1} si $i < k$, ou vers e_1 si $i = k$.

Un exemple de tampon circulaire contenant les valeurs 10, −4, 5 et 42 est représenté ci-après :



(a) Définir un type structuré permettant de représenter un élément d'un tampon circulaire.

(b) Écrire une fonction prenant en arguments un tableau d'entiers et sa taille (supposée non nulle), et construisant un tampon circulaire contenant ces entiers, dans le même ordre que dans le tableau. Chaque élément de cette structure doit être alloué dynamiquement en mémoire. La fonction doit retourner un pointeur vers le premier élément de la structure de données nouvellement créée.

(c) Écrire une fonction qui calcule la somme de toutes les valeurs contenues dans un tampon circulaire, donné par un pointeur vers son premier élément.

(d) Écrire une fonction qui libère tous les éléments d'un tampon circulaire, donné par un pointeur vers son premier élément.

26. *(Examen de juin 2021)*

(a) Écrire un fragment de code définissant un type structuré permettant de représenter une sphère dans un espace à trois dimensions. Une sphère est caractérisée par les coordonnées (x, y, z) de son centre et par son rayon r. Les valeurs de x, y, z et r sont réelles.

(b) Écrire une fonction prenant en arguments quatre réels x, y, z et r, et retournant un pointeur vers une représentation de la sphère correspondante, nouvellement allouée. Si la valeur fournie pour le rayon r est négative ou nulle, alors la fonction doit retourner un code d'erreur, sous la forme du pointeur vide.

(c) Écrire une fonction prenant en arguments un tableau `s` de pointeurs vers des représentations de sphères et le nombre `nb` d'éléments de ce tableau, supposé non nul. Cette fonction doit retourner un pointeur vers la plus petite sphère pointée par les éléments de `s`. (Dans le cas où plusieurs sphères sont de plus petite taille, la fonction peut choisir arbitrairement l'une d'entre elles.)

27. *(Examen d'août 2021)*

(a) Écrire un fragment de code définissant un type structuré représentant un tableau bidimensionnel de chaînes de 100 caractères ou moins. La dimension de ce tableau, c'est-à-dire son nombre de lignes et son nombre de colonnes, doit être retenue dans la structure.

(b) Écrire une fonction prenant en arguments un nombre de lignes n et un nombre de colonnes m, et retournant un pointeur vers une structure nouvellement allouée (de même type que votre réponse à (a)), contenant un tableau de $n \times m$ chaînes initialement vides.

(c) Écrire une fonction prenant en arguments deux pointeurs vers des représentations de tableaux (de même type que votre réponse à (a)), et retournant une valeur booléenne indiquant si ces tableaux sont égaux. Deux tableaux sont égaux s'ils possèdent la même dimension, et si pour chaque position (i, j), les chaînes figurant à cette position dans les deux tableaux ont un contenu identique.

28. *(Examen de janvier 2022)*

(a) Écrire un fragment de code définissant un type structuré capable de représenter un tableau bidimensionnel d'entiers dont chaque ligne est de taille quelconque. (Cela signifie en particulier que deux lignes distinctes de ce tableau peuvent être de taille différente.) La taille de chaque ligne du tableau doit être retenue dans la structure.

(b) Écrire une fonction prenant en arguments un vecteur `tailles` d'entiers non signés et un entier non signé `nb`, et retournant un pointeur vers une représentation nouvellement allouée d'un tableau à `nb` lignes, dans lequel la taille de la ligne d'indice i est égale à `tailles[`i`]`, pour $i = 0, 1, 2, \ldots,$ `nb` $- 1$. Tous les éléments de ce tableau doivent être initialisés à 0. En cas d'erreur, la fonction doit retourner le pointeur vide.

(c) Écrire une fonction permettant de libérer une représentation d'un tableau créée par la fonction obtenue au point (b).

(d) Écrire le prototype d'une fonction capable de déterminer si deux tableaux bidimensionnels d'entiers possèdent la même forme, c'est-à-dire le même nombre de lignes et des lignes mutuellement de même taille. Les deux tableaux à comparer sont fournis sous la forme de pointeurs vers leur représentation, du type obtenu au point (a), et la fonction doit retourner une valeur booléenne. On souhaite que la portée de cette fonction soit limitée au fichier source dans lequel elle est définie. On ne demande pas de fournir l'implémentation de cette fonction.

29. *(Examen de juin 2022)*

Pour représenter un vecteur d'entiers dont le nombre d'éléments peut être très variable, on le décompose en blocs d'éléments consécutifs, et on place ces blocs dans une liste liée. En d'autres termes, chaque bloc possède un pointeur vers le bloc suivant, à l'exception du dernier bloc pour lequel ce pointeur est vide. Les blocs apparaissent dans la liste liée dans le même ordre que dans le vecteur. Chaque bloc contient exactement 1000 éléments du vecteur, sauf le dernier qui est plus petit dans le cas où le nombre total d'éléments du vecteur n'est pas un multiple exact de 1000.

(a) Écrire un fragment de code définissant un type structuré capable de représenter un tel bloc.

(b) Écrire une fonction prenant en arguments un vecteur d'entiers (fourni sous la forme d'un tableau) et le nombre d'éléments de ce vecteur, supposé non nul, et construisant la représentation de ce vecteur en allouant dynamiquement les blocs qui la composent. La fonction doit retourner un pointeur vers le premier bloc de la représentation obtenue, ou le pointeur vide en cas d'erreur.

(c) Écrire une fonction prenant en argument un pointeur vers le premier bloc de la représentation d'un vecteur, et retournant la taille de celui-ci (c'est-à-dire le nombre total d'entiers qu'il contient).

30. *(Examen d'août 2022)*

Un traitement de textes représente une ligne de texte sous la forme d'une liste simplement liée de mots. Chaque élément de cette liste contient un pointeur vers une chaîne de caractères représentant un mot, ainsi qu'un pointeur vers l'élément suivant (qui est vide dans le cas du dernier élément).

On considère qu'une ligne de texte est formée par la concaténation de tous les mots qui la composent, chaque paire de mots consécutifs étant séparés par un espace.

Par exemple, la ligne de texte composée des quatre mots `"introduction"`, `"a"`, `"l"` et `"informatique"` est `"introduction a l informatique"`.

(a) Écrire un fragment de code définissant un type structuré pour un élément d'une liste liée représentant une ligne de texte.

(b) Écrire une fonction qui prend en argument un pointeur vers le premier élément d'une liste liée représentant une ligne de texte, et qui retourne la longueur de cette ligne, c'est-à-dire son nombre total de caractères.

(c) Écrire une fonction qui prend en argument un pointeur vers le premier élément d'une liste liée représentant une ligne de texte, et qui retourne une chaîne de caractères nouvellement allouée contenant cette ligne de texte, ou le pointeur vide en cas d'erreur.

Notes : Pour résoudre ces problèmes, il est permis d'exploiter les fonctions de la bibliothèque standard vues au cours. Il est évidemment aussi permis d'utiliser au point (c) la fonction obtenue au point (b).

31. *(Examen de janvier 2023)*

(a) Écrire un fragment de code définissant un type structuré capable de représenter une séquence de mots. Une instance de ce type est composée d'un champ `nb_mots` représentant le nombre de mots qui appartiennent à la séquence, et d'un champ `mots` pointant vers un vecteur de `nb_mots` pointeurs vers des chaînes de caractères, correspondant chacune à un mot de la séquence.

(b) Écrire une fonction prenant en argument une chaîne de caractères, et retournant un pointeur vers une représentation nouvellement allouée de la séquence des mots qui

composent cette chaîne. On considère que ces mots sont délimités dans la chaîne par le caractère d'espacement `' '`. Par exemple, la chaîne de caractères contenant `"Introduction à l'informatique"` est composée des mots `"Introduction"`, `"à"` et `"l'informatique"`. Le nombre de mots et leur longueur ne sont pas connus à l'avance, et peuvent grandement varier d'une chaîne à une autre. En cas d'erreur, la fonction doit retourner le pointeur vide.

Notes :

— Vous pouvez programmer des fonctions ou des types de données supplémentaires si votre solution le nécessite.

— Il est permis d'employer la fonction `strncpy(dest, src, nb)` de la bibliothèque standard, qui recopie la chaîne de caractères pointée par `src` à l'endroit donné par `dest`, en écrivant au plus `nb` caractères. Le fichier d'en-tête correspondant est `string.h`.

(c) Écrire une fonction permettant de libérer une représentation d'une séquence de mots créée par la fonction obtenue au point (b).

32. *(Examen de juin 2023)*

(a) Écrire un fragment de code définissant un type structuré représentant une date, composé de trois champs numériques correspondant à un jour, un mois et une année.

(b) Écrire une fonction recevant en arguments deux pointeurs vers des dates (représentées grâce à la structure définie au point (a)), et retournant une valeur booléenne indiquant si la première de ces dates est antérieure à la seconde. *Note :* On ne demande pas de vérifier que les dates reçues en entrée sont valides.

33. *(Examen d'août 2023)*

On décide de représenter une séquence de chaînes de caractères par une structure de données composée d'un tableau de pointeurs vers ces chaînes et du nombre de chaînes contenues dans la séquence. La taille du tableau est fixée et le nombre de chaînes peut varier d'une séquence à une autre. L'ordre des pointeurs dans le tableau correspond à l'ordre des chaînes dans la séquence représentée.

(a) Écrire un fragment de code définissant un type structuré correspondant à cette structure de données. La taille du tableau peut être librement choisie.

(b) Écrire une fonction recevant en arguments deux pointeurs vers des séquences de

chaînes de caractères, représentées grâce à la structure définie au point (a), et retournant une valeur booléenne indiquant si ces séquences sont identiques. Deux séquences sont considérées comme étant identiques si elles sont composées du même nombre de chaînes, et si ces chaînes sont égales deux à deux (c'est-à-dire, si les premières chaînes de chaque séquence sont égales, si les deuxièmes sont égales, et ainsi de suite).

Note : Pour comparer les chaînes de caractères, il est permis d'utiliser la fonction `strcmp` de la bibliothèque standard : `strcmp(s1, s2)` retourne 0 si les chaînes pointées par `s1` and `s2` sont égales, et une valeur non nulle sinon. Le fichier d'en-tête correspondant est `string.h`.

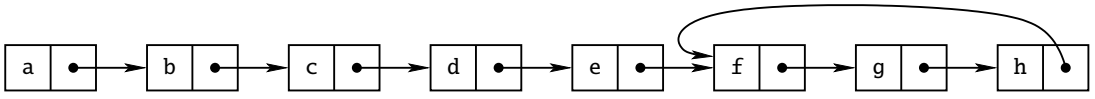
34. *(Examen de janvier 2024)*

Un outil de manipulation d'automates utilisé pour une application d'informatique théorique doit construire des *structures de lasso*, définies de la façon suivante : Une structure de lasso est caractérisée par deux chaînes de caractères b et p non vides, respectivement appelées *base* et *période*, et dont les longueurs sont notées $\ell(b)$ et $\ell(p)$. La structure est composée de $\ell(b) + \ell(p)$ éléments $e_1, e_2, \ldots, e_{\ell(b)+\ell(p)}$.

Le premier élément e_1 est appelé l'élément *initial*. Chaque élément contient un caractère et un pointeur. Les caractères contenus dans $e_1, e_2, \ldots, e_{\ell(b)}$ sont ceux de la base b, dans le même ordre que dans cette chaîne. Les caractères contenus dans $e_{\ell(b)+1}, e_{\ell(b)+2}, \ldots, e_{\ell(b)+\ell(p)}$ sont ceux de la période p, également dans le même ordre.

Chaque élément e_i tel que $1 \leq i < \ell(b) + \ell(p)$ contient un pointeur vers l'élément suivant e_{i+1}. Le dernier élément $e_{\ell(b)+\ell(p)}$ contient quant à lui un pointeur vers $e_{\ell(b)+1}$, c'est-à-dire vers le premier élément associé à la période.

Par exemple, le diagramme suivant représente une structure de lasso correspondant à la base `"abcde"` et à la période `"fgh"`. L'élément initial est celui situé à gauche.



(a) Écrire un fragment de code définissant un type structuré pour un élément d'une structure de lasso. *Note :* Vous avez la liberté d'insérer dans les éléments des données supplémentaires de votre choix. La base et la période d'une structure de lasso doivent cependant être uniquement représentées par les caractères figurant dans les éléments de cette structure, et non comme des chaînes de caractères mémorisées séparément.

(b) Écrire une fonction prenant en arguments une base et une période, fournies sous la

forme de deux chaînes de caractères *b* et *p*. Cette fonction doit retourner un pointeur vers l'élément initial d'une structure de lasso nouvellement allouée, correspondant à cette base et cette période. (Par exemple, appeler cette fonction avec les arguments `"abcde"` et `"fgh"` doit conduire à créer une structure de lasso identique à celle du diagramme précédent.) Le type des éléments de la structure doit bien sûr être celui de votre réponse au point (a).

En cas d'erreur, la fonction doit retourner le pointeur vide. Il est permis de supposer que les chaînes *b* et *p* passées en arguments sont non vides. Vous pouvez si vous le souhaitez utiliser la fonction `strlen` de la bibliothèque standard pour calculer la longueur de chaînes de caractères.

(c) Écrire une fonction prenant en argument un pointeur vers le premier élément d'une structure de lasso créée par la fonction développée en réponse au point (b), et libérant la mémoire occupée par cette structure.

35. *(Examen de juin 2024)*

Une application de gestion d'itinéraires de randonnée représente un *parcours* sous la forme d'une liste simplement liée dont chaque élément (appelé *lieu*) est composé d'une chaîne de caractères donnant le nom du lieu, ainsi qu'un pointeur vers le prochain lieu à visiter. Un parcours se termine toujours par son point de départ, c'est-à-dire que le dernier élément de la liste doit pointer vers le premier.

(a) Écrire un fragment de code définissant un type structuré capable de représenter un lieu d'un parcours.

Note : Une instance de ce type ne doit pas mémoriser le contenu de la chaîne de caractères décrivant le lieu, mais doit seulement contenir un pointeur vers cette chaîne.

(b) Écrire une fonction prenant en argument une chaîne de caractères, et retournant un pointeur vers une représentation nouvellement allouée du parcours dont les noms de lieux figurent successivement dans la chaîne de caractères, séparés par un espace. Par exemple, pour la chaîne `"B4 B7b Montefiore"`, le parcours doit être composé de trois lieux correspondant à `"B4"`, `"B7b"` et `"Montefiore"`, dans cet ordre.

La fonction doit retourner un pointeur vers le premier lieu de la représentation qu'elle a construit, de même type que votre réponse au point (a), ou le pointeur vide en cas d'erreur.

Notes :
— Vous pouvez programmer des fonctions ou des types de données supplémentaires si votre solution le nécessite.

— Vous pouvez si vous le souhaitez modifier le contenu de la chaîne fournie en argument, et l'utiliser comme destination des pointeurs contenus dans les éléments de la liste liée.

(c) Écrire une fonction prenant en argument un pointeur vers le premier lieu d'un parcours créé par la fonction du point (b). Cette fonction doit retourner le nombre de lieux qui composent ce parcours, et libérer la représentation de celui-ci.

36. *(Examen d'août 2024)*

Une application de gestion de notes d'étudiants représente ces étudiants comme les éléments d'une liste simplement liée. Chacun de ces éléments est composé du matricule d'un étudiant, formé par 8 caractères, et d'un pointeur vers l'élément suivant dans la liste, ce pointeur étant vide pour le dernier élément. La liste liée retient uniquement un pointeur vers son premier élément (qui est vide si cet élément n'existe pas), ainsi que le nombre d'éléments qu'elle contient.

(a) Écrire un fragment de code définissant deux types structurés : le premier pour un élément de liste liée représentant un étudiant, et le second pour une liste simplement liée représentant une séquence d'étudiants.

(b) Écrire une fonction prenant en arguments un pointeur vers la représentation d'une liste liée ℓ d'étudiants, et un tableau de caractères contenant le matricule m d'un étudiant supplémentaire. Si le matricule m n'est pas déjà présent dans ℓ, alors la fonction doit ajouter l'étudiant correspondant à la fin de cette liste et retourner 0. Sinon, la fonction doit laisser ℓ inchangée et retourner -1.

Notes : Les types des éléments de la liste liée et de la liste liée elle-même doivent correspondre à votre réponse au point (a). Vous pouvez programmer des fonctions ou des types de données supplémentaires si votre solution le nécessite.

(c) Écrire une fonction prenant en argument un pointeur vers la représentation d'une liste liée, compatible avec votre réponse au point (a), et libérant l'ensemble de la mémoire allouée à cette liste et à ses éléments.

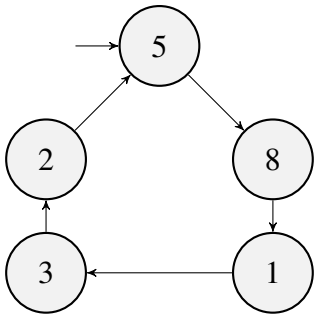
37. *(Examen de janvier 2025)*

Un *graphe cyclique* de taille $n > 0$ est composé d'un ensemble de n *nœuds*, chacun d'entre eux étant caractérisé par un *identifiant* entier qui lui est propre et par un pointeur vers un nœud appelé son *successeur*. Un des nœuds du graphe est distingué comme étant *initial*.

Pour être valide, le graphe doit former un unique cycle. Cela signifie que si on le

parcourt en passant répétitivement d'un nœud à son successeur, en commençant à partir d'un nœud quelconque, on finira toujours par visiter tous les nœuds.

Par exemple, la figure suivante montre un graphe cyclique de taille 5, dont les identifiants des nœuds forment l'ensemble $\{1, 2, 3, 5, 8\}$. Le successeur de chaque nœud est représenté par une flèche, et la flèche sans origine indique que le nœud initial est celui d'identifiant 5.



(a) Écrire des fragments de code définissant deux types structurés permettant de représenter un nœud d'un graphe cyclique, et un graphe cyclique. On demande que chaque champ de ces types structurés corresponde soit à l'identifiant d'un nœud, soit à un pointeur vers un nœud.

(b) Écrire une fonction prenant en arguments un tableau d'identifiants et la longueur de ce tableau, et retournant un pointeur vers une représentation nouvellement allouée d'un graphe cyclique dont les nœuds successifs correspondent aux éléments du tableau. Le premier élément du tableau définit le nœud initial. Par exemple, si la fonction reçoit en entrée le tableau $[\,5, 8, 1, 3, 2\,]$, alors elle doit produire le graphe cyclique de la figure précédente. Le type de la représentation générée doit bien sûr être compatible avec votre réponse au point (a).

(c) Écrire une procédure acceptant en argument un pointeur vers une représentation d'un graphe cyclique, chargée d'afficher dans l'ordre l'identifiant de tous les nœuds visités lors d'un parcours de ce graphe. Par exemple, pour le graphe de la figure précédente, elle doit afficher "`5 8 1 3 2`". On demande également que cette procédure libère tout l'espace mémoire occupé par la représentation du graphe.

Note : Vous pouvez programmer des fonctions ou des types de données supplémentaires si votre solution le nécessite.

38. *(Examen de juin 2025)*

Une liste simplement liée contient une séquence $e_1; e_2; \ldots; e_n$ d'éléments e_i composés chacun d'une valeur entière et d'un pointeur vers l'élément suivant e_{i+1} dans la séquence. Ce pointeur est vide dans le cas où $i = n$.

(a) Écrire un fragment de code définissant un type structuré pour un élément de liste liée.

(b) Écrire une fonction prenant en argument une liste liée ℓ, donnée sous la forme d'un pointeur vers son premier élément, ou d'un pointeur vide dans le cas où ℓ ne contient aucun élément. Cette fonction doit construire une nouvelle liste liée ℓ' en allouant dynamiquement chacun de ses éléments, de façon à ce que ℓ' soit une copie conforme de ℓ. En d'autres termes, ℓ' doit être composée du même nombre d'éléments que ℓ, et les valeurs entières contenues dans ces deux listes doivent correspondre, dans le même ordre. La fonction doit retourner un pointeur vers le premier élément de ℓ', ou un pointeur vide si ℓ' ne contient aucun élément.

(c) Écrire une fonction prenant en argument une liste liée ℓ, donnée de la même façon qu'au point précédent, et retournant la somme des valeurs entières contenues dans les éléments de ℓ. On demande également que cette procédure libère tout l'espace mémoire occupé par les éléments de ℓ, en supposant qu'ils ont été précédemment alloués dynamiquement.

Note : Vous pouvez programmer des fonctions ou des types de données supplémentaires si votre solution le nécessite.

39. *(Examen d'août 2025)*

Un navigateur WWW enregistre l'historique d'une session en mémorisant les pages Web qui ont été visitées, ainsi que le moment où elles ont été visitées. Chaque page Web est identifiée par une chaîne de caractères appelée URL (*Uniform Resource Locator*), par exemple `"https://www.uliege.be"`. La date et l'heure d'une visite sont représentées par un seul nombre entier de type `unsigned long` correspondant au nombre de secondes écoulées depuis une date de référence fixée au premier janvier 1970 à 0h00. De plus, pour chaque page faisant partie de l'historique, on conserve un moyen d'accès à la page précédente et à la page suivante (par rapport à l'ordre chronologique où elles ont été visitées), de façon à pouvoir naviguer en arrière ou en avant.

(a) Écrire un fragment de code définissant un type structuré capable de représenter une page Web contenue dans un historique. Ce type structuré doit comprendre

— un pointeur vers une chaîne de caractères donnant l'URL de la page,

— la représentation de la date et l'heure de visite de la page,
— deux pointeurs vers la page précédente et la page suivante dans l'historique. (Si l'une de ces pages n'existe pas, le pointeur concerné est alors vide.)

(b) Écrire une fonction prenant en arguments un tableau `url` de chaînes de caractères constituant les URLs de pages visitées, dans l'ordre chronologique de leur visite, un tableau `temps` d'entiers contenant les dates et heures de visite de ces pages (en d'autres termes, `temps`[*i*] contient le moment où la page `url`[*i*] a été visitée), et la taille `n` commune de ces deux tableaux, supposée non nulle. On demande que la fonction crée un nouvel historique de navigation pour les `n` pages fournies, en allouant dynamiquement la mémoire nécessaire, et retourne un pointeur vers la représentation de la première page faisant partie de cet historique.

Note : Vous pouvez programmer des fonctions ou des types de données supplémentaires si votre solution le nécessite, et utiliser des fonctions issues de la bibliothèque standard.

(c) Écrire une fonction prenant en argument un pointeur vers n'importe quelle page d'un historique de navigation, et qui retourne le nombre de secondes écoulées entre les moments où la première et la dernière page de cet historique ont été visitées.

40. *(Examen de janvier 2026)*

Un jeu vidéo représente son aire de jeu par une matrice de n lignes et m colonnes, ces valeurs étant supposées non nulles. Chaque élément de cette matrice contient la couleur d'un pixel, composée de trois valeurs entières r, g et b comprises entre 0 et 255.

(a) Écrire un fragment de code définissant deux types structurés `couleur` et `aire` capables de représenter respectivement la couleur d'un pixel et une aire de jeu de dimension quelconque. Les nombres de lignes et de colonnes d'une aire de jeu doivent faire partie de sa représentation.

(b) Écrire une fonction prenant en arguments un nombre de lignes, un nombre de colonnes et une couleur, et retournant un pointeur vers une représentation nouvellement allouée d'une aire de jeu possédant ce nombre de lignes et de colonnes, dont tous les pixels sont initialisés à cette couleur. En cas d'erreur, la fonction doit retourner le pointeur nul.

(c) Écrire une fonction prenant en arguments un pointeur vers la représentation d'une aire de jeu, les indices d'une ligne et d'une colonne (supposés numérotés à partir de 0) et

une couleur, et attribuant cette couleur au pixel de l'aire de jeu situé à cette ligne et cette colonne. La couleur des autres pixels de l'aire de jeu reste inchangée.

(d) Écrire une fonction prenant en arguments un pointeur vers la représentation d'une aire de jeu, et retournant 1 si tous les pixels qu'elle contient sont de la même couleur, et 0 sinon.

Note : Vous pouvez définir des fonctions et types de données supplémentaires de votre choix, ainsi qu'utiliser des fonctions de la bibliothèque standard.

41. *(Examen de juin 2026)*

Un vecteur creux est un vecteur contenant beaucoup d'éléments nuls. Afin d'économiser de l'espace, on souhaite représenter un tel vecteur en ne mémorisant que ses éléments non nuls. Plus précisément la représentation d'un vecteur creux $\vec{v}$ est une structure de données composée de

— la dimension n de $\vec{v}$,
— le nombre m d'éléments non nuls de $\vec{v}$,
— un tableau t de taille m contenant ces éléments. Chacun d'entre eux est caractérisé par une paire (i, r) formée par sa position $i \in [0, n-1]$ dans $\vec{v}$ et sa valeur $r \in \mathbb{R}$. Les éléments de $\vec{v}$ apparaissent dans t par ordre strictement croissant de position.

Par exemple, pour le vecteur $\vec{v} = [0, 0, (3{,}5), 0, 0, 0, (-8{,}3)]$, on a $n = 7$, $m = 2$ et $t = [(2, (3{,}5)), (6, (-8{,}3))]$.

(a) Écrire un fragment de code définissant un type structuré permettant de représenter un vecteur creux.

(b) Écrire une fonction prenant en arguments un tableau de réels et sa taille, et retournant une représentation nouvellement allouée de ce tableau sous la forme d'un vecteur creux.

(c) Écrire une fonction prenant en argument un pointeur vers la représentation d'un vecteur creux, et retournant la somme de tous ses éléments. Par exemple, pour le vecteur précédemment donné en illustration, cette fonction doit retourner $-4{,}8$.

Note : Il est permis de définir des types ou des fonctions supplémentaires si votre solution le nécessite.

Chapitre 7

Les piles et les files

Dans ce chapitre, nous n'allons pas introduire de nouveau mécanisme de programmation, mais appliquer ceux qui ont été étudiés aux chapitres précédents afin de développer deux structures de données importantes : les piles et les files. Nous allons montrer en particulier que ces structures peuvent être implémentées de différentes façons, tout en présentant la même interface au code qui les utilise.

7.1 Les structures de données

Dans le contexte particulier du langage C, nous avons introduit les données structurées au chapitre 6. En informatique, le concept de *structure de données* est plus général, et peut être vu comme une extension de la notion de *type* discutée pour la première fois au chapitre 2.

Le fait d'attribuer un type à une variable précise deux choses : la nature des données qui peuvent être placées dans cette variable (par exemple, en langage C, une variable de type `double` peut retenir la valeur approximée d'un nombre réel), et l'ensemble des opérations que l'on peut appliquer à ces données (par exemple, l'opérateur "`/`" appliqué à des valeurs de type `double` indique une division réelle).

De la même façon, la définition d'une *structure de données* spécifie la nature des données mémorisées dans les instances de cette structure, ainsi que la liste des opérations qu'il est possible de leur appliquer.

Une structure de données possède une *interface* et une *implémentation*, tout comme les fonctions étudiées au chapitre 4. L'interface est une documentation permettant d'exploiter la structure de données, en caractérisant rigoureusement les données qu'elle représente et en fournissant le mode d'emploi des opérations qu'elle permet d'effectuer. L'implémentation précise quant à elle

tous les détails internes de la représentation des données au sein de la structure et des opérations qu'elle définit. Pour utiliser une structure de données, il est suffisant de connaître son interface.

7.2 Les piles

7.2.1 Définition

Une *pile (stack)* est une structure de données capable de retenir une collection de valeurs. Cette structure offre la possibilité d'ajouter un nouvel élément à la collection, de retirer un élément, et d'obtenir certaines informations sur le contenu courant de la collection. Une pile est caractérisée par le fait que les valeurs qu'elle contient sont accessibles selon une politique *LIFO (Last In First Out)*. Cela signifie qu'on ne peut à chaque instant retirer de la pile que l'élément qui, parmi tous ceux qu'elle contient, a été ajouté en dernier lieu. En d'autres termes, une structure de pile permet d'accéder à ses éléments dans l'ordre inverse où ils y sont placés.

Pour bien comprendre le principe de fonctionnement d'une pile, on peut considérer l'analogie entre une telle structure et une pile de crêpes : lorsqu'on cuisine des crêpes, on effectue deux opérations de base qui consistent à cuire des crêpes et à les manger. On peut respectivement comparer ces opérations à l'ajout et au retrait d'éléments à la structure.

Si l'on ne dispose que d'une seule assiette pour y placer les crêpes, le seul endroit où l'on peut placer une crêpe que l'on vient de cuire est au-dessus des crêpes déjà présentes sur l'assiette, ou bien directement sur l'assiette s'il s'agit de la première crêpe. Pour manger une crêpe, la seule possibilité consiste à prendre celle qui se situe au-dessus de toutes les autres. Il s'agit d'une politique LIFO : on mange toujours la crêpe cuisinée en dernier lieu parmi celles présentes sur l'assiette.

7.2.2 Interface

Une structure de pile possède l'interface suivante. Nous ne faisons pas d'hypothèse sur la nature des valeurs que la pile retient, qui peuvent être de n'importe quel type. Les opérations que l'on peut appliquer à une pile *s* sont les suivantes.

— `push(`*s*`,` *v*`)` : *Empile* la valeur *v*, en d'autres termes, ajoute *v* à la collection de valeurs retenue par la pile *s*.

— `pop(`*s*`)` : *Dépile* une valeur, en d'autres termes, retire une valeur de la pile *s* et retourne cette valeur. Conformément à la politique d'accès LIFO, la valeur extraite est, parmi celles présentes sur la pile, celle qui a été empilée en dernier lieu. Cette opération signale

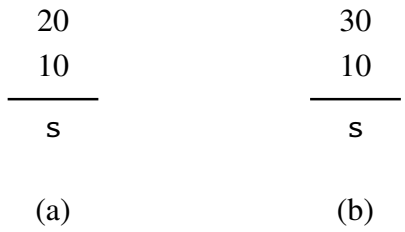


FIGURE 7.1 – Exemple de contenus de pile

une erreur si elle est appliquée à une pile vide.

— `top(`*s*`)` : Retourne la valeur située au *sommet* de la pile *s*, c'est-à-dire, celle empilée en dernier lieu parmi les valeurs présentes sur la pile, mais sans la dépiler. Cette opération ne modifie donc pas le contenu de la pile. Elle signale une erreur si elle est appliquée à une pile vide.

— `size(`*s*`)` : Retourne le nombre de valeurs contenues dans la pile *s*.

— `is_empty(`*s*`)` : Retourne une valeur booléenne indiquant si la pile *s* est vide.

Pour illustrer le fonctionnement d'une pile, nous considérons la séquence suivante d'opérations, impliquant une pile `s` que nous supposons être initialement vide.

```
push(s, 10)
push(s, 20)
pop(s)
push(s, 30)
pop(s)
pop(s)
```

Les deux premières opérations empilent successivement les valeurs 10 et 20 ; après ces opérations, le contenu de la pile correspond à celui montré à la figure 7.1 (a). L'opération suivante dépile une valeur, et retourne donc celle qui avait été empilée en dernier lieu, c'est-à-dire 20. Ensuite, on empile la valeur 30, ce qui mène au contenu de la pile de la figure 7.1 (b). Les deux dernières opérations dépilent les deux valeurs encore présentes sur la pile, dans l'ordre inverse où elles ont été empilées, et retournent donc successivement 30 et 10.

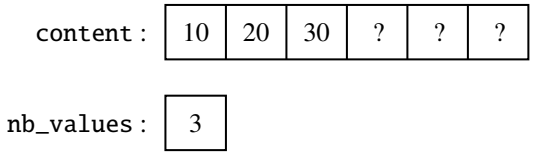


FIGURE 7.2 – Représentation d'une pile basée sur un vecteur

7.2.3 Implémentation à l'aide d'un vecteur

Nous nous intéressons maintenant à la question d'implémenter une structure de pile, c'est-à-dire développer un procédé pour représenter son contenu en mémoire, ainsi que programmer des fonctions pour chacune des cinq opérations qui ont été définies. Ce problème admet plusieurs solutions, possédant différents avantages et inconvénients. Nous en présentons une première ici, et une alternative sera ensuite proposée à la section 7.2.5.

Une première façon de représenter une pile consiste à placer ses éléments successifs dans un vecteur. La taille de ce vecteur correspondra à la capacité maximale de la pile, c'est-à-dire au plus grand nombre d'éléments qu'elle peut contenir à chaque instant. Lorsque la pile atteint cette capacité maximale, il n'est plus possible d'effectuer une opération `push`. Cette opération échouera alors en signalant une erreur.

Représenter les éléments de la pile dans ceux d'un vecteur n'est pas suffisant ; il faut aussi retenir une donnée supplémentaire indiquant la position du sommet de la pile dans ce vecteur, ou de façon équivalente le nombre d'éléments contenus dans la pile. La figure 7.2 montre la représentation d'une structure décrivant une pile de capacité maximale égale à 6, contenant les valeurs 10, 20 et 30 empilées dans cet ordre. Le champ `content` correspond au tableau d'éléments, et le champ `nb_values` au nombre d'éléments de la pile.

Dans la structure de la figure 7.2, les valeurs appartenant à la pile sont directement placées dans les éléments du vecteur `content`. Le problème de cette approche est que le type de ces valeurs doit correspondre au type défini pour les éléments de ce vecteur. Si l'on souhaite programmer une structure de pile capable d'accepter des valeurs de n'importe quel type, une solution consiste à placer dans les éléments du vecteur des pointeurs vers les valeurs à retenir plutôt que ces valeurs. En effet, nous avons vu à la section 6.2.2 qu'il est possible de définir des pointeurs vers n'importe quelle donnée en employant le type `void *`. Ce type sera donc celui spécifié pour les éléments du vecteur `content`.

Un programme implémentant une structure de pile à l'aide d'un vecteur est donné aux figures 7.3 et 7.4. Comme d'habitude, son code source est réparti dans un fichier d'en-tête `stack.h` qui définit le type de la structure de données et le prototype des fonctions permettant de la manipuler, et un fichier `stack.c` qui contient l'implémentation de ces fonctions.

stack.c:

```
#include <stdlib.h>
#include "stack.h"

/*  Retourne une nouvelle pile vide, ou NULL en cas d'erreur.  */

stack *stack_new(void)
{
  stack *s;

  s = malloc(sizeof(stack));
  if (!s)
    return NULL;

  s -> nb_values = 0;

  return s;
}

/*  Libère la pile <s>.  */

void stack_free(stack *s)
{
  free(s);
}

/*  Empile le pointeur <p> sur la pile <s>.  Retourne 0 en cas de succès,
    ou -1 si la pile est saturée.  */

int stack_push(stack *s, void *p)
{
  if (s -> nb_values >= MAX_STACK_SIZE)
    return -1;

  s -> content[s -> nb_values++] = p;
  return 0;
}

/*  Dépile un pointeur depuis la pile <s> et retourne ce pointeur,
    ou NULL si la pile est vide.  */

void *stack_pop(stack *s)
{
  if (!(s -> nb_values))
    return NULL;

  return s -> content[--(s -> nb_values)];
}

/*  Retourne le pointeur au sommet de la pile <s>, sans le dépiler,
    ou NULL si la pile est vide.  */

void *stack_top(stack *s)
{
  if (!(s -> nb_values))
    return NULL;

  return s -> content[s -> nb_values - 1];
}
```

FIGURE 7.3 – Implémentation d'une pile basée sur un vecteur (1/2)

```
   /*  Retourne le nombre d'éléments présents dans la pile <s>.  */

   unsigned stack_size(stack *s)
   {
     return s -> nb_values;
   }

   /*  Détermine si la pile <s> est vide.  */

   int stack_is_empty(stack *s)
   {
     return s -> nb_values == 0;
   }
```

stack.h:

```
   #define MAX_STACK_SIZE  1000

   typedef struct
   {
     void    *content[MAX_STACK_SIZE];
     unsigned nb_values;
   } stack;

   stack   *stack_new(void);
   void     stack_free(stack *);
   int      stack_push(stack *, void *);
   void    *stack_pop(stack *);
   void    *stack_top(stack *);
   unsigned stack_size(stack *);
   int      stack_is_empty(stack *);
```

FIGURE 7.4 – Implémentation d'une pile basée sur un vecteur (2/2)

```c
#include <stdio.h>
#include "stack.h"

int main()
{
  int    i = 10, i2;
  double d = 3.1416, d2;
  stack *s;

  s = stack_new();
  if (!s || stack_push(s, &i) || stack_push(s, &d))
    return -1;

  d2 = *((double *) stack_pop(s));
  i2 = *((int *) stack_pop(s));

  printf("valeurs dépilées: %lf, %d.\n", d2, i2);

  stack_free(s);

  return 0;
}
```

FIGURE 7.5 – Programme de test de la structure de pile

En addition aux cinq fonctions discutées à la section 7.2.2, ce programme définit des fonctions `stack_new` et `stack_free` qui permettent respectivement d'allouer une nouvelle pile (initialement vide) et de libérer une pile. La capacité de la pile est fixée par une constante `MAX_STACK_SIZE` définie dans le fichier d'en-tête `stack.h`.

Les détails de ce programme appellent peu de commentaires. En cas d'erreur, les fonctions `stack_new`, `stack_pop` et `stack_top` retournent le pointeur vide, et la fonction `stack_push` un entier non nul.

Un exemple de programme de test de cette structure de données est fourni à la figure 7.5. Ce programme alloue une pile par un appel à `stack_new`, et y empile ensuite la valeur entière 10 et la valeur réelle 3.1416. Ces opérations sont effectuées en passant à la fonction `stack_push` des pointeurs vers deux variables `i` et `d` contenant ces valeurs. Les situations d'erreur sont gérées grâce au mécanisme d'évaluation en circuit court de l'opérateur "`||`" : si l'appel à `stack_new` retourne le pointeur vide, ou si l'un des appels à `stack_push` retourne un entier non nul, alors la

fonction `main` interrompt immédiatement [1] son exécution et retourne le code de diagnostic -1. Sinon, le programme dépile deux valeurs qu'il place dans des variables `d2` et `i2`, dont il affiche ensuite le contenu. La dernière opération consiste à invoquer `stack_free` afin de libérer les ressources liées à la pile.

7.2.4 Application : appariement de parenthèses

Nous venons de voir ce qu'est une pile et comment l'implémenter, mais cela ne nous apprend pas quelle peut bien être l'utilité d'une telle structure de données. On utilise une pile dans les situations où l'on est amené à produire et à consommer des données, et où la nature du problème à résoudre fait que ces données sont toujours consommées dans l'ordre inverse de leur production.

Pour illustrer cela, nous considérons le problème qui consiste à déterminer si différentes formes de parenthèses (c'est-à-dire des parenthèses simples "`(`" et "`)`", des crochets "`[`" et "`]`" et des accolades "`{`" et "`}`") sont correctement appariées dans un texte fourni en entrée. Par exemple, la réponse à cette question est positive pour le texte

```
[- (b) + sqrt(4 * (a) * (c))] / (2 * a)
```

mais négative pour

```
((x) + (y)] / 2.
```

En effet, dans ce dernier texte, la première parenthèse ouvrante "`(`" est fermée par le symbole "`]`" au lieu d'une parenthèse fermante "`)`".

Une structure de pile est bien adaptée pour résoudre ce problème, car on ferme les parenthèses dans l'ordre inverse où on les ouvre. Plus précisément, en parcourant le texte de gauche à droite, un symbole fermant "`)`", "`]`" ou "`}`" se rapporte au symbole ouvrant "`(`", "`[`" ou "`{`" le plus récent qui n'a pas déjà été fermé. Une solution simple consiste donc à lire le texte caractère par caractère, et à maintenir une structure de pile qui contient à chaque instant les parenthèses ouvrantes qui ont été rencontrées et qui n'ont pas déjà été fermées. On obtient l'algorithme suivant.

1. On crée une pile, initialement vide.

2. On examine les caractères du texte, dans l'ordre. Pour chaque caractère c :

1. Remarquons que dans le cas où l'appel à `stack_push` échoue, ce code ne libère pas explicitement la pile allouée. Cette pratique n'est pas idéale, mais reste acceptable pour un simple programme de test.

— Si c n'est pas une parenthèse, alors on l'ignore.

— Si c est une parenthèse ouvrante, alors on l'empile.

— Si c est une parenthèse fermante, alors :

2.1 On dépile un caractère c'.

2.2 Si cette opération échoue (car la pile est vide), ou si c' n'est pas le caractère symétrique de c, alors on signale une erreur.

3. S'il reste un ou plusieurs caractères dans la pile après avoir examiné la totalité du texte, alors on signale une erreur.

Un programme implémentant cet algorithme est donné aux figures 7.6 et 7.7. Bien sûr, ce programme nécessite une implémentation de la structure de pile, et doit donc être compilé avec le fichier source `stack.c` des figures 7.3 et 7.4. Le programme inclut le fichier d'en-tête `stack.h` afin de disposer de la définition du type `stack` et du prototype des fonctions de manipulation d'une pile.

Pour pouvoir traiter n'importe quel texte, le programme lit celui-ci depuis un *fichier*, dont le nom lui est fourni par l'intermédiaire d'un argument d'exécution. Le programme commence donc par vérifier si le nombre d'arguments d'exécution, qui est égal à la valeur du paramètre `argc` de la fonction `main`, vaut 2, ce qui correspond à un seul argument d'exécution à la suite du nom du programme. Dans le cas contraire, le programme affiche un message d'erreur rappelant comment l'utiliser correctement, et termine son exécution en retournant un code de diagnostic égal à -1.

La bibliothèque standard fournit des structures de données et des fonctions permettant de manipuler des fichiers. Les définitions et déclarations correspondantes se trouvent dans le fichier d'en-tête standard `stdio.h`. Pour ouvrir un fichier, une possibilité consiste à invoquer la fonction `fopen` de la bibliothèque standard, en lui passant comme arguments le nom de ce fichier et la chaîne constante "`r`" indiquant un mode de lecture [2]. Cette fonction retourne un pointeur vers une structure de données de la bibliothèque, de type `FILE`, représentant le fichier qui vient d'être ouvert, ou le pointeur vide dans le cas d'une erreur. Dans le programme, la fonction `fopen` est invoquée avec comme premier argument la chaîne de caractères présente dans `argv[1]`, qui correspond à l'argument d'exécution fourni au programme lors de son lancement. Après avoir ouvert le fichier, la fonction `main` invoque la fonction `verifie_fichier` chargée de vérifier si son contenu apparie correctement les parenthèses, et ferme ensuite le fichier (c'est-à-dire, libère les ressources qui lui sont liées) en invoquant la fonction `fclose` de la bibliothèque standard.

2. D'autres valeurs de cette chaîne permettent notamment de spécifier un mode d'écriture ("`w`") ou d'ajout ("`a`").

```
#include <stdio.h>
#include <stdlib.h>
#include "stack.h"

static void verifie_parenthese(stack *s, char c, unsigned ligne, unsigned col)
{
  char *p;

  p = stack_pop(s);
  if (!p || *p != c)
    printf("erreur: Symbole incorrect ligne %u, colonne %u.\n", ligne, col);
  free(p);
}

static void verifie_fichier(FILE *f)
{
  stack    *s;
  unsigned  num_ligne, num_col;
  char      c, *p;

  s = stack_new();
  if (!s)
    exit(-1);

  for (num_ligne = num_col = 1;; num_col++)
    {
      c = fgetc(f);  /* Lecture d'un caractère dans le fichier. */
      switch (c)
        {
        case EOF:    /* Fin du fichier. */
          if (!stack_is_empty(s))
            {
              printf("erreur: %d symbole(s) encore ouvert(s) en fin de fichier.\n",
                     stack_size(s));
              while (stack_size(s))
                free(stack_pop(s));
            }

          stack_free(s);
          return;

        case '\n':
          num_ligne++;
          num_col = 0;
          break;

        case '(':
        case '[':
        case '{':
          p = malloc(1);
          if (!p)
            exit(-1);

          *p = c;

          if (stack_push(s, p))
            exit(-1);
          break;
```

FIGURE 7.6 – Appariement de parenthèses (1/2)

```
        case ')':
          verifie_parenthese(s, '(', num_ligne, num_col);
          break;

        case ']':
          verifie_parenthese(s, '[', num_ligne, num_col);
          break;

        case '}':
          verifie_parenthese(s, '{', num_ligne, num_col);
          break;
        }
    }
}

int main(int argc, char *argv[])
{
  FILE *f;

  if (argc != 2)
    {
      printf("usage: %s <nom-fichier>\n", argv[0]);
      exit(-1);
    }

  f = fopen(argv[1], "r");  /* Ouverture d'un fichier en lecture. */
  if (!f)
    {
      printf("impossible d'ouvrir le fichier [%s]\n", argv[1]);
      exit(-1);
    }

  verifie_fichier(f);

  fclose(f);  /* Fermeture du fichier. */
  exit(0);
}
```

FIGURE 7.7 – Appariement de parenthèses (2/2)

Le reste du programme implémente directement l'algorithme que nous avons développé. Pour lire le prochain caractère du fichier, on emploie la fonction `fgetc` de la bibliothèque standard ; celle-ci retourne un entier égal à la valeur numérique du caractère lu, ou à la constante `EOF` lorsque la fin du fichier est atteinte. Le programme maintient deux variables `num_ligne` et `num_colonne` qui contiennent à chaque instant le numéro de la ligne et de la colonne du caractère courant ; cela permet d'afficher en cas d'erreur d'appariement de parenthèses la position exacte du caractère problématique. La variable `num_colonne` est incrémentée à chaque lecture d'un nouveau caractère. Elle est remise à zéro, et la variable `num_ligne` est incrémentée, à chaque lecture d'un saut de ligne "`\n`". De cette façon, la valeur de `num_col` deviendra égale à 1 pour le caractère suivant, étant donné qu'elle est incrémentée à chaque itération de la boucle `for`.

Dans notre implémentation de la structure de pile, la fonction `stack_push` reçoit en argument un pointeur vers la donnée qui doit être empilée. Cette donnée correspond ici au symbole d'une parenthèse ouvrante "`(`", "`[`" ou "`{`". Lorsque le programme rencontre un tel symbole, il alloue un bloc de mémoire de taille 1 par un appel à la fonction `malloc`, recopie le caractère concerné dans ce bloc de mémoire, et transmet un pointeur vers ce bloc à `stack_push`. Réciproquement, lorsqu'une valeur est extraite de la pile grâce à `stack_pop`, elle est interprétée comme un pointeur vers un bloc, qui est libéré par `free` après avoir été lu. Quand la fin du fichier est atteinte, tous les blocs correspondant à des valeurs encore présentes sur la pile sont libérés, avant de libérer la pile elle-même.

7.2.5 Implémentation à l'aide d'une liste liée

La plupart des structures de données peuvent être implémentées de différentes façons, parfois basées sur des principes très différents, et la structure de pile ne fait pas exception. La solution développée à la section 7.2.3 a l'avantage d'être simple, mais présente l'inconvénient de devoir estimer a priori le nombre d'éléments à allouer pour le vecteur représentant le contenu de la pile. Cela peut conduire à gaspiller de la mémoire si ce nombre est surestimé, ou à atteindre prématurément la limite de capacité de la structure s'il est sous-estimé. Nous présentons ici une autre solution qui ne possède pas cet inconvénient. L'idée est que cette implémentation alternative présente exactement la même interface que la précédente, afin de pouvoir l'utiliser dans une application telle que celle des figures 7.6 et 7.7 sans devoir en modifier le code source. Il s'agit d'une illustration d'un mécanisme de *modularité* : il est possible de modifier l'implémentation d'une partie d'un programme sans que cela n'affecte le reste du projet, à condition d'en préserver l'interface. Ce mécanisme est essentiel pour le développement de logiciels de grande envergure.

Nous avons déjà élaboré à la section 6.1.3 un algorithme capable de lire un ensemble de valeurs et de les restituer en ordre inverse, sans devoir connaître a priori le nombre de ces valeurs. Cet algorithme reposait sur la construction d'une *liste simplement liée* dont chaque élément mémorisait une valeur. La lecture d'une valeur s'effectuait en créant un nouvel élément placé en tête de la liste, et la restitution d'une valeur consistait à retirer l'élément situé en tête de liste. Le même principe permet d'implémenter les opérations `push` et `pop` d'une pile. L'avantage de

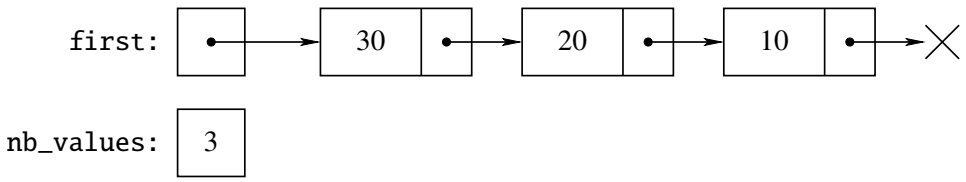


FIGURE 7.8 – Représentation d'une pile basée sur une liste liée

cette approche est que les éléments de la liste liée peuvent être alloués dynamiquement en fonction des besoins. L'inconvénient d'une liste liée est que l'opération qui consiste à compter le nombre d'éléments qu'elle contient est inefficace, car elle demande de la parcourir entièrement. Une solution permettant de pallier cet inconvénient consiste à prévoir un champ supplémentaire contenant un compteur du nombre d'éléments présents dans la liste à chaque instant.

Une illustration de cette structure de pile basée sur une liste simplement liée est donnée à la figure 7.8, pour une pile contenant les valeurs 10, 20 et 30 empilées dans cet ordre. Elle est composée d'un champ `first` pointant vers la tête de la liste liée, et un champ `nb_values` retenant le nombre courant d'éléments de la pile. Une implémentation de cette structure est fournie aux figures 7.9, 7.10 et 7.11. Comme pour l'implémentation basée sur un vecteur développée à la section 7.2.3, les éléments de la pile sont représentés par des pointeurs vers des données de n'importe quel type.

7.3 Les files

Nous abordons à présent l'étude d'une nouvelle structure de données, la *file*, ou *file d'attente (queue, FIFO buffer)*.

7.3.1 Définition

La file possède plusieurs points communs avec la structure de pile introduite à la section 7.2. Une file est une structure de données permettant de retenir une collection de valeurs, en disposant d'opérations permettant d'y ajouter ou d'en retirer une valeur. À la différence d'une pile, l'accès aux éléments d'une file s'effectue selon une politique FIFO (*First In First Out*). Cela signifie que les éléments d'une file sont extraits dans l'ordre où ils y ont été placés : l'opération d'extraction retourne toujours l'élément qui, parmi ceux présents dans la file, s'y trouve depuis le plus longtemps. On peut établir une analogie avec la file d'attente qui permet d'accéder à un guichet ou au comptoir d'un magasin : les personnes qui quittent cette file pour être servies le font dans

stack.c:

```
#include <stdlib.h>
#include "stack.h"

/*  Retourne une nouvelle pile vide, ou NULL en cas d'erreur.  */

stack *stack_new(void)
{
  stack *s;

  s = malloc(sizeof(stack));
  if (!s)
    return NULL;

  s -> first = NULL;
  s -> nb_values = 0;

  return s;
}

/*  Libère la pile <s>.  */

void stack_free(stack *s)
{
  stack_element *e, *e_next;
  for (e = s -> first; e; e = e_next)
    {
      e_next = e -> next;
      free(e);
    }

  free(s);
}

/*  Empile le pointeur <p> sur la pile <s>.  Retourne 0 en cas de succès,
    ou -1 en cas d'insuffisance mémoire.  */

int stack_push(stack *s, void *p)
{
  stack_element *e;

  e = malloc(sizeof(stack_element));
  if (!e)
    return -1;

  e -> content = p;
  e -> next = s -> first;
  s -> first = e;
  s -> nb_values++;

  return 0;
}
```

FIGURE 7.9 – Implémentation d'une pile basée sur une liste liée (1/3)

```
/*  Dépile un pointeur depuis la pile <s> et retourne ce pointeur,
    ou NULL si la pile est vide.  */

void *stack_pop(stack *s)
{
  stack_element *e;
  void          *p;

  if (!(s -> first))
    return NULL;

  e = s -> first;
  p = e -> content;

  s -> first = e -> next;
  s -> nb_values--;

  free(e);

  return p;
}

/*  Retourne le pointeur au sommet de la pile <s>, sans le dépiler,
    ou NULL si la pile est vide.  */

void *stack_top(stack *s)
{
  if (!(s -> first))
    return NULL;

  return s -> first -> content;
}

/*  Retourne le nombre d'éléments présents dans la pile <s>.  */

unsigned stack_size(stack *s)
{
  return s -> nb_values;
}

/*  Détermine si la pile <s> est vide.  */

int stack_is_empty(stack *s)
{
  return s -> nb_values == 0;
}
```

FIGURE 7.10 – Implémentation d'une pile basée sur une liste liée (2/3)

stack.h:

```
  typedef struct _stack_element
  {
    void                  *content;
    struct _stack_element *next;
  } stack_element;

  typedef struct
  {
    stack_element *first;
    unsigned       nb_values;
  } stack;

  stack   *stack_new(void);
  void     stack_free(stack *);
  int      stack_push(stack *, void *);
  void    *stack_pop(stack *);
  void    *stack_top(stack *);
  unsigned stack_size(stack *);
  int      stack_is_empty(stack *);
```

FIGURE 7.11 – Implémentation d'une pile basée sur une liste liée (3/3)

l'ordre où elles sont arrivées et se sont ajoutées à la queue. En informatique, les files d'attente sont notamment utilisées pour gérer les situations où l'on reçoit des requêtes pour accéder à une ressource partagée, par exemple une imprimante, et où ces requêtes doivent être traitées dans l'ordre où elles arrivent.

7.3.2 Interface

L'interface d'une file q est composée des fonctions suivantes. Tout comme pour le cas d'une pile, nous considérons que les valeurs v placées dans la file peuvent être de n'importe quel type.

— `send(`q`,` v`)` : Ajoute la valeur v à la collection des valeurs retenues par la file q.

— `receive(`q`)` : Extrait une valeur de la file q et retourne cette valeur. Selon la politique d'accès FIFO, la valeur extraite est, parmi celles présentes dans la file, celle qui a été ajoutée en premier lieu. Dans la cas où la file est vide, cette opération signale une erreur.

— `front(`q`)` : Retourne la valeur située en *tête* de la file q, c'est-à-dire celle qui a été ajoutée en premier lieu parmi celles présentes dans la file, sans l'extraire. Cette opération ne modifie donc pas le contenu de la file. Elle signale une erreur si elle est appliquée à une file vide.

— `size(`q`)` : Retourne le nombre de valeurs contenues dans la file q.

— `is_empty(`q`)` : Retourne une valeur booléenne indiquant si la file q est vide.

Nous illustrons ces opérations à l'aide de la séquence suivante, que l'on applique à une file q supposée initialement vide.

```
send(q, 10)
send(q, 20)
receive(q)
send(q, 30)
receive(q)
receive(q)
```

Les deux premières opérations ajoutent successivement les valeurs 10 et 20 à la file ; après les avoir effectuées, le contenu de la file est celui de la figure 7.12 (a), où la tête de la file est située à gauche. L'opération suivante extrait une valeur de la file, et retourne donc celle qui a été ajoutée en premier lieu, c'est-à-dire 10. Ensuite, on ajoute la valeur 30, ce qui mène au contenu de la file montré à la figure 7.12 (b). Les deux dernières opérations extraient les valeurs encore



FIGURE 7.12 – Exemple de contenus de file

présentes dans la file dans l'ordre où elles y ont été ajoutées, et produisent donc successivement les valeurs 20 et 30.

7.3.3 Implémentation à l'aide d'un vecteur

Tout comme pour une pile, on peut implémenter une file en plaçant les valeurs qu'elle contient dans les éléments consécutifs d'un vecteur de taille donnée. Cette taille correspondra à la capacité maximale de la file. La façon de manipuler ce vecteur est cependant plus compliquée pour une file que pour une pile : pour une pile, les opérations `push` et `pop` peuvent être implémentées en modifiant uniquement le voisinage du sommet de la pile, sans que les valeurs situées à la base de celle-ci ne soient affectées. Pour une file, la situation est différente : une opération `receive` modifie la tête de la file, et une opération `send` son autre extrémité.

Si l'on considère le cas particulier d'une file contenant n de valeurs, à laquelle on applique répétitivement une opération `send` suivie d'une opération `receive`, on souhaite éviter de devoir décaler les n éléments du vecteur qui représentent le contenu de la file, ce qui est inefficace si n est grand. Une solution consiste à maintenir deux indices f ("*front*") et r ("*rear*") vers les éléments du vecteur, définis de la façon suivante. L'indice f repère la valeur la plus ancienne parmi celles contenues dans la file, c'est-à-dire celle qui a été ajoutée en premier lieu. L'indice r repère l'élément du vecteur qui suit immédiatement la valeur la plus récente, c'est-à-dire l'emplacement où sera écrite la prochaine valeur que l'on ajoutera à la file dans le futur. Les opérations `send` et `receive` s'effectuent alors en déplaçant ces indices d'une position vers la droite : `send` déplace r et `receive` déplace f, l'autre indice restant inchangé. Ce mécanisme est illustré à la figure 7.13, qui montre l'effet d'une opération `send` suivie d'une opération `receive` appliquées trois fois de suite à une file de capacité 10 contenant initialement 6 valeurs. Dans cet exemple, les valeurs successivement ajoutées par `send` sont les entiers 7, 8 et 9, et celles récupérées par `receive` sont 1, 2 et 3.

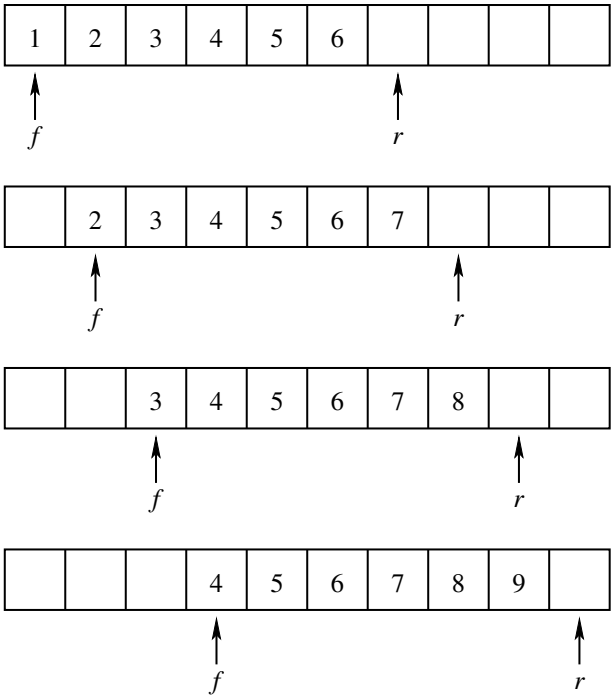


FIGURE 7.13 – Index repérant les éléments d'une file dans un vecteur

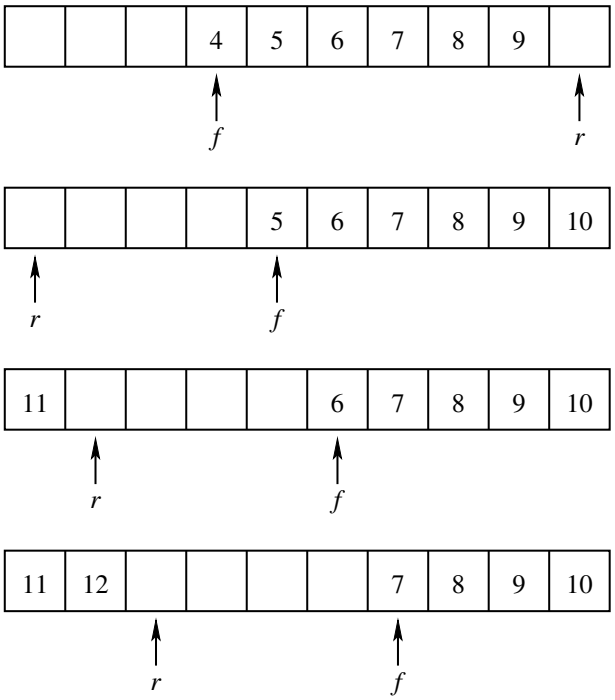


FIGURE 7.14 – Mécanisme de tableau circulaire

Avec cette approche, les valeurs contenues dans la file sont représentées par les éléments du vecteur qui commencent à l'indice f et se terminent immédiatement avant l'indice r. Un problème est que les indices f et r se déplacent toujours vers la droite lorsqu'on effectue des opérations `receive` et `send`, ce qui peut les conduire à sortir du vecteur. On résout ce problème en considérant que les éléments du vecteur sont organisés de façon *circulaire*. Pour un vecteur de taille m, possédant donc des éléments indicés de 0 à $m - 1$, cela signifie que si un indice devient égal à m, alors on le ramène à 0. En d'autres termes, on considère que l'élément d'indice $m - 1$ est suivi par l'élement d'indice 0. Ce mécanisme est illustré à la figure 7.14, qui montre la suite de la séquence d'opérations entamée à la figure 7.13, pour les entiers 10, 11 et 12 ajoutés à la file.

Il reste une difficulté à résoudre. La situation particulière où l'on a $f = r$ est ambiguë : elle correspond à la fois au cas d'une file vide, c'est-à-dire ne contenant aucune valeur, mais aussi à celui d'une file saturée, c'est-à-dire contenant un nombre de valeurs égal à la taille du vecteur. Par exemple, on arriverait à une telle situation en appliquant 4 opérations `send` ou bien 6 opérations `receive` à n'importe laquelle des configurations des figures 7.13 et 7.14.

Il est cependant indispensable de pouvoir distinguer le cas d'une file vide de celui d'une file saturée. En particulier, une opération `send` doit retourner une erreur dans le deuxième cas mais pas dans le premier, au contraire d'une opération `receive` qui ne doit signaler une erreur que si la file est vide. L'opération `size` nécessite aussi de pouvoir déterminer le nombre de valeurs contenues dans la file.

Ce problème admet deux solutions simples. La première consiste à retenir séparément le nombre d'éléments contenus dans la file, comme nous l'avons fait à la section 7.2.3 pour la structure de pile. Une autre stratégie consiste à interdire d'utiliser tous les éléments du vecteur ; pour un vecteur de taille m, cela revient à considérer que la file est saturée dès qu'elle contient $m - 1$ valeurs.

Une implémentation basée sur cette seconde solution est données aux figures 7.15 et 7.16. De façon similaire à la structure de pile développée à la section 7.2.3, les valeurs contenues dans la file sont de type `void *`. La fonction `queue_send` vérifie que la file n'est pas saturée, et écrit ensuite la nouvelle valeur à l'endroit désigné par le champ `rear`. Ce champ est ensuite incrémenté de façon circulaire. De même, la fonction `queue_receive` commence par tester si la file est vide, ce qui se produit lorsque les champs `front` et `rear` sont égaux. Elle lit ensuite la valeur désignée par `front`, et incrémente ce champ de façon circulaire. La fonction `queue_size` détermine le nombre de valeurs contenues dans la file en calculant la différence entre `rear` et `front`, modulo la taille `MAX_QUEUE_SIZE` du vecteur. Remarquons que l'on évalue

```
(MAX_QUEUE_SIZE + q -> rear - q -> front) % MAX_QUEUE_SIZE
```

et non

```
(q -> rear - q -> front) % MAX_QUEUE_SIZE
```

.

queue.c:

```
#include <stdlib.h>
#include "queue.h"

/*  Retourne une nouvelle file vide, ou NULL en cas d'erreur.  */

queue *queue_new(void)
{
  queue *q;

  q = malloc(sizeof(queue));
  if (!q)
    return NULL;

  q -> front = 0;
  q -> rear = 0;

  return q;
}

/*  Libère la file <q>.  */

void queue_free(queue *q)
{
  free(q);
}

/*  Ajoute le pointeur <p> à la file <q>.  Retourne 0 en cas de succès,
    ou -1 si la file est saturée.  */

int queue_send(queue *q, void *p)
{
  if (queue_size(q) >= MAX_QUEUE_SIZE - 1)
    return -1;

  q -> contents[q -> rear] = p;
  q -> rear = (q -> rear + 1) % MAX_QUEUE_SIZE;

  return 0;
}

/*  Retire un pointeur depuis la file <q> et retourne ce pointeur,
    ou NULL si la file est vide.  */

void *queue_receive(queue *q)
{
  void *p;

  if (q -> front == q -> rear)
    return NULL;

  p = q -> contents[q -> front];
  q -> front = (q -> front + 1) % MAX_QUEUE_SIZE;

  return p;
}
```

FIGURE 7.15 – Implémentation d'une file basée sur un vecteur (1/2)

```
/*  Retourne le pointeur en tête de la file <q>, sans le retirer,
    ou NULL si la file est vide.  */

void *queue_front(queue *q)
{
  if (q -> front == q -> rear)
    return NULL;

  return q -> contents[q -> front];
}

/*  Retourne le nombre d'éléments présents dans la file <q>.  */

unsigned queue_size(queue *q)
{
  return (MAX_QUEUE_SIZE + q -> rear - q -> front) % MAX_QUEUE_SIZE;
}

/*  Détermine si la file <q> est vide.  */

int queue_is_empty(queue *q)
{
  return q -> front == q -> rear;
}
```

queue.h:

```
#define MAX_QUEUE_SIZE  1000

typedef struct
{
  void    *contents[MAX_QUEUE_SIZE];
  unsigned front, rear;
} queue;

queue   *queue_new(void);
void     queue_free(queue *);
int      queue_send(queue *, void *);
void    *queue_receive(queue *);
void    *queue_front(queue *);
unsigned queue_size(queue *);
int      queue_is_empty(queue *);
```

FIGURE 7.16 – Implémentation d'une file basée sur un vecteur (2/2)

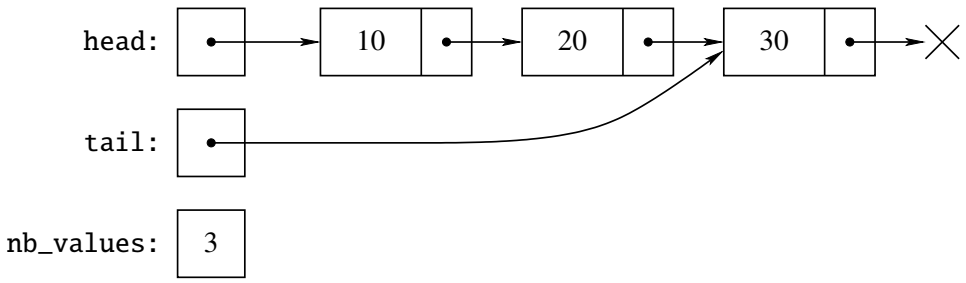


FIGURE 7.17 – Représentation d'une file basée sur une liste liée

Cela permet de garantir que la première opérande de l'opérateur "`%`" possède une valeur positive ou nulle, ce qui est nécessaire pour que cet opérateur corresponde à la notion mathématique de modulo, comme nous l'avons vu à la section 2.3.1.

7.3.4 Implémentation à l'aide d'une liste liée

On peut chercher, comme dans le cas de la structure de pile, à développer une implémentation de la file qui serait plus dynamique, c'est-à-dire capable d'adapter sa capacité aux besoins de l'application pendant l'exécution du programme. Pour la pile, nous avons obtenu à la section 7.2.5 une implémentation basée sur la notion de liste simplement liée. Nous montrons à présent comment implémenter une structure de file à l'aide d'une telle liste liée.

L'idée consiste à représenter le contenu d'une file par une liste liée dont le premier élément correspond à la tête de la file, et où chaque élément suit celui qui a été ajouté immédiatement avant lui. On peut alors effectuer une opération `send` en ajoutant un nouvel élément à la fin de la liste, et une opération `receive` en retirant l'élément situé en tête de liste. La seule difficulté technique est celle de localiser la fin de la liste. Pour éviter de devoir résoudre ce problème en parcourant tous les éléments de la liste, ce qui est inefficace, on gère la liste en maintenant deux pointeurs : un pointeur `head` vers son premier élément, et un pointeur `tail` vers son dernier élément. Dans le cas particulier où la liste est vide, les pointeurs `head` et `tail` sont tous deux vides. Enfin, on définit un champ supplémentaire retenant le nombre de valeurs contenues dans la file, encore une fois dans le but d'éviter de devoir compter explicitement le nombre d'éléments qui font partie de la liste liée. La figure 7.17 illustre cette structure de données dans le cas d'une file contenant les valeurs [3] 10, 20 et 30, ajoutées dans cet ordre.

3. Nous montrons ici le cas d'une file contenant des entiers, dans un but de simplicité. Comme cela a déjà été expliqué, notre implémentation définira des files plus générales, dont les valeurs seront de type `void *`.

queue.c:

```
#include <stdlib.h>
#include "queue.h"

/*  Retourne une nouvelle file vide, ou NULL en cas d'erreur.  */

queue *queue_new(void)
{
  queue *q;

  q = malloc(sizeof(queue));
  if (!q)
    return NULL;

  q -> head = NULL;
  q -> tail = NULL;
  q -> nb_values = 0;

  return q;
}

/*  Libère la file <q>.  */

void queue_free(queue *q)
{
  queue_element *e, *e_next;
  for (e = q -> head; e; e = e_next)
    {
      e_next = e -> next;
      free(e);
    }

  free(q);
}

/*  Ajoute le pointeur <p> à la file <q>.  Retourne 0 en cas de succès,
    ou -1 en cas d'insuffisance mémoire.  */

int queue_send(queue *q, void *p)
{
  queue_element *e;

  e = malloc(sizeof(queue_element));
  if (!e)
    return -1;

  e -> content = p;
  e -> next = NULL;

  if (q -> tail)
    q -> tail -> next = e;
  else
    q -> head = e;

  q -> tail = e;
  q -> nb_values++;

  return 0;
}
```

FIGURE 7.18 – Implémentation d'une file basée sur une liste liée (1/3)

```
/*  Extrait un pointeur de la file <q> et retourne ce pointeur,
    ou NULL si la file est vide.  */

void *queue_receive(queue *q)
{
  queue_element *e;
  void          *p;

  if (!(q -> head))
    return NULL;

  e = q -> head;
  p = e -> content;

  q -> head = e -> next;
  if (!--(q -> nb_values))
    q -> tail = NULL;

  free(e);

  return p;
}

/*  Retourne le pointeur en tête de la file <q>, sans le retirer,
    ou NULL si la file est vide.  */

void *queue_front(queue *q)
{
  if (!(q -> head))
    return NULL;

  return q -> head -> content;
}

/*  Retourne le nombre d'éléments présents dans la file <q>.  */

unsigned queue_size(queue *q)
{
  return q -> nb_values;
}

/*  Détermine si la file <q> est vide.  */

int queue_is_empty(queue *q)
{
  return q -> nb_values == 0;
}
```

FIGURE 7.19 – Implémentation d'une file basée sur une liste liée (2/3)

queue.h:

```
  typedef struct _queue_element
  {
    void                  *content;
    struct _queue_element *next;
  } queue_element;

  typedef struct
  {
    queue_element *head, *tail;
    unsigned       nb_values;
  } queue;

  queue   *queue_new(void);
  void     queue_free(queue *);
  int      queue_send(queue *, void *);
  void    *queue_receive(queue *);
  void    *queue_front(queue *);
  unsigned queue_size(queue *);
  int      queue_is_empty(queue *);
```

FIGURE 7.20 – Implémentation d'une file basée sur une liste liée (3/3)

Une implémentation basée sur ces principes est donnée aux figures 7.18, 7.19 et 7.20. La fonction `queue_send` alloue un nouvel élément *e* destiné à être placé à la fin de la liste liée. Son champ `next` est donc initialisé à l'aide du pointeur vide. Il y a deux cas à considérer : si la file est vide, alors les champs `head` et `tail` doivent tous deux pointer vers *e*. Si la file n'est pas vide, alors le champ `next` de l'élément pointé par `tail` doit maintenant pointer vers *e*. Ensuite, `tail` doit être modifié pour pointer également vers *e*.

La fonction `queue_receive` doit retirer de la liste l'élément pointé par `head`, ce qui implique que `head` doit devenir égal à la valeur du champ `next` de cet élément. Un cas particulier doit être traité : si la file devient vide après l'opération, alors `head` prendra pour valeur le pointeur vide, mais il faut également veiller à ce que `tail` prenne aussi cette valeur. On peut indifféremment tester si la file est vide en détérminant si `head` est le pointeur vide, si `tail` l'est, ou si la valeur courante de `nb_values` est nulle.

7.4 Exercices avancés

Les exercices de cette section ne font pas partie de la matière du cours d'*introduction à l'informatique*. Leur intérêt est de montrer quelques applications des structures de données développées dans ce chapitre, ainsi que de fournir des exemples de problèmes d'algorithmique et de programmation plus difficiles mais plus stimulants que ceux du cours.

1. Un *arbre binaire* est une structure de données composée d'un certain nombre de *nœuds*. Chaque nœud retient une valeur, ainsi que deux pointeurs vers des nœuds qui sont appelés ses *fils gauche* et *fils droit*. Chacun de ces pointeurs peut être vide. Si l'arbre n'est pas vide, alors il possède un nœud particulier appelé sa *racine*, qui est le seul à ne pas être le fils (gauche ou droit) d'un nœud. À partir de la racine, il doit être possible d'atteindre tous les nœuds de l'arbre en suivant une certaine succession de pointeurs vers le fils gauche ou le fils droit du nœud courant.

   (a) Écrire une bibliothèque permettant de représenter et de manipuler des arbres binaires retenant des valeurs entières. Il doit être possible de créer un arbre composé d'une racine *r* retenant un entier donné. Les fils gauche et droit de *r* doivent correspondre à la racine de deux arbres donnés, chacun d'entre eux pouvant être vide. On suppose que ces deux arbres contiennent des nœuds distincts, ainsi que distincts du nœud *r*.

   (b) Écrire une fonction récursive permettant d'afficher toutes les valeurs contenues dans les nœuds d'un arbre binaire donné, en parcourant celui-ci selon un ordre *infixe*. Cela signifie que pour chaque nœud *n*, les valeurs situées dans les nœuds accessibles via son fils gauche doivent être affichées avant celle associée à *n*. Les valeurs retenues dans les nœuds accessibles par l'intermédiaire du fils droit de *n* doivent être affichées

après celle du nœud *n*.

(c) Écrire une fonction itérative qui effectue le même travail que la fonction obtenue au point (a). *Suggestion :* Utiliser une pile pour retenir les nœuds dont les successeurs doivent encore être explorés.

2. Un *graphe dirigé* est une structure de données qui, comme l'arbre binaire, est composée d'un certain nombre de *nœuds* retenant chacun une valeur. De chaque nœud sont issus un certain nombre d'*arcs* qui mènent chacun vers un nœud du graphe. S'il n'est pas vide, un graphe possède un nœud particulier appelé son nœud initial.

(a) Écrire une bibliothèque permettant de représenter et de manipuler des graphes dirigés retenant des valeurs entières. Il doit être possible

— de créer un graphe vide.

— d'ajouter un nouveau nœud à un graphe donné, contenant une valeur donnée. Le premier nœud ainsi créé devient le nœud initial.

— d'ajouter un arc dans un graphe donné, depuis un nœud donné vers un nœud donné.

Suggestion : Utiliser des listes simplement liées pour représenter l'ensemble des nœuds d'un graphe, ainsi que l'ensemble des arcs issus de chaque nœud.

(b) Un nœud d'un graphe est dit *accessible* s'il peut être atteint depuis le nœud initial en suivant un certain nombre d'arcs.

Écrire une fonction qui affiche la valeur retenue dans chaque nœud accessible d'un graphe donné, dans un ordre arbitraire.

Suggestions :

— Prévoir dans chaque nœud un champ retenant si ce nœud a déjà été visité ou non.

— Utiliser une pile ou une file contenant les nœuds dont les successeurs doivent encore être explorés.

3. (a) Définir une structure de données permettant de représenter des nombres entiers de taille arbitraire (c'est-à-dire s'écrivant avec un nombre de chiffres non borné a priori).

(b) Développer, et programmer sous la forme d'une bibliothèque de fonctions, des algo-

rithmes pour appliquer des opérations arithmétiques (addition, soustraction, multiplication et division entière) aux nombres représentés par la structure obtenue au point (a).

(c) Ajouter à cette bibliothèque une fonction capable de calculer la racine carrée entière d'un nombre. *Suggestion :* Implémenter l'algorithme de Héron discuté au chapitre 1 à l'aide des opérations obtenues au point (b).

4. Ce problème consiste à permuter deux parties d'un vecteur. On reçoit en entrée

   — un vecteur v et son nombre d'éléments n.

   — un indice k tel que $0 < k < n$.

   L'opération consiste à déplacer le bloc $v[0 : k-1]$, c'est-à-dire les k éléments de v commençant à l'indice 0, vers la position d'indice $n-k$ du vecteur, et à déplacer le bloc $v[k : n-1]$, c'est-à-dire les $n-k$ éléments de v commençant à l'indice k, vers la position d'indice 0. En d'autres termes, cela revient à permuter les blocs $v[0 : k-1]$ et $v[k : n-1]$ du vecteur.

   (a) Écrire en pseudo-code un algorithme capable d'effectuer cette opération, en utilisant le moins de mémoire externe possible. *Note :* Il existe une solution dont la complexité en espace (sans compter les données d'entrée) est $O(1)$.

   (b) Traduire cet algorithme en langage C, en supposant que les éléments du vecteur v sont entiers.

5. On considère une matrice M de nombres réels possédant n lignes et m colonnes, avec $n > 0$ et $m > 0$. Cette matrice est représentée sous la forme d'un tableau unidimensionnel t de nombres, de taille nm : les m premiers éléments de t représentent la première ligne de M, puis les m éléments suivants sa deuxième ligne, et ainsi de suite.

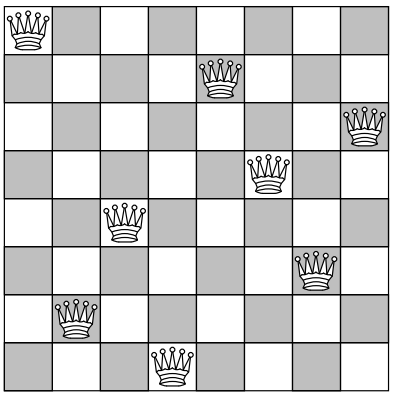
   (a) Écrire en pseudo-code un algorithme capable de transposer une matrice M représentée de cette façon, étant donnés le tableau t et les valeurs de m et n correspondantes. Le résultat doit être écrit dans le tableau t. Comme dans l'exercice précédent, on souhaite utiliser le moins de mémoire externe possible. *Note :* Il existe une solution dont la complexité en espace (sans compter les données d'entrée) est $O(1)$.

   (b) Traduire cet algorithme en langage C.

6. *(Problème des huit reines)*

Ce problème consiste à trouver le moyen de placer 8 reines sur un échiquier (ou, plus généralement, n reines sur un plateau de jeu de taille $n \times n$, avec $n \geq 1$), de façon à ce qu'aucune d'entre elles ne soit en prise. Cela signifie que deux reines ne peuvent pas être situées dans une même rangée, une même colonne, ou sur une même diagonale de l'échiquier.

Une solution à ce problème est donnée ci-après pour le cas $n = 8$. Bien sûr, l'objectif consiste à développer un algorithme capable de générer une telle solution de façon autonome. (Un problème alternatif intéressant est celui de compter le nombre de solutions distinctes que le problème des n reines admet.)



(a) Écrire en pseudo-code un algorithme capable de trouver une solution au problème des n reines. *Suggestion :* Procéder de façon récursive, en exploitant les résultats obtenus pour le sous-ensemble formé par les reines situées dans les colonnes 1 à k afin de résoudre le problème pour les reines des colonnes de 1 à $k+1$, pour $k = 1, 2, \ldots, n-1$.

(b) Traduire cet algorithme en langage C, et évaluer expérimentalement jusqu'à quelle valeur de n il est utilisable en pratique.

7. *(Ronde du cavalier)*

Aux échecs, le mouvement d'un cavalier est particulier : il se déplace de deux cases dans une direction donnée (horizontale ou verticale), puis d'une case dans la direction perpendiculaire à celle-ci.

Le problème consiste à calculer une séquence de mouvements pour un cavalier qui permet de visiter toutes les cases de l'échiquier une et une seule fois. *Note :* Quand un cavalier effectue un mouvement, on considère que seules les cases de départ et d'arrivée sont visitées, et pas les cases intermédiaires survolées par ce mouvement.

(a) Écrire en pseudo-code un algorithme permettant de résoudre ce problème, pour un échiquier standard ou pour un plateau de jeu de taille $n \times n$ avec $n \geq 1$ donné.

(b) Traduire cet algorithme en langage C.

8. Ce problème consiste à trouver, dans un vecteur d'entiers signés donné, une sous-séquence d'éléments consécutifs qui présente la plus grande somme possible. Par exemple, dans le tableau $[1, -2, 4, 2, -2, 6, 3, -1, 4, -5]$, il s'agit de la sous-séquence $[4, 2, -2, 6, 3, -1, 4]$ dont la somme des éléments vaut 16.

   (a) Écrire en pseudo-code un algorithme permettant de résoudre ce problème, et en déterminer la complexité en temps. *Note :* Il existe une solution (*algorithme de Kadane*) de complexité en temps $O(n)$, où n est la taille du vecteur.

   (b) Traduire cet algorithme en langage C.

9. Ce problème consiste à trouver le premier endroit où une chaîne de caractères *cible* apparaît dans une autre chaîne *texte*. Par exemple, si l'on a *cible* = `"mat"` et *texte* = `"informatique"`, le résultat de cette opération est égal à 5, car la première occurrence de *cible* dans *texte* commence à l'indice 5 de cette dernière chaîne. Dans le cas particulier où *texte* ne contient pas d'occurrence de *cible*, l'opération doit retourner -1.

   (a) Écrire en pseudo-code un algorithme permettant de résoudre ce problème. *Note :* La difficulté est d'obtenir un algorithme qui reste efficace lorsque *cible* est une longue chaîne de caractères. Il existe une solution (*algorithme de Knuth-Morris-Pratt*) dont la complexité en temps est $O(n + m)$, où n et m sont respectivement la longueur des chaînes *texte* et *cible*.

   (b) Traduire cet algorithme en langage C, et en évaluer expérimentalement les performances pour de grandes chaînes de caractères.

10. Ce problème consiste à trouver la plus grande sous-séquence commune de caractères (pas nécessairement consécutifs, mais apparaissant dans le même ordre) dans deux chaînes de caractères données. Par exemple pour les chaînes

    ```
    "le cours d'introduction a l'informatique"
    ```

    et

    ```
    "l'informatique a l'universite de liege",
    ```

le résultat de cette opération est

```
"l'inoti a l'nrtie"
```

qui est de longueur 17. Cette séquence peut être mise en évidence dans les deux chaînes de départ :

```
"le cours d'introduction a l'informatique"
```

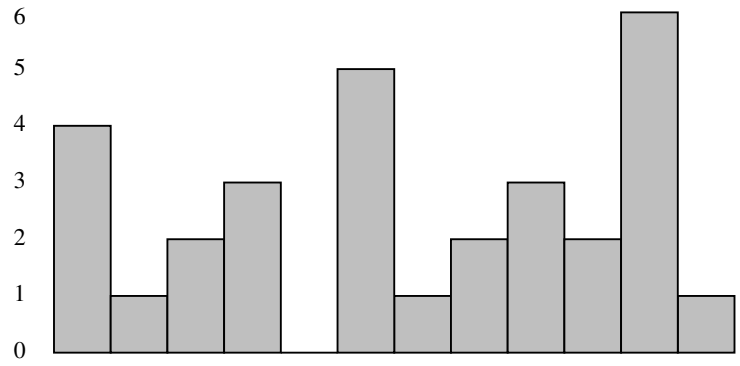
et

```
"l'informatique a l'universite de liege".
```

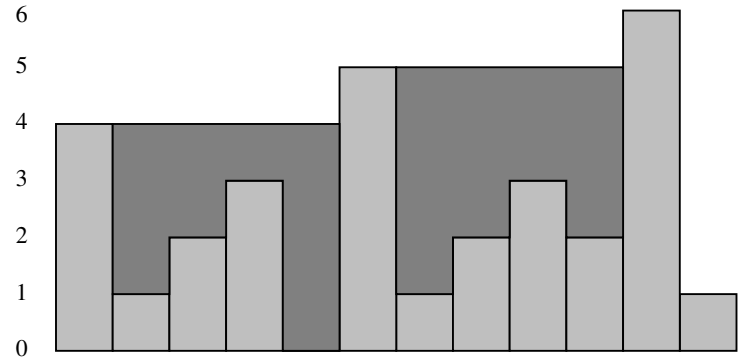
(a) Écrire en pseudo-code un algorithme permettant de résoudre ce problème, et en déterminer la complexité en temps et en espace.

(b) Traduire cet algorithme en langage C.

11. Un *histogramme* est représenté par un vecteur h d'entiers non négatifs, de taille n. On considère qu'à chaque indice $i \in [0, n-1]$ de ce vecteur, la valeur de $h[i]$ représente la hauteur d'un rectangle de largeur unitaire posé sur le sol. Les rectangles associés aux valeurs consécutives de i sont adjacents. Par exemple, l'histogramme $[4, 1, 2, 3, 0, 5, 1, 2, 3, 2, 6, 1]$ possède la représentation graphique suivante.



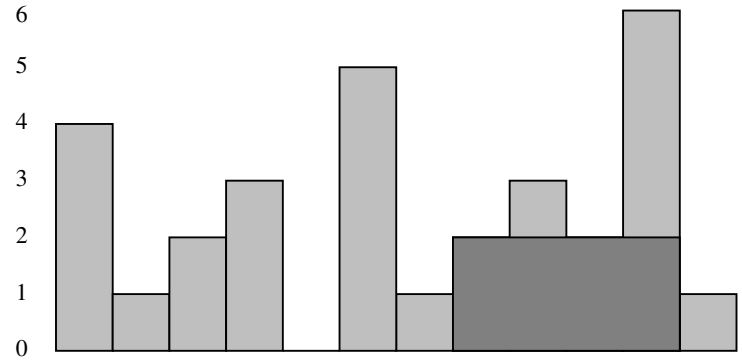
Pour ce problème, on reçoit en entrée la représentation vectorielle d'un histogramme, et l'on suppose que les rectangles qui composent ce dernier sont construits dans un matériau imperméable. Le problème consiste alors à calculer la quantité maximale d'eau que cet histogramme peut contenir. Par exemple, pour l'histogramme de l'exemple précédent, cette quantité d'eau est égale à 22, ce qui correspond à l'aire totale des parties foncées sur le diagramme suivant.



(a) Écrire en pseudo-code un algorithme permettant de résoudre ce problème, et en déterminer la complexité en temps. *Note :* Il existe une solution de complexité en temps $O(n)$.

(b) Traduire cet algorithme en langage C.

12. Étant donné un histogramme comme celui du problème précédent, on cherche à présent à calculer l'aire du plus grand rectangle qui est entièrement contenu dans sa représentation graphique. Par exemple, pour l'histogramme $[4, 1, 2, 3, 0, 5, 1, 2, 3, 2, 6, 1]$ qui illustre le problème précédent, le résultat doit être égal à 8, comme le montre le diagramme suivant.



(a) Écrire en pseudo-code un algorithme permettant de résoudre ce problème, et en déterminer la complexité en temps. *Note :* Tout comme pour le problème précédent, il existe une solution à ce problème de complexité en temps $O(n)$, où n est la taille de l'histogramme.

(b) Traduire cet algorithme en langage C.

13. On reçoit en entrée une matrice rectangulaire M contenant des valeurs binaires (c'est-à-dire, 0 ou 1). Le problème consiste à calculer le nombre d'éléments contenus dans la plus grande sous-matrice de M composée uniquement d'éléments égaux à 1. Par exemple, pour la matrice ci-dessous, le résultat doit être égal à 10. (La sous-matrice de plus grande taille est mise en évidence.)

$$\begin{bmatrix} 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$$

(a) Écrire en pseudo-code un algorithme permettant de résoudre ce problème, et en déterminer la complexité en temps. *Note :* Il existe une solution de complexité en temps $O(nm)$, où n et m sont respectivement le nombre de lignes et de colonnes de la matrice. Une approche suprenante mais élégante consiste à exploiter la solution développée pour le problème précédent.

(b) Traduire cet algorithme en langage C.

14. On demande de concevoir un algorithme recevant en entrée un vecteur d'entiers, capable de déterminer efficacement si ce vecteur contient une valeur majoritaire, c'est-à-dire présente dans au moins la moitié des éléments du vecteur. Dans le cas où elle existe, on souhaite également connaître cette valeur majoritaire. *Note :* Il est facile de résoudre ce problème avec une complexité en temps $O(n)$, où n est la taille du vecteur. Le défi consiste à minimiser la consommation de mémoire de l'algorithme. Il existe une solution (*algorithme MJRTY*) de complexité en espace égale à $O(1)$ (sans compter les données d'entrée).

15. Déterminer et décrire de la façon la plus simple possible l'opération effectuée par la fonction suivante [4].

```
unsigned f(unsigned n)
{
  unsigned r, s, u, v;

  for (r = 1, u = 1; v = u, r < n; r++)
     for (s = 1; u += v, s++ < r;);

  return u;
}
```

16. Déterminer et décrire de la façon la plus simple possible l'opération effectuée par la fonction suivante (*fonction* 91 *de McCarthy* [5]).

```
int m(int n)
{
  if (n > 100)
    return n - 10;

  return m(m(n + 11));
}
```

4. Source : Alan Turing, 1949. *Checking a large routine.* Report of a Conference on High Speed Automatic Calculating Machines, pp. 67-–69, University Mathematical Laboratory, Cambridge, UK.

5. Source : Zohar Manna, John McCarthy, 1970. *Properties of programs and partial function logic.* Machine Intelligence, Vol. 5, Edinburgh University Press.